

CCAR-F — Claude Certified Architect Foundations Study Pack

Contents

CCAR-F — Claude Certified Architect – Foundations: Study Pack Index	8
The exam (from the official Exam Guide v1.0, July 2026 — authoritative; download from the Pearson VUE Anthropic program page)	8
Files — priority-labeled for CCAR-F	9
Suggested study order (weight-driven)	11
How these notes were made	11
Related (pointer only)	11
Claude 101 — Study Notes	11
Overview	12
Key concepts map	12
Deep notes by topic	13
Decision trees / comparison tables	22
Common exam traps	22
Expanded self-check (Q&A)	23
Quick review checklist	25
Glossary	25
Reading tips	26
Claude 101 — Study Notes	26
Overview	26
Key concepts map	27
Deep notes by topic	28
Decision trees / comparison tables	36
Common exam traps	37
Expanded self-check (Q&A)	38
Quick review checklist	40
Glossary	40
Reading tips	41
AI Fluency: Framework & Foundations — Study Notes	41
Overview	41
Related titles in the AI Fluency collection (brief)	42
Key concepts map	42
Deep notes by topic	44
Decision trees / comparison tables	50
Common exam traps	52
Expanded self-check (Q&A)	52

Quick review checklist	54
Glossary	55
Study plan	55
AI Fluency: Framework & Foundations — Study Notes	56
Overview	56
Related titles in the AI Fluency collection (brief)	57
Key concepts map	57
Deep notes by topic	58
Decision trees / comparison tables	65
Common exam traps	67
Expanded self-check (Q&A)	67
Quick review checklist	69
Glossary	69
Study plan	70
Building with the Claude API — Exam Prep Study Notes	70
Overview	71
Key concepts (cheat sheet)	71
Deep notes by topic	72
Comparison tables (exam quick hits)	84
Exam tips	84
Exam traps (common wrong answers)	85
Self-check Q&A	86
CCAR-F Domain 4 gap card (added 2026-08-23)	87
G1. Schema design against fabrication (task 4.3)	87
G2. Validation, retry, and feedback loops (task 4.4)	87
G3. Multi-instance & multi-pass review (task 4.6)	87
G4. Gap-card Q&A	88
Quick review checklist	89
Glossary	89
Quick lab checklist (practice without copying course labs)	90
Source URLs (verify before exam day)	90
Building with the Claude API — Exam Prep Study Notes	90
Overview	91
Key concepts (cheat sheet)	91
Deep notes by topic	92
Comparison tables (exam quick hits)	103
Exam tips	104
Exam traps (common wrong answers)	105
Self-check Q&A	105
CCAR-F Domain 4 gap card (added 2026-08-23)	106
G1. Schema design against fabrication (task 4.3)	107
G2. Validation, retry, and feedback loops (task 4.4)	107
G3. Multi-instance & multi-pass review (task 4.6)	107
G4. Gap-card Q&A	107
Quick review checklist	108

Glossary	109
Quick lab checklist (practice without copying course labs)	109
Source URLs (verify before exam day)	110
Introduction to Model Context Protocol — Exam Prep Study Notes — Simplified Technical English	110
Overview	110
Key concepts (cheat sheet)	111
Deep notes by topic	111
Comparison tables (exam quick hits)	119
Exam tips	120
Exam traps	120
Self-check Q&A	121
Quick review checklist	122
Glossary	123
Study drills (original practice ideas)	123
Source URLs	124
CCAR-F Domain 2 mechanics supplement (added 2026-08-23)	124
S1. Structured error responses for MCP tools (task 2.2)	124
S2. MCP servers in Claude Code: scoping and configuration (task 2.4)	125
S3. Built-in tools: selection criteria (task 2.5)	126
S4. Supplement Q&A	126
S5. Supplement checklist	127
Introduction to Model Context Protocol — Exam Prep Study Notes	127
Overview	128
Key concepts (cheat sheet)	128
Deep notes by topic	128
Comparison tables (exam quick hits)	136
Exam tips	137
Exam traps	137
Self-check Q&A	138
Quick review checklist	139
Glossary	140
Study drills (original practice ideas)	140
Source URLs	141
CCAR-F Domain 2 mechanics supplement (added 2026-08-23)	141
S1. Structured error responses for MCP tools (task 2.2)	141
S2. MCP servers in Claude Code: scoping and configuration (task 2.4)	142
S3. Built-in tools: selection criteria (task 2.5)	143
S4. Supplement Q&A	143
S5. Supplement checklist	144
Claude Code in Action — Primary Study Notes	144
1. Course map (public objectives)	145
2. Steering long sessions	145
3. Configuration surfaces (advisory vs enforced)	147
Never include bypass-by-default in a team plugin	149

4. Permission modes (match mode to risk)	149
5. Hooks: unskippable control	150
6. Automation spectrum	150
7. Verify unsupervised work	151
High: auth, payments, migrations, IAM → mandatory human review regardless of green CI	152
8. Practical end-to-end workflow (cert checklist)	152
9. Anti-patterns (high-yield exam recognition)	152
10. Comparison: control surfaces deep dive	152
11. Exam tips	153
Headless without hooks is an incomplete answer for unattended safety.	154
12. Self-check Q&A (with answers)	154
13. Review checklist (before exam)	155
14. Glossary	155
15. Extra depth: subagents, MCP, and team rollout	156
16. Reading tip	157
17. Quick reference card (print-friendly)	157
CCAR-F Domain 3 mechanics supplement (added 2026-08-23)	157
S1. CLAUDE.md hierarchy, @import, and /memory (task 3.1)	158
S2. Custom slash commands and skills (task 3.2)	158
S3. Path-specific rules — .claude/rules/ (task 3.3)	159
S4. Plan mode, direct execution, and the Explore subagent (task 3.4)	159
S4b. Iterative refinement techniques (task 3.5)	160
S5. CI/CD integration flags (task 3.6)	160
S6. Supplement Q&A	161
S7. Supplement checklist	162
Claude Code in Action — Primary Study Notes	162
1. Course map (public objectives)	163
2. Steering long sessions	163
3. Configuration surfaces (advisory vs enforced)	165
Never ship bypass-by-default in a team plugin	167
4. Permission modes (match mode to risk)	167
5. Hooks: unskippable control	168
6. Automation spectrum	168
7. Verify unsupervised work	169
High: auth, payments, migrations, IAM → mandatory human review regardless of green CI	169
8. Practical end-to-end workflow (cert checklist)	169
9. Anti-patterns (high-yield exam recognition)	170
10. Comparison: control surfaces deep dive	170
11. Exam tips	171
Headless without hooks is an incomplete answer for unattended safety.	171
12. Self-check Q&A (with answers)	171
13. Review checklist (before exam)	173
14. Glossary	173
15. Extra depth: subagents, MCP, and team rollout	173
16. Reading tip	174
17. Quick reference card (print-friendly)	174
CCAR-F Domain 3 mechanics supplement (added 2026-08-23)	175

S1. CLAUDE.md hierarchy, @import, and /memory (task 3.1)	175
S2. Custom slash commands and skills (task 3.2)	176
S3. Path-specific rules — .claude/rules/ (task 3.3)	177
S4. Plan mode, direct execution, and the Explore subagent (task 3.4)	177
S4b. Iterative refinement techniques (task 3.5)	177
S5. CI/CD integration flags (task 3.6)	178
S6. Supplement Q&A	178
S7. Supplement checklist	179

Claude with Amazon Bedrock — Primary Study Notes 180

1. Course map (public modules)	180
2. Access patterns on Bedrock	181
3. Converse API vs InvokeModel (high-yield)	182
4. IAM essentials (least privilege)	184
For residency: combine profile choice with IAM Region conditions where policy requires it	184
5. Regions and data residency	184
6. Claude features through Bedrock (course themes)	185
Agent patterns still apply: parallelize, chain, route, and debug with traces.	186
7. Comparison snapshot: Bedrock vs Anthropic API vs Vertex	186
8. Exam traps (Bedrock-specific)	187
9. Exam tips	188
10. Minimal practice sketch (conceptual)	188
Write five eval cases. Grade with code asserts plus one model rubric.	188
11. Self-check Q&A (with answers)	188
12. Review checklist (before exam)	190
13. Glossary	190
14. Deeper decision trees	190
15. Shared skill themes mapped to Bedrock fields	191
16. Study rhythm	192
<i>Aligned to https://academy.claude.com/courses/claude-with-amazon-bedrock. Check live model IDs, Regions, and IAM examples in AWS Bedrock documentation before production use.</i>	192
17. Worked mini-scenarios (exam style)	192
18. Quick reference card	193
19. Extra Q&A (stretch)	193
20. Study close	193

Claude with Amazon Bedrock — Primary Study Notes 194

1. Course map (public modules)	194
2. Access patterns on Bedrock	194
3. Converse API vs InvokeModel (high-yield)	195
4. IAM essentials (least privilege)	197
For residency: combine profile choice with IAM Region conditions where policy requires it	198
5. Regions and data residency	198
6. Claude features through Bedrock (course themes)	199
Agent patterns still apply: parallelize, chain, route, debug with traces.	200
7. Comparison snapshot: Bedrock vs Anthropic API vs Vertex	200
8. Exam traps (Bedrock-specific)	201
9. Exam tips	201

10. Minimal practice sketch (conceptual)	201
Write five eval cases and grade with code asserts + one model rubric.	202
11. Self-check Q&A (with answers)	202
12. Review checklist (before exam)	203
13. Glossary	203
14. Deeper decision trees	204
15. Shared skill themes mapped to Bedrock fields	205
16. Study rhythm	205
<i>Aligned to https://academy.claude.com/courses/claude-with-amazon-bedrock. Verify live model IDs, Regions, and IAM examples in AWS Bedrock documentation before production use.</i>	205
17. Worked mini-scenarios (exam style)	205
18. Quick reference card	206
19. Extra Q&A (stretch)	206
20. Study close	207

Claude with Google Cloud's Vertex AI — Primary Study Notes **207**

1. Course map (public modules)	207
2. Setup and authentication	208
3. How the Vertex Claude API differs from Anthropic API	209
4. Regions and endpoints (high-value exam topic)	210
5. Model IDs on Vertex	211
Copy-paste across clouds is an intentional distractor.	211
6. Feature support (what to expect)	211
7. Course skill themes (shared with Bedrock track)	212
8. Vertex vs Bedrock vs Anthropic API (comparison sheet)	212
9. Exam traps (Vertex-specific)	213
10. Exam tips	213
11. Minimal practice sketch (conceptual)	214
12. Self-check Q&A (with answers)	214
13. Review checklist (before exam)	215
14. Glossary	216
15. Worked mini-scenarios	216
16. Deeper endpoint comparison table	217
17. Study rhythm	217

Rule to memorize: Enable in Garden, authenticate with Google, select the endpoint for residency, use publisher IDs, assume Messages-not-Converse, verify parity before you promise features.

18. Mapping shared Claude skills to Vertex request fields	217
19. Failure triage tree	218
20. Extra self-check Q&A	218
21. Production readiness mini-rubric (Vertex)	219
22. Side-by-side request anatomy (conceptual)	219
23. Team onboarding one-pager (what to paste in a wiki)	219
24. Quick reference card	220
25. Study close	220
26. Final flash drills	220

Claude with Google Cloud's Vertex AI — Primary Study Notes **221**

1. Course map (public modules)	221
2. Setup and authentication	221
3. How the Vertex Claude API differs from Anthropic API	222
4. Regions and endpoints (high-value exam topic)	223
5. Model IDs on Vertex	224
Copy-paste across clouds is an intentional distractor.	224
6. Feature support (what to expect)	224
7. Course skill themes (shared with Bedrock track)	225
8. Vertex vs Bedrock vs Anthropic API (comparison sheet)	226
9. Exam traps (Vertex-specific)	226
10. Exam tips	227
11. Minimal practice sketch (conceptual)	227
12. Self-check Q&A (with answers)	227
13. Review checklist (before exam)	229
14. Glossary	229
15. Worked mini-scenarios	230
16. Deeper endpoint comparison table	230
17. Study rhythm	230
Mantra: Enable in Garden, auth with Google, pick endpoint for residency, use publisher IDs, assume Messages-not-Converse, verify parity before promising features.	231
18. Mapping shared Claude skills to Vertex request fields	231
19. Failure triage tree	231
20. Extra self-check Q&A	232
21. Production readiness mini-rubric (Vertex)	232
22. Side-by-side request anatomy (conceptual)	232
23. Team onboarding one-pager (what to paste in a wiki)	233
24. Quick reference card	233
25. Study close	234
26. Final flash drills	234
Agentic Architecture & Orchestration — Domain 1 (27%)	234
Overview	234
Key map (task statement ↔ exam verbs)	235
Deep notes	235
Decision trees	239
Exam traps	240
Self-check Q&A	240
Pre-exam checklist	242
Glossary	242
Agentic Architecture & Orchestration — Domain 1 (27%)	243
Overview	243
Key map (task statement ↔ exam verbs)	244
Deep notes	244
Decision trees	248
Exam traps	249
Self-check Q&A	249
Pre-exam checklist	251
Glossary	251

Context Management & Reliability — Domain 5 (15%)	252
Overview	252
Key map	253
Deep notes	253
Decision trees	256
Exam traps	257
Self-check Q&A	257
Pre-exam checklist	259
Glossary	259
Context Management & Reliability — Domain 5 (15%)	260
Overview	260
Key map	260
Deep notes	261
Decision trees	264
Exam traps	264
Self-check Q&A	265
Pre-exam checklist	266
Glossary	267
CCAR-F Pro Tips — how to take this exam well	267
The numbers that drive strategy	268
If the stem says X → think Y	268
Terminology discipline	269
Distractor archetypes to expect	269
Freshness watchlist (re-verify the week of your exam)	269
The 48-hour plan	269
Day-of (online proctored)	269

CCAR-F — Claude Certified Architect – Foundations: Study Pack Index

Original exam-prep study notes. **Not** official Anthropic material; written in the author’s own words for focused reading and certification review.

Note: this folder (CCAR-F-architect-foundations, renamed 2026-08-23) covers **CCAR-F (Claude Certified Architect – Foundations)** — cert name and code confirmed against the official Pearson VUE Anthropic program listing. The separate CCAR-P-architect-professional directory covers the Professional exam.

The exam (from the official Exam Guide v1.0, July 2026 — authoritative; download from the Pearson VUE Anthropic program page)

Item	Value
Credential / code	Claude Certified Architect – Foundations / CCAR-F
Items	60 (MC + multiple-response; each item states how many to select)

Item	Value
Structure	4 scenarios drawn at random from a bank of 6 (all 6 printed in the guide)
Time	120 minutes
Pass	720 scaled (100–1,000); score report includes per-domain %
Fee / validity	\$125 USD / 12 months (free non-proctored on-time renewal)
Retakes	14 / 30 / 90-day waits; max 4 attempts per rolling 12 months
Delivery	Pearson VUE (online proctored or test center)

Blueprint (weights are study-time law)

#	Domain	Weight	Primary file(s)
1	Agentic Architecture & Orchestration	27%	08 (+ 03 §agent loop)
2	Tool Design & MCP Integration	18%	04 + its Domain 2 supplement (+ 03 §tools)
3	Claude Code Configuration & Workflows	20%	05 + its Domain 3 supplement
4	Prompt Engineering & Structured Output	20%	03 (schemas, tool_choice, batches, few-shot)
5	Context Management & Reliability	15%	09 (+ 05 §compaction)

Explicitly out of scope (don't spend exam-prep time here): cloud-provider configurations (AWS/GCP/Azure), streaming/SSE implementation, rate limits & pricing, auth protocols, prompt-caching implementation detail, Constitutional AI/RLHF, embeddings/vector DBs, computer use, vision, fine-tuning, MCP server hosting/deployment, token counting.

The 6 scenarios (4 appear on your form)

1. Customer Support Resolution Agent (D1, D2, D5)
2. Code Generation with Claude Code (D3, D5)
3. Multi-Agent Research System (D1, D2, D5)
4. Developer Productivity with Claude (D2, D3, D1)
5. Claude Code for CI (D3, D4)
6. Structured Data Extraction (D4, D5)

Files — priority-labeled for CCAR-F

#	File	Covers	CCAR-F priority
08	08-agentic-orchestration-agent-sdk.md	Domain 1 (27%): agentic loops, coordinator–subagent, Task tool/AgentDefinition, hooks, decomposition, sessions/forking	PRIMARY — study first
09	09-context-reliability.md	Domain 5 (15%): case facts, lost-in-the-middle, escalation, error propagation, scratchpads/manifests, calibration, provenance	PRIMARY
05	05-claude-code-in-action.md + Domain 3 supplement	Domain 3 (20%): steering/config concepts + CLAUDE.md hierarchy, @import,.claude/rules/ commands, SKILL.md frontmatter, -p/–output-format/–json-schema	PRIMARY
03	03-building-with-claude-api.md	Domain 4 (20%) + D1/D2 API halves: tool_use, tool_choice, schemas, batches, agent loop	PRIMARY
04	04-introduction-to-mcp.md + Domain 2 supplement	Domain 2 (18%): MCP concepts + isError, error taxonomy,.mcp.json vs ~/.claude.json, built-in tools	PRIMARY
01	01-claude-101.md	claude.ai product basics	Background only — CCAO-F territory, low CCAR-F yield
02	02-ai-fluency.md	4D fluency framework	Background only — low CCAR-F yield
06	06-claude-with-amazon-bedrock.md	Bedrock access, IAM, Converse	OUT OF SCOPE for CCAR-F (cloud-provider config excluded by the guide) — keep for platform work, skip for this exam
07	07-claude-with-google-cloud-vertex-ai.md	Vertex AI setup, ADC, endpoints	OUT OF SCOPE for CCAR-F — same as 06

(Files 01–07 originated as Claude Academy course notes — Claude 101, AI Fluency, Building with the Claude API, Intro to MCP, Claude Code in Action, Bedrock, Vertex. Files 08–09 and the supplements in 04/05 were added 2026-08-23 to close the pack’s gaps against the exam-guide blueprint.)

Exam craft (pacing, the 6-scenario strategy, reflex table, distractor patterns): [PRO-TIPS.md](#) — read after the content files.

Suggested study order (weight-driven)

1. **File 08** — Domain 1 is 27% and drives 3 of 6 scenarios; nothing else pays better.
2. **File 05 + its Domain 3 supplement** — 20%; the supplement holds the exact mechanics (rules globs, commands scope, CI flags) two of the twelve sample questions test.
3. **File 03** — Domain 4 (20%): tool_use schemas, tool_choice auto/any/forced, nullable fields, retry-with-feedback themes, Batches (50% / 24h / custom_id / no multi-turn tool calling).
4. **File 04 + its Domain 2 supplement** — 18%: descriptions, error taxonomy, isError, scoping.
5. **File 09** — Domain 5 (15%), then re-skim 08 §handoff (they interlock).
6. Walk all **12 sample questions** in the exam guide PDF cold; every one is answerable from files 03/04/05/08/09 — a miss tells you which file to re-read.
7. Do the guide’s **4 preparation exercises** hands-on if time allows (they map 1:1 to the scenario bank).

How these notes were made

- Files 01–07: public course pages used for titles, module lists, and stated learning objectives; prose is original synthesis.
- Files 08–09 + supplements: written directly against the official exam-guide task statements, with SDK/CLI mechanics verified against current public Claude Code / Agent SDK docs (2026-08). Names drift — verify — resume, SKILL.md frontmatter, and Agent SDK option spellings in live docs before exam day. Notably: the exam guide says **Task tool**; current Claude Code renamed it **Agent** — answer exams with the guide’s term.
- Currency fixes 2026-08-23: temperature/sampling-param removal on current frontier models (03, 06, 07), adaptive thinking vs deprecated thinking budgets (06), mid-conversation system messages note (03).

Related (pointer only)

- Advanced MCP (after intro notes in 04–...): [Model Context Protocol: Advanced Topics](#)

Claude 101 — Study Notes

Note: This edition follows the ASD-STE100 writing rules: simple present tense, active voice, one word = one meaning, sentences of 20–25 words or less. Technical names are exceptions and stay as written. The names include Claude, Cowork, Projects, Artifacts, Skills, Connectors, Enterprise Search, Research, Claude Code, Claude Tag, Claude Design, MCP, RAG, 4D, Delegation, Description, Discernment, and Diligence. Product UIs and plan gates change. Check current Help Center articles before the exam.

Disclaimer: These notes are original study notes for exam prep. They are **not** official Anthropic material. Course outline source: <https://academy.claude.com/courses/claude-101>. Public Anthropic Help Center articles provide the feature details. Product UIs and plan gates can change. Check them before exam day.

Overview

Claude 101 is Anthropic's starter path. It helps you bring Claude into everyday work. Public course pages describe **13 lessons and 1 quiz** (about 2.5 hours). The path ends with a completion badge. The course is for anyone with a Claude account (Free through Enterprise). A few features (for example Cowork and Enterprise Search) need a paid plan or a Team/Enterprise plan for hands-on use. The ideas still matter for exams.

What you can do after you study this material

- Explain what Claude is and run a useful first conversation
- Write clearer prompts, iterate when results do not meet the goal, and connect prompting to the AI Fluency 4Ds.
- Select among three “shapes of work” in the desktop app: turn-by-turn Chat, handing work off (Cowork), and building software (Code)
- Use Projects, Artifacts, and Skills to organize context and repeatable work
- Expand reach with Connectors, Enterprise Search, and Research
- Map Claude to common roles and know adjacent products (Claude Code, Claude Tag, Claude Design, Claude for Microsoft 365, Claude in Chrome)

Typical module arc (aligned to public outlines)

1. Meet Claude — what it is. First conversation
2. Getting better results — prompting. Chat / Cowork / Code
3. Organizing work & knowledge — Projects, Artifacts, Skills
4. Expanding reach — Connectors, Enterprise Search, Research
5. Putting it together — role use cases. Other ways to work with Claude

These notes rewrite concepts in original language for reading and Claude cert prep. They are **not** a transcript of the course. The notes keep original study wording in expanded form. Do not treat this file as Academy lesson prose.

How to use this file as a primary study source

1. Read the Key concepts map one time
2. Read Deep notes in order (features build on each other)
3. Practice Decision trees / comparison tables until you can select the correct tool without notes
4. Read Common exam traps
5. Answer Self-check without looking at the answers, then check answers
6. Use the Quick review checklist the night before the exam
7. Pair with 02-ai-fluency.md — Claude 101 applies the framework. Fluency explains it

Key concepts map

Idea	One-line meaning	Fluency link
Prompt pattern	Give role/stage, task, and rules (context, format, constraints)	Description
Iteration	The first output is a draft. Refine it with specific feedback.	Description–Discernment loop
Chat	Turn-by-turn collaboration in conversation	Augmentation default
Cowork	Hand off multi-step work with files/apps (plan-gated)	Agency-leaning automation
Code	Software-build shape of work	Implementation agency
Projects	Workspace with knowledge + instructions across chats	Standing Description
Artifacts	Standalone outputs in a side panel (docs, code, apps)	Visible deliverable
Skills	Reusable <i>how-to</i> procedures Claude can run	Process Description
Connectors	Links to web services or local desktop tools	Platform reach
Enterprise Search	Company-knowledge retrieval (Team/Enterprise)	Internal grounding
Research	Broader, multi-source investigation. Not a single chat guess.	Multi-step search agency
4D Fluency	Delegation, Description, Discernment, Diligence	Whole practice
RAG (projects)	Retrieve relevant chunks when knowledge grows large	Capacity without overload
Progressive disclosure (skills)	Load only relevant skill bits when needed	Context hygiene

Memory peg: *Shape → Context → Deliverable → Procedure → Reach*

- Shape = Chat / Cowork / Code
- Context = Projects (+ instructions)
- Deliverable = Artifacts
- Procedure = Skills
- Reach = Connectors / Enterprise Search / Research

Deep notes by topic

1. What Claude is (and is not)

Claude is Anthropic’s AI assistant for writing, analysis, coding help, research, and collaborative problem-solving. Treat Claude as a **capable collaborator**. Do not treat Claude as an oracle. Claude predicts helpful responses from patterns in training. Claude also uses what you put in the current conversation and in connected context.

Exam-style takeaway: Confidence in tone is not the same as correctness. High-stakes facts need independent verification, citations, or web grounding.

Practical tip: Start every new domain with a short “contract”: goal, audience, constraints, and what “done” looks like. That sets Description (see AI Fluency). Do this before you try clever wording.

Pitfall: You send a vague request (“write something about Q3”) and then blame the model. Vague input gives generic output.

Worked example (original) — first contract

Bad: “Help with onboarding.”

Better: “You are helping a new CS manager onboard. Deliver a 1-page checklist for week 1. Audience: managers who hate filler. Must include: access requests, buddy meeting, first ticket shadow. Avoid: motivational quotes. Ask me 3 clarifying questions before drafting if anything is ambiguous.”

If the exam asks X, think Y

- X: “Why did Claude invent a statistic?” → Y: LLMs generate fluent continuations. If you do not ground or tools they can invent facts. Verify.
- X: “Is Claude always right when confident?” → Y: No. Tone is not the same as truth. You must have Discernment.

2. Effective first conversations

A strong first message usually includes:

1. **Who you are / role** (or who Claude should act as)
2. **What you need** (concrete deliverable)
3. **Context** (audience, stakes, prior decisions)
4. **Format and limits** (length, structure, tone, must-include / must-avoid)

Public lesson material often uses a simple pattern such as **role + task + context + format**. Another pattern is a three-part frame of stage, task, and rules. Memorize the meaning of the pattern. Do not memorize a brand slogan: **orient** → **specify** → **constrain**.

Self-check habit: Before you send, ask “Could a smart new hire succeed with only this message?” If the answer is no, add context.

Anti-patterns

Anti-pattern	Why it fails	Repair
One-word asks	No deliverable	Name the artifact
Hidden constraints	Wrong tone/length	State must-avoid
Paste of whole email threads	Noise is larger than signal	Summarize stakes + paste only key excerpts
“Act as expert” with no task	Role without a task	Role + concrete output
Never iterating	You stay with draft 1	Precise, targeted feedback

Worked example (original) — email rewrite

Prompt skeleton:

Role: You are my editor for customer emails.

Task: Rewrite the draft below into a calm apology.

Context: Customer waited 6 days; we missed an SLA; refund already approved.

Format: 120–150 words; subject line + body; no legalese; end with one clear next step.

Rules: Do not blame the customer; do not invent policy; flag any claim you cannot verify
[paste draft]

3. Getting better results: common fixes

When output misses the goal, diagnose the failure mode:

Symptom	Likely cause	Fix
Too generic	Thin context	Add audience, constraints, history
Wrong length	Unstated expectation	Set word/paragraph limits
Wrong structure	Format not shown	Give a skeleton or example
Wrong tone	Default “helpful professional”	Name tone. Paste a sample
Plausible but wrong	Hallucination risk	Verify. Ask for sources/confidence. Use search/Research
Over-agreeable	Sycophancy	Ask for critique / opposing view
Thread chaos	Too many pivots	New chat with cleaned prompt

Iteration mindset

- Treat draft 1 as a starting point
- Feedback must be **precise** (“cut para 1. Make CTA specific”). Do not only say “make better”
- If the thread is mixed and unclear, **start a clean chat** with a clearer prompt

Lightweight evals (Discernment in practice)

For work that repeats:

1. Collect 5–10 past good examples you already trust
2. Write prompts that recreate them
3. Compare Claude’s output to your gold examples
4. Record gaps. Add missing rules into prompts, Projects, or Skills
5. Only then trust the workflow on new data

This is the same idea as a “delegation diligence loop”: test on known answers before you trust new ones.

Worked example (original) — eval for weekly metrics blurb

- Gold set: last 4 Friday write-ups you already published
- Input: same raw CSV + same Project instructions
- Score: factual match, length, chart callouts, no invented drivers
- Gap found: Claude invents “seasonality” without data → add rule “Only claim drivers present in the numbers or my notes”
- Add the rule into Project instructions. Test one more time. Then use the workflow on live data.

If the exam asks X, think Y

- X: “Output is generic” → Y: Weak Description / missing audience
 - X: “How do you improve without better ‘prompt unspecified mechanism’?” → Y: Iteration + evals + Projects/Skills
-

4. Three shapes of work (desktop app)

Public objectives stress three modes of collaboration:

1. **Turn by turn (Chat)** — You draft, ask, and analyze in conversation, one turn at a time. Best when you want tight control each step.
2. **Handing work off (Cowork)** — Longer sessions where Claude works with files and applications with more autonomy. Best for multi-step operational tasks. Often plan-gated (paid plans). Sessions can organize files, synthesize research, run scheduled tasks, and use projects scoped for Cowork.
3. **Building software (Code / Claude Code)** — Implementation-oriented work: repos, agents, coding workflows.

Exam tip: Match the *shape of the task*. Do not select a mode only because it looks new. Exploratory idea work → Chat. “Clean this folder of invoices and summarize” → Cowork-style handoff. “Ship a feature in the repo” → Code.

Pitfall: You use agentic handoff for a task that still needs turn-by-turn judgment (sensitive client email, legal wording).

Decision tree — select a shape

Is the primary goal writing/judging tone sentence-by-sentence?

YES → Chat

NO ↓

Is the primary goal multi-step file/app ops you can leave running?

YES → Cowork (if plan allows)

NO ↓

Is the primary goal software in a repo / coding agent workflow?

YES → Code

NO → Default Chat, then escalate if the task grows

Comparison — Chat vs Cowork vs Code

Dimension	Chat	Cowork	Code
Control per step	Highest	Medium (plan then run)	Medium-high when you steer
Best for	Drafts, Q&A, judgment	Ops across files/apps	Implementation
Autonomy	Low	Higher	Higher in repo
Plan gates	Broadest access	Often paid	Separate coding surface
Risk if misused	Wasted time	Side effects without review	Bad commits / scope grows without control
Diligence focus	Review text	Review actions + files	Review diffs + tests

Anti-patterns

- You start Cowork for a one-paragraph rewrite
- You use Code mode to “think about strategy” with no repo
- You leave Cowork without review on irreversible actions, and you set no checkpoints

5. Projects — knowledge hubs

Projects are dedicated workspaces with:

- Their own chat histories
- **Project knowledge** (uploaded docs Claude can reuse)
- **Project instructions** (tone, expertise, standing rules)
- Optional team sharing of the same shared context (Team/Enterprise)

Public Help Center framing: Projects are available broadly (including Free, with a small project count limit on Free). On paid plans, large knowledge bases can shift into **RAG mode**. Claude retrieves relevant chunks. Claude does not put everything into the window. This expands capacity while it keeps quality.

Use Projects when: You have a stable domain (product docs, brand voice, client briefs) that many chats should share.

Do not confuse Projects with Skills: Projects store *what* Claude should know. Skills encode *how* to run a procedure. Help Center contrast: Projects = static background knowledge loaded in that workspace. Skills = specialized procedures that activate dynamically and can work across Claude.

Worked example (original) — “Acme Support” Project

- Knowledge: FAQ PDF, severity matrix, refund policy, tone samples
- Instructions: “Never invent policy. Quote policy titles. Escalate legal claims to human. Tone: calm, plain.”
- Chats inside Project: ticket rewrites, macro drafts, weekly theme summaries
- Outside Project: do **not** expect the same policy grounding

Anti-patterns

- One Project for every client mixes client contexts.
- You upload secrets that you would not put in a shared drive
- You write no instructions, then you blame Claude: “Claude forgot our voice”
- You treat Project knowledge as always fully loaded when RAG may retrieve selectively. Ask pointed questions. Cite files.

If the exam asks X, think Y

- X: “How do I reuse brand voice across chats?” → Y: Project knowledge + instructions
 - X: “Knowledge got huge—what happens conceptually?” → Y: Retrieval of relevant chunks (RAG-style), especially on paid plans
-

6. Artifacts — standalone outputs

Artifacts appear in a dedicated panel beside chat. Claude does not hide a long HTML, code, or document output in the thread. Claude renders something you can preview, copy, download, or iterate on.

Claude tends to create an artifact when content is significant, self-contained, editable/reusable, and likely to be referenced later. The content is often longer or more structured than a short chat reply.

Common types (conceptually):

- Documents (markdown/text reports, plans, posts)
- Code snippets ready to copy
- Interactive / visual pieces (simple apps, charts, calculators, SVG/diagrams, React components)

Tips

- Ask explicitly: “Create this as an artifact” if the content stays stuck in chat
- Toggle preview vs underlying code when you debug
- Use Artifacts for shareable deliverables. Keep chat for negotiation and critique
- Version selector helps you explore directions without losing prior work
- Publishing/sharing and advanced behaviors (AI-powered artifacts, MCP inside artifacts, persistent storage) depend on plan and settings. Know that they exist. Check live docs for exam specifics
- Code execution / file creation capabilities are often required for modern artifact workflows

Worked example (original)

Ask for a “pricing comparison calculator” as an artifact. Iterate on UI in the panel. Download or share when the artifact is stable. Keep negotiation (“should we include annual discount?”) in chat.

Anti-patterns

- You leave a 200-line HTML app only in chat bubbles
 - You paste the whole file again every time when you edit. You do not target the artifact.
 - You publish without a check of personal vs shared storage (privacy)
-

7. Skills — procedural machines

Skills are reusable instruction sets (often folders of instructions, scripts, and resources) for multi-step workflows. Examples include brand process, PDF packaging, call-preparation procedure, and blog

pipeline. Skills improve consistency via **progressive disclosure**. Claude loads what is relevant. Claude does not load every skill into context.

Types to recognize at a high level:

- Anthropic-maintained skills (e.g., stronger document creation flows)
- Custom skills you author (Markdown instructions. Optional scripts)
- Organization-provisioned skills (Team/Enterprise)
- Partner skills in a directory (often paired with MCP connectors)

Projects vs Skills (exam favorite)

	Projects	Skills
Purpose	Store knowledge	Define processes
Best for	Long-term context, team reference	Repeatable methodology
Persis- tence	Across chats in the project	When the skill is invoked / relevant
Metaphor	Library + standing rules	Playbook / SOP
Loading	Background in that workspace	Dynamic / on-demand

They combine well. A “customer call prep” Skill can pull profiles from a Project’s knowledge base.

Skills vs Connectors/MCP (short)

- Connectors/MCP = *access* to tools and data
- Skills = *how* to use those tools for a workflow
- Use both: MCP opens access. Skills teach the workflow.

Skills vs custom instructions

- Custom instructions = broad, always-on preferences
- Skills = task-specific, loaded when relevant

Pitfall: You encode single-use trivia as a Skill. Or you put a giant SOP into a Project with no procedure.

Worked example (original) — “Weekly status” Skill

Procedure steps: pull themes from notes → group by risk → draft bullets → ask a human to confirm blockers → format for Slack. Attach the Skill only if the procedure repeats weekly.

If the exam asks X, think Y

- X: “Same procedure every time across chats” → Y: Skill
- X: “Same docs and voice every time in a workspace” → Y: Project
- X: “Need live Asana data” → Y: Connector/MCP (+ Skill for the workflow)

8. Connectors — expanding reach

Connectors link Claude to external tools.

- **Web connectors** — Cloud apps (Drive, Notion, Slack, Asana, Gmail, etc.), often via MCP-style integrations
- **Desktop extensions** — Local files and native apps via the desktop app

Setup pattern: find in directory → connect → authenticate → grant least privilege → test with “Can you see X?”

Diligence angle: Install only trusted connectors. Permissions are revocable. Treat connected data as in-scope for privacy reviews.

Anti-patterns

- You connect everything without a current need
- You ask Claude to email externally without an approval gate
- You assume connector access equals unlimited org-wide visibility (auth and scopes matter)

9. Enterprise Search and Research

- **Enterprise Search** — Reach into company knowledge (typically Team/Enterprise). It is often a dedicated org knowledge workspace (for example, an “Ask Your Org”-style project). It has guided connectors and search-optimized behavior. Use it when the answer is in internal docs, not the open web. It is optimized for relatively quick internal retrieval with citations.
- **Research** — Multi-step, agentic investigation across sources when a single chat guess is not enough. Research is good for literature-style or competitive scans with grounding. It is typically on a paid plan. Web search must be available. Research can use web plus connected internal context. It produces thorough answers with checkable citations. It can consume usage limits faster.

Compare (exam favorite)

Enterprise Search	Research	Ordinary web search in chat
Main Find internal knowledge fast job	Deep multi-step synthesis	Quick external facts
Typ- Connected company tools i-cal sources	Web + integrations, many steps	Web
DepthTargeted retrieval	Broad investigation	Light
Plan Team/Enterprise notes	Paid plans common	Depends on settings

Exam tip: Use Research/Search when freshness or source grounding matters. Still apply Discernment. Sources can be incomplete or conflicting.

Worked example (original)

- “Where is our PTO policy?” → Enterprise Search / internal connectors
- “Compare three competitors’ pricing pages and summarize changes since last quarter” → Research
- “What time is the Singapore GP this year?” → Web search in chat may suffice

If the exam asks X, think Y

- X: “Internal Slack + Drive answer” → Y: Enterprise Search / connectors
- X: “Multi-angle report with citations in minutes” → Y: Research
- X: “One quick public fact” → Y: Web search, not full Research

10. Role-based use and product tour

Course wrap-ups typically show Claude across roles (marketing calendars, follow-ups, analysis, ops). They also point to specialized surfaces:

- **Claude Code** — coding agents / software work
- **Claude Tag / Design** — tagged or design-oriented workflows (product naming may evolve. Know they exist as adjacent tools)
- **Claude for Microsoft 365** — work inside M365 surfaces
- **Claude in Chrome** — browser-context assistance

For exams: know **when** each shape helps. You do not need every UI click.

Role mapping cheat sheet (original)

Role need	Prefer
Draft + edit copy	Chat + Artifacts
Recurring brand SOP	Skill (+ Project knowledge)
Clean folders / multi-app ops	Cowork
Ship code	Code
Company wiki answers	Enterprise Search
Competitive landscape brief	Research
Browser page help	Claude in Chrome
Outlook/Teams embedded help	Claude for Microsoft 365

11. Mapping Claude 101 → AI Fluency 4Ds

Feature / habit	Strongest D	Why
Select Chat vs Cowork vs Code	Delegation	Task split + platform awareness
Prompt contracts, Projects, Skills	Description	Goals, process, behavior
Evals, citation checks, “fix with Claude”	Discernment	Critique outputs/process
Connector least privilege, disclosure, human owns the final	Diligence	Responsibility

Remember: Features are **infrastructure**. They are not a substitute for judgment.

Decision trees / comparison tables

Master chooser — “I need Claude to...”

Need reusable domain knowledge across many chats?

→ Project (+ instructions)

Need a repeatable multi-step method?

→ Skill

Need a tangible deliverable to preview/share?

→ Artifact

Need live external/app data?

→ Connector / MCP

Need internal company answer quickly?

→ Enterprise Search

Need deep multi-source investigation?

→ Research

Need tight editorial control?

→ Chat

Need hands-off multi-step ops?

→ Cowork

Need repo/implementation work?

→ Code

Projects vs Skills vs Artifacts vs Connectors

Need	Reach for
What Claude should know here	Project
How Claude should run a procedure	Skill
What Claude produced as a thing	Artifact
What systems Claude can touch	Connector
Where company answers are kept	Enterprise Search
How to investigate broadly	Research

Failure → feature map

Failure	First fix
Generic answers	Better prompt / Project context
Inconsistent weekly procedure	Skill
Lost long HTML in scrollbar	Artifact
Cannot see Drive/Slack	Connector
Invented market numbers	Research/web + verify
Wrong tool for autonomy level	Select Chat/Cowork/Code again

Common exam traps

1. **Projects = Skills** — False. Knowledge/workspace vs procedure.

2. **Artifacts are just pretty chat** — False. Artifacts are standalone panel deliverables.
 3. **More autonomy = less oversight** — False. Cowork/Code/Research need *more* Discernment and Diligence.
 4. **Confident tone = verified fact** — False.
 5. **Free plan can do every hands-on demo** — False. Cowork / Enterprise Search / some Research behaviors are plan-gated. Definitions are still testable.
 6. **Connectors replace Skills** — False. Access is not the same as procedure.
 7. **Enterprise Search = Research** — False. Quick internal retrieval vs deep multi-step investigation.
 8. **Custom instructions = Skills** — False. Always-on broad prefs vs task-specific loaded procedures.
 9. **RAG means Claude read every file fully every time** — Misleading. Retrieval pulls relevant chunks when knowledge is large.
 10. **One Project for all clients is fine** — Risky. Client contexts mix. Prefer separate Projects for sensitive matters.
 11. **“Make it better” is good feedback** — Weak. Give precise, targeted feedback.
 12. **Publishing an artifact has no privacy angle** — False. Shared storage / org sharing rules matter.
 13. **Fluency is only prompt tricks** — False. Fluency is a 4D collaboration skill (see course 02).
 14. **You can skip human accountability if Claude drafted it** — False. You own the work product.
-

Expanded self-check (Q&A)

Q1. A teammate says Projects and Skills are the same because both “give Claude context.” How do you correct them?

A1. Projects mainly hold reusable *knowledge and standing instructions* across chats in a workspace. Skills encode *repeatable procedures*. Claude invokes or loads a Skill when it is relevant. Knowledge is not the same as process. They complement each other.

Q2. You need a one-page client apology email and must control every sentence. Which desktop “shape of work” fits best and why?

A2. Chat (turn-by-turn). High judgment and tone control favor conversational iteration. Do not use a long handoff.

Q3. Claude’s market stats look polished but you cannot verify them. What should you do?

A3. Treat the stats as possibly wrong. Verify them independently. Ask for sources and confidence. Enable web or Research grounding for current facts.

Q4. Name one symptom of weak Description and one fix.

A4. Example: generic output → add audience, constraints, and a format example.

Q5. What is a lightweight eval for a weekly metrics write-up?

A5. Take last month’s known-good write-ups. Prompt Claude on the same raw inputs. Compare the output to your originals. Then add missing rules into the prompt, Project, or Skill. Do this before you use the workflow on new data.

Q6. When project knowledge gets huge, what conceptual shift helps capacity?

A6. Shift from loading all files into context to retrieving only relevant chunks (RAG-style). Answers stay grounded. The window does not overflow. Paid plans emphasize this.

Q7. Web connector vs desktop extension—key difference?

A7. Web connectors link cloud services. Desktop extensions run locally through the desktop app for files and native apps.

Q8. Map “I am accountable for the final report and will disclose AI help if asked” to which Fluency D?

A8. Diligence (responsibility, transparency, ethical use).

Q9. When should you prefer Research over a single web-search answer?

A9. Prefer Research when you need multi-step, multi-source synthesis with systematic exploration and citations. Do not use full Research for one quick fact.

Q10. How does Enterprise Search differ from a normal Project?

A10. Enterprise Search is specialized for org knowledge retrieval. It is often auto-provisioned, search-optimized, and paired with guided connectors. Normal Projects are general-purpose workspaces. You can customize their instructions and knowledge.

Q11. Give an example of a wrong use of Cowork.

A11. You hand off a sensitive legal letter that still needs sentence-level human judgment, and you set no tight checkpoints. Or you run irreversible file operations with no review.

Q12. Skills vs MCP connectors in one sentence each.

A12. Skills teach *how* to perform a workflow. MCP/connectors provide *access* to external tools and data.

Q13. What should you put in Project instructions vs Project knowledge?

A13. Instructions = standing rules/tone/role behavior. Knowledge = reference files and facts Claude should draw from.

Q14. Name three common artifact content types.

A14. Any three: documents, code snippets, HTML pages, SVG/diagrams, interactive components/calculators.

Q15. Why start a new chat instead of a messy thread that continues?

A15. Mixed, unclear context causes drift. A clean prompt with clarified goals often works better than endless patches.

Q16. Map Artifacts to a Fluency idea.

A16. Artifacts make the *product* of Description visible and editable. This supports Discernment on a concrete deliverable.

Q17. Free-plan common exam error: which ideas still matter even if you cannot click every feature?

A17. Definitions and fit-for-task reasoning for Cowork, Enterprise Search, Research, and similar features. The exam can still test these ideas.

Q18. What is progressive disclosure in Skills?

A18. Claude loads only relevant skill information for the task. This reduces context overload.

Q19. You need Claude to follow the same PDF packaging steps every Friday. Project or Skill—primary choice?

A19. Skill (procedure). Optionally keep brand assets in a Project.

Q20. “AI wrote it, so I’m not responsible.” Which D fails?

A20. Diligence—humans remain accountable for AI-assisted work.

Q21. Pick a tool: summarize themes across connected Slack + Drive for “what did we decide about pricing?”

A21. Enterprise Search / org knowledge connectors (internal). Use Research if you also need deep external competitive synthesis.

Q22. What feedback is better than “make it better”?

A22. Precise, targeted instructions: what to cut, what to add, tone target, length, must-keep facts.

Q23. Why separate client matters into different Projects?

A23. Reduce context bleed. Keep histories, files, and instructions separate for accuracy and confidentiality.

Q24. Code vs Chat for “explain this stack trace and propose a patch,” when you are not in a repo agent workflow?

A24. Chat can analyze and propose. Switch to Code when you want agentic implementation in the actual repository.

Q25. How do you combine Project + Skill + Artifact in one workflow?

A25. Example: the Project holds product docs. The Skill runs the “release notes ritual.” The Artifact is the final markdown notes that you preview and share.

Quick review checklist

- ☐ I can explain Claude as a collaborator, not an oracle
 - ☐ I can write orient → specify → constrain first messages
 - ☐ I can diagnose generic / wrong tone / hallucination and fix them
 - ☐ I can select Chat vs Cowork vs Code with a reason
 - ☐ Projects = knowledge + instructions. Skills = procedures. Artifacts = deliverables
 - ☐ Connectors = access. Skills = how. Do not mix these ideas
 - ☐ Enterprise Search (internal quick) ≠ Research (deep multi-step)
 - ☐ RAG / progressive disclosure = capacity without putting every file into context
 - ☐ Plan gates exist. Definitions still matter on exams
 - ☐ Map features to Delegation / Description / Discernment / Diligence
 - ☐ Human owns the final work product
 - ☐ I reviewed anti-patterns and exam traps once aloud
-

Glossary

Term	Short definition
Artifact	Standalone, previewable output in a side panel

Term	Short definition
Chat	Turn-by-turn conversational shape of work
Connector	Integration that links Claude to external or local tools
Cowork	Hands-off / multi-step operational shape of work (often paid)
Code (shape)	Software-build collaboration surface
Custom instructions	Broad always-on preferences
Enterprise Search	Org-knowledge retrieval workspace/capability
Hallucination	Fluent but false content
MCP	Model Context Protocol — tool/data connection standard
Progressive disclosure	Load only relevant instructions/resources
Project	Workspace with chats, knowledge, instructions
Prompt contract	Goal, audience, constraints, definition of done
RAG	Retrieval Augmented Generation — fetch relevant chunks
Research	Agentic multi-step investigation with citations
Skill	Reusable procedural how-to package
Sycophancy	You agree too much instead of seeking truth
4Ds	Delegation, Description, Discernment, Diligence

Reading tips

- Read the Key concepts map before Deep notes
- After each deep topic, answer one self-check aloud
- Practice the master chooser decision tree from memory
- Pair with the AI Fluency notes (02-ai-fluency.md)—Claude 101 applies the framework. Fluency explains it
- Night before: checklist + glossary only

End of Claude 101 study notes.

Claude 101 — Study Notes

Disclaimer: These are original study notes for exam prep — **not** official Anthropic material. Course outline source: <https://academy.claude.com/courses/claude-101>. Feature details cross-checked against public Anthropic Help Center articles; product UIs and plan gates can change—verify before exam day.

Overview

Claude 101 is Anthropic’s starter path for bringing Claude into everyday work. Public course pages describe **13 lessons and 1 quiz** (about 2.5 hours), ending with a completion badge. It is aimed at anyone with a Claude account (Free through Enterprise). A few features (for example Cowork and Enterprise Search) need a paid or Team/Enterprise plan to try hands-on; the ideas still matter for exams.

What you should be able to do after studying this material

- Explain what Claude is and run a useful first conversation

- Write clearer prompts, iterate when results miss, and connect prompting to the AI Fluency 4Ds
- Choose among three “shapes of work” in the desktop app: turn-by-turn Chat, handing work off (Cowork), and building software (Code)
- Use Projects, Artifacts, and Skills to organize context and repeatable work
- Expand reach with Connectors, Enterprise Search, and Research
- Map Claude to common roles and know adjacent products (Claude Code, Claude Tag, Claude Design, Claude for Microsoft 365, Claude in Chrome)

Typical module arc (aligned to public outlines)

1. Meet Claude — what it is; first conversation
2. Getting better results — prompting; Chat / Cowork / Code
3. Organizing work & knowledge — Projects, Artifacts, Skills
4. Expanding reach — Connectors, Enterprise Search, Research
5. Putting it together — role use cases; other ways to work with Claude

These notes rewrite concepts in original language for reading and Claude cert prep. They are **not** a transcript of the course. Keep ORIGINAL study wording expanded here; do not treat this as Academy lesson prose.

How to use this file as a primary study source

1. Skim the Key concepts map once
2. Read Deep notes in order (features build on each other)
3. Drill Decision trees / comparison tables until you can pick the right tool cold
4. Walk Common exam traps
5. Answer Self-check without peeking, then check answers
6. Use the Quick review checklist the night before the exam
7. Pair with 02-ai-fluency.md — Claude 101 applies the framework; Fluency explains it

Key concepts map

Idea	One-line meaning	Fluency link
Prompt pattern	Give role/stage, task, and rules (context, format, constraints)	Description
Iteration	First output is a draft; refine with specific feedback	Description–Discernment loop
Chat	Turn-by-turn collaboration in conversation	Augmentation default
Cowork	Hand off multi-step work with files/apps (plan-gated)	Agency-leaning automation
Code	Software-building shape of work	Implementation agency
Projects	Workspace with knowledge + instructions across chats	Standing Description
Artifacts	Standalone outputs in a side panel (docs, code, apps)	Visible deliverable
Skills	Reusable <i>how-to</i> procedures Claude can run	Process Description
Connectors	Links to web services or local desktop tools	Platform reach
Enterprise Search	Company-knowledge retrieval (Team/Enterprise)	Internal grounding

Idea	One-line meaning	Fluency link
Research	Broader, multi-source investigation beyond a single chat guess	Multi-step search agency
4D Fluency	Delegation, Description, Discernment, Diligence	Whole practice
RAG (projects)	Retrieve relevant chunks when knowledge grows large	Capacity without overload
Progressive disclosure (skills)	Load only relevant skill bits when needed	Context hygiene

Memory peg: *Shape → Context → Deliverable → Procedure → Reach*

- Shape = Chat / Cowork / Code
- Context = Projects (+ instructions)
- Deliverable = Artifacts
- Procedure = Skills
- Reach = Connectors / Enterprise Search / Research

Deep notes by topic

1. What Claude is (and is not)

Claude is Anthropic’s AI assistant for writing, analysis, coding help, research, and collaborative problem-solving. Treat it as a **capable collaborator**, not an oracle. It predicts helpful responses from patterns in training plus whatever you put in the current conversation and connected context.

Exam-style takeaway: Confidence in tone ≠ correctness. High-stakes facts need independent verification, citations, or web grounding.

Practical tip: Start every new domain with a short “contract”: goal, audience, constraints, and what “done” looks like. That sets Description (see AI Fluency) before you chase clever wording.

Pitfall: Dumping a vague ask (“write something about Q3”) and blaming the model. Vague in → generic out.

Worked example (original) — first contract

Bad: “Help with onboarding.”

Better: “You are helping a new CS manager onboard. Deliver a 1-page checklist for week 1. Audience: managers who hate fluff. Must include: access requests, buddy meeting, first ticket shadow. Avoid: motivational quotes. Ask me 3 clarifying questions before drafting if anything is ambiguous.”

If the exam asks X, think Y

- X: “Why did Claude invent a statistic?” → Y: LLMs generate fluent continuations; without grounding/tools they can invent. Verify.
- X: “Is Claude always right when confident?” → Y: No. Tone ≠ truth. Discernment required.

2. Effective first conversations

A strong first message usually includes:

1. **Who you are / role** (or who Claude should act as)
2. **What you need** (concrete deliverable)
3. **Context** (audience, stakes, prior decisions)
4. **Format and limits** (length, structure, tone, must-include / must-avoid)

Public lesson material often frames a simple pattern such as **role + task + context + format**, or a three-part framing of stage, task, and rules. Memorize the *spirit*, not a brand slogan: **orient** → **specify** → **constrain**.

Self-check habit: Before sending, ask “Could a smart new hire succeed with only this message?” If not, add context.

Anti-patterns

Anti-pattern	Why it fails	Repair
One-word asks	No deliverable	Name the artifact
Hidden constraints	Wrong tone/length	State must-avoid
Dumping whole email threads	Noise > signal	Summarize stakes + paste only key excerpts
“Act as expert” with no task	Role theater	Role + concrete output
Never iterating	Stuck with draft 1	Surgical feedback

Worked example (original) — email rewrite

Prompt skeleton:

Role: You are my editor for customer emails.

Task: Rewrite the draft below into a calm apology.

Context: Customer waited 6 days; we missed an SLA; refund already approved.

Format: 120–150 words; subject line + body; no legalese; end with one clear next step.

Rules: Do not blame the customer; do not invent policy; flag any claim you cannot verify
[paste draft]

3. Getting better results: common fixes

When output misses, diagnose the failure mode:

Symptom	Likely cause	Fix
Too generic	Thin context	Add audience, constraints, history

Symptom	Likely cause	Fix
Wrong length	Unstated expectation	Set word/paragraph limits
Wrong structure	Format not shown	Give a skeleton or example
Wrong tone	Default “helpful professional”	Name tone; paste a sample
Plausible but wrong	Hallucination risk	Verify; ask for sources/confidence; use search/Research
Over-agreeable	Sycophancy	Ask for critique / opposing view
Thread chaos	Too many pivots	New chat with cleaned prompt

Iteration mindset

- Treat draft 1 as a starting point
- Feedback should be **surgical** (“cut para 1; make CTA specific”) not only “make better”
- If the thread is tangled, **start a clean chat** with a clearer prompt

Lightweight evals (Discernment in practice)

For recurring work:

1. Collect 5–10 past good examples you already trust
2. Write prompts that would recreate them
3. Compare Claude’s output to your gold examples
4. Note gaps; bake missing rules into prompts, Projects, or Skills
5. Only then trust the workflow on new data

This is the same idea as a “delegation diligence loop”: test on known answers before you trust new ones.

Worked example (original) — eval for weekly metrics blurb

- Gold set: last 4 Friday write-ups you already published
- Input: same raw CSV + same Project instructions
- Score: factual match, length, chart callouts, no invented drivers
- Gap found: Claude invents “seasonality” without data → add rule “Only claim drivers present in the numbers or my notes”
- Bake rule into Project instructions; re-test once; then use live

If the exam asks X, think Y

- X: “Output is generic” → Y: Weak Description / missing audience
- X: “How do you improve without better ‘prompt magic’?” → Y: Iteration + evals + Projects/Skills

4. Three shapes of work (desktop app)

Public objectives stress three modes of collaboration:

1. **Turn by turn (Chat)** — Back-and-forth drafting, Q&A, analysis in conversation. Best when you want tight control each step.
2. **Handing work off (Cowork)** — Longer sessions where Claude works with files and applications more autonomously. Best for multi-step operational tasks. Often plan-gated (paid plans). Sessions can organize files, synthesize research, run scheduled tasks, and use projects scoped for Cowork.
3. **Building software (Code / Claude Code)** — Implementation-oriented work: repos, agents, coding workflows.

Exam tip: Match the *shape of the task*, not the shiny mode. Exploratory brainstorming → Chat. “Clean this folder of invoices and summarize” → Cowork-style handoff. “Ship a feature in the repo” → Code.

Pitfall: Using agentic handoff for a task that still needs turn-by-turn judgment (sensitive client email, legal wording).

Decision tree — pick a shape

Is the primary goal writing/judging tone sentence-by-sentence?

YES → Chat

NO ↓

Is the primary goal multi-step file/app ops you can leave running?

YES → Cowork (if plan allows)

NO ↓

Is the primary goal software in a repo / coding agent workflow?

YES → Code

NO → Default Chat, then escalate if the task grows

Comparison — Chat vs Cowork vs Code

Dimension	Chat	Cowork	Code
Control per step	Highest	Medium (plan then run)	Medium–high via steering
Best for	Drafts, Q&A, judgment	Ops across files/apps	Implementation
Autonomy	Low	Higher	Higher in repo
Plan gates	Broadest access	Often paid	Separate coding surface
Risk if misused	Wasted time	Unwatched side effects	Bad commits / scope creep
Diligence focus	Review text	Review actions + files	Review diffs + tests

Anti-patterns

- Starting Cowork for a one-paragraph rewrite
- Using Code mode to “think about strategy” with no repo
- Leaving Cowork unsupervised on irreversible actions without checkpoints

5. Projects — knowledge hubs

Projects are dedicated workspaces with:

- Their own chat histories
- **Project knowledge** (uploaded docs Claude can reuse)
- **Project instructions** (tone, expertise, standing rules)
- Optional team sharing of the same shared context (Team/Enterprise)

Public Help Center framing: Projects are available broadly (including Free, with a small project count limit on Free). On paid plans, large knowledge bases can shift into **RAG mode**—Claude retrieves relevant chunks instead of stuffing everything into the window, expanding capacity while keeping quality.

Use Projects when: You have a stable domain (product docs, brand voice, client briefs) that many chats should share.

Don't confuse with Skills: Projects store *what* Claude should know; Skills encode *how* to run a procedure. Help Center contrast: Projects = static background knowledge loaded in that workspace; Skills = specialized procedures that activate dynamically and can work across Claude.

Worked example (original) — “Acme Support” Project

- Knowledge: FAQ PDF, severity matrix, refund policy, tone samples
- Instructions: “Never invent policy. Quote policy titles. Escalate legal claims to human. Tone: calm, plain.”
- Chats inside Project: ticket rewrites, macro drafts, weekly theme summaries
- Outside Project: do **not** expect the same policy grounding

Anti-patterns

- One mega-Project for every client (context bleed)
- Uploading secrets you would not put in a shared drive
- No instructions, then blaming “Claude forgot our voice”
- Treating Project knowledge as always fully “in mind” when RAG may retrieve selectively—ask pointed questions; cite files

If the exam asks X, think Y

- X: “How do I reuse brand voice across chats?” → Y: Project knowledge + instructions
- X: “Knowledge got huge—what happens conceptually?” → Y: Retrieval of relevant chunks (RAG-style), especially on paid plans

6. Artifacts — standalone outputs

Artifacts appear in a dedicated panel beside chat. Instead of burying a long HTML/code/doc blob in the thread, Claude renders something you can preview, copy, download, or iterate on.

Claude tends to create an artifact when content is significant, self-contained, editable/reusable, and likely to be referenced later (often roughly longer / more structured than a short chat reply).

Common types (conceptually):

- Documents (markdown/text reports, plans, posts)
- Code snippets ready to copy
- Interactive / visual pieces (simple apps, charts, calculators, SVG/diagrams, React components)

Tips

- Ask explicitly: “Create this as an artifact” if it stays stuck in chat
- Toggle preview vs underlying code when debugging
- Use Artifacts for shareable deliverables; keep chat for negotiation and critique
- Version selector helps explore directions without losing prior work
- Publishing/sharing and advanced behaviors (AI-powered artifacts, MCP inside artifacts, persistent storage) are plan- and settings-dependent—know they exist; verify live docs for exam specifics
- Code execution / file creation capabilities are often required for modern artifact workflows

Worked example (original)

Ask for a “pricing comparison calculator” as an artifact → iterate on UI in the panel → download or share when stable. Keep negotiation (“should we include annual discount?”) in chat.

Anti-patterns

- Leaving a 200-line HTML app only in chat bubbles
- Editing by re-pasting the whole file every time instead of targeting the artifact
- Publishing without checking whether storage is personal vs shared (privacy)

7. Skills — procedural machines

Skills are reusable instruction sets (often folders of instructions, scripts, and resources) for multi-step workflows: brand process, PDF packaging, call-prep ritual, blog pipeline, and so on. They improve consistency via **progressive disclosure**—Claude loads what is relevant rather than dumping every skill into context.

Types to recognize at a high level:

- Anthropic-maintained skills (e.g., stronger document creation flows)
- Custom skills you author (Markdown instructions; optional scripts)
- Organization-provisioned skills (Team/Enterprise)
- Partner skills in a directory (often paired with MCP connectors)

Projects vs Skills (exam favorite)

	Projects	Skills
Purpose	Store knowledge	Define processes
Best for	Long-term context, team reference	Repeatable methodology
Persistence	Across chats in the project	When the skill is invoked / relevant
Metaphor	Library + house rules	Playbook / SOP
Loading	Background in that workspace	Dynamic / on-demand

They combine well: a “customer call prep” Skill can pull profiles from a Project’s knowledge base.

Skills vs Connectors/MCP (short)

- Connectors/MCP = *access* to tools and data
- Skills = *how* to use those tools for a workflow
- Use both: MCP opens the door; Skills teach the dance

Skills vs custom instructions

- Custom instructions = broad, always-on preferences
- Skills = task-specific, loaded when relevant

Pitfall: Encoding one-off trivia as a Skill, or stuffing a giant SOP into a Project with no procedure.

Worked example (original) — “Weekly status” Skill

Procedure steps: pull themes from notes → group by risk → draft bullets → ask human to confirm blockers → format for Slack. Attach only if the ritual repeats weekly.

If the exam asks X, think Y

- X: “Same procedure every time across chats” → Y: Skill
 - X: “Same docs and voice every time in a workspace” → Y: Project
 - X: “Need live Asana data” → Y: Connector/MCP (+ Skill for the workflow)
-

8. Connectors — expanding reach

Connectors link Claude to external tools.

- **Web connectors** — Cloud apps (Drive, Notion, Slack, Asana, Gmail, etc.), often via MCP-style integrations
- **Desktop extensions** — Local files and native apps via the desktop app

Setup pattern: find in directory → connect → authenticate → grant least privilege → test with “Can you see X?”

Diligence angle: Only install trusted connectors; permissions are revocable; treat connected data as in-scope for privacy reviews.

Anti-patterns

- Connecting everything “just in case”
 - Asking Claude to email externally without an approval gate
 - Assuming connector access = unlimited org-wide visibility (auth and scopes matter)
-

9. Enterprise Search and Research

- **Enterprise Search** — Reach into company knowledge (typically Team/Enterprise). Often presented as a dedicated org knowledge workspace (e.g., an “Ask Your Org”-style project) with guided connectors and search-optimized behavior. Useful when the answer lives in internal docs, not the open web. Optimized for relatively quick internal retrieval with citations.
- **Research** — Multi-step, agentic investigation across sources when a single chat guess is not enough; good for literature-style or competitive scans with grounding. Typically paid-plan; web search must be available. Can use web + connected internal context; produces thorough answers with checkable citations; can consume usage limits faster.

Compare (exam favorite)

Enterprise Search	Research	Ordinary web search in chat
Main Find internal knowledge fast job	Deep multi-step synthesis	Quick external facts
Typ- Connected company tools i-cal sources	Web + integrations, many steps	Web
DepthTargeted retrieval	Broad investigation	Light
Plan Team/Enterprise notes	Paid plans common	Depends on settings

Exam tip: Use Research/Search when freshness or source grounding matters; still apply Discernment—sources can be incomplete or conflicting.

Worked example (original)

- “Where is our PTO policy?” → Enterprise Search / internal connectors
- “Compare three competitors’ pricing pages and summarize changes since last quarter” → Research
- “What time is the Singapore GP this year?” → Web search in chat may suffice

If the exam asks X, think Y

- X: “Internal Slack + Drive answer” → Y: Enterprise Search / connectors
- X: “Multi-angle report with citations in minutes” → Y: Research
- X: “One quick public fact” → Y: Web search, not full Research

10. Role-based use and product tour

Course wrap-ups typically show Claude across roles (marketing calendars, follow-ups, analysis, ops) and point to specialized surfaces:

- **Claude Code** — coding agents / software work
- **Claude Tag / Design** — tagged or design-oriented workflows (product naming may evolve; know they exist as adjacent tools)
- **Claude for Microsoft 365** — work inside M365 surfaces
- **Claude in Chrome** — browser-context assistance

For exams: know **when** each shape helps, not every UI click.

Role mapping cheat sheet (original)

Role need	Prefer
Draft + edit copy	Chat + Artifacts
Recurring brand SOP	Skill (+ Project knowledge)

Role need	Prefer
Clean folders / multi-app ops	Cowork
Ship code	Code
Company wiki answers	Enterprise Search
Competitive landscape brief	Research
Browser page help	Claude in Chrome
Outlook/Teams embedded help	Claude for Microsoft 365

11. Mapping Claude 101 → AI Fluency 4Ds

Feature / habit	Strongest D	Why
Choosing Chat vs Cowork vs Code	Delegation	Task split + platform awareness
Prompt contracts, Projects, Skills	Description	Goals, process, behavior
Evals, citation checks, “fix with Claude”	Discernment	Critique outputs/process
Connector least privilege, disclosure, human owns final	Diligence	Responsibility

Remember: Features are **infrastructure**, not a substitute for judgment.

Decision trees / comparison tables

Master chooser — “I need Claude to...”

Need reusable domain knowledge across many chats?

→ Project (+ instructions)

Need a repeatable multi-step method?

→ Skill

Need a tangible deliverable to preview/share?

→ Artifact

Need live external/app data?

→ Connector / MCP

Need internal company answer quickly?

→ Enterprise Search

Need deep multi-source investigation?

→ Research

Need tight editorial control?

→ Chat

Need hands-off multi-step ops?

→ Cowork
 Need repo/implementation work?
 → Code

Projects vs Skills vs Artifacts vs Connectors

Need	Reach for
What Claude should know here	Project
How Claude should run a ritual	Skill
What Claude produced as a thing	Artifact
What systems Claude can touch	Connector
Where company truth lives	Enterprise Search
How to investigate broadly	Research

Failure → feature map

Failure	First fix
Generic answers	Better prompt / Project context
Inconsistent weekly ritual	Skill
Lost long HTML in scrollback	Artifact
Can't see Drive/Slack	Connector
Invented market numbers	Research/web + verify
Wrong tool for autonomy level	Re-pick Chat/Cowork/Code

Common exam traps

1. **Projects = Skills** — False. Knowledge/workspace vs procedure.
2. **Artifacts are just pretty chat** — False. Standalone panel deliverables.
3. **More autonomy = less oversight** — False. Cowork/Code/Research need *more* Discernment and Diligence.
4. **Confident tone = verified fact** — False.
5. **Free plan can do every hands-on demo** — False. Cowork / Enterprise Search / some Research behaviors are plan-gated; definitions still testable.
6. **Connectors replace Skills** — False. Access ≠ procedure.
7. **Enterprise Search = Research** — False. Quick internal retrieval vs deep multi-step investigation.
8. **Custom instructions = Skills** — False. Always-on broad prefs vs task-specific loaded procedures.
9. **RAG means Claude read every file fully every time** — Misleading. Retrieval pulls relevant chunks when knowledge is large.
10. **One Project for all clients is fine** — Risky. Context bleed; prefer separation for sensitive matters.
11. **"Make it better" is good feedback** — Weak. Be surgical.
12. **Publishing an artifact has no privacy angle** — False. Shared storage / org sharing rules matter.
13. **Fluency is only prompt tricks** — False. 4D collaboration skill (see course 02).

14. **You can skip human accountability if Claude drafted it** — False. You own the work product.
-

Expanded self-check (Q&A)

Q1. A teammate says Projects and Skills are the same because both “give Claude context.” How do you correct them?

A1. Projects mainly hold reusable *knowledge and standing instructions* across chats in a workspace. Skills encode *repeatable procedures* invoked or loaded when relevant. Knowledge vs process; they complement each other.

Q2. You need a one-page client apology email and must control every sentence. Which desktop “shape of work” fits best and why?

A2. Chat (turn-by-turn). High judgment and tone control favor conversational iteration over long handoff.

Q3. Claude’s market stats look polished but you cannot verify them. What should you do?

A3. Treat as possibly wrong; verify independently; ask for sources/confidence; enable web/Research grounding for current facts.

Q4. Name one symptom of weak Description and one fix.

A4. Example: generic output → add audience, constraints, and a format example.

Q5. What is a lightweight eval for a weekly metrics write-up?

A5. Take last month’s known-good write-ups, prompt Claude on the same raw inputs, compare to your originals, then bake missing rules into the prompt/Project/Skill before using on new data.

Q6. When project knowledge gets huge, what conceptual shift helps capacity?

A6. From loading all files into context toward retrieving only relevant chunks (RAG-style), so answers stay grounded without overflowing the window (especially emphasized on paid plans).

Q7. Web connector vs desktop extension—key difference?

A7. Web connectors link cloud services; desktop extensions run locally through the desktop app for files/native apps.

Q8. Map “I am accountable for the final report and will disclose AI help if asked” to which Fluency D?

A8. Diligence (responsibility, transparency, ethical use).

Q9. When should you prefer Research over a single web-search answer?

A9. When you need multi-step, multi-source synthesis with systematic exploration and citations—not just one quick fact.

Q10. How does Enterprise Search differ from a normal Project?

A10. Enterprise Search is specialized for org knowledge retrieval (often auto-provisioned, search-optimized, guided connectors). Normal Projects are general-purpose workspaces with user-customizable instructions and knowledge.

Q11. Give an example of misusing Cowork.

A11. Handing off a sensitive legal letter that still needs sentence-level human judgment without tight checkpoints—or running irreversible file ops with no review.

Q12. Skills vs MCP connectors in one sentence each.

A12. Skills teach *how* to perform a workflow; MCP/connectors provide access to external tools and data.

Q13. What should you put in Project instructions vs Project knowledge?

A13. Instructions = standing rules/tone/role behavior. Knowledge = reference files and facts Claude should draw from.

Q14. Name three common artifact content types.

A14. Any three: documents, code snippets, HTML pages, SVG/diagrams, interactive components/calculators.

Q15. Why start a new chat instead of continuing a messy thread?

A15. Tangled context causes drift; a clean prompt with clarified goals often outperforms endless patches.

Q16. Map Artifacts to a Fluency idea.

A16. They make the *product* of Description visible and editable—supporting Discernment on a concrete deliverable.

Q17. Free-plan exam trap: which ideas still matter even if you cannot click every feature?

A17. Definitions and fit-for-task reasoning for Cowork, Enterprise Search, Research, etc., remain fair game.

Q18. What is progressive disclosure in Skills?

A18. Claude loads only relevant skill information for the task, reducing context overload.

Q19. You need Claude to follow the same PDF packaging steps every Friday. Project or Skill—primary choice?

A19. Skill (procedure). Optionally keep brand assets in a Project.

Q20. “AI wrote it, so I’m not responsible.” Which D fails?

A20. Diligence—humans remain accountable for AI-assisted work.

Q21. Pick a tool: summarize themes across connected Slack + Drive for “what did we decide about pricing?”

A21. Enterprise Search / org knowledge connectors (internal). Research if you also need deep external competitive synthesis.

Q22. What feedback is better than “make it better”?

A22. Surgical instructions: what to cut, what to add, tone target, length, must-keep facts.

Q23. Why separate client matters into different Projects?

A23. Reduce context bleed; keep histories, files, and instructions walled for accuracy and confidentiality.

Q24. Code vs Chat for “explain this stack trace and propose a patch,” when you are not in a repo agent workflow?

A24. Chat can analyze and propose; switch to Code when you want agentic implementation in the actual repository.

Q25. How do you combine Project + Skill + Artifact in one workflow?

A25. Example: Project holds product docs; Skill runs “release notes ritual”; Artifact is the final markdown notes you preview and share.

Quick review checklist

- ☐ I can explain Claude as collaborator ≠ oracle
 - ☐ I can write orient → specify → constrain first messages
 - ☐ I can diagnose generic / wrong tone / hallucination and fix them
 - ☐ I can pick Chat vs Cowork vs Code with a reason
 - ☐ Projects = knowledge + instructions; Skills = procedures; Artifacts = deliverables
 - ☐ Connectors = access; Skills = how; don't conflate
 - ☐ Enterprise Search (internal quick) ≠ Research (deep multi-step)
 - ☐ RAG / progressive disclosure = capacity without stuffing everything
 - ☐ Plan gates exist; definitions still matter on exams
 - ☐ Map features to Delegation / Description / Discernment / Diligence
 - ☐ Human owns the final work product
 - ☐ I reviewed anti-patterns and exam traps once aloud
-

Glossary

Term	Short definition
Artifact	Standalone, previewable output in a side panel
Chat	Turn-by-turn conversational shape of work
Connector	Integration linking Claude to external/local tools
Cowork	Hands-off / multi-step operational shape of work (often paid)
Code (shape)	Software-building collaboration surface
Custom instructions	Broad always-on preferences
Enterprise Search	Org-knowledge retrieval workspace/capability
Hallucination	Fluent but false content
MCP	Model Context Protocol — tool/data connection standard
Progressive disclosure	Load only relevant instructions/resources
Project	Workspace with chats, knowledge, instructions
Prompt contract	Goal, audience, constraints, definition of done
RAG	Retrieval Augmented Generation — fetch relevant chunks
Research	Agentic multi-step investigation with citations
Skill	Reusable procedural how-to package
Sycophancy	Over-agreeing instead of truth-seeking
4Ds	Delegation, Description, Discernment, Diligence

Reading tips

- Skim the Key concepts map before deep notes
- After each deep topic, answer one self-check aloud
- Rehearse the master chooser decision tree from memory
- Pair with the AI Fluency notes (02-ai-fluency.md)—Claude 101 applies the framework; Fluency explains it
- Night before: checklist + glossary only

End of Claude 101 study notes.

AI Fluency: Framework & Foundations — Study Notes

Note: This edition follows the ASD-STE100 writing rules: simple present tense, active voice, one word = one meaning, sentences of 20–25 words or less. Technical names are exceptions and stay as written. Examples: AI Fluency, 4D, Delegation, Description, Discernment, Diligence, Automation, Augmentation, Agency, Claude, RAG, LLM, Rick Dakan, Joseph Feller, Anthropic.

Disclaimer: These notes are original study notes for exam prep. They are **not** official Anthropic material. Course outline source: <https://academy.claude.com/courses/ai-fluency-framework-foundations>. Collection context: <https://academy.claude.com/collections/ai-fluency>. Public sources credit Rick Dakan and Joseph Feller for the framework, with Anthropic collaboration. These notes paraphrase the terminology for study. The authors did not copy lesson prose.

Overview

AI Fluency: Framework & Foundations is Anthropic’s primary fluency course. Public pages list **14 lessons · 1 quiz · ~4 hours**. You get a completion badge after the assessment. The course comes from work between Anthropic educators and two professors. **Rick Dakan** works at Ringling College of Art and Design. **Joseph Feller** works at University College Cork. The framework comes from 2023–2024 work by Rick Dakan and Joseph Feller.

The course is not about the newest prompting techniques. The course gives a stable way to work with AI: **effectively, efficiently, ethically, and safely**. These four “E/S” outcomes often pair with the 4D competencies.

Stated learning outcomes (paraphrased from the public course page)

- Define AI Fluency and why it matters
- Apply the **4D framework**: Delegation, Description, Discernment, Diligence
- Distinguish **Automation, Augmentation, and Agency** as engagement modes
- Explain generative AI / LLM basics, strengths, and limits
- Make Delegation choices. Use problem awareness, platform awareness, and task split
- Communicate with Product / Process / Performance Description and solid prompting
- Critique outputs. Refine them through the **Description–Discernment loop**
- Practice Creation, Transparency, and Deployment Diligence

These notes are original study material for reading and Claude cert prep. They are not official Anthropic content. They are not a lesson transcript.

How to use this file as a primary study source

- 1. Memorize the Key concepts map (10–15 minutes)
- 2. Read Deep notes on the 4Ds and the three engagement modes twice
- 3. Drill Decision trees until you can label scenarios without notes
- 4. Read Common exam traps aloud
- 5. Complete Self-check closed-book, then check your answers
- 6. Night before: Quick review checklist + glossary
- 7. Pair with 01-claude-101.md to connect framework → product features

Public diligence modeling note (concept, not a list of quotes to memorize): The course materials show Transparency Diligence. They disclose how Claude helped with the draft. Humans keep judgment and final responsibility. Use that as a *pattern* for your own disclosures. Do not memorize the text word for word.

Related titles in the AI Fluency collection (brief)

Read these titles quickly. Then you know where to study next if the exam asks. Source collection: <https://academy.claude.com/collections/ai-fluency>

Title	Why it matters
The 4 Properties of AI (short tutorial)	Fast mental model of what makes AI strong or weak
AI Capabilities and Limitations (full course)	Deeper model of next-token prediction, knowledge, memory, steerability, context limits
4 Ds — Behavioral Indicators	Concrete day-to-day behaviors for each D
Short trust tutorials	“What happens when you talk to AI?”, trust, privacy, hallucination, sycophancy, bias
Writing an AI diligence statement	Practice Transparency Diligence
Tokens / How context affects performance & cost	Practical cost–quality tradeoffs
Editions “for your world”	Builders, educators, pK–12, nonprofits, small businesses, students
Teach & facilitate tracks	Teach AI Fluency. Train-the-trainer. Education Report / Fluency Index guides

Study the flagship (primary) course first. Use the collection as a map of nearby skills.

Key concepts map

Term	Short definition	Core question
AI Fluency	Skillful collaboration with AI: effective, efficient, ethical, safe	Can I work <i>with</i> AI well?
Delegation	Decide whether/when/how to engage AI. Split work	Who does which part?
Description	Communicate goals, process, and desired behavior	How do I say what a good result is?
Discernment	Critically evaluate outputs, process, and behavior	How good is this, really?
Diligence	Responsibility for creation, transparency, deployment	What am I accountable for?
Automation	AI executes specified tasks on instruction	"Do this."
Augmentation	Human + AI co-create / co-think	"Work with me."
Agency	You shape AI to act more independently on your behalf	"Act toward these aims."
Description–Discernment loop	Describe → output → critique → re-describe	How do we improve?
Problem awareness	Understand the real goal, risks, and success criteria	What is actually at stake?
Platform awareness	Know this tool's strengths and limits	What can <i>this</i> system do?
Task delegation	Split the workflow into human / shared / AI-primary	Keep / share / give?
Product Description	Specify the deliverable	What should exist at the end?
Process Description	Specify how work should proceed	What steps/tools/stop points?
Performance Description	Specify interaction behavior	How should AI behave with me?
Creation Diligence	Care while you build with AI	Is the quality bar right?
Transparency Diligence	Disclose AI assistance where it matters	Who needs to know AI helped?
Deployment Diligence	Care before you release work into the world	Who could be harmed if this is released?

Memory pegs

- Outcomes: **Effective** · **Efficient** · **Ethical** · **Safe**
- Competencies: **Delegate** · **Describe** · **Discern** · **Diligent**
- Modes: **Automate** · **Augment** · **Agency**
- Description layers: **Product** · **Process** · **Performance**
- Delegation lenses: **Problem** · **Platform** · **Task**
- Diligence strands: **Creation** · **Transparency** · **Deployment**

Deep notes by topic

1. Why AI Fluency (not just prompting)

Tools change quickly. Fluency is a **stable competency layer**. You use judgment about when to use AI. You steer it. You check it. You own the result. Without fluency, people over-trust fluent nonsense. Or they under-use valuable help.

Exam framing: Fluency is about *thinking with* AI. Do not treat AI as only a spelling checker.

What fluency is not

- Memorize model version numbers
- Collect popular prompt templates alone
- Blind trust in polished paragraphs
- Avoid AI entirely from fear, and never learn evaluation

Worked contrast (original)

- Prompt-only user: pastes a clever template. Sends the first draft. Stakeholders then find errors
- Fluent user: decides the split (Delegation), writes a clear contract (Description), checks against criteria (Discernment), discloses and owns the result (Diligence)

If the exam asks X, think Y

- X: “Why not just learn prompting?” → Y: Prompting $\subset$ Description. Fluency also covers whether/when, evaluation, and responsibility
-

2. The 4D Framework

Remember the 4Ds with a story sequence:

1. **Delegation** — Should AI be in this work at all, and on which parts?
2. **Description** — How do I tell it what a good result is?
3. **Discernment** — How do I tell if the result (and the process) is good?
4. **Diligence** — How do I stay responsible as I create, disclose, and release?

The Ds are **interconnected**. Weak Description breaks Discernment (“I do not know what I asked for”). Weak Discernment harms Delegation (“I will automate something I never validated”). Diligence covers the whole practice.

Delegation Think with care. Decide what humans do, what AI does, and how they pass work. Public framing stresses this: set goals. Decide whether, when, and how to engage with AI.

Objectives name these helpful sub-skills:

- **Problem awareness** — Understand the real goal, constraints, risks, and success criteria
- **Platform awareness** — Know what *this* model/tool can and cannot do (and plan limits)
- **Task delegation** — Split the workflow: draft vs decide, clean vs interpret, explore vs approve

Practical tip: For any recurring task, write a one-sentence “keep / share / give” list. Keep (must stay human). Share (pair). Give (AI-primary with review).

Pitfall: You delegate interpretation of high-stakes outcomes. You only check formatting.

Worked example (original) — hiring scorecard

- Keep: final hire decision, legal compliance calls
- Share: structure rubrics, clarify role requirements
- Give: format candidate comparison tables from notes you already wrote
- Checkpoint: a human verifies every factual claim about a candidate against source notes—no invented experience

Description Clear communication with AI systems. Public framing often splits:

- **Product Description** — What should the *output* be? (audience, structure, examples of good/bad)
- **Process Description** — How should AI *work*? (steps, tools, order, stop points)
- **Performance Description** — How should AI *behave* in the interaction? (tone, ask-before-assuming, cite uncertainty)

Foundational prompting is part of Description: role/context, task clarity, constraints, examples, and iteration instructions.

Pitfall: You describe only the product (“write a strategy”). You give no process or performance rules. Then you get confident overreach.

Mini templates (original wording)

PRODUCT: Deliverable, audience, length, must-include, must-avoid, example of good.

PROCESS: Steps to follow, tools to use, when to ask me questions, what to cite.

PERFORMANCE: Tone, uncertainty handling, challenge my assumptions if weak.

Discernment Evaluate:

- **Outputs** — accuracy, completeness, usefulness, tone fit
- **Processes** — Did the approach make sense? Missing steps? Invented data?
- **Behaviors** — sycophancy, overconfidence, bias patterns, refusal quality

Build eval habits. Compare to known-good work. Spot-check facts. Ask the model to show reasoning or sources, *then verify*.

Pitfall: You treat “I like how this sounds” as “this is correct.”

Diligence Responsible use. Courses often teach three strands:

- **Creation Diligence** — Care in how you build with AI (quality bar, appropriate use)
- **Transparency Diligence** — Disclose AI assistance where it matters (clients, academia, teammates)
- **Deployment Diligence** — Care before you release AI-touched work into the world (safety, fairness, compliance)

If the exam asks X, think Y

- X: “Which D is missing if I automate before validating?” → Y: Discernment (and poor Delegation)
- X: “Which D is disclosure?” → Y: Transparency Diligence
- X: “Pretty but wrong citations” → Y: Discernment failure (+ Diligence if published)

3. Three ways of engaging: Automation, Augmentation, Agency

Mode	Human role	AI role	Example
Automation	Specify task. Review	Execute defined work	"Format these rows into a table"
Augmentation	Co-think, co-edit	Partner on ideas and drafts	Strategy brainstorm + critique
Agency	Set goals, knowledge, guardrails	Work more independently toward aims	Ongoing research agent with standing instructions

Exam tip: Agency is not a setup that you never check again. You still shape knowledge and behavior. Then you apply Discernment and Diligence on outcomes. More autonomy → **more** need for clear Description and stronger checks—not less.

Pitfall: You call something “automation” when you need augmentation (creative work that needs judgment).

Decision tree — choose a mode

Is the task fully specified with a clear success check?

YES → Automation (+ review)

NO ↓

Do you need ongoing co-thinking and judgment tradeoffs?

YES → Augmentation

NO ↓

Are you configuring standing goals/tools so AI can pursue work with less turn-by-turn steering?

YES → Agency (with stronger guardrails + monitoring)

ELSE → Start in Augmentation; tighten into Automation once stable

Anti-patterns by mode

Mode	Anti-pattern	Better
Automation	Vague task, no acceptance test	Specify + sample output
Augmentation	You accept flattery as collaboration	Demand critique criteria
Agency	Unbounded autonomy on email/send	Guardrails, approvals, logs

4. Generative AI & LLM fundamentals (mental model)

You do not need a PhD for the quiz. You need a **usable mental model**:

- Modern assistants use **large language models**. The models train to predict useful next tokens in context
- They combine **learned patterns** with **current context** (your messages, files, tool results)
- Strengths often include: drafting, transforming, summarizing, coding assistance, structured brainstorming, explaining
- Limits often include: they invent plausible falsehoods. Weak live knowledge without tools. Context window bounds. Uneven niche expertise. Sensitivity to framing

Related collection titles go deeper: next-token prediction, working memory, steerability, tokens, and cost/quality tradeoffs.

Delegation link: Platform awareness means you match the task type to these strengths and limits.

Trust topics to recognize (collection)

- **Hallucination** — Fluent but false content. Spot-check, ground with sources/tools
- **Sycophancy** — The model agrees with you because you seem to want it. Ask for critique explicitly
- **Bias** — Skewed associations from data/systems. Watch stereotypes and skewed recommendations
- **Privacy** — The system may process what you paste. Minimize sensitive data. Follow org policy

Tokens & context (exam-aware, light)

- You measure usage in tokens. Longer context can raise cost. It can also dilute attention
- Everything Claude “knows” for your answer is in training, or it is in current context/tools
- Choose context with care: more context is not always better. Use relevant context. Do not add extra unused text

Worked example (original) — platform awareness

Task: “What is our CEO’s exact quote from yesterday’s all-hands?” - Without connectors/transcripts: high hallucination risk → do not Delegate as factual recall - With meeting notes in a Project or Enterprise Search: better platform fit - Still Discern: the quote must match source text

5. Strategic Delegation in projects

When you plan a project with AI:

1. Clarify the **problem** (who is harmed if this is wrong?)
2. Inventory **platform** capabilities (chat only? tools? company search? coding agent?)
3. Split **tasks** into AI-suitable vs human-required
4. Define checkpoints where Discernment is mandatory
5. Write Diligence rules (disclosure, data handling, approval gates)

Worked micro-example (original): You publish a customer FAQ update.

- Human keeps: legal claims, pricing promises, escalation policy
- AI drafts: rewrites from ticket themes. Proposes structure
- Checkpoint: a human verifies every factual claim against source docs
- Disclosure: internal note that AI assisted drafting

Second worked example (original): Student literature survey

- Problem awareness: academic integrity rules. Originality required
- Platform awareness: the model may invent citations
- Task split: AI helps outline and explain papers you already found. The human reads sources and writes claims
- Diligence: follow school disclosure policy. Never cite unread papers

Keep / share / give worksheet (original)

Step in workflow	Keep	Share	Give	Checkpoint
Define success	✓			Stakeholder sign-off
Gather sources		✓		Source list review
Draft synthesis		✓		Fact audit
Format tables			✓	Spot-check
Publish	✓			Final human approve

6. Description in practice & prompting techniques

Write Product / Process / Performance as a prompt skeleton (original wording):

PRODUCT: Deliverable, audience, length, must-include, must-avoid, example of good.

PROCESS: Steps to follow, tools to use, when to ask me questions, what to cite.

PERFORMANCE: Tone, uncertainty handling, challenge my assumptions if weak.

Then iterate. Description is not a single prompt that you never change. It is a **conversation skill**.

Effective habits

- Show examples. Do not only tell (examples work better than adjectives)
- Separate “generate options” from “pick a winner”
- Ask for assumptions listed explicitly
- Prefer precise feedback over a full restart (unless the thread is lost)
- Put evaluation criteria in the prompt when possible (“score yourself against this rubric, then improve”)

Anti-patterns

Anti-pattern	Failure	Fix
Adjective pile (“brilliant, epic, viral”)	Vague product	Concrete audience + example
No stop points	Overconfident completion	“Ask before assuming X”
Hidden stakes	Wrong risk stance	State who is harmed if the work is wrong
One giant mega-prompt forever	The prompt fails easily	Iterate via Discernment loop

If the exam asks X, think Y

- X: “Only product described” → Y: Add process + performance
- X: “Model never asks clarifying questions” → Y: Performance/process rules that require questions when the task is ambiguous

7. The Description–Discernment loop

Core cycle:

1. Describe what you want (and how)
2. Receive output / observe behavior
3. Discern gaps against criteria
4. Update Description (or Delegation decision)
5. Repeat until you meet the quality bar—or decide AI is the wrong tool

This loop is the fix for two errors: blind acceptance and aimless re-prompting.

Exam phrase to remember: Better Description is often the *result* of Discernment, not only its input.

Worked loop (original) — landing-page hero

1. Describe: “Hero copy for privacy-first notes app. 12 words max.”
2. Output: clever but implies “military-grade encryption” you do not offer
3. Discern: marketing claim risk
4. Re-describe: “No security claims beyond ‘encrypted on device’. Friendly tone. 12 words.”
5. Output passes the claim check → proceed. A human still approves before you release (Deployment Diligence)

Loop failure modes

- You skip criteria → debates about taste with no end
 - You change the goal every turn → no learning
 - You never change Delegation when the tool is wrong → wasted loops
-

8. Diligence in the real world

Creation: Do not release first-draft quality on regulated or reputation-critical work. Choose an appropriate involvement mode. Keep a quality bar.

Transparency: Use diligence statements, footnotes, or client notes that AI helped. Match the note to the context (school policies and enterprise rules differ). Collection tutorial “Writing an AI diligence statement” is practical homework.

Deployment: Consider who is affected, failure modes, and how you monitor after release (especially for agentic workflows).

Sample diligence statement pattern (original, short)

AI assistance: Claude helped outline and edit sections 2–3.

Human responsibility: I verified all facts against sources dated ____, removed unsupported claims, and approve this version for [audience].

Data note: No customer PII was pasted into the chat.

If the exam asks X, think Y

- X: “Course discloses AI help in making materials” → Y: This shows Transparency Diligence
 - X: “Agent sends emails unsupervised” → Y: Deployment Diligence (+ Delegation/Agency risk)
-

9. How the 4Ds show up across modes

- **Automation:** Strong Description of the task. Discernment on outputs. Diligence on data you feed in
- **Augmentation:** All four Ds, with heavy Description–Discernment looping
- **Agency:** Heavy upfront Delegation + standing Description. Continuous Discernment of autonomous steps. Diligence before and after deployment

Claude product features (Projects, Skills, Connectors—see Claude 101 notes) are **infrastructure for Description and Delegation**. They do not replace judgment.

Claude 101 feature	Fluency job
Projects	Standing Description (knowledge + rules)
Skills	Process Description reusable
Artifacts	Make Product visible for Discernment
Connectors	Expand platform options for Delegation
Research / Enterprise Search	Grounding helps Discernment—still verify
Chat vs Cowork vs Code	Mode / Delegation choice

10. Behavioral indicators (study translation)

Public collection includes “4 Ds — Behavioral Indicators.” For exams, remember *examples of behaviors*. Do not memorize an official list word for word:

D	Looks like in daily work
Delegation	You pause and ask “should AI do this?”. You write keep/share/give
Description	You add examples, constraints, success criteria. You clarify ambiguity
Discernment	You spot-check facts. You reject sycophancy. You compare to gold outputs
Diligence	You disclose assistance. You protect data. You refuse to release unverified claims

Anti-behaviors

- Delegation: “Let AI decide the legal call”
- Description: “You know what I mean”
- Discernment: “Sounds smart, ship it”
- Diligence: “Nobody needs to know AI wrote this” when policy requires disclosure

Decision trees / comparison tables

Scenario → primary D

Scenario	Start with
Should we use AI at all on this?	Delegation
Output misses the brief	Description (then loop)
Output is fluent but wrong	Discernment
Ready to publish / send to a client	Diligence
Model only agrees with you	Discernment + Performance Description
Choosing Automation vs Agency	Delegation (+ platform awareness)

Automation vs Augmentation vs Agency

Question	Automation	Augmentation	Agency
Who steers moment-to-moment?	Human spec	Both	Standing config + monitoring
Typical risk	Silent wrong execution	The pair drifts from the goal	Side effects with no watch
Description weight	Task clarity	Ongoing dialogue	Goals + guardrails
Discernment timing	After output	Continuously	During and after autonomous steps

Product vs Process vs Performance

Layer	Asks	Example ingredient
Product	What is the thing?	"1-page FAQ. Audience = new users"
Process	How to make it?	"Use only attached policy. List assumptions first"
Performance	How to behave?	"If unsure, ask. Never invent policy"

Problem vs Platform vs Task (Delegation lenses)

Lens	Failure if skipped	Fix
Problem	Solve the wrong problem	Restate success + harm
Platform	Ask impossible recall / ignore tools	Match task to capabilities
Task	AI decides what humans must own	Keep/share/give split

Trust failure → response

Failure	Response mix
Hallucination	Grounding tools + verify sources + Discernment
Sycophancy	Ask for critique / strongest opposing view
Bias	Diversify examples. Inspect recommendations for stereotypes
Privacy leak risk	Minimize data. Follow policy. Prefer approved environments

Common exam traps

1. **Fluency = prompt engineering** — Too narrow. Prompting supports Description. Fluency includes all 4Ds + modes.
2. **Agency means no oversight** — Opposite. More autonomy needs more Description, Discernment, Diligence.
3. **Automation is always faster and better** — Only when the task is well-specified and checked.
4. **Discernment is only fact-checking outputs** — Also process and behavior (sycophancy, bias, refusals).
5. **Diligence is only “be nice / ethical mood”** — Includes creation care, disclosure, and deployment checks.
6. **Product Description alone is enough** — Missing process/performance causes overreach.
7. **Platform awareness is optional if you are an expert user** — Experts still mismatch tools to tasks.
8. **If AI helped, you can deny responsibility** — False. Humans own outcomes.
9. **Description–Discernment loop is optional extra work** — It is the main method to improve results.
10. **Hallucination means the model is “broken” only** — It is an expected generative failure. Mitigate it. Do not treat it as a myth.
11. **Sycophancy is the same as hallucination** — Related trust issues. Sycophancy is over-agreement that aims to please.
12. **Collection electives do not matter** — Exams may ask where to go deeper (capabilities course, diligence statement, tokens).
13. **4Ds are a strict waterfall once** — They interact continuously.
14. **Effective automatically means ethical** — Four outcomes. You can be effective at the wrong thing.
15. **Dakan–Feller framework is “Anthropic-only trivia”** — Know high-level origin. Focus on how you apply competencies.

Expanded self-check (Q&A)

Q1. Define AI Fluency in one sentence. Use the four outcome words.

A1. AI Fluency is the ability to engage with AI systems in ways that are effective, efficient, ethical, and safe.

Q2. List the 4Ds and give one verb for each.

A2. Delegation (decide/split), Description (communicate/specify), Discernment (evaluate/critique), Diligence (own/disclose/protect).

Q3. How does Augmentation differ from Automation?

A3. Automation has AI execute a specified task. Augmentation is ongoing partnership on thinking and doing. Automation is “do this.” Augmentation is “work with me on this.”

Q4. You ask AI to “make my argument stronger,” and it only agrees louder. Which failure mode is likely, and what Description fix helps?

A4. Sycophancy. Ask it to state the strongest opposing view, list weaknesses, and only then suggest improvements. This is Performance Description that rewards critique.

Q5. Name the three Description types and one prompt ingredient for each.

A5. Product (deliverable + example). Process (steps + when to ask questions). Performance (tone + uncertainty rules).

Q6. What is the Description–Discernment loop?

A6. Iterative cycle: describe needs → review results critically → refine description (or change delegation) → repeat.

Q7. Give an example of Transparency Diligence.

A7. Add a short note that AI assisted drafting of a report section, per school or company policy.

Q8. Why does platform awareness matter for Delegation?

A8. You cannot assign work well without knowledge of tool limits (no live web, context size, hallucination risk, missing connectors, and similar limits).

Q9. Agency increases autonomy. Does that reduce the need for Discernment?

A9. No. It usually increases the need for clearer guardrails and stronger evaluation of independent actions.

Q10. Name two related collection resources beyond this flagship course.

A10. Any two: AI Capabilities and Limitations. 4 Properties of AI. Hallucination/sycophancy/bias tutorials. Tokens/context tutorials. Diligence statement tutorial. Role-specific Fluency editions.

Q11. Map “I automated grading comments without sampling accuracy first” to the broken Ds.

A11. Weak Discernment before Delegation into Automation. Diligence risk if you deploy the comments to students.

Q12. What are the three Delegation lenses?

A12. Problem awareness, platform awareness, and task delegation (split).

Q13. Creation vs Deployment Diligence—what is the difference?

A13. Creation = care while you produce with AI. Deployment = care before and as you release into real-world impact (users, public, systems).

Q14. Give a keep/share/give split for writing a public incident report.

A14. Keep: final severity language and legal claims. Share: structure and clarity edits. Give: format timeline tables from approved facts. Checkpoint: every factual sentence traces to an approved source.

Q15. Why are the 4Ds described as interconnected?

A15. Failures pass from one D to the next. Weak Description harms Discernment. Weak Discernment leads to reckless Delegation. Diligence fails if the other three Ds are careless.

Q16. What is a usable LLM mental model for this course?

A16. Next-token / pattern prediction plus current context/tools. Strong at draft/transform/explain. Weak at guaranteed truth without grounding.

Q17. When should you exit the Description–Discernment loop and change tools or stop?

A17. When criteria cannot be met with reliability (wrong platform, missing data, stakes too high for generative drafting)—change Delegation.

Q18. How do Projects/Skills from Claude 101 support Fluency?

A18. They encode standing Description (knowledge/process). They improve Delegation quality. They do not replace Discernment or Diligence.

Q19. Name the three Diligence strands.

A19. Creation, Transparency, Deployment.

Q20. “Sounds confident and cites DOIs.” Is that enough to trust?

A20. No. Discernment requires verification. Citations can be invented or mismatched.

Q21. Efficient but unethical—is that fluency?

A21. Incomplete. Fluency requires effective, efficient, ethical, *and* safe.

Q22. Who developed the AI Fluency Framework (high level)?

A22. Professors Rick Dakan and Joseph Feller (publicly credited). They also work with Anthropic on the Academy course.

Q23. What is the badge path on the public course page?

A23. Finish lessons → final assessment → completion badge opportunity.

Q24. Bias mitigation habit (one)?

A24. Inspect outputs for stereotyped assumptions. Ask for alternative framings. Diversify examples in Description.

Q25. Privacy diligence micro-habit?

A25. Minimize sensitive data pasted into chats. Follow org/school rules. Prefer approved enterprise controls when required.

Quick review checklist

- ☐ Fluency = effective, efficient, ethical, safe collaboration
- ☐ 4Ds recited with one example behavior each
- ☐ Automation / Augmentation / Agency differences without notes
- ☐ Product / Process / Performance Description ingredients
- ☐ Problem / Platform / Task Delegation lenses

- ☐ Description–Discernment loop steps
- ☐ Creation / Transparency / Deployment Diligence
- ☐ Hallucination, sycophancy, bias, privacy—define + mitigate
- ☐ More agency → more oversight, not less
- ☐ Features (Projects/Skills/etc.) support Description/Delegation. Judgment remains human
- ☐ Know collection “where next” at title level
- ☐ Human accountability always remains

Glossary

Term	Short definition
Agency	Configuring AI to pursue goals more independently under your guardrails
AI Fluency	Effective, efficient, ethical, safe human–AI collaboration skill
Augmentation	Human–AI partnership on thinking and doing
Automation	AI executes a specified task on instruction
Bias	Skewed patterns/associations that distort outputs
Creation Diligence	Care and quality bar while you build with AI
Delegation	Deciding whether/when/how to use AI and how to split work
Deployment Diligence	Responsibility checks before/as work enters the world
Description	Communicating product, process, and performance clearly
Description–Discernment loop	Iterate specify → critique → respecify
Discernment	Critical evaluation of outputs, processes, behaviors
Diligence	Taking responsibility for AI-assisted work
Hallucination	Fluent falsehood
Platform awareness	Knowing a given tool’s capabilities and limits
Problem awareness	Understanding goals, stakes, constraints, success
Performance Description	Desired interaction behavior
Process Description	Desired method/steps/tools
Product Description	Desired deliverable
Sycophancy	Over-agreeable behavior that aims to please
Task delegation	Keep / share / give workflow splitting
Transparency Diligence	Appropriate disclosure of AI assistance
Tokens	Units of text processing/cost/context

Study plan

1. Memorize Key concepts map (10–15 minutes)
2. Read Deep notes on 4Ds + modes twice
3. Read fundamentals + trust topics quickly
4. Rehearse decision trees from memory
5. Drill self-check without looking at answers
6. Optional: read collection titles quickly so the pathways are familiar
7. Pair with 01–claude–101.md to connect framework → product features

8. Night before: checklist + glossary only

End of AI Fluency study notes.

AI Fluency: Framework & Foundations — Study Notes

Disclaimer: These are original study notes for exam prep — **not** official Anthropic material. Course outline source: <https://academy.claude.com/courses/ai-fluency-framework-foundations>. Collection context: <https://academy.claude.com/collections/ai-fluency>. Framework origin credited publicly to Rick Dakan and Joseph Feller with Anthropic collaboration; terminology here is paraphrased for study, not copied lesson prose.

Overview

AI Fluency: Framework & Foundations is Anthropic’s flagship fluency course (public pages: **14 lessons · 1 quiz · ~4 hours**, completion badge after assessment). It grew from collaboration between Anthropic educators and professors **Rick Dakan** (Ringling College of Art and Design) and **Joseph Feller** (University College Cork), who developed the AI Fluency Framework in 2023–2024.

The course is less about “prompt tricks of the month” and more about a durable way to collaborate with AI: **effectively, efficiently, ethically, and safely**—the four “E/S” outcomes often paired with the 4D competencies.

Stated learning outcomes (paraphrased from the public course page)

- Define AI Fluency and why it matters
- Apply the **4D framework**: Delegation, Description, Discernment, Diligence
- Distinguish **Automation, Augmentation, and Agency** as engagement modes
- Explain generative AI / LLM basics, strengths, and limits
- Make Delegation choices using problem awareness, platform awareness, and task split
- Communicate via Product / Process / Performance Description and solid prompting
- Critique outputs and refine through the **Description–Discernment loop**
- Practice Creation, Transparency, and Deployment Diligence

These notes are original study material for reading and Claude cert prep—not official Anthropic content and not a lesson transcript.

How to use this file as a primary study source

1. Memorize the Key concepts map (10–15 minutes)
2. Read Deep notes on the 4Ds and the three engagement modes twice
3. Drill Decision trees until you can label scenarios cold
4. Walk Common exam traps aloud
5. Complete Self-check closed-book, then verify
6. Night-before: Quick review checklist + glossary
7. Pair with 01–claude–101.md to connect framework → product features

Public diligence modeling note (concept, not a quote bank): The course materials themselves demonstrate Transparency Diligence by disclosing how Claude assisted drafting while humans retained judgment and final responsibility. Use that as a *pattern* for your own disclosures—not as text to memorize verbatim.

Related titles in the AI Fluency collection (brief)

Skim these so exam questions about “where to go next” do not surprise you. Source collection: <https://academy.claude.com/collections/ai-fluency>

Title	Why it matters
The 4 Properties of AI (short tutorial)	Fast mental model of what makes AI strong or weak
AI Capabilities and Limitations (full course)	Deeper model of next-token prediction, knowledge, memory, steerability, context limits
4 Ds — Behavioral Indicators	Concrete day-to-day behaviors tied to each D
Short trust tutorials	“What happens when you talk to AI?”, trust, privacy, hallucination, sycophancy, bias
Writing an AI diligence statement	Practice Transparency Diligence
Tokens / How context affects performance & cost	Practical cost–quality tradeoffs
Editions “for your world”	Builders, educators, pK–12, nonprofits, small businesses, students
Teach & facilitate tracks	Teaching AI Fluency; train-the-trainer; Education Report / Fluency Index guides

Flagship first; use the collection as a map of adjacent skills.

Key concepts map

Term	Short definition	Core question
AI Fluency	Skillful collaboration with AI: effective, efficient, ethical, safe	Can I work <i>with</i> AI well?
Delegation	Decide whether/when/how to engage AI; split work	Who does which part?
Description	Communicate goals, process, and desired behavior	How do I say what good looks like?
Discernment	Critically evaluate outputs, process, and behavior	How good is this, really?
Diligence	Responsibility for creation, transparency, deployment	What am I accountable for?
Automation	AI executes specified tasks on instruction	“Do this.”
Augmentation	Human + AI co-create / co-think	“Work with me.”
Agency	You shape AI to act more independently on your behalf	“Act toward these aims.”
Description–Discernment loop	Describe → output → critique → re-describe	How do we improve?

Term	Short definition	Core question
Problem awareness	Understand real goal, risks, success criteria	What's actually at stake?
Platform awareness	Know this tool's strengths/limits	What can <i>this</i> system do?
Task delegation	Slice workflow into human / shared / AI-primary	Keep / share / give?
Product Description	Specify the deliverable	What should exist at the end?
Process Description	Specify how work should proceed	What steps/tools/stop points?
Performance Description	Specify interaction behavior	How should AI behave with me?
Creation Diligence	Care while building with AI	Is the quality bar right?
Transparency Diligence	Disclose AI assistance where it matters	Who needs to know AI helped?
Deployment Diligence	Care before releasing into the world	Who could be harmed if this ships?

Memory pegs

- Outcomes: **Effective** • **Efficient** • **Ethical** • **Safe**
- Competencies: **Delegate** • **Describe** • **Discern** • **Diligent**
- Modes: **Automate** • **Augment** • **Agency**
- Description layers: **Product** • **Process** • **Performance**
- Delegation lenses: **Problem** • **Platform** • **Task**
- Diligence strands: **Creation** • **Transparency** • **Deployment**

Deep notes by topic

1. Why AI Fluency (not just prompting)

Tools change quickly. Fluency is a **stable competency layer**: judgment about when to use AI, how to steer it, how to check it, and how to own the result. Without it, people either over-trust fluent nonsense or under-use valuable help.

Exam framing: Fluency is about *thinking with* AI, not treating it as a fancy spellchecker.

What fluency is not

- Memorizing model version numbers
- Collecting viral prompt templates alone
- Blind trust in polished paragraphs
- Avoiding AI entirely out of fear without learning evaluation

Worked contrast (original)

- Prompt-only user: pastes a clever template; ships first draft; surprises stakeholders with errors

- Fluent user: decides the split (Delegation), writes a clear contract (Description), checks against criteria (Discernment), discloses and owns the result (Diligence)

If the exam asks X, think Y

- X: “Why not just learn prompting?” → Y: Prompting $\subset$ Description; fluency also covers whether/when, evaluation, and responsibility
-

2. The 4D Framework

Remember with a story arc:

1. **Delegation** — Should AI be in this work at all, and on which parts?
2. **Description** — How do I tell it what good looks like?
3. **Discernment** — How do I tell if the result (and the process) is good?
4. **Diligence** — How do I stay responsible as I create, disclose, and ship?

The Ds are **interconnected**. Weak Description breaks Discernment (“I don’t know what I asked for”). Weak Discernment poisons Delegation (“I’ll automate something I never validated”). Diligence wraps the whole practice.

Delegation Thoughtfully decide what humans do, what AI does, and how they hand off. Public framing emphasizes setting goals and deciding whether, when, and how to engage with AI.

Helpful sub-skills called out in objectives:

- **Problem awareness** — Understand the real goal, constraints, risks, and success criteria
- **Platform awareness** — Know what *this* model/tool can and cannot do (and plan limits)
- **Task delegation** — Split the workflow: draft vs decide, clean vs interpret, explore vs approve

Practical tip: For any recurring task, write a one-sentence “keep / share / give” list: keep (must stay human), share (pair), give (AI-primary with review).

Pitfall: Delegating interpretation of high-stakes outcomes while only checking formatting.

Worked example (original) — hiring scorecard

- Keep: final hire decision, legal compliance calls
- Share: structuring rubrics, clarifying role requirements
- Give: formatting candidate comparison tables from notes you already wrote
- Checkpoint: human verifies every factual claim about a candidate against source notes—no invented experience

Description Clear communication with AI systems. Public framing often splits:

- **Product Description** — What should the *output* be? (audience, structure, examples of good/bad)
- **Process Description** — How should AI *work*? (steps, tools, order, stop points)
- **Performance Description** — How should AI *behave* in the interaction? (tone, ask-before-assuming, cite uncertainty)

Foundational prompting lives here: role/context, task clarity, constraints, examples, and iteration instructions.

Pitfall: Only describing the product (“write a strategy”) with no process or performance rules—then being surprised by confident overreach.

Mini templates (original wording)

PRODUCT: Deliverable, audience, length, must-include, must-avoid, example of good.

PROCESS: Steps to follow, tools to use, when to ask me questions, what to cite.

PERFORMANCE: Tone, uncertainty handling, challenge my assumptions if weak.

Discernment Evaluate:

- **Outputs** — accuracy, completeness, usefulness, tone fit
- **Processes** — Did the approach make sense? Missing steps? Invented data?
- **Behaviors** — sycophancy, overconfidence, bias patterns, refusal quality

Build eval habits: compare to known-good work; spot-check facts; ask the model to show reasoning or sources *then verify*.

Pitfall: Mistaking “I like how this sounds” for “this is correct.”

Diligence Responsible use, often taught as three strands:

- **Creation Diligence** — Care in how you build with AI (quality bar, appropriate use)
- **Transparency Diligence** — Disclose AI assistance where it matters (clients, academia, teammates)
- **Deployment Diligence** — Care before releasing AI-touched work into the world (safety, fairness, compliance)

If the exam asks X, think Y

- X: “Which D is missing if I automate before validating?” → Y: Discernment (and poor Delegation)
- X: “Which D is disclosure?” → Y: Transparency Diligence
- X: “Pretty but wrong citations” → Y: Discernment failure (+ Diligence if published)

3. Three ways of engaging: Automation, Augmentation, Agency

Mode	Human role	AI role	Example
Automation	Specify task; review	Execute defined work	“Format these rows into a table”
Augmentation	Co-think, co-edit	Partner on ideas and drafts	Strategy brainstorm + critique
Agency	Set goals, knowledge, guardrails	Work more independently toward aims	Ongoing research agent with standing instructions

Exam tip: Agency is not “set and forget forever.” You still shape knowledge and behavior, then apply Discernment and Diligence on outcomes. More autonomy → **more** need for clear Description and stronger checks—not less.

Pitfall: Calling something “automation” when you actually need augmentation (judgment-heavy creative work).

Decision tree — pick a mode

Is the task fully specified with a clear success check?

YES → Automation (+ review)

NO ↓

Do you need ongoing co-thinking and judgment tradeoffs?

YES → Augmentation

NO ↓

Are you configuring standing goals/tools so AI can pursue work with less turn-by-turn steering?

YES → Agency (with stronger guardrails + monitoring)

ELSE → Start in Augmentation; tighten into Automation once stable

Anti-patterns by mode

Mode	Anti-pattern	Better
Automation	Vague task, no acceptance test	Specify + sample output
Augmentation	Accepting flattery as collaboration	Demand critique criteria
Agency	Unbounded autonomy on email/send	Guardrails, approvals, logs

4. Generative AI & LLM fundamentals (mental model)

You do not need a PhD for the quiz; you need a **usable mental model**:

- Modern assistants are built on **large language models** trained to predict useful next tokens in context
- They combine **learned patterns** with **current context** (your messages, files, tool results)
- Strengths often include: drafting, transforming, summarizing, coding assistance, structured brainstorming, explaining
- Limits often include: inventing plausible falsehoods, weak live knowledge without tools, context window bounds, uneven niche expertise, sensitivity to framing

Related collection pieces deepen: next-token prediction, working memory, steerability, tokens, and cost/quality tradeoffs.

Delegation link: Platform awareness means matching task type to these strengths/limits.

Trust topics to recognize (collection)

- **Hallucination** — Fluent but false content; spot-check, ground with sources/tools
- **Sycophancy** — Agreeing with you because you seem to want it; ask for critique explicitly
- **Bias** — Skewed associations from data/systems; watch stereotypes and skewed recommendations
- **Privacy** — What you paste may be processed; minimize sensitive data; follow org policy

Tokens & context (exam-aware, light)

- Usage is measured in tokens; longer context can raise cost and sometimes dilute attention

- Everything Claude “knows” for your answer is either baked into training or present in current context/tools
- Be intentional: more context is not always better—relevance beats dump

Worked example (original) — platform awareness

Task: “What’s our CEO’s exact quote from yesterday’s all-hands?” - Without connectors/transcripts: high hallucination risk → don’t Delegate as factual recall - With meeting notes in a Project or Enterprise Search: better platform fit - Still Discern: quote must match source text

5. Strategic Delegation in projects

When planning a project with AI:

1. Clarify the **problem** (who hurts if this is wrong?)
2. Inventory **platform** capabilities (chat only? tools? company search? coding agent?)
3. Slice **tasks** into AI-suitable vs human-required
4. Define checkpoints where Discernment is mandatory
5. Write Diligence rules (disclosure, data handling, approval gates)

Worked micro-example (original): Publishing a customer FAQ update.

- Human keeps: legal claims, pricing promises, escalation policy
- AI drafts: rewrites from ticket themes; proposes structure
- Checkpoint: human verifies every factual claim against source docs
- Disclosure: internal note that AI assisted drafting

Second worked example (original): Student literature survey

- Problem awareness: academic integrity rules; originality required
- Platform awareness: model may invent citations
- Task split: AI helps outline and explain papers you already found; human reads sources and writes claims
- Diligence: follow school disclosure policy; never cite unread papers

Keep / share / give worksheet (original)

Step in workflow	Keep	Share	Give	Checkpoint
Define success	✓			Stakeholder sign-off
Gather sources		✓		Source list review
Draft synthesis		✓		Fact audit
Format tables			✓	Spot-check
Publish	✓			Final human approve

6. Description in practice & prompting techniques

Translate Product / Process / Performance into a prompt skeleton (original wording):

PRODUCT: Deliverable, audience, length, must-include, must-avoid, example of good.
 PROCESS: Steps to follow, tools to use, when to ask me questions, what to cite.
 PERFORMANCE: Tone, uncertainty handling, challenge my assumptions if weak.

Then iterate. Description is not a one-shot ritual; it is a **conversation craft**.

Effective habits

- Show, don't only tell (examples beat adjectives)
- Separate "generate options" from "pick a winner"
- Ask for assumptions listed explicitly
- Prefer precise feedback over restart-everything rage (unless the thread is lost)
- Put evaluation criteria in the prompt when possible ("score yourself against this rubric, then improve")

Anti-patterns

Anti-pattern	Failure	Fix
Adjective pile ("brilliant, epic, viral")	Vague product	Concrete audience + example
No stop points	Overconfident completion	"Ask before assuming X"
Hidden stakes	Wrong risk posture	State who gets hurt if wrong
One giant mega-prompt forever	Brittle	Iterate via Discernment loop

If the exam asks X, think Y

- X: "Only product described" → Y: Add process + performance
- X: "Model never asks clarifying questions" → Y: Performance/process rules that require questions when ambiguous

7. The Description–Discernment loop

Core cycle:

1. Describe what you want (and how)
2. Receive output / observe behavior
3. Discern gaps against criteria
4. Update Description (or Delegation decision)
5. Repeat until the bar is met—or decide AI is the wrong tool

This loop is the antidote to both blind acceptance and aimless re-prompting.

Exam phrase to remember: Better Description is often the *result* of Discernment, not only its input.

Worked loop (original) — landing-page hero

1. Describe: “Hero copy for privacy-first notes app; 12 words max.”
2. Output: clever but implies “military-grade encryption” you don’t offer
3. Discern: marketing claim risk
4. Re-describe: “No security claims beyond ‘encrypted on device’; friendly tone; 12 words.”
5. Output passes claim check → proceed; still human-approves before ship (Deployment Diligence)

Loop failure modes

- Skipping criteria → endless taste debates
 - Changing the goal every turn → no learning
 - Never changing Delegation when the tool is wrong → wasted loops
-

8. Diligence in the real world

Creation: Don’t ship first draft vibes on regulated or reputation-critical work. Choose appropriate involvement mode; maintain a quality bar.

Transparency: Diligence statements, footnotes, or client notes that AI helped—proportionate to context (school policies and enterprise rules differ). Collection tutorial “Writing an AI diligence statement” is practical homework.

Deployment: Consider who is affected, failure modes, and monitoring after release (especially for agentic workflows).

Sample diligence statement pattern (original, short)

AI assistance: Claude helped outline and edit sections 2–3.

Human responsibility: I verified all facts against sources dated ____, removed unsupported claims, and approve this version for [audience].

Data note: No customer PII was pasted into the chat.

If the exam asks X, think Y

- X: “Course discloses AI help in making materials” → Y: Models Transparency Diligence
 - X: “Agent sends emails unsupervised” → Y: Deployment Diligence (+ Delegation/Agency risk)
-

9. How the 4Ds show up across modes

- **Automation:** Strong Description of the task; Discernment on outputs; Diligence on data you feed in
- **Augmentation:** All four, with heavy Description–Discernment looping
- **Agency:** Heavy upfront Delegation + standing Description; continuous Discernment of autonomous steps; Diligence before and after deployment

Claude product features (Projects, Skills, Connectors—see Claude 101 notes) are **infrastructure for Description and Delegation**, not a substitute for judgment.

Claude 101 feature	Fluency job
Projects	Standing Description (knowledge + rules)

Claude 101 feature	Fluency job
Skills	Process Description reusable
Artifacts	Make Product visible for Discernment
Connectors	Expand platform options for Delegation
Research / Enterprise Search	Grounding aids Discernment—still verify
Chat vs Cowork vs Code	Mode / Delegation choice

10. Behavioral indicators (study translation)

Public collection includes “4 Ds — Behavioral Indicators.” For exams, remember *examples of behaviors*, not a memorized official list:

D	Looks like in daily work
Delegation	Pausing to ask “should AI do this?”; writing keep/share/give
Description	Adding examples, constraints, success criteria; clarifying ambiguity
Discernment	Spot-checking facts; rejecting sycophancy; comparing to gold outputs
Diligence	Disclosing assistance; protecting data; refusing to ship unverified claims

Anti-behaviors

- Delegation: “Let AI decide the legal call”
- Description: “You know what I mean”
- Discernment: “Sounds smart, ship it”
- Diligence: “Nobody needs to know AI wrote this” when policy requires disclosure

Decision trees / comparison tables

Scenario → primary D

Scenario	Start with
Should we use AI at all on this?	Delegation
Output misses the brief	Description (then loop)
Output is fluent but wrong	Discernment
About to publish / client-send	Diligence
Model only agrees with you	Discernment + Performance Description
Choosing Automation vs Agency	Delegation (+ platform awareness)

Automation vs Augmentation vs Agency

Question	Automation	Augmentation	Agency
Who steers moment-to-moment?	Human spec	Both	Standing config + monitoring
Typical risk	Silent wrong execution	Collaborative drift	Unwatched side effects
Description weight	Task clarity	Ongoing dialogue	Goals + guardrails
Discernment timing	After output	Continuously	During and after autonomous steps

Product vs Process vs Performance

Layer	Asks	Example ingredient
Product	What is the thing?	"1-page FAQ; audience = new users"
Process	How to make it?	"Use only attached policy; list assumptions first"
Performance	How to behave?	"If unsure, ask; never invent policy"

Problem vs Platform vs Task (Delegation lenses)

Lens	Failure if skipped	Fix
Problem	Solve the wrong pain	Restate success + harm
Platform	Ask impossible recall / ignore tools	Match task to capabilities
Task	AI decides what humans must own	Keep/share/give split

Trust failure → response

Failure	Response mix
Hallucination	Grounding tools + verify sources + Discernment
Sycophancy	Ask for critique / steelman opposition
Bias	Diversify examples; inspect recommendations for stereotypes
Privacy leak risk	Minimize data; follow policy; prefer approved environments

Common exam traps

1. **Fluency = prompt engineering** — Too narrow. Prompting supports Description; fluency includes all 4Ds + modes.
 2. **Agency means no oversight** — Opposite. More autonomy needs more Description, Discernment, Diligence.
 3. **Automation is always faster and better** — Only when the task is well-specified and checked.
 4. **Discernment is only fact-checking outputs** — Also process and behavior (sycophancy, bias, refusals).
 5. **Diligence is only “be nice / ethical vibes”** — Includes creation care, disclosure, and deployment checks.
 6. **Product Description alone is enough** — Missing process/performance causes overreach.
 7. **Platform awareness is optional if you’re a power user** — Experts still mismatch tools to tasks.
 8. **If AI helped, you can deny responsibility** — False. Humans own outcomes.
 9. **Description–Discernment loop is optional polish** — It’s the core improvement engine.
 10. **Hallucination means the model is “broken” only** — It’s an expected failure mode of generative systems; mitigate, don’t mythologize.
 11. **Sycophancy is the same as hallucination** — Related trust issues; sycophancy is over-agreement/people-pleasing.
 12. **Collection electives don’t matter** — Exams may ask where to deepen (capabilities course, diligence statement, tokens).
 13. **4Ds are a strict waterfall once** — They interact continuously.
 14. **Effective automatically means ethical** — Four outcomes; you can be effective at the wrong thing.
 15. **Dakan–Feller framework is “Anthropic-only IP trivia”** — Know high-level origin; focus on applying competencies.
-

Expanded self-check (Q&A)

Q1. Define AI Fluency in one sentence using the four outcome words.

A1. AI Fluency is the ability to engage with AI systems in ways that are effective, efficient, ethical, and safe.

Q2. List the 4Ds and give one verb for each.

A2. Delegation (decide/split), Description (communicate/specify), Discernment (evaluate/critique), Diligence (own/disclose/protect).

Q3. How does Augmentation differ from Automation?

A3. Automation has AI execute a specified task; Augmentation is ongoing partnership on thinking and doing. Automation is “do this”; Augmentation is “work with me on this.”

Q4. You ask AI to “make my argument stronger,” and it only agrees louder. Which failure mode is likely, and what Description fix helps?

A4. Sycophancy. Ask it to steelman the opposing view, list weaknesses, and only then suggest improvements—Performance Description that rewards critique.

Q5. Name the three Description types and one prompt ingredient for each.

A5. Product (deliverable + example); Process (steps + when to ask questions); Performance (tone + uncertainty rules).

Q6. What is the Description–Discernment loop?

A6. Iterative cycle: describe needs → review results critically → refine description (or change delegation) → repeat.

Q7. Give an example of Transparency Diligence.

A7. Adding a short note that AI assisted drafting of a report section, per school or company policy.

Q8. Why does platform awareness matter for Delegation?

A8. You cannot wisely assign work without knowing tool limits (no live web, context size, hallucination risk, missing connectors, etc.).

Q9. Agency increases autonomy. Does that reduce the need for Discernment?

A9. No—it usually increases the need for clearer guardrails and stronger evaluation of independent actions.

Q10. Name two related collection resources beyond this flagship course.

A10. Any two: AI Capabilities and Limitations; 4 Properties of AI; hallucination/sycophancy/bias tutorials; tokens/context tutorials; diligence statement tutorial; role-specific Fluency editions.

Q11. Map “I automated grading comments without sampling accuracy first” to the broken Ds.

A11. Weak Discernment before Delegation into Automation; Diligence risk if deployed to students.

Q12. What are the three Delegation lenses?

A12. Problem awareness, platform awareness, and task delegation (split).

Q13. Creation vs Deployment Diligence—difference?

A13. Creation = care while producing with AI; Deployment = care before/as you release into real-world impact (users, public, systems).

Q14. Give a keep/share/give split for writing a public incident report.

A14. Keep: final severity language and legal claims. Share: structure and clarity edits. Give: formatting timeline tables from approved facts. Checkpoint: every factual sentence traced to an approved source.

Q15. Why are the 4Ds described as interconnected?

A15. Failures cascade: bad Description undermines Discernment; bad Discernment leads to reckless Delegation; Diligence fails if the other three are sloppy.

Q16. What is a usable LLM mental model for this course?

A16. Next-token / pattern prediction plus current context/tools; strong at draft/transform/explain; weak at guaranteed truth without grounding.

Q17. When should you exit the Description–Discernment loop and change tools or stop?

A17. When criteria cannot be met reliably (wrong platform, missing data, stakes too high for generative drafting)—change Delegation.

Q18. How do Projects/Skills from Claude 101 support Fluency?

A18. They encode standing Description (knowledge/process), improving Delegation quality—not replacing Discernment or Diligence.

Q19. Name the three Diligence strands.

A19. Creation, Transparency, Deployment.

Q20. “Sounds confident and cites DOIs.” Enough to trust?

A20. No. Discernment requires verification; citations can be invented or mismatched.

Q21. Efficient but unethical—fluency?

A21. Incomplete. Fluency requires effective, efficient, ethical, *and* safe.

Q22. Who developed the AI Fluency Framework (high level)?

A22. Professors Rick Dakan and Joseph Feller (publicly credited), later partnered with Anthropic for the Academy course.

Q23. Badge path on the public course page?

A23. Finish lessons → final assessment → completion badge opportunity.

Q24. Bias mitigation habit (one)?

A24. Inspect outputs for stereotyped assumptions; ask for alternative framings; diversify examples in Description.

Q25. Privacy diligence micro-habit?

A25. Minimize sensitive data pasted into chats; follow org/school rules; prefer approved enterprise controls when required.

Quick review checklist

- ☐ Fluency = effective, efficient, ethical, safe collaboration
 - ☐ 4Ds recited with one example behavior each
 - ☐ Automation / Augmentation / Agency differences cold
 - ☐ Product / Process / Performance Description ingredients
 - ☐ Problem / Platform / Task Delegation lenses
 - ☐ Description–Discernment loop steps
 - ☐ Creation / Transparency / Deployment Diligence
 - ☐ Hallucination, sycophancy, bias, privacy—define + mitigate
 - ☐ More agency → more oversight, not less
 - ☐ Features (Projects/Skills/etc.) support Description/Delegation; judgment remains human
 - ☐ Know collection “where next” at title level
 - ☐ Human accountability never disappears
-

Glossary

Term	Short definition
Agency	Configuring AI to pursue goals more independently under your guardrails
AI Fluency	Effective, efficient, ethical, safe human–AI collaboration skill
Augmentation	Human–AI partnership on thinking and doing
Automation	AI executes a specified task on instruction
Bias	Skewed patterns/associations that distort outputs
Creation Diligence	Care and quality bar while building with AI
Delegation	Deciding whether/when/how to use AI and how to split work
Deployment Diligence	Responsibility checks before/as work enters the world
Description	Communicating product, process, and performance clearly
Description–Discernment loop	Iterate specify → critique → respecify
Discernment	Critical evaluation of outputs, processes, behaviors
Diligence	Taking responsibility for AI-assisted work
Hallucination	Fluent falsehood
Platform awareness	Knowing a given tool’s capabilities and limits
Problem awareness	Understanding goals, stakes, constraints, success
Performance Description	Desired interaction behavior
Process Description	Desired method/steps/tools
Product Description	Desired deliverable
Sycophancy	Over-agreeable, people-pleasing behavior
Task delegation	Keep / share / give workflow splitting
Transparency Diligence	Appropriate disclosure of AI assistance
Tokens	Units of text processing/cost/context

Study plan

1. Memorize Key concepts map (10–15 minutes)
2. Read Deep notes on 4Ds + modes twice
3. Skim fundamentals + trust topics
4. Rehearse decision trees from memory
5. Drill self-check without peeking
6. Optional: skim collection titles so pathways feel familiar
7. Pair with 01–claude–101.md to connect framework → product features
8. Night before: checklist + glossary only

End of AI Fluency study notes.

Building with the Claude API — Exam Prep Study Notes

Note: This edition follows the ASD-STE100 writing rules: simple present tense, active voice, one word = one meaning, sentences of 20–25 words or less. Technical names (Claude, MCP, prompting, caching, effort, p95) are exceptions and stay as written. Model IDs and prices change. Learn the decision rules. Check the current docs before the exam.

Disclaimer: These are **original study notes** for exam and certification prep. They are **not** official Anthropic course material. They are **not** transcripts. They are **not** verbatim lesson

content. Align the topics to the public outline of [Building with the Claude API](#). Check current API details in the official Anthropic docs ([platform.claude.com](#) / [docs.anthropic.com](#)).

Primary sources to check (this file does not copy them): Messages API, tool use, streaming, structured outputs, prompt caching, Message Batches, vision/files, rate limits/errors.

Overview

The public course page is a production path for software engineers who already know Python and JSON. Stated outcomes include these skills. You authenticate and call the Anthropic API. You run single-turn chats and multi-turn chats. You use system prompts, temperature, streaming, and structured outputs. You evaluate prompts in a systematic way. You call tools (custom and built-in). You build RAG pipelines. You use extended features (thinking, vision, PDFs, citations, caching). You integrate MCP. You design agent workflows (parallelize, chain, route).

Course shape (public modules): Accessing Claude with the API → Prompt engineering → Tool use → RAG. Also, agentic search → Model Context Protocol → Anthropic apps (Claude Code, Computer Use) → Agents and workflows.

How to use these notes: Treat each section as a flashcard cluster. After you read a section, answer the Self-check Q&A. Do not look back. Prefer official docs for parameter names when APIs change.

Exam posture: Graders check if you can operate the Messages loop correctly. This includes roles, stop reasons, tool_result ids, and stream assembly. They also check if you diagnose cost and latency. Caching, batches, and model choice matter here. They check if you select workflow vs. agent patterns with intent. They do not grade marketing slogans.

Key concepts (cheat sheet)

Idea	Remember
Messages API	Conversation turns with roles user / assistant. There is no system role in messages. Use top-level system.
Auth	API key via env / secret store. Send required version headers. Never commit keys.
max_tokens	Required upper bound on generated tokens. Truncation vs. complete answers.
Temperature	Higher → more varied. Lower → more deterministic. Pair with evals.
Streaming	SSE event stream. Assemble deltas. Special handling for tool input_json_delta.
Stop reasons	end_turn, max_tokens, tool_use, stop_sequence, pause_turn, refusal. Each needs a different next action.
Tools	Model emits tool_use. Your app runs the tool. Return tool_result in the next turn.
Parallel tools	Multiple tool_use blocks in one assistant turn. Return one result per id.

Idea	Remember
Structured outputs	Schema-constrained JSON (and/or strict tools) for reliable pipelines.
RAG	Retrieve relevant chunks → inject into context → generate with citations/grounds.
Prompt caching	Cache stable prefixes (tools → system → messages) to cut latency and cost.
Batches	Async Message Batches for large offline jobs. Typically discounted vs. sync.
Agents	Workflows vs. free-form agents. Parallel / chain / route patterns.
MCP	Standardize tools/resources/prompts across hosts (see companion MCP notes).

Current notes (2026-08): (1) *Temperature* — Current frontier models remove sampling controls (temperature/top_p/top_k). Opus 4.7+, Opus 5, Sonnet 5, and Fable 5 return a 400. They stay valid on 4.6-and-older. Determinism advice moves to schemas and structured outputs. (2) *System role* — “no system role in messages” stays the foundations-exam answer. The newest Opus/Fable-tier models do accept **mid-conversation** {"role": "system"} messages appended to messages. This is a cache-friendly operator channel. messages[0] still cannot be a system message.

Deep notes by topic

1. Accessing Claude with the API

Authentication & setup

- Create an API key in the Anthropic Console. Store it in environment variables or a secrets manager (ANTHROPIC_API_KEY). Never put it in source control. Never put it in client-side apps.
- Use the official SDKs (Python / TypeScript) when you can. They handle headers, retries, and streaming helpers.
- HTTP clients must send the API key and an API version header (commonly anthropic-version). Treat version headers as part of the contract. Wrong or missing headers fail with a clear error.
- Typical request fields: model, max_tokens, messages, optional system, temperature, tools, stream, tool_choice, caching controls, thinking config, and similar fields.
- Select models by capability vs. cost and latency. For cert prep, know why you select a faster or cheaper model for classification or routing. Know why you pick a stronger model for hard reasoning or tool orchestration.
- **Server-side vs. client-side keys:** Browser apps must never hold long-lived API keys. Put Claude calls behind your backend.

Original example — minimal mental request shape

```
POST /v1/messages
Headers: x-api-key, anthropic-version, content-type: application/json
Body: {
  model: "<current-model-id>",
```



```

max_tokens: 1024,
system: "You are a careful support triage bot. Reply in JSON.",
messages: [
  { role: "user", content: "Order #4421 never arrived." }
]
}

```

Models & selection heuristics

Workload	Prefer	Why
Intent routing / cheap classify	Smaller / faster model	High volume, low ambiguity
Hard reasoning, long tool chains	Stronger model	Fewer tool mistakes, better planning
Batch offline labeling	Strong enough + Batches API	Cost + throughput
Interactive UI chat	Mid/strong + streaming	Latency perception + quality

Common exam error: “always use the model with the most capability.” The correct answer is to match the model size to the task. Measure with evals.

Messages & multi-turn context

- Each message has a role and content (string or content blocks).
- Alternate user/assistant turns. Your app owns history. Trim, summarize, or drop old turns to stay within context limits.
- Content can be multimodal: text plus images or document inputs where the API supports them.
- Common error: put a “system” message into messages with role system. The Messages API expects the top-level system parameter instead. *(Current note: newest Opus/Fable-tier models accept mid-conversation system messages appended to messages. The exam-level rule still stands. Put system instructions in top-level system.)*
- When you continue after tools, append the full assistant turn (including tool_use blocks). Then add a user turn that holds tool_result blocks. Do not invent fake assistant text in place of tool results.

History hygiene patterns

1. **Sliding window:** keep last N turns. Drop older user/assistant pairs.
2. **Summarize-and-replace:** replace early turns with a compact summary message at set intervals.
3. **Pin critical facts:** put durable policy in system. Put session facts in a short pinned user/assistant preface.
4. **Never silently rewrite tool transcripts** if you still need auditability. Store raw tool logs offline even if you summarize for the model.

System prompts

- System text sets role, rules, output format, safety boundaries, and tool-use policy.
- Keep system prompts stable when you use prompt caching. Put volatile user data in the user message.

- Prefer explicit instructions (“always return JSON that matches schema X”). Do not use vague lines (“be helpful”).
- Structure long systems with labeled sections (XML-style tags are a common pattern): `<role>`, `<rules>`, `<output_format>`, `<examples>`.
- Tool policy belongs here too: when to call tools, when to ask the user, when to refuse.

Temperature, max_tokens, and output control

- Temperature (and related sampling controls) trades creativity for consistency — **on models where they still exist**. Current frontier models (Opus 4.7+, Opus 5, Sonnet 5, Fable 5) have **removed** temperature/top_p/top_k. If you send them, you get a 400. They stay valid on 4.6-and-older models.
- For grading, extraction, and tool argument generation, prefer schemas and structured outputs as the primary consistency control. Add lower temperature only on older models that support it.
- Always set max_tokens with care. Too low truncates mid-JSON. Too high wastes budget if you do not need long answers.
- max_tokens is an **upper bound**, not a target. Models often stop earlier with end_turn.
- Custom stop_sequences can halt generation on sentinel strings. Then stop_reason becomes stop_sequence. Use them sparingly. Schemas are usually better than brittle stop strings for structured work.

Streaming (SSE)

- Set stream: true to receive server-sent events as the model generates.
- Typical event flow: message_start → (content_block_start → deltas → content_block_stop)* → message_delta → message_stop (plus ping / error events as applicable).
- For UX: render text deltas immediately. Buffer tool JSON until the block completes. Or follow fine-grained tool streaming rules with care.
- Exam tip: streaming does **not** change the final semantic result. It changes delivery. It changes how you assemble the message.
- Tool streaming detail: tool_use blocks often start with empty input: {}. Arguments arrive as input_json_delta / partial_json fragments. Concatenate fragments. Then parse after content_block_stop.
- Thinking streams (when enabled) may emit thinking_delta and a signature_delta before the thinking block closes. Treat signatures as integrity metadata. Do not treat them as user-facing prose.
- SDKs usually provide helpers (text_stream, get_final_message / finalMessage). You do not hand-parse SSE unless you build a raw HTTP client.
- Handle unknown event types with care (versioning may add events).

Stop reasons (must-know table)

stop_reason	Meaning	Typical next action
end_turn	Natural completion	Show / use the response
max_tokens	Hit your output cap	Raise max_tokens or continue generation
tool_use	Model wants client tools	Execute tools. Send tool_results

stop_reason	Meaning	Typical next action
stop_sequence	Hit your custom stop string	Handle sentinel. Maybe continue
pause_turn	Server-side loop paused (e.g. long server tools / thinking)	Continue turn per docs (often re-send assistant content)
refusal	Policy refusal	Surface safely. Do not retry without a check with the same prompt

Common exam error: treat every unfinished answer as a model failure. First check `stop_reason` and usage.

Structured outputs

- Use schema-constrained JSON so downstream code can parse without brittle regex.
- Strict tool schemas similarly reduce invalid tool arguments.
- Combine structured outputs with evals. Assert shape *and* semantic correctness.
- Prefer official structured-output / JSON-schema features when they are available. Do not rely on “please reply as JSON” alone.
- If schema fails in production, log raw text. Fail closed for critical paths. Fall back to a repair prompt only when that path is safe.

Errors, retries, and rate limits

- Expect HTTP-class failures: auth (401), bad request (400), not found (404), rate limit (429), overloaded / capacity (e.g. 529-class), 5xx.
- On 429: respect `retry-after` when it is present. Use exponential backoff + jitter. Isolate per-tenant budgets if you multi-tenant.
- Distinguish **request rate** vs. **token rate** limits (RPM vs. input/output tokens per minute). A “small” chat app can still exceed ITPM with huge RAG contexts.
- Idempotency rule: retries must not double-charge side effects. Make tool execution idempotent. Or dedupe by `tool_use id`.
- Streaming errors can arrive as SSE error events mid-stream. Your client must abort cleanly. Then decide whether to resume.

Message Batches (async scale)

- Message Batches API submits many Messages-style requests asynchronously. It is good for offline eval sets, bulk classification, and overnight enrichment.
- Batches typically cost less than synchronous Messages for the same tokens. You retrieve results later. This is not a live chat UX.
- Each item needs a `custom_id` so you can join results back to source rows.
- Still subject to batch-specific rate / queue limits. Requests can expire if the system does not process them in time. Docs cite long outer windows such as ~24h processing bounds. Check current numbers before exams.
- Feature parity is broad (tools, vision, caching often supported). Cache **pre-warm with `max_tokens: 0`** is a poor fit inside batches. Ephemeral cache may expire before follow-ups run.

- Design pattern: sync path for interactive users. Batch path for backfills and eval sweeps.
-

2. Prompt engineering & evaluation

Techniques that show up on exams

- **Clear directives:** say what to do, what not to do, and the output contract.
- **XML / tagged sections:** separate instructions, context, examples, and user input so the model can attend to the right block.
- **Few-shot examples:** show input→output pairs that match the desired style and edge cases.
- **Chain of thought (when appropriate):** ask for reasoned steps for hard problems. For structured extraction, prefer schemas over open-ended rambling.
- **Role prompting:** put expertise and constraints in system.
- **Delimiter hygiene:** never concatenate untrusted user text next to instructions without clear boundaries. This reduces instruction-injection risk.

Original mini-pattern (study only)

system:

```
<role>You extract invoice fields.</role>
<rules>If a field is missing, use null. Never invent tax IDs.</rules>
<output>JSON object with keys vendor, total, currency, due_date.</output>
```

user:

```
<document>
... raw OCR text...
</document>
```

Evaluation approach

- Do not judge prompts only by inspection in production. Build a small dataset of cases (standard path, edge cases, adversarial).
- Automate grading: exact match, rubric scores, or model-as-judge with a fixed rubric.
- Iterate: change one prompt variable at a time. Track regressions.
- Generate synthetic test cases. Then hand-spot-check for realism.
- Separate **prompt regressions** from **model version** regressions. Pin model ids in eval runs.
- Track cost and latency alongside quality. A 2% quality gain that triples tokens may be a product loss.

Eval loop (memorize)

1. Define task success criteria in writing.
 2. Collect 30–200 labeled cases (start small).
 3. Freeze a grader (script or rubric).
 4. Run baseline → change one thing → compare.
 5. Promote only if quality ↑ and cost/latency acceptable.
-

3. Tool use with Claude

Core loop

1. Send user message + tool definitions (name, description, JSON input_schema).
2. Model may return one or more tool_use blocks (id, name, input).
3. Your application executes the tools (and enforces authz!).
4. Append tool_result blocks tied to each tool_use id. Continue the conversation until the model answers in plain text (or calls more tools).

Design tips

- Descriptions are part of the prompt. Be precise about when to call the tool and what each field means.
- Prefer many small, composable tools over one oversized tool with ambiguous arguments.
- Handle parallel/batch tool calls. Execute safely (idempotency, timeouts, partial failure).
- Built-in / server tools (e.g. web search, code execution, computer-use variants that depend on the product surface) differ from client-executed tools. Know which side runs the code.
- tool_choice can guide or force tool use (auto / any / specific tool / none — confirm exact enum in current docs).
- For strict argument validity, enable schema-strict tool modes when the API offers them.

Parallel tools checklist

- One assistant message may contain multiple tool_use blocks.
- Return **one** tool_result per id in the following user message (can be same user message content array).
- Failures: still return a result with an error payload so the model can recover. Do not omit the id.
- Concurrency: only parallelize tools that are safe together (no conflicting writes).

Client tools vs. server tools

Kind	Who executes?	Your job
Client tool	Your app / MCP server / backend	Run function. Send tool_result
Server tool	Anthropic infrastructure	Often appears as server tool use + result blocks in-stream. You may still need to continue on pause_turn

Common failures

- Forgetting to return results for every tool_use id.
- Executing tools the user is not allowed to run.
- Feeding huge raw tool dumps back without a summary when context is tight.
- Treating hallucinated tool names as real — validate against your registry.
- Parsing partial JSON mid-stream before content_block_stop.
- Infinite tool loops with no max-iteration / budget guard.

Original example — weather tool round-trip (conceptual)

Assistant content:

```
[tool_use id=toolu_1 name=get_weather input={location:"Singapore"}]
```

Your next user content:

```
[tool_result tool_use_id=toolu_1 content="31°C, thunderstorms"]
```

Then model produces end_turn text for the user.

4. RAG and agentic search

Why RAG: Context windows are finite. Private corpora change. You want grounded answers with sources.

Pipeline building blocks

- **Chunking:** split documents by size/structure (headings, paragraphs). Overlap can help continuity.
- **Embeddings + vector search:** semantic similarity for fuzzy queries.
- **BM25 / lexical search:** strong for exact terms, IDs, rare tokens.
- **Hybrid retrieval:** combine lexical + dense. Often add **reranking**.
- **Contextual retrieval:** enrich chunks with document-level context so retrieval is less ambiguous.
- **Multi-index:** separate indexes by corpus or modality. Route queries.

Injection pattern

system: Answer ONLY using <sources>. If missing, say you don't know.

user:

```
<question>...</question>
<sources>
[1]...chunk...
[2]...chunk...
</sources>
```

Agentic search pattern

- Instead of one-shot retrieve→answer, let the model (or a planner) issue follow-up searches. Refine queries. Stop when evidence is enough.
- Always keep a citation trail: which chunk IDs supported which claims.
- Budget the loop: max searches, max tokens, max wall time.

Evaluation split (exam favorite)

Layer	Metric ideas
Retrieval	Recall@k, MRR, citation hit-rate
Generation	Faithfulness, answer correctness, format validity
System	Latency p95, cost / query, tool error rate

Pitfalls

- Chunks too large (noise) or too small (lost meaning).
 - You retrieve relevant text, but a weak prompt does not use it well.
 - No evaluation of retrieval hit-rate separately from generation quality.
 - Updating docs in the CMS but forgetting to re-index.
 - Do not retrieve Letting the model cite sources that (hallucinated citations).
-

5. Extended Claude features

- **Extended thinking / reasoning modes:** useful for hard multi-step problems. Understand cost/latency tradeoffs. Whether thinking content is separable from the final answer depends on the API surface.
- **Images & PDFs:** pass as content blocks. Ask for structured extraction. Watch token/cost implications of large documents.
- **Vision inputs:** common source types include base64, URL, or Files API references. Media types typically include jpeg/png/gif/webp. Confirm current limits.
- **Documents / PDFs:** prefer upload-once via Files API and reference by id for repeated analysis. Page limits and token costs matter for exam “what breaks at scale?” questions.
- **Citations:** when available, prefer features that ground quotes to source spans for trustworthy UX.
- **Prompt caching:** mark stable prefixes. Measure cache hit rates. Do not put per-request secrets or unique IDs in the cached prefix if that defeats reuse.

Prompt caching — operational detail

- Cacheable prefix order to remember: **tools** → **system** → **messages** (and included content blocks up to a cache breakpoint).
- Place `cache_control` on the **last shared block** you expect to reuse. Do not put it on a unique placeholder user message if that would key the cache poorly.
- Monitor usage fields such as cache creation vs. cache read tokens (names like `cache_creation_input_tokens` / `cache_read_input_tokens` in responses).
- TTL / ephemeral behavior: know that cache entries expire. High-churn prefixes waste money on repeated writes.
- Pre-warming exists in some forms (including `max_tokens: 0` patterns for cache population). Use it only when follow-up traffic will hit soon enough.

Files, vision, and documents — design checklist

1. Resize/compress huge images before upload when quality allows.
 2. Prefer file ids over repeatedly shipping megabyte base64 payloads.
 3. For PDFs: extract text when layout is simple. Use document/vision pathways when layout matters.
 4. Ask for structured fields + page references when you build audit trails.
 5. Redact secrets in screenshots before they enter logs or prompts.
-

6. MCP (course module — high level)

This API course also covers building MCP servers/clients so tools and resources are reusable across hosts. For depth, study the dedicated MCP notes (`04-introduction-to-mcp.md`). Exam-relevant mapping:

- Host app ↔ MCP client ↔ MCP server.
- Primitives: **tools** (actions), **resources** (read-only data), **prompts** (reusable instruction templates).
- Prefer MCP when many apps need the same integrations. Prefer inline tools for single-use prototypes.

- Lifecycle: initialize / negotiate capabilities → list primitives → call tools / read resources → stop cleanly.
 - Security: user consent, least privilege, and clear boundaries still apply when tools arrive via MCP instead of hard-coded JSON schemas.
-

7. Anthropic apps: Claude Code & Computer Use

- **Claude Code:** agentic coding assistant in the terminal/IDE. It accelerates repo tasks. It often integrates MCP for project tools.
 - **Computer Use:** model-driven UI automation (screenshots + actions). It is powerful and risky. Sandbox. Use least privilege. Get human approval for sensitive steps.
 - Cert framing: know *use cases* and *safety boundaries*. Do not memorize product marketing copy.
 - Map to API knowledge: Computer Use is an extreme form of tool use + vision. Claude Code is an agent host with permissions, hooks, and MCP connectors.
-

8. Agents and workflows

Workflow vs. agent

- **Workflow:** you define the graph (steps, branches). The model fills in nodes. Deterministic control. Easier to test.
- **Agent:** model decides next actions with tools until a stop condition. Flexible. Harder to evaluate.

Core patterns

- **Parallelization:** fan-out independent subtasks. Merge results.
- **Chaining:** output of step N becomes input to N+1 (extract → transform → validate).
- **Routing:** classify intent, then send to a specialized prompt/model/tool path.
- **Debugging:** log every tool call, intermediate state, and token usage. Add timeouts and max-iteration caps to prevent unbounded loops.

Comparison table

Pattern	Control	Best for	Failure mode
Chain	High	ETL-like LLM steps	Error cascades if no validators
Parallel	Medium	Map-reduce style tasks	Merge conflicts / inconsistent schemas
Route	High	Multi-domain assistants	Mis-route → wrong specialist
Free agent	Low	Open-ended research	Loops, spend spikes, hard audits

Production guardrails (write these on the exam)

1. Max tool iterations / max wall clock.
2. Per-tool authz and allowlists.
3. Human approval for irreversible actions.

4. Structured logs (request id, tool id, tokens, stop_reason).
5. Dead-letter queue for failed tool side effects.
6. Eval suite before you enable new tools.

Original routing sketch

User message

- cheap classifier (billing | technical | cancel)
 - billing prompt + CRM tools
 - technical prompt + docs RAG
 - cancel workflow (deterministic steps + confirmation)
-

9. End-to-end request anatomy (study diagram)

Think in layers so exam scenarios do not blur together:

1. **Transport/auth layer** — HTTPS, API key, version header, optional beta headers.
2. **Sampling layer** — model id, temperature, max_tokens, stop sequences, thinking config.
3. **Context layer** — system, prior messages, retrieved RAG chunks, images/files.
4. **Capability layer** — tools / MCP-discovered tools, tool_choice, structured output schema.
5. **Delivery layer** — stream on/off. Client event assembly. Timeouts.
6. **Control-plane layer** — your retries, rate-limit budgets, agent iteration caps, authz.

If something fails, locate the layer first. A parse error in the client is not a “model quality” problem. A tool_use without results is not a temperature problem.

10. Multi-turn conversation patterns worth memorizing

A. Plain chat User → Assistant (end_turn) → User → ...

B. Tool-interrupted turn User → Assistant (tool_use) → User(tool_result) → Assistant (end_turn or more tools)

C. Parallel tools User → Assistant(tool_use A + tool_use B) → User(tool_result A + tool_result B) → ...

D. RAG-augmented turn Retriever(user query) → User(question + sources) → Assistant grounded answer

E. Routed workflow Classifier → branch-specific system/tool set → specialized answer

F. Batch offline Enqueue N independent Messages params → poll/fetch results by custom_id

Original practice cue: given a bug report (“assistant asked for weather but never answered”), map it to pattern B. Ask which hop broke. Options: model did not call tool / app did not execute / app did not return result / second model call missing.

11. Prompt caching worked example (conceptual)

Suppose every support ticket shares:

- the same 3 tools (CRM lookup, refund calculator, policy search),
- the same 2,000-token policy system prompt,

- but a unique ticket body.

Good: cache tools + system (and maybe a static policy resource). Put ticket text only in the varying user message after the breakpoint.

Bad: put Ticket `#{{id}}` inside the cached system string. Each ticket writes a new prefix.

Measurement habit: log cache-write tokens vs. cache-read tokens. A feature that “uses caching” but shows near-zero cache reads gives no real benefit.

Invalidation intuition: a change to tool schemas, system text, thinking/effort settings, or breakpoint placement can miss the prior cache entry. Keep cached bytes byte-stable across requests you want to hit.

12. Structured outputs & tools — choosing a contract

Need	Prefer	Notes
Final answer must be parseable JSON	Structured outputs / JSON schema mode	Best for API responses to clients
Model must call your function	Tools + input_schema (+ strict if available)	Best for side effects
Both	Tools for actions + structured final text/schema	Common agent pattern
Soft formatting only	Prompt instructions	Fragile. Use only for prototypes

Common exam error: “tools are only for side effects, never for JSON.” Historically, developers also used tools to force JSON shapes. Modern structured-output features may be cleaner for final answers. Know both stories. Check current docs.

13. Rate limits, errors, and resilience playbook

Classify the failure

Signal	Likely class	First response
401/403	Auth/permission	Fix key/scopes. Do not retry without a check
400	Request shape / params	Fix payload. Log body (redacted)
404	Bad path / missing resource	Fix endpoint or file id
429	Rate limit	Backoff. Shed load. Reduce tokens
5xx / overloaded	Capacity	Retry with jitter. Fallback model if policy allows
SSE error event	Mid-stream failure	Abort UI cleanly. Optional resume strategy

Resilience patterns

- Token bucket per tenant.
- Queue interactive traffic separately from batch traffic.

- Circuit breaker when error rate spikes.
- Fallback to a smaller model for non-critical paths.
- Idempotency keys around refunds/emails triggered by tools.

14. RAG production checklist (beyond the happy path)

1. **Corpus governance:** who can add docs? PII scrubbing?
2. **Chunking strategy recorded:** size, overlap, splitter version — so you can reproduce indexes.
3. **Embedding model version pinned.**
4. **Hybrid weights tuned** on a labeled query set, not vague preference.
5. **Rerank top-n** before prompting.
6. **Prompt forces citations** from provided ids only.
7. **Refuse when retrieval empty.**
8. **Freshness:** re-index on publish. Show doc updated-at to users when relevant.
9. **Poisoning awareness:** untrusted user-uploaded docs can instruct the model. Sandbox them. Label them.
10. **Cost guard:** cap retrieved tokens. Do not dump entire manuals.

15. Agent debugging scenarios (exam-style)

Scenario 1 — Spend spike. Logs show 40 tool calls/turn. Fix: max iterations, tool allowlist, require plan-before-act for expensive tools.

Scenario 2 — Fails intermittently JSON. stop_reason=max_tokens mid-object. Fix: raise cap, shrink schema, or continue. Validate before DB write.

Scenario 3 — Wrong tool chosen. Descriptions overlap (“search” vs “search_users”). Fix: rename, narrow descriptions, add “when NOT to use” notes, reduce tool count.

Scenario 4 — Grounding misses. Answers sound confident without sources. Fix: retrieval eval. Strengthen “sources only” system rule. Add citation feature if available.

Scenario 5 — Cache never hits. Breakpoint on unique user turn. Fix: move breakpoint to shared system/tools.

16. Security & compliance notes for API builders

- Treat prompts + tool results as **data stores**. They may contain secrets. Scrub logs.
- Separate **model trust** from **tool trust**. Even if the model asks to delete_all_users, your authz layer must refuse.
- Prompt injection via retrieved docs or web-tool results is real. Prefer allowlisted tools. Confirm irreversible actions.
- For Computer Use / browser tools: dedicated VM, no production credentials. Record sessions for audit when required.
- Data retention: know whether your org requires zero-retention options or regional processing. Cloud courses cover Bedrock/Vertex residency separately.

17. Mapping course modules → skills you must demo

Module	Skill to demo from memory
Accessing API	Auth, messages, system, stream, structured out
Prompt engineering	Tagged prompts + eval loop
Tool use	Full client tool loop + parallel + server-tool awareness
RAG / agentic search	Hybrid retrieve + cite + iterate
MCP	When to externalize tools/resources/prompts
Claude Code / Computer Use	Use cases + safety boundaries
Agents / workflows	Chain / parallel / route + guardrails

Comparison tables (exam quick hits)

Sync Messages vs. Batches

	Sync Messages	Message Batches
Latency	Seconds	Minutes–hours
UX	Interactive	Offline / jobs
Cost	Standard	Typically discounted
Streaming	Yes	No live SSE to user
Best for	Chat, copilots	Evals, backfills

Streaming vs. non-streaming

	Non-stream	Stream SSE
Time-to-first-token	Worse	Better
Client complexity	Lower	Higher (assemble events)
Final message	Direct JSON	Accumulated equivalent
Tool JSON	Complete object	Often partial deltas first

Inline tools vs. MCP tools

	Inline tool defs	MCP
Setup speed	Fastest	More moving parts
Reuse across hosts	Copy/paste schemas	Shared servers
Ops	App owns everything	Server processes / remote endpoints
When	Single app prototype	Platform / many clients

Exam tips

1. **Messages vs. Completions working model:** Claude’s public API path is Messages-centric. System is top-level.
2. **Tool loop completeness:** every `tool_use` needs a matching `tool_result` (or a deliberate abort strategy).

3. **Streaming assembly:** text deltas vs. partial JSON for tools. Do not json.loads mid-stream unless you know it is complete.
4. **Evals are better than vague judgment:** expect questions on datasets, graders, and regression when you change prompts.
5. **RAG is two systems:** retrieval quality and generation quality fail differently. Diagnose them separately.
6. **Caching & cost:** know *what* to cache (stable prefix) and *why* (latency + spend).
7. **Safety:** keys, tool authz, computer-use sandboxing, and PII in logs are valid exam topics for “what is wrong with this design?” questions.
8. **MCP pointer:** if a question mentions standardized reusable tools across hosts, think MCP. Do not think ad-hoc duplicate schemas.
9. **Always read stop_reason before retrying.**
10. **Batches ≠ streaming chat.** Different product shape.
11. **Parallel tools:** match ids, not names alone.
12. **Rate limits:** backoff + jitter. Separate RPM from token budgets.

Exam traps (common wrong answers)

Trap	Why it is wrong	Better answer
Put role: "system" inside messages	Wrong shape for Messages API	Top-level system
Assume stream changes answer quality	Delivery ≠ sampling semantics	Same request params → same final content intent
Drop failed tool ids	Model waits forever / errors	Return error tool_result for that id
Cache the unique user question as prefix	Destroys hit rate	Cache tools/system/static docs
One oversized do_anything tool	Ambiguous args, hard authz	Small tools with clear schemas
Only measure final answer accuracy for RAG	Hides retrieval failures	Measure retrieval and generation separately
No max agent iterations	Cost/unbounded risk	Hard caps + stop control
Put API key in a mobile app	Key theft	Backend proxy

(Row 1 current note: mid-conversation {"role": "system"}) The newest Opus/Fable-tier models now accept mid-conversation system messages. “Top-level system” remains the correct foundations-exam

answer for where system instructions live.)

Self-check Q&A

Q1. Where do you put system instructions in the Messages API? **A:** In the top-level system parameter — not as a message with role system.

Q2. What must your app do after Claude returns tool_use blocks? **A:** Execute authorized tools. Then continue the conversation with matching tool_result content blocks (by tool use id).

Q3. Name one reason to stream responses. **A:** Lower perceived latency / progressive UI updates. The final assembled message should match non-streaming semantics.

Q4. Why combine BM25 with embeddings in RAG? **A:** Lexical search catches exact tokens/IDs. Dense search catches semantic paraphrases. Hybrid plus reranking is often better than either alone.

Q5. What is a prompt evaluation workflow in one sentence? **A:** Fixed test cases + automated scoring so prompt changes are measurable and regressions are caught.

Q6. When is a deterministic workflow better than an open-ended agent? **A:** When steps are known, compliance/auditability matters, or you need reliable SLAs and cheaper debugging.

Q7. What belongs in a prompt cache prefix? **A:** Stable content reused across requests (long system prompts, tool defs, static docs). Not unique per-user payloads that destroy hit rate.

Q8. How does MCP relate to the API course? **A:** It standardizes how hosts discover and call external tools/resources/prompts instead of reinventing schemas per app.

Q9. What does stop_reason: "max_tokens" imply? **A:** Output hit your cap. Raise max_tokens or continue generation. Do not treat the truncated JSON as complete.

Q10. What is input_json_delta for? **A:** Streaming partial JSON fragments that assemble into a tool's input object after the content block stops.

Q11. Client tool vs. server tool—who runs the code? **A:** Client tools run in your environment. Server tools run on Anthropic's side (your app may still need to continue turns).

Q12. Why require max_tokens? **A:** It bounds generation cost/length. Models may stop earlier, but you always set the ceiling.

Q13. Name two rate-limit dimensions. **A:** Requests per minute and tokens per minute (input/output). 429s need backoff.

Q14. When would you use Message Batches? **A:** You do not need Large asynchronous jobs (evals, backfills) where interactive latency and discounted pricing helps.

Q15. How do you handle parallel tool_use ids? **A:** Execute each authorized tool. Return a tool_result for every id in the follow-up user turn.

Q16. What is a safe temperature default for extraction? **A:** On older models (4.6 and earlier): low temperature plus a schema. On current frontier models the API removes sampling params. The schema or structured output is the determinism control.

Q17. How should vision images be supplied? **A:** As image content blocks via supported sources (e.g. base64, URL, or Files API id). Not as pretend text paths.

Q18. What is contextual retrieval trying to fix? A: Ambiguous chunks that lack document-level context, which hurts embedding match quality.

Q19. Why pin model ids in evals? A: So prompt changes are not confounded with silent model upgrades.

CCAR-F Domain 4 gap card (added 2026-08-23)

Compact supplement for **exam guide tasks 4.3/4.4/4.6** specifics the notes above touch only lightly. Original synthesis against the official task statements.

G1. Schema design against fabrication (task 4.3)

- **Nullable/optional fields prevent fabrication:** if a source document may lack a field, a **required** schema field forces the model to invent a value. Make it **optional/nullable** so the model can return null. This is the exam's canonical "model fabricates values" fix.
- **Extensible enums:** add "unclear" for ambiguous cases. Add "other" **plus a detail string field** for categories outside the enum. Closed enums force misclassification.
- **Strict schemas via tool_use eliminate syntax errors only.** They never eliminate *semantic* errors. Examples: line items that do not sum to the stated total, or correct values in wrong fields. Semantic checks are validation-layer work (G2).
- Include **format-normalization rules in the prompt** alongside strict output schemas when source formatting varies.

G2. Validation, retry, and feedback loops (task 4.4)

- **Retry-with-error-feedback:** on validation failure, send a follow-up that contains **the original document, the failed extraction, and the specific validation errors**. The model self-corrects against named errors far better than against a bare "try again."
- **Limits of retry (tested):** retries fix **format/structural** errors. They are **ineffective when the information is absent from the source** (or lives in a document you did not provide). Diagnose which failure class you have before you use more retries.
- **Self-correction flows:** extract `calculated_total` alongside `stated_total` to flag discrepancies. Add `conflict_detected` booleans for inconsistent source data.
- **detected_pattern fields:** tag each finding with the code construct/pattern that triggered it. Then you can analyze dismissed findings in a systematic way for false-positive patterns (feeds the 4.1 precision loop).

G3. Multi-instance & multi-pass review (task 4.6)

- **Self-review limitation:** a model that generated code **retains its reasoning context**. It is unlikely to question its own decisions in the same session. Neither "review your work carefully" instructions nor extended thinking substitute for independence.
- **Independent review instance:** a second Claude instance **without the generator's reasoning context** catches subtle issues self-review misses. This pairs with the CI session-isolation point in file 05's supplement.

- **Multi-pass review:** split large reviews into **per-file local passes + a cross-file integration pass**. Attention dilution, not context size, causes inconsistent single-pass reviews (see 08 §6).
- **Calibrated routing:** have the reviewer self-report **confidence alongside each finding** to enable review routing. Then calibrate those scores before you trust them (see 09 §5).

G4. Gap-card Q&A

GQ1. The model fills in an invoice field the document does not contain. Schema fix? **A.** Make the field optional/nullable. Required fields force fabrication.

GQ2. What errors do strict tool-use schemas eliminate, and what survives? **A.** JSON syntax errors are eliminated. Semantic errors (wrong sums, misplaced values) survive and need validation logic.

GQ3. Describe retry-with-error-feedback. **A.** Follow-up request = original document + failed extraction + the specific validation errors. This guides targeted self-correction.

GQ4. When will retries never fix an extraction? **A.** When the required information is absent from the provided source. That is a data problem, not a format problem.

GQ5. Why is an independent instance better than “review your own code thoroughly”? **A.** The generator retains its reasoning context and will not question its own decisions. Independence removes that bias.

GQ6. Purpose of a detected_pattern field in review findings? **A.** Enables systematic analysis of which constructs trigger dismissed findings. This is the raw material for cutting false-positive categories.

Q20. What is a routing workflow? **A:** Classify the request. Then send it to a specialized prompt/model/tool path.

Q21. What should you log for an agent turn? **A:** Message ids, stop_reason, tool names/ids/inputs (redacted), results status, token usage, latency.

Q22. Cache prefix order to remember? **A:** Tools, then system, then messages—up to the cache breakpoint.

Q23. What is pause_turn signaling? **A:** A long-running server-side turn paused. Continue according to docs rather than treating it as a hard failure.

Q24. Why are tool descriptions “prompt engineering”? **A:** The model uses them to decide *when* and *how* to call tools. Vague descriptions cause wrong calls.

Q25. What is wrong with returning a huge SQL dump as tool_result? **A:** Context bloat and distraction. Summarize or page results. Keep raw data server-side.

Q26. How do refusals differ from empty answers? **A:** stop_reason: "refusal" shows a policy refusal. Handle with safe UX. Do not infinite-retry the same content.

Q27. Name one Computer Use safety control. **A:** Sandbox / least privilege / human approval for sensitive UI actions.

Q28. What is the difference between chaining and parallelization? **A:** Chaining is sequential dependencies. Parallelization fans out independent work then merges.

Q29. Why separate retrieval metrics from answer metrics? **A:** Bad answers may be good writing over bad chunks—or good chunks with a weak prompt. Fixes differ.

Q30. What is the first debugging step when tool use does not complete? **A:** Check whether every `tool_use` id received a `tool_result`. Inspect `stop_reason` / iteration caps.

Quick review checklist

- ☐ I can sketch a Messages request with `model`, `max_tokens`, `system`, `messages`.
 - ☐ I never put `system` as a message role.
 - ☐ I can name ≥ 4 stop reasons and the next action for each.
 - ☐ I can assemble SSE text deltas and `tool_partial_json` safely.
 - ☐ I can complete a tool loop including parallel tools and error results.
 - ☐ I can explain structured outputs vs. “please return JSON”.
 - ☐ I can design a tiny eval set and grader.
 - ☐ I can describe hybrid RAG + why to measure retrieval separately.
 - ☐ I know what to put in a prompt-cache prefix (and what not to).
 - ☐ I know when to use Batches vs. sync Messages.
 - ☐ I can compare workflow vs. agent and give parallel/chain/route examples.
 - ☐ I can map MCP host/client/server at a high level.
 - ☐ I treat API keys, tool authz, and computer-use sandboxing as exam-critical safety topics.
-

Glossary

Term	Plain meaning
Messages API	Chat-style API with user/assistant turns and top-level system
Content block	Typed piece of message content (text, image, <code>tool_use</code> , <code>tool_result</code> , ...)
SSE	Server-Sent Events stream used for incremental responses
<code>stop_reason</code>	Why generation stopped
Tool use	Model requests a function call via <code>tool_use</code> blocks
Server tool	Tool executed by the platform rather than your client
Structured outputs	Schema-constrained model output for parsers
Prompt caching	Reusing a stable prompt prefix to save tokens/latency
Message Batches	Async bulk Messages processing API
RAG	Retrieve documents, then generate grounded answers
BM25.	Common lexical ranking function.
Reranker	Second-stage model/function that reorders retrieved hits
Agentic search	Multi-step retrieve/refine loop driven by the model
Workflow	Developer-defined graph of LLM/tool steps
Agent	Model-directed loop over tools until a stop condition
MCP	Open protocol for reusable tools/resources/prompts across hosts
Computer Use	Vision + UI action tools for controlling a desktop/browser
ITPM / OTPM	Input/output tokens-per-minute style rate limits
<code>custom_id</code>	Caller-assigned id joining a batch request to its result

Quick lab checklist (practice without copying course labs)

Use this as a private drill list while you study official docs/SDKs:

1. **Hello Messages:** one-shot user message → print assistant text. Then add a second user turn with prior assistant message preserved.
2. **System ablations:** same user question with three different system prompts. Note tone and constraint adherence.
3. **Stream assemble:** concatenate text deltas. Assert equality with non-streamed output for the same seed/settings when possible.
4. **Tool round-trip:** fake weather tool. Verify ids match on `tool_result`. Force a parallel double call if the model emits two tools.
5. **Structured parse:** schema for `{ "sentiment": enum, "confidence": number }`. Fail the build if parse throws.
6. **Mini-RAG:** three markdown files → chunk → embed or BM25 → answer with “sources:” lines.
7. **Cache sanity:** put a long static system prompt behind cache controls. Compare latency on request 1 vs. request 2.
8. **Agent budget:** max 5 tool iterations. On loop, return “budget exceeded” instead of hanging.
9. **Stop-reason drill:** intentionally set tiny `max_tokens` and confirm you detect truncation.
10. **Batch dry-run design:** sketch 100 classification rows with `custom_ids` and a results joiner (even without calling the API).
11. **429 simulator:** write a retry helper with exponential backoff + jitter and a max-retry cap.
12. **Vision extract:** one screenshot/receipt image → structured JSON fields. Verify you did not hardcode fake OCR.

These drills reinforce exam mechanics. They are not substitutes for Anthropic’s graded quizzes.

Source URLs (verify before exam day)

- Course outline: <https://academy.claude.com/courses/building-with-the-claude-api>
- Anthropic docs hub: <https://docs.anthropic.com> (also platform.claude.com/docs)
- Topics to re-open live: Messages, streaming, stop reasons, tool use, structured outputs, prompt caching, batch processing, vision/files, rate limits/errors, MCP overview

End of notes. Cross-check parameters against current Anthropic platform docs before an exam sitting.

Building with the Claude API — Exam Prep Study Notes

Disclaimer: These are **original study notes** written for exam / certification prep. They are **not** official Anthropic course material, transcripts, or verbatim lesson content. Align topics to the public outline of [Building with the Claude API](#). Always verify current API details in the official Anthropic docs (platform.claude.com / docs.anthropic.com).

Primary sources to cross-check (not copied here): Messages API, tool use, streaming, structured outputs, prompt caching, Message Batches, vision/files, rate limits/errors.

Overview

The public course page positions this as a production-oriented path for software engineers who already know Python and JSON. Stated outcomes include: authenticate and call the Anthropic API; run single- and multi-turn chats; use system prompts, temperature, streaming, and structured outputs; evaluate prompts systematically; call tools (custom and built-in); build RAG pipelines; use extended features (thinking, vision, PDFs, citations, caching); integrate MCP; and design agent workflows (parallelize, chain, route).

Course shape (public modules): Accessing Claude with the API → Prompt engineering → Tool use → RAG and agentic search → Model Context Protocol → Anthropic apps (Claude Code, Computer Use) → Agents and workflows.

How to use these notes: Treat each section as a flashcard cluster. After reading, answer the Self-check Q&A without looking back. Prefer official docs for parameter names when APIs change.

Exam posture: Graders care whether you can *operate* the Messages loop correctly (roles, stop reasons, tool_result ids, stream assembly), diagnose cost/latency (caching, batches, model choice), and choose workflow vs. agent patterns deliberately—not whether you memorized marketing slogans.

Key concepts (cheat sheet)

Idea	Remember
Messages API	Conversation turns with roles user / assistant; no system role in messages — use top-level system
Auth	API key via env / secret store; send required version headers; never commit keys
max_tokens	Required upper bound on generated tokens; truncation vs. complete answers
Temperature	Higher → more varied; lower → more deterministic; pair with evals
Streaming	SSE event stream; assemble deltas; special handling for tool input_json_delta
Stop reasons	end_turn, max_tokens, tool_use, stop_sequence, pause_turn, refusal — each needs a different next action
Tools	Model emits tool_use; your app runs the tool; return tool_result in next turn
Parallel tools	Multiple tool_use blocks in one assistant turn; return one result per id
Structured outputs	Schema-constrained JSON (and/or strict tools) for reliable pipelines
RAG	Retrieve relevant chunks → inject into context → generate with citations/grounds
Prompt caching	Cache stable prefixes (tools → system → messages) to cut latency/cost
Batches	Async Message Batches for large offline jobs; typically discounted vs. sync

Idea	Remember
Agents	Workflows vs. free-form agents; parallel / chain / route patterns
MCP	Standardize tools/resources/prompts across hosts (see companion MCP notes)

Currency notes (2026-08): (1) *Temperature* — sampling knobs (temperature/top_p/top_k) are **removed on current frontier models** (Opus 4.7+, Opus 5, Sonnet 5, Fable 5 return a 400); they remain valid on 4.6-and-older. Determinism advice shifts to schemas/structured outputs. (2) *System role* — “no system role in messages” remains the foundations-exam answer, but the newest Opus/Fable-tier models do accept **mid-conversation** {"role": "system"} messages appended to messages (cache-friendly operator channel); messages[0] still can't be one.

Deep notes by topic

1. Accessing Claude with the API

Authentication & setup

- Create an API key in the Anthropic Console; store it in environment variables or a secrets manager (ANTHROPIC_API_KEY), never in source control or client-side apps.
- Use the official SDKs (Python / TypeScript) when possible: they handle headers, retries, and streaming helpers.
- HTTP clients must send the API key and an API version header (commonly anthropic-version). Treat version headers as part of the contract: wrong/missing headers fail loudly.
- Typical request fields: model, max_tokens, messages, optional system, temperature, tools, stream, tool_choice, caching controls, thinking config, etc.
- Choose models by capability vs. cost/latency. For cert prep: know *why* you'd pick a faster/cheaper model for classification or routing and a stronger model for hard reasoning or tool orchestration.
- **Server-side vs. client-side keys:** browser apps must never hold long-lived API keys. Put Claude calls behind your backend.

Original example — minimal mental request shape

```
POST /v1/messages
Headers: x-api-key, anthropic-version, content-type: application/json
Body: {
  model: "<current-model-id>",
  max_tokens: 1024,
  system: "You are a careful support triage bot. Reply in JSON.",
  messages: [
    { role: "user", content: "Order #4421 never arrived." }
  ]
}
```

Models & selection heuristics

Workload	Prefer	Why
Intent routing / cheap classify	Smaller / faster model	High volume, low ambiguity
Hard reasoning, long tool chains	Stronger model	Fewer tool mistakes, better planning
Batch offline labeling	Strong enough + Batches API	Cost + throughput
Interactive UI chat	Mid/strong + streaming	Latency perception + quality

Exam trap: “always use the biggest model.” Correct answer is **right-size** for the task and measure with evals.

Messages & multi-turn context

- Each message has a role and content (string or content blocks).
- Alternate user/assistant turns. Your app owns history: trim, summarize, or drop old turns to stay within context limits.
- Content can be multimodal: text plus images or document inputs where the API supports them.
- Pitfall: stuffing a “system” message into messages with role system — Messages API expects the top-level system parameter instead. *(Current note: newest Opus/Fable-tier models accept mid-conversation system messages appended to messages; the exam-level rule — system instructions go in top-level system — still stands.)*
- When continuing after tools, append the full assistant turn (including tool_use blocks) and a user turn that holds tool_result blocks. Do not invent fake assistant text in place of tool results.

History hygiene patterns

1. **Sliding window:** keep last N turns; drop older user/assistant pairs.
2. **Summarize-and-replace:** periodically replace early turns with a compact summary message.
3. **Pin critical facts:** put durable policy in system; put session facts in a short pinned user/assistant preface.
4. **Never silently rewrite tool transcripts** if you still need auditability—store raw tool logs offline even if you summarize for the model.

System prompts

- System text sets role, rules, output format, safety boundaries, and tool-use policy.
- Keep system prompts stable when using prompt caching; put volatile user data in the user message.
- Prefer explicit instructions (“always return JSON matching schema X”) over vague vibes (“be helpful”).
- Structure long systems with labeled sections (XML-style tags are a common pattern): <role>, <rules>, <output_format>, <examples>.
- Tool policy belongs here too: when to call tools, when to ask the user, when to refuse.

Temperature, max_tokens, and output control

- Temperature (and related sampling knobs) trade creativity for consistency — **on models where they still exist**. Current frontier models (Opus 4.7+, Opus 5, Sonnet 5, Fable 5) have **removed** temperature/top_p/top_k (sending them returns a 400); they remain valid on 4.6-and-older models.
- For grading, extraction, and tool argument generation, prefer **schemas / structured outputs** as the primary consistency lever (plus lower temperature only on older models that support it).
- Always set max_tokens thoughtfully: too low truncates mid-JSON; too high wastes budget if you don't need long answers.
- max_tokens is an **upper bound**, not a target. Models often stop earlier with end_turn.
- Custom stop_sequences can halt generation on sentinel strings; then stop_reason becomes stop_sequence. Use sparingly—schemas usually beat brittle stop strings for structured work.

Streaming (SSE)

- Set stream: true to receive server-sent events as the model generates.
- Typical event flow: message_start → (content_block_start → deltas → content_block_stop)* → message_delta → message_stop (plus ping / error events as applicable).
- For UX: render text deltas immediately; buffer tool JSON until the block completes (or follow fine-grained tool streaming rules carefully).
- Exam tip: streaming does **not** change the final semantic result; it changes delivery and how you assemble the message.
- Tool streaming detail: tool_use blocks often start with empty input: {}; arguments arrive as input_json_delta / partial_json fragments. Concatenate fragments, then parse after content_block_stop.
- Thinking streams (when enabled) may emit thinking_delta and a signature_delta before the thinking block closes—treat signatures as integrity metadata, not user-facing prose.
- SDKs usually provide helpers (text_stream, get_final_message / finalMessage) so you don't hand-parse SSE unless building a raw HTTP client.
- Handle unknown event types gracefully (versioning may add events).

Stop reasons (must-know table)

stop_reason	Meaning	Typical next action
end_turn	Natural completion	Show / use the response
max_tokens	Hit your output cap	Raise max_tokens or continue generation
tool_use	Model wants client tools	Execute tools; send tool_results
stop_sequence	Hit your custom stop string	Handle sentinel; maybe continue
pause_turn	Server-side loop paused (e.g. long server tools / thinking)	Continue turn per docs (often re-send assistant content)
refusal	Policy refusal	Surface safely; don't retry blindly with same prompt

Exam trap: treating every unfinished answer as a model failure. First check stop_reason and usage.

Structured outputs

- Use schema-constrained JSON so downstream code can parse without brittle regex.
- Strict tool schemas similarly reduce invalid tool arguments.
- Combine structured outputs with evals: assert shape *and* semantic correctness.
- Prefer official structured-output / JSON-schema features when available over “please reply as JSON” alone.
- If schema fails in production, log raw text, fail closed for critical paths, and fall back to a repair prompt only when safe.

Errors, retries, and rate limits

- Expect HTTP-class failures: auth (401), bad request (400), not found (404), rate limit (429), overloaded / capacity (e.g. 529-class), 5xx.
- On 429: respect `retry-after` when present; use exponential backoff + jitter; isolate per-tenant budgets if you multi-tenant.
- Distinguish **request rate** vs. **token rate** limits (RPM vs. input/output tokens per minute). A “small” chat app can still blow ITPM with huge RAG contexts.
- Idempotency mindset: retries must not double-charge side effects—make tool execution idempotent or dedupe by `tool_use id`.
- Streaming errors can arrive as SSE error events mid-stream; your client must abort cleanly and decide whether to resume.

Message Batches (async scale)

- Message Batches API submits many Messages-style requests asynchronously—good for offline eval sets, bulk classification, overnight enrichment.
- Typically priced lower than synchronous Messages for the same tokens; results are retrieved later (not a live chat UX).
- Each item needs a `custom_id` so you can join results back to source rows.
- Still subject to batch-specific rate / queue limits; requests can expire if not processed in time (docs cite long outer windows such as ~24h processing bounds—verify current numbers before exams).
- Feature parity is broad (tools, vision, caching often supported), but cache **pre-warm with `max_tokens: 0`** is a poor fit inside batches because ephemeral cache may expire before follow-ups run.
- Design pattern: sync path for interactive users; batch path for backfills and eval sweeps.

2. Prompt engineering & evaluation

Techniques that show up on exams

- **Clear directives:** say what to do, what not to do, and the output contract.
- **XML / tagged sections:** separate instructions, context, examples, and user input so the model can attend to the right block.
- **Few-shot examples:** show input→output pairs that match the desired style and edge cases.
- **Chain of thought (when appropriate):** ask for reasoned steps for hard problems; for structured extraction, prefer schemas over open-ended rambling.
- **Role prompting:** put expertise and constraints in system.

- **Delimiter hygiene:** never concatenate untrusted user text next to instructions without clear boundaries—reduces instruction-injection risk.

Original mini-pattern (study only)

system:

```
<role>You extract invoice fields.</role>
<rules>If a field is missing, use null. Never invent tax IDs.</rules>
<output>JSON object with keys vendor, total, currency, due_date.</output>
```

user:

```
<document>
... raw OCR text...
</document>
```

Evaluation mindset

- Don't "eyeball" prompts in production. Build a small dataset of cases (happy path, edge cases, adversarial).
- Automate grading: exact match, rubric scores, or model-as-judge with a fixed rubric.
- Iterate: change one prompt variable at a time; track regressions.
- Generate synthetic test cases, then hand-spot-check for realism.
- Separate **prompt regressions** from **model version** regressions—pin model ids in eval runs.
- Track cost and latency alongside quality; a 2% quality bump that triples tokens may be a product loss.

Eval loop (memorize)

1. Define task success criteria in writing.
 2. Collect 30–200 labeled cases (start small).
 3. Freeze a grader (script or rubric).
 4. Run baseline → change one thing → compare.
 5. Promote only if quality ↑ and cost/latency acceptable.
-

3. Tool use with Claude

Core loop

1. Send user message + tool definitions (name, description, JSON input_schema).
2. Model may return one or more tool_use blocks (id, name, input).
3. Your application executes the tools (and enforces authz!).
4. Append tool_result blocks tied to each tool_use id; continue the conversation until the model answers in plain text (or calls more tools).

Design tips

- Descriptions are part of the prompt: be precise about when to call the tool and what each field means.
- Prefer many small, composable tools over one mega-tool with ambiguous arguments.
- Handle parallel/batch tool calls: execute safely (idempotency, timeouts, partial failure).
- Built-in / server tools (e.g. web search, code execution, computer use variants depending on product surface) differ from client-executed tools: know which side runs the code.

- `tool_choice` can nudge or force tool use (auto / any / specific tool / none—confirm exact enum in current docs).
- For strict argument validity, enable schema-strict tool modes when offered.

Parallel tools checklist

- One assistant message may contain multiple `tool_use` blocks.
- Return **one** `tool_result` per id in the following user message (can be same user message content array).
- Failures: still return a result with an error payload so the model can recover—don't omit the id.
- Concurrency: only parallelize tools that are safe together (no conflicting writes).

Client tools vs. server tools

Kind	Who executes?	Your job
Client tool	Your app / MCP server / backend	Run function; send <code>tool_result</code>
Server tool	Anthropic infrastructure	Often appears as server tool use + result blocks in-stream; you may still need to continue on <code>pause_turn</code>

Common failures

- Forgetting to return results for every `tool_use` id.
- Executing tools the user is not allowed to run.
- Feeding huge raw tool dumps back without summarizing when context is tight.
- Treating hallucinated tool names as real — validate against your registry.
- Parsing partial JSON mid-stream before `content_block_stop`.
- Infinite tool loops with no max-iteration / budget guard.

Original example — weather tool round-trip (conceptual)

Assistant content:

```
[tool_use id=toolu_1 name=get_weather input={location:"Singapore"}]
```

Your next user content:

```
[tool_result tool_use_id=toolu_1 content="31°C, thunderstorms"]
```

Then model produces `end_turn` text for the user.

4. RAG and agentic search

Why RAG: Context windows are finite; private corpora change; you want grounded answers with sources.

Pipeline building blocks

- **Chunking:** split documents by size/structure (headings, paragraphs); overlap can help continuity.
- **Embeddings + vector search:** semantic similarity for fuzzy queries.

- **BM25 / lexical search:** strong for exact terms, IDs, rare tokens.
- **Hybrid retrieval:** combine lexical + dense; often add **reranking**.
- **Contextual retrieval:** enrich chunks with document-level context so retrieval is less ambiguous.
- **Multi-index:** separate indexes by corpus or modality; route queries.

Injection pattern

system: Answer ONLY using <sources>. If missing, say you don't know.

user:

<question>...</question>

<sources>

[1]...chunk...

[2]...chunk...

</sources>

Agentic search pattern

- Instead of one-shot retrieve→answer, let the model (or a planner) issue follow-up searches, refine queries, and stop when evidence is enough.
- Always keep a citation trail: which chunk IDs supported which claims.
- Budget the loop: max searches, max tokens, max wall time.

Evaluation split (exam favorite)

Layer	Metric ideas
Retrieval	Recall@k, MRR, citation hit-rate
Generation	Faithfulness, answer correctness, format validity
System	Latency p95, cost / query, tool error rate

Pitfalls

- Chunks too large (noise) or too small (lost meaning).
- Retrieving relevant text but burying it under a weak prompt (“use the context if helpful”).
- No evaluation of retrieval hit-rate separately from generation quality.
- Updating docs in the CMS but forgetting to re-index.
- Letting the model cite sources that weren’t retrieved (hallucinated citations).

5. Extended Claude features

- **Extended thinking / reasoning modes:** useful for hard multi-step problems; understand cost/latency tradeoffs and that thinking content may be separable from the final answer depending on API surface.
- **Images & PDFs:** pass as content blocks; ask for structured extraction; watch token/cost implications of large documents.
- **Vision inputs:** common source types include base64, URL, or Files API references; media types typically include jpeg/png/gif/webp—confirm current limits.
- **Documents / PDFs:** prefer upload-once via Files API and reference by id for repeated analysis; page limits and token costs matter for exam “what breaks at scale?” questions.

- **Citations:** when available, prefer features that ground quotes to source spans for trustworthy UX.
- **Prompt caching:** mark stable prefixes; measure cache hit rates; don't put per-request secrets or unique IDs in the cached prefix if that defeats reuse.

Prompt caching — operational detail

- Cacheable prefix order to remember: **tools** → **system** → **messages** (and included content blocks up to a cache breakpoint).
- Place `cache_control` on the **last shared block** you expect to reuse—not on a unique placeholder user message if that would key the cache poorly.
- Monitor usage fields such as cache creation vs. cache read tokens (names like `cache_creation_input_tokens` / `cache_read_input_tokens` in responses).
- TTL / ephemeral behavior: know that cache entries expire; high-churn prefixes waste money on repeated writes.
- Pre-warming exists in some forms (including `max_tokens: 0` patterns for cache population)—use only when follow-up traffic will hit soon enough.

Files, vision, and documents — design checklist

1. Resize/compress huge images before upload when quality allows.
2. Prefer file ids over repeatedly shipping megabyte base64 payloads.
3. For PDFs: extract text when layout is simple; use document/vision pathways when layout matters.
4. Ask for structured fields + page references when building audit trails.
5. Redact secrets in screenshots before they enter logs or prompts.

6. MCP (course module — high level)

This API course also covers building MCP servers/clients so tools and resources are reusable across hosts. For depth, study the dedicated MCP notes (`04-introduction-to-mcp.md`). Exam-relevant mapping:

- Host app ↔ MCP client ↔ MCP server.
- Primitives: **tools** (actions), **resources** (read-only data), **prompts** (reusable instruction templates).
- Prefer MCP when many apps need the same integrations; prefer inline tools for one-off prototypes.
- Lifecycle: initialize / negotiate capabilities → list primitives → call tools / read resources → shut down cleanly.
- Security: user consent, least privilege, and clear boundaries still apply when tools arrive via MCP instead of hard-coded JSON schemas.

7. Anthropic apps: Claude Code & Computer Use

- **Claude Code:** agentic coding assistant in the terminal/IDE; accelerates repo tasks; often integrates MCP for project tools.
- **Computer Use:** model-driven UI automation (screenshots + actions); powerful and risky — sandbox, least privilege, human approval for sensitive steps.
- Cert framing: know *use cases* and *safety boundaries*, not product marketing copy.

- Map to API knowledge: Computer Use is an extreme form of tool use + vision; Claude Code is an agent host with permissions, hooks, and MCP connectors.

8. Agents and workflows

Workflow vs. agent

- **Workflow:** you define the graph (steps, branches); the model fills in nodes. Deterministic control, easier to test.
- **Agent:** model decides next actions with tools until a stop condition. Flexible, harder to evaluate.

Core patterns

- **Parallelization:** fan-out independent subtasks; merge results.
- **Chaining:** output of step N becomes input to N+1 (extract → transform → validate).
- **Routing:** classify intent, then send to specialized prompt/model/tool path.
- **Debugging:** log every tool call, intermediate state, token usage; add timeouts and max-iteration caps to prevent runaway loops.

Comparison table

Pattern	Control	Best for	Failure mode
Chain	High	ETL-like LLM steps	Error cascades if no validators
Parallel	Medium	Map-reduce style tasks	Merge conflicts / inconsistent schemas
Route	High	Multi-domain assistants	Mis-route → wrong specialist
Free agent	Low	Open-ended research	Loops, spend spikes, hard audits

Production guardrails (write these on the exam)

1. Max tool iterations / max wall clock.
2. Per-tool authz and allowlists.
3. Human approval for irreversible actions.
4. Structured logs (request id, tool id, tokens, stop_reason).
5. Dead-letter queue for failed tool side effects.
6. Eval suite before enabling new tools.

Original routing sketch

User message

- cheap classifier (billing | technical | cancel)
- billing prompt + CRM tools
- technical prompt + docs RAG
- cancel workflow (deterministic steps + confirmation)

9. End-to-end request anatomy (study diagram)

Think in layers so exam scenarios don't blur together:

1. **Transport/auth layer** — HTTPS, API key, version header, optional beta headers.
2. **Sampling layer** — model id, temperature, max_tokens, stop sequences, thinking config.
3. **Context layer** — system, prior messages, retrieved RAG chunks, images/files.
4. **Capability layer** — tools / MCP-discovered tools, tool_choice, structured output schema.
5. **Delivery layer** — stream on/off; client event assembly; timeouts.
6. **Control-plane layer** — your retries, rate-limit budgets, agent iteration caps, authz.

If something fails, locate the layer first. A parse error in the client is not a “model quality” problem; a tool_use without results is not a temperature problem.

10. Multi-turn conversation patterns worth memorizing

A. Plain chat User → Assistant (end_turn) → User → ...

B. Tool-interrupted turn User → Assistant (tool_use) → User(tool_result) → Assistant (end_turn or more tools)

C. Parallel tools User → Assistant(tool_use A + tool_use B) → User(tool_result A + tool_result B) → ...

D. RAG-augmented turn Retriever(user query) → User(question + sources) → Assistant grounded answer

E. Routed workflow Classifier → branch-specific system/tool set → specialized answer

F. Batch offline Enqueue N independent Messages params → poll/fetch results by custom_id

Original practice cue: given a bug report (“assistant asked for weather but never answered”), map it to pattern B and ask which hop broke (model didn't call tool / app didn't execute / app didn't return result / second model call missing).

11. Prompt caching worked example (conceptual)

Suppose every support ticket shares:

- the same 3 tools (CRM lookup, refund calculator, policy search),
- the same 2,000-token policy system prompt,
- but a unique ticket body.

Good: cache tools + system (and maybe a static policy resource). Put ticket text only in the varying user message after the breakpoint.

Bad: put Ticket #{{id}} inside the cached system string—each ticket writes a new prefix.

Measurement habit: log cache-write tokens vs. cache-read tokens. A feature that “uses caching” but shows near-zero reads is theater.

Invalidation intuition: changing tool schemas, system text, thinking/effort settings, or breakpoint placement can miss the prior cache entry. Keep cached bytes byte-stable across requests you want to hit.

12. Structured outputs & tools — choosing a contract

Need	Prefer	Notes
Final answer must be parseable JSON	Structured outputs / JSON schema mode	Best for API responses to clients
Model must call your function	Tools + input_schema (+ strict if available)	Best for side effects
Both	Tools for actions + structured final text/schema	Common agent pattern
Soft formatting only	Prompt instructions	Fragile; use only for prototypes

Exam trap: “tools are only for side effects, never for JSON.” Historically, tools were *a/so* used to force JSON shapes; modern structured-output features may be cleaner for final answers—know both stories and check current docs.

13. Rate limits, errors, and resilience playbook

Classify the failure

Signal	Likely class	First response
401/403	Auth/permission	Fix key/scopes; don't retry blindly
400	Request shape / params	Fix payload; log body (redacted)
404	Bad path / missing resource	Fix endpoint or file id
429	Rate limit	Backoff; shed load; reduce tokens
5xx / overloaded	Capacity	Retry with jitter; fallback model if policy allows
SSE error event	Mid-stream failure	Abort UI cleanly; optional resume strategy

Resilience patterns

- Token bucket per tenant.
- Queue interactive traffic separately from batch traffic.
- Circuit breaker when error rate spikes.
- Fallback to a smaller model for non-critical paths.
- Idempotency keys around refunds/emails triggered by tools.

14. RAG production checklist (beyond the happy path)

1. **Corpus governance:** who can add docs? PII scrubbing?
2. **Chunking strategy recorded:** size, overlap, splitter version—so you can reproduce indexes.
3. **Embedding model version pinned.**
4. **Hybrid weights tuned** on a labeled query set, not vibes.
5. **Rerank top-n** before prompting.
6. **Prompt forces citations** from provided ids only.
7. **Refuse when retrieval empty.**
8. **Freshness:** re-index on publish; show doc updated-at to users when relevant.

9. **Poisoning awareness:** untrusted user-uploaded docs can instruct the model—sandbox and label them.
10. **Cost guard:** cap retrieved tokens; don't dump entire manuals.

15. Agent debugging scenarios (exam-style)

Scenario 1 — Spend spike. Logs show 40 tool calls/turn. Fix: max iterations, tool allowlist, require plan-before-act for expensive tools.

Scenario 2 — Flaky JSON. stop_reason=max_tokens mid-object. Fix: raise cap, shrink schema, or continue; validate before DB write.

Scenario 3 — Wrong tool chosen. Descriptions overlap (“search” vs “search_users”). Fix: rename, narrow descriptions, add “when NOT to use” notes, reduce tool count.

Scenario 4 — Grounding misses. Answers sound confident without sources. Fix: retrieval eval; strengthen “sources only” system rule; add citation feature if available.

Scenario 5 — Cache never hits. Breakpoint on unique user turn. Fix: move breakpoint to shared system/tools.

16. Security & compliance notes for API builders

- Treat prompts + tool results as **data stores**: they may contain secrets; scrub logs.
- Separate **model trust** from **tool trust**: even if the model asks to delete_all_users, your authz layer must refuse.
- Prompt injection via retrieved docs or web-tool results is real—prefer allowlisted tools and confirmations for irreversible actions.
- For Computer Use / browser tools: dedicated VM, no production credentials, record sessions for audit when required.
- Data retention: know whether your org requires zero-retention options or regional processing (cloud courses cover Bedrock/Vertex residency separately).

17. Mapping course modules → skills you must demo

Module	Skill to demo from memory
Accessing API	Auth, messages, system, stream, structured out
Prompt engineering	Tagged prompts + eval loop
Tool use	Full client tool loop + parallel + server-tool awareness
RAG / agentic search	Hybrid retrieve + cite + iterate
MCP	When to externalize tools/resources/prompts
Claude Code / Computer Use	Use cases + safety boundaries
Agents / workflows	Chain / parallel / route + guardrails

Comparison tables (exam quick hits)

Sync Messages vs. Batches

	Sync Messages	Message Batches
Latency	Seconds	Minutes–hours
UX	Interactive	Offline / jobs
Cost	Standard	Typically discounted
Streaming	Yes	No live SSE to user
Best for	Chat, copilots	Evals, backfills

Streaming vs. non-streaming

	Non-stream	Stream SSE
Time-to-first-token	Worse	Better
Client complexity	Lower	Higher (assemble events)
Final message	Direct JSON	Accumulated equivalent
Tool JSON	Complete object	Often partial deltas first

Inline tools vs. MCP tools

	Inline tool defs	MCP
Setup speed	Fastest	More moving parts
Reuse across hosts	Copy/paste schemas	Shared servers
Ops	App owns everything	Server processes / remote endpoints
When	Single app prototype	Platform / many clients

Exam tips

1. **Messages vs. Completions mental model:** Claude’s public API path is Messages-centric; system is top-level.
2. **Tool loop completeness:** every `tool_use` needs a matching `tool_result` (or a deliberate abort strategy).
3. **Streaming assembly:** text deltas vs. partial JSON for tools — don’t `json.loads` mid-stream unless you know it’s complete.
4. **Evals beat vibes:** expect questions on datasets, graders, and regression when changing prompts.
5. **RAG is two systems:** retrieval quality and generation quality fail differently — diagnose separately.
6. **Caching & cost:** know *what* to cache (stable prefix) and *why* (latency + spend).
7. **Safety:** keys, tool authz, computer-use sandboxing, and PII in logs are fair game for “what’s wrong with this design?” questions.
8. **MCP pointer:** if a question mentions standardized reusable tools across hosts, think MCP — not ad-hoc duplicate schemas.
9. **Always read `stop_reason` before retrying.**
10. **Batches ≠ streaming chat.** Different product shape.
11. **Parallel tools:** match ids, not names alone.

12. **Rate limits:** backoff + jitter; separate RPM from token budgets.

Exam traps (common wrong answers)

Trap	Why it's wrong	Better answer
Put role: "system" inside messages	Wrong shape for Messages API	Top-level system
Assume stream changes answer quality	Delivery $\neq$ sampling semantics	Same request params $\rightarrow$ same final content intent
Drop failed tool ids	Model waits forever / errors	Return error <code>tool_result</code> for that id
Cache the unique user question as prefix	Destroys hit rate	Cache tools/system/static docs
One mega-tool <code>do_anything</code>	Ambiguous args, hard authz	Small tools with clear schemas
Only measure final answer accuracy for RAG	Hides retrieval failures	Measure retrieval and generation separately
No max agent iterations	Cost/runaway risk	Hard caps + kill switch
Put API key in a mobile app	Key theft	Backend proxy

(Row 1 currency note: mid-conversation `{"role": "system"}` messages are now accepted on the newest Opus/Fable-tier models — but “top-level system” remains the correct foundations-exam answer for where system instructions live.)

Self-check Q&A

Q1. Where do you put system instructions in the Messages API? **A:** In the top-level system parameter — not as a message with role system.

Q2. What must your app do after Claude returns `tool_use` blocks? **A:** Execute authorized tools, then continue the conversation with matching `tool_result` content blocks (by tool use id).

Q3. Name one reason to stream responses. **A:** Lower perceived latency / progressive UI updates; final assembled message should match non-streaming semantics.

Q4. Why combine BM25 with embeddings in RAG? **A:** Lexical search catches exact tokens/IDs; dense search catches semantic paraphrases; hybrid + rerank often beats either alone.

- Q5. What's a prompt evaluation workflow in one sentence? A:** Fixed test cases + automated scoring so prompt changes are measurable and regressions are caught.
- Q6. When is a deterministic workflow better than an open-ended agent? A:** When steps are known, compliance/auditability matters, or you need reliable SLAs and cheaper debugging.
- Q7. What belongs in a prompt cache prefix? A:** Stable content reused across requests (long system prompts, tool defs, static docs) — not unique per-user payloads that destroy hit rate.
- Q8. How does MCP relate to the API course? A:** It standardizes how hosts discover and call external tools/resources/prompts instead of reinventing schemas per app.
- Q9. What does stop_reason: "max_tokens" imply? A:** Output hit your cap; raise max_tokens or continue generation—don't treat the truncated JSON as complete.
- Q10. What is input_json_delta for? A:** Streaming partial JSON fragments that assemble into a tool's input object after the content block stops.
- Q11. Client tool vs. server tool—who runs the code? A:** Client tools run in your environment; server tools run on Anthropic's side (your app may still need to continue turns).
- Q12. Why require max_tokens? A:** It bounds generation cost/length; models may stop earlier, but you always set the ceiling.
- Q13. Name two rate-limit dimensions. A:** Requests per minute and tokens per minute (input/output); 429s need backoff.
- Q14. When would you use Message Batches? A:** Large asynchronous jobs (evals, backfills) where interactive latency isn't required and discounted pricing helps.
- Q15. How do you handle parallel tool_use ids? A:** Execute each authorized tool and return a tool_result for every id in the follow-up user turn.
- Q16. What's a safe temperature default for extraction? A:** On older models (4.6 and earlier): low temperature plus a schema. On current frontier models sampling params are removed — the schema/structured output is the determinism lever.
- Q17. How should vision images be supplied? A:** As image content blocks via supported sources (e.g. base64, URL, or Files API id)—not as pretend text paths.
- Q18. What is contextual retrieval trying to fix? A:** Ambiguous chunks that lack document-level context, which hurts embedding match quality.
- Q19. Why pin model ids in evals? A:** So prompt changes aren't confounded with silent model upgrades.
-

CCAR-F Domain 4 gap card (added 2026-08-23)

Compact supplement for **exam guide tasks 4.3/4.4/4.6** specifics the notes above touch only lightly. Original synthesis against the official task statements.

G1. Schema design against fabrication (task 4.3)

- **Nullable/optional fields prevent fabrication:** if a source document may lack a field, a **required** schema field forces the model to invent a value to satisfy the schema. Make it **optional/nullable** so the model can return null — the exam’s canonical “model fabricates values” fix.
- **Extensible enums:** add "unclear" for ambiguous cases and "other" **plus a detail string field** for categories outside the enum — closed enums force misclassification.
- **Strict schemas via tool_use eliminate syntax errors only** — never *semantic* errors (line items that don’t sum to the stated total, correct values in wrong fields). Semantic checks are validation-layer work (G2).
- Include **format-normalization rules in the prompt** alongside strict output schemas when source formatting varies.

G2. Validation, retry, and feedback loops (task 4.4)

- **Retry-with-error-feedback:** on validation failure, send a follow-up containing **the original document, the failed extraction, and the specific validation errors** — the model self-corrects against named errors far better than against a bare “try again.”
- **Limits of retry (tested):** retries fix **format/structural** errors; they are **ineffective when the information is absent from the source** (or lives in a document you didn’t provide). Diagnose *which* failure class you have before burning retries.
- **Self-correction flows:** extract `calculated_total` alongside `stated_total` to flag discrepancies; add `conflict_detected` booleans for inconsistent source data.
- **detected_pattern fields:** tag each finding with the code construct/pattern that triggered it, so dismissed findings can be analyzed systematically for false-positive patterns (feeds the 4.1 precision loop).

G3. Multi-instance & multi-pass review (task 4.6)

- **Self-review limitation:** a model that generated code **retains its reasoning context** and is unlikely to question its own decisions in the same session. Neither “review your work carefully” instructions nor extended thinking substitute for independence.
- **Independent review instance:** a second Claude instance **without the generator’s reasoning context** catches subtle issues self-review misses (pairs with the CI session-isolation point in file 05’s supplement).
- **Multi-pass review:** split large reviews into **per-file local passes + a cross-file integration pass** — attention dilution, not context size, causes inconsistent single-pass reviews (see 08 §6).
- **Calibrated routing:** have the reviewer self-report **confidence alongside each finding** to enable review routing — then calibrate those scores before trusting them (see 09 §5).

G4. Gap-card Q&A

GQ1. The model fills in an invoice field the document doesn’t contain. Schema fix? **A.** Make the field optional/nullable — required fields force fabrication.

GQ2. What errors do strict tool-use schemas eliminate, and what survives? **A.** JSON syntax errors are eliminated; semantic errors (wrong sums, misplaced values) survive and need validation logic.

GQ3. Describe retry-with-error-feedback. **A.** Follow-up request = original document + failed extraction + the specific validation errors, guiding targeted self-correction.

Q4. When will retries never fix an extraction? **A:** When the required information is absent from the provided source — that’s a data problem, not a format problem.

Q5. Why is an independent instance better than “review your own code thoroughly”? **A:** The generator retains its reasoning context and won’t question its own decisions; independence removes that bias.

Q6. Purpose of a `detected_pattern` field in review findings? **A:** Enables systematic analysis of which constructs trigger dismissed findings — the raw material for cutting false-positive categories.

Q20. What is a routing workflow? **A:** Classify the request, then send it to a specialized prompt/model/tool path.

Q21. What should you log for an agent turn? **A:** Message ids, `stop_reason`, tool names/ids/inputs (redacted), results status, token usage, latency.

Q22. Cache prefix order to remember? **A:** Tools, then system, then messages—up to the cache breakpoint.

Q23. What is `pause_turn` signaling? **A:** A long-running server-side turn paused; continue according to docs rather than treating it as a hard failure.

Q24. Why are tool descriptions “prompt engineering”? **A:** The model uses them to decide *when* and *how* to call tools—vague descriptions cause wrong calls.

Q25. What’s wrong with returning a huge SQL dump as `tool_result`? **A:** Context bloat and distraction; summarize or page results, keep raw data server-side.

Q26. How do refusals differ from empty answers? **A:** `stop_reason: "refusal"` indicates policy refusal—handle with safe UX, don’t infinite-retry the same content.

Q27. Name one Computer Use safety control. **A:** Sandbox / least privilege / human approval for sensitive UI actions.

Q28. What’s the difference between chaining and parallelization? **A:** Chaining is sequential dependencies; parallelization fans out independent work then merges.

Q29. Why separate retrieval metrics from answer metrics? **A:** Bad answers may be good writing over bad chunks—or good chunks with a weak prompt; fixes differ.

Q30. What’s the first debugging step when tool use “hangs”? **A:** Check whether every `tool_use` id received a `tool_result` and inspect `stop_reason` / iteration caps.

Quick review checklist

- ☐ I can sketch a Messages request with `model`, `max_tokens`, `system`, `messages`.
- ☐ I never put `system` as a message role.
- ☐ I can name ≥ 4 stop reasons and the next action for each.
- ☐ I can assemble SSE text deltas and tool `partial_json` safely.
- ☐ I can complete a tool loop including parallel tools and error results.
- ☐ I can explain structured outputs vs. “please return JSON”.
- ☐ I can design a tiny eval set and grader.
- ☐ I can describe hybrid RAG + why to measure retrieval separately.
- ☐ I know what to put in a prompt-cache prefix (and what not to).

- ☐ I know when to use Batches vs. sync Messages.
- ☐ I can compare workflow vs. agent and give parallel/chain/route examples.
- ☐ I can map MCP host/client/server at a high level.
- ☐ I treat API keys, tool authz, and computer-use sandboxing as exam-critical safety topics.

Glossary

Term	Plain meaning
Messages API	Chat-style API with user/assistant turns and top-level system
Content block	Typed piece of message content (text, image, tool_use, tool_result, ...)
SSE	Server-Sent Events stream used for incremental responses
stop_reason	Why generation stopped
Tool use	Model requests a function call via tool_use blocks
Server tool	Tool executed by the platform rather than your client
Structured outputs	Schema-constrained model output for parsers
Prompt caching	Reusing a stable prompt prefix to save tokens/latency
Message Batches	Async bulk Messages processing API
RAG	Retrieve documents, then generate grounded answers
BM25	Classic lexical ranking function
Reranker	Second-stage model/function that reorders retrieved hits
Agentic search	Multi-step retrieve/refine loop driven by the model
Workflow	Developer-defined graph of LLM/tool steps
Agent	Model-directed loop over tools until a stop condition
MCP	Open protocol for reusable tools/resources/prompts across hosts
Computer Use	Vision + UI action tools for controlling a desktop/browser
ITPM / OTPM	Input/output tokens-per-minute style rate limits
custom_id	Caller-assigned id joining a batch request to its result

Quick lab checklist (practice without copying course labs)

Use this as a private drill list while studying official docs/SDKs:

1. **Hello Messages:** one-shot user message → print assistant text; then add a second user turn with prior assistant message preserved.
2. **System ablations:** same user question with three different system prompts; note tone and constraint adherence.
3. **Stream assemble:** concatenate text deltas; assert equality with non-streamed output for the same seed/settings when possible.
4. **Tool round-trip:** fake weather tool; verify ids match on tool_result; force a parallel double call if the model emits two tools.
5. **Structured parse:** schema for { "sentiment": enum, "confidence": number }; fail the build if parse throws.
6. **Mini-RAG:** three markdown files → chunk → embed or BM25 → answer with “sources:” lines.
7. **Cache sanity:** put a long static system prompt behind cache controls; compare latency on request 1 vs. request 2.

8. **Agent budget:** max 5 tool iterations; on loop, return “budget exceeded” instead of hanging.
9. **Stop-reason drill:** intentionally set tiny max_tokens and confirm you detect truncation.
10. **Batch dry-run design:** sketch 100 classification rows with custom_ids and a results joiner (even without calling the API).
11. **429 simulator:** write a retry helper with exponential backoff + jitter and a max-retry cap.
12. **Vision extract:** one screenshot/receipt image → structured JSON fields; verify you didn’t hard-code fake OCR.

These drills reinforce exam mechanics; they are not substitutes for Anthropic’s graded quizzes.

Source URLs (verify before exam day)

- Course outline: <https://academy.claude.com/courses/building-with-the-claude-api>
- Anthropic docs hub: <https://docs.anthropic.com> (also platform.claude.com/docs)
- Topics to re-open live: Messages, streaming, stop reasons, tool use, structured outputs, prompt caching, batch processing, vision/files, rate limits/errors, MCP overview

End of notes. Cross-check parameters against current Anthropic platform docs before an exam sitting.

Introduction to Model Context Protocol — Exam Prep Study Notes — Simplified Technical English

Note: This edition follows the ASD-STE100 writing rules: simple present tense, active voice, one word = one meaning, sentences of 20–25 words or less. Technical names MCP, Claude, JSON-RPC, STDIO, SSE stay as written. These notes are **original exam-prep notes**. They are **not** official Anthropic course material. They are **not** verbatim lesson text. The official course is the source of truth. Spec versions evolve. Topics follow the public outline of [Introduction to Model Context Protocol](#).

Advanced follow-on (pointer only): Take [Model Context Protocol: Advanced Topics](#) after this intro course. It covers sampling, progress/logging, and roots. It covers JSON-RPC message details and STDIO handshake depth. It covers Streamable HTTP / SSE and stateful vs. stateless scaling.

Check live docs: modelcontextprotocol.io (architecture, primitives, transports, lifecycle). Spec versions evolve. Prefer concepts. Do not memorize one date stamp.

Overview

MCP (Model Context Protocol) is an open standard. It connects AI hosts (Claude Desktop, IDEs, custom agents) to external tools and data. You do **not** write a unique JSON tool schema by hand for every app and service pair.

The intro course public learning goals emphasize these topics. Architecture: you shift tool definition and execution to specialized servers. Messaging does not depend on one transport. You learn the end-to-end request flow. You build servers with the Python SDK (decorators vs. raw JSON schemas). You use document-style tools. You use the MCP Inspector. You use **resources** (direct + templated).

You read resources on the client side and handle MIME types. You use **prompts** as reusable workflows. You select among tools / resources / prompts. You use integration patterns such as autocomplete and context injection.

Public module shape: Introduction → Hands-on MCP servers → Connecting with MCP clients.

Prereqs the public course lists: basic Python, async/await, and API literacy.

How these notes relate to the API course: Building with the Claude API teaches the Messages/tool loop inside *one* app. MCP teaches how many apps share the same external capabilities through one common protocol. On exams, if you see “reuse across hosts” or “Inspector,” think MCP.

Key concepts (cheat sheet)

Concept	One-liner
Host	The AI application the user sees (it orchestrates clients, UI, and model calls)
Client	Connector inside the host. 1:1 with a single MCP server
Server	Process or service that exposes tools, resources, and/or prompts
Tools	Model-controlled actions (side effects are allowed. Like function calling)
Resources	App-controlled, mostly read-only data through URIs
Prompts	User-controlled, pre-crafted instruction templates
Transport	How JSON-RPC messages move (commonly STDIO locally. Streamable HTTP / SSE remotely)
Data layer	JSON-RPC methods, lifecycle, primitives, notifications
Inspector	Browser-based helper that lists and calls tools and helps you debug a server
Capability negotiation	Initialize handshake declares what each side supports

Working model: The host asks the model what to do. The model may select a tool. The client calls the MCP server. The server talks to real systems. Results flow back into the model context.

Deep notes by topic

1. Why MCP exists

Without MCP, every AI app copies integration code for GitHub, Slack, databases, filesystems, and similar systems. With MCP:

- **N servers** (one per system or capability domain) + **M hosts/clients** is better than **M×N** custom adapters.
- Servers own auth to the backend, input validation, and domain logic.
- Hosts keep their focus on UX, policy, and model orchestration.

Exam phrase to remember: MCP **shifts** tool definition and execution from your one large app server toward **specialized MCP servers**.

Analogy: USB standard vs. proprietary cables. Hosts speak MCP. Servers speak MCP. Backends stay behind the server adapter.

What MCP is not

- Not a replacement for the Claude Messages API (the host still calls the model).
- Not a database or an embedding store by itself.
- Not a guarantee of safety. Servers with a wide scope are still dangerous.
- Not “only for Anthropic”. It is an open protocol. Many hosts can implement it.

2. Architecture: hosts, clients, servers

User ↔ Host (LLM app) ↔ Client₁ ↔ Server₁ (e.g. docs)
↔ Client₂ ↔ Server₂ (e.g. calendar)

- **Host responsibilities:** Start and manage clients. Present prompts/resources to users. Decide what context reaches the model. Enforce user consent and security policy. Run or call the LLM.
- **Client responsibilities:** Run capability negotiation and session lifecycle. Translate host needs into MCP requests. Isolate one server from another.
- **Server responsibilities:** Advertise and implement primitives. Keep a narrow focus. A single domain is easier to secure and test.

1:1 rule: each client talks to exactly one server. A host that uses three servers runs three clients. This isolation is a frequent exam topic.

Local vs. remote servers

	Local (often STDIO)	Remote (often Streamable HTTP)
Process	Host starts a subprocess	Independent service
Typical fan-out	One client per process	Many clients to one service
Auth story	OS user privileges	HTTP auth / OAuth / tokens
Failure mode	A crash stops that server	Network partitions, version skew

Security boundary reminder: a compromised or overly powerful server is dangerous. Use least privilege. Set clear roots/paths. Get user approval for sensitive tools. These rules still apply in “local” STDIO setups.

3. Layers: data vs. transport

Official architecture splits MCP into:

1. **Data layer** — JSON-RPC 2.0 message semantics: lifecycle, tools/resources/prompts, notifications, and client features like sampling/elicitation (advanced).
2. **Transport layer** — how bytes move: STDIO vs. Streamable HTTP (with optional SSE), framing, and transport auth.

Intro exam level: you can explain that the **same** JSON-RPC methods travel on different transports. The Advanced course goes deeper on production HTTP scaling.

4. Message flow (request → Claude → tools → back)

A typical turn:

1. The user asks a question in the host.
2. The host assembles context (chat history, selected resources, chosen prompt template).
3. The host calls Claude with available tool definitions that it discovered from MCP servers.
4. Claude returns a tool call (name + arguments).
5. The host/client sends an MCP tools/call to the correct server.
6. The server executes (read file, edit doc, query API) and returns structured results.
7. The host adds tool results into the conversation and calls Claude again.
8. Claude produces a user-visible answer (or more tool calls).

Know the difference between **request/result** pairs (RPC calls) and **notifications** (one-way events). A deeper message taxonomy appears in the Advanced Topics course.

Original sequence diagram (text)

```
User → Host: "Summarize Q3 plan doc"
Host → Client(docs): resources/read docs://q3-plan
Client → Server: resources/read
Server → Client: text/markdown body
Host → Claude: messages + tool defs + injected resource
Claude → Host: tool_use edit_section(...)
Host → Client: tools/call edit_section
Server → Client: ok
Host → Claude: tool_result
Claude → Host: final summary for user
```

5. Lifecycle (intro depth)

MCP sessions must not call methods in a random order. Conceptual lifecycle:

1. **Connect** transport (start a STDIO process or open an HTTP session).
2. **Initialize** — exchange protocol version, `clientInfo` / `serverInfo`, and capabilities.
3. **Initialized notification** — the client signals it is ready.
4. **Operate** — `tools/list`, `tools/call`, `resources/list`, `resources/read`, `prompts/list`, `prompts/get`, and similar methods.
5. **Stop** — close streams / stop the subprocess. Do not leave child processes running.

Capability negotiation intuition

- The server may declare support for tools, resources, and prompts. It may also declare whether lists can change (`listChanged`).

- The client may declare support for features that servers might request later (sampling, elicitation—advanced).
- A call to an unsupported method is incorrect protocol use. Discover first.

Exam rule: if the question asks “what happens before tools/list?”, answer **initialize / negotiate**. Do not answer “call the tool immediately.”

6. Building MCP servers (Python SDK mindset)

Intro course emphasis: use the SDK so you **do not** write large JSON schemas by hand.

- Decorators / type hints / Field descriptions become tool metadata that the model sees.
- Good descriptions improve tool selection greatly. Treat them as prompt engineering.
- Example domain from the public outline: **document management** — tools to read and edit documents.
- Keep tools narrow: `read_document(id)` and `update_section(id, section, text)` are better than a vague `do_anything(payload)`.

Original decorator-style sketch (illustrative, not an SDK dump)

```
# Conceptual only – verify current SDK APIs before coding
@server.tool()
def read_document(doc_id: str) -> str:
    """Return the full text of a document by id. Use when the user asks what's inside a doc.
    ...

@server.tool()
def update_section(doc_id: str, section: str, text: str) -> str:
    """Replace one named section. Prefer this over rewriting the entire document."""
    ...
```

Description writing checklist

- What the tool does, in one sentence.
- When to use it / when not to use it.
- Units, id formats, side effects.
- Failure modes the model should expect.

Inspector: run the server and open the MCP Inspector in a browser. List tools, run calls, and inspect payloads before you connect a full host. This is a strong exam answer for “how do you debug a server?”

7. Resources: direct vs. templated

Resources expose read-oriented data. The *application* (not the model alone) decides to fetch the data and inject it.

- **Direct resources:** static URI, e.g. `docs://index` or `config://app-settings`.
- **Templated resources:** URI templates with parameters, e.g. `docs://{doc_id}` — the client fills params.

Client reading checklist:

- Request the resource by URI.

- Handle **MIME types** (e.g. `application/json` vs. `text/plain`) so you parse correctly.
- Inject content into the model context with a clear plan (whole doc vs. excerpt).

When resource vs. tool? If the work is mainly “fetch data for context” with no meaningful side effect → resource. If it changes state or performs an action → tool.

URI design tips

- Use a clear scheme (`docs://`, `db://`, `config://`) per server domain.
- Keep ids URL-safe. Document template parameters.
- Prefer immutable snapshots for audit when you inject into prompts.

MIME handling examples

MIME	Client behavior
<code>text/plain</code>	Inject as text. Maybe truncate
<code>text/markdown</code>	Same, keep headings if possible
<code>application/json</code>	Parse. Maybe pretty-print selected fields
<code>image/png</code>	The host may route to vision-capable message blocks

Common exam error: you ignore MIME and always run `json.loads`. Binary or plain text then fails.

8. Prompts as MCP primitives

Prompts are reusable, high-quality instruction templates for common workflows. The course publicly mentions examples like document formatting.

- **User-controlled:** you typically select them from a menu (“Format selection as RFC-style memo”). The model does not invent them without user action.
- You can parameterize slots (title, tone, audience).
- They help teams share best-practice instructions across hosts.

Original prompt template sketch

Name: `format_as_adr`

Args: `title`, `status`, `context`

Body:

Write an Architecture Decision Record titled `{title}`.

Status: `{status}`.

Context: `{context}`.

Use sections: Context, Decision, Consequences.

Host UX: the user selects a prompt. The user fills args. The host expands the prompt. The host sends it to the model. The host may attach related resources.

9. Choosing tools vs. resources vs. prompts

Primitive	Who steers?	Typical use	Side effects?
Tool	Model	Search, edit, send, compute	Often yes

Primitive	Who steers?	Typical use	Side effects?
Resource	Application / host logic	Load file, config, dataset into context	Generally no
Prompt	User	Start a known workflow with vetted instructions	No (instructions only)

Common exam error: you call everything a “tool.” Graders look for the control-plane distinction above.

Sorting drill (memorize)

- “Load README into context” → **resource**
- “Create GitHub issue” → **tool**
- “Apply ADR template” → **prompt**
- “Search docs then edit” → **tools** (+ maybe resources for known URIs)
- “Show company style guide” → **resource** or **prompt**, depending on raw data vs. a workflow template

10. Transports (intro-level)

MCP is **transport-agnostic** at the message layer (JSON-RPC style messages). Common options:

- **STDIO:** the client starts the server as a subprocess. They communicate over stdin/stdout. This is ideal for local desktop/IDE integrations. You need a correct **initialization handshake** before ordinary calls (the Advanced Topics course stresses the details more).
- **HTTP + SSE / Streamable HTTP:** for remote/networked servers. Servers can push events. Think about sessions, auth, and scaling tradeoffs. Modern docs emphasize **Streamable HTTP** (HTTP POST client→server, optional SSE for streaming). They often recommend OAuth for tokens.

Intro exam level: name the transports. Know when you select local STDIO vs. remote HTTP. Leave dual-connection SSE details and stateless horizontal scaling to the advanced course.

Transport choice memo (original)

Use STDIO when the server must touch the local filesystem or a developer laptop secret store. Use STDIO when the host can manage a child process. Use Streamable HTTP when multiple hosts across a company share one managed integration. That path needs real auth, audit logs, and central updates.

11. Practical integration patterns

- **Context injection:** the host reads resources → attaches relevant text to the model request.
- **Autocomplete / discovery:** list tools, resources, and prompts so UIs can suggest actions.
- **Lifecycle:** connect → initialize/negotiate → operate → stop cleanly (do not leave subprocesses running).
- **Composition:** one host, many servers (docs + issue tracker + DB), each behind its own client.
- **Notifications:** if a server declares list-changed capability, it may notify clients to refresh tools/list (and similar). Hosts should update the model’s available tools.
- **Error surfacing:** tool failures should become structured errors the model can see. They should not become silent host crashes.

12. Client features (pointer — advanced)

Intro awareness only (depth is in Advanced Topics):

- **Sampling:** the server asks the host/client to run an LLM completion (model-agnostic servers).
- **Elicitation:** the server asks the host to collect user input / confirmation.
- **Logging / progress:** the server emits logs or progress for long work.

Do not claim extra depth on these on an intro quiz. Know they exist. Know which side exposes them.

13. Security considerations (intro must-pass)

1. **User consent:** hosts should approve sensitive tool calls, especially on first use.
2. **Least privilege:** filesystem servers get a directory root. They do not get the entire \$HOME disk when that is possible.
3. **Secret handling:** servers should use OS keychains / env / secret managers. Do not hardcode tokens in tool responses.
4. **Confused deputy:** a malicious document could try to coerce the model into calling delete_repo. You still need policy gates.
5. **Supply chain:** treat third-party MCP servers like software you install. Review permissions.
6. **Transport auth:** remote servers need real authentication. STDIO inherits the user’s OS privileges. It is still not “safe by default.”
7. **Audit:** log tool name, args (redacted), actor, and timestamp.

14. End-to-end document-management mental lab

Match the course public project theme. Do not copy lab prose:

1. The server exposes list_docs, read_document, and update_section tools.
2. The server exposes a docs://{doc_id} templated resource and a docs://index direct resource.
3. The server exposes a format_document prompt.
4. Use Inspector to call read_document before you wire Claude.
5. The host lists prompts for a “Format” menu. On select, it expands the prompt and optionally injects a resource.
6. The model may still call update_section as a tool when the user asks to edit.

This single domain exercises **all three primitives**.

15. MCP vs. plain Messages tool defs

Concern	Plain tool JSON in Messages	MCP
Discovery	Hardcoded in the app	*/list from servers
Reuse	Copy schemas	Shared servers across hosts
Resources / prompts	You build your own	First-class primitives
Ops	In-process functions	Local processes or remote services
Best when	One app, few tools	Ecosystem / many hosts

Bridge insight: hosts often **translate** MCP tool schemas into the model provider’s tool format internally. MCP does not remove the Claude tool loop. It standardizes the outer integration.

16. JSON-RPC mental model (lightweight)

Enough for intro exams:

- **Request:** has id, method, params → expects **result** or **error** with the same id.
- **Notification:** has method (and maybe params) but **no id** → no response.
- Methods look like initialize, tools/list, tools/call, resources/read, prompts/get.
- Unknown methods / bad params → structured JSON-RPC errors. They are not HTTP HTML pages. Even on HTTP transports, the payload is still RPC-shaped.

17. Common failure modes & debugging

Symptom	Likely cause	Fix direction
Host shows no tools	Init failed / capabilities missing / list not called	Check initialize + tools/list in Inspector
Tool call looks like 404 / unknown tool	Stale list after a server change	Handle list_changed. Refresh
MIME parse error	Assumed JSON for a text resource	Branch on MIME
Hung STDIO server	Handshake incomplete / logs written to stdout	Keep protocol on stdout. Log to stderr
Remote 401	Missing bearer/OAuth	Fix transport auth
Model never calls your tool	Weak description / too many overlapping tools	Rewrite descriptions. Reduce the set
Child processes left running	Host did not close the lifecycle	Stop clients on exit

18. Host implementation checklist (what “good” looks like)

When you build or review a host that speaks MCP, graders look for more than one successful connect. Production reviewers look for the same.

1. **Config surface:** the user can add/remove servers (command + args for STDIO. URL + auth for HTTP).
2. **Consent UX:** first-time tool use prompts. Distinguish read-only vs. mutating tools.
3. **Unified registry:** merge tools from all clients with namespaced collision handling (docs.read vs. db.read).
4. **Model translation:** map MCP tool schemas into the provider tool format your LLM API expects.
5. **Result hygiene:** truncate huge tool outputs. Keep enough detail for the model to continue.
6. **Cancellation:** the user can abort long tool calls. The client cancels in-flight RPC when supported.
7. **Observability:** per-server latency, error rates, and which prompts/resources were injected.
8. **Graceful degradation:** if one server dies, other servers keep working.

19. Server design patterns (document domain + beyond)

Pattern A — Thin adapter MCP server wraps an existing REST API. Tools ≈ endpoints. Resources ≈ GETs that are safe to inject.

Pattern B — Domain facade Fewer, smarter tools (`search_and_summarize_locally`) that hide backend complexity. Easier for models. Harder to keep descriptions honest.

Pattern C — Read/write split Expose reads as resources. Expose writes as tools that need confirmation. This matches the control-plane triad cleanly.

Pattern D — Prompt pack The server mainly ships prompts + a few resources (style guides). This is strong for teams that standardize workflows without write power for the model.

Anti-pattern — do-everything server One server that runs arbitrary shell commands, reads all files, and sends email. This fails least-privilege. Host permission UIs then show high risk.

20. Mapping Claude Academy modules → study actions

Public module	What to practice in this session
Introduction	Draw architecture. Recite the triad
Hands-on servers	Build 2–3 tools. Open Inspector
Connecting with clients	Read a resource + MIME branch. List prompts. Inject context

Link back to Building with the Claude API. After MCP returns tool output, you still perform the `Messages tool_result` continuation. MCP did not erase that loop.

21. Worked story: autocomplete + context injection

Autocomplete: The host calls `prompts/list` and `tools/list` on idle. The UI shows “Format as ADR”, “Search docs”, “Create issue”. The user selects “Format as ADR” → the host calls `prompts/get` with args → expands text into the composer.

Context injection: The user @-mentions a doc. The host resolves the `docs://{id}` resource and checks MIME. The host injects a truncated body into the next user message inside tagged `<doc>` delimiters. It *separately* keeps write tools available if the user asks to edit.

Exam insight: autocomplete is **discovery UX**. Injection is **context assembly**. Both are host responsibilities, even though data comes from servers.

22. Versioning & compatibility mindset

- Protocol versions appear during initialize. Incompatible versions should fail immediately.
- Servers and hosts upgrade independently. Capability flags exist so optional features do not break required behavior.
- For exams: prefer the answer “negotiate capabilities and degrade gracefully.” Do not invent a single forever-version number.

Comparison tables (exam quick hits)

Control plane triad

Question	Tools	Resources	Prompts
Who decides to use it?	Model	App/host	User

Question	Tools	Resources	Prompts
Primary verb	Call / execute	Read / inject	Select / expand
Typical risk	Side effects	Data exfil via context	Instruction exfil / social engineering
Discovery method family	tools/*	resources/*	prompts/*

STDIO vs Streamable HTTP

Dimension	STDIO	Streamable HTTP
Best for	Desktop, IDE, local files	Team/shared integrations
Auth	OS user	Tokens / OAuth / headers
Scaling	Per-user processes	Central service
Ops complexity	Process mgmt	Network + auth + versioning

Host vs Client vs Server (blame chart)

Bug	First place to look
Wrong tool chosen	Tool descriptions + model prompt (host)
Tool not listed	Client discovery / server capabilities
Wrong file edited	Server implementation / authz roots
UI missing prompt menu	Host did not list/present prompts
MIME crash	Client resource reader

Exam tips

1. Draw the **Host** → **Client** → **Server** triangle without notes. Mark 1:1 client-server.
2. Memorize the triad: **tools = model-controlled**, **resources = app-controlled**, **prompts = user-controlled**.
3. SDK decorators exist so you avoid brittle hand-written schemas. Remember this fact.
4. Inspector = first debugging stop for servers.
5. MIME type handling on resource reads is an easy trap question.
6. Know that an **Advanced Topics** course exists for sampling, roots, progress, and production transports. Do not claim intro-course depth on those topics.
7. Security: local STDIO is not “no security”. It still runs with user privileges.
8. Initialize before list/call.
9. Notifications ≠ requests (no response id).
10. MCP complements Messages tool use. It does not delete the tool_result loop.

Exam traps

Trap	Fix
"Client can talk to many servers"	No—the host has many clients. Each client is 1:1
Everything is a tool	Use resource/prompt when the control plane differs
Skip Inspector, debug only in the prod host	Inspector first
Ignore MIME	Always branch
Assume STDIO is sandboxed	It inherits user privileges
Memorize one HTTP dialect forever	Concepts: Streamable HTTP / SSE exist.
Put secrets in resource bodies without care	Check the current spec
Forget shutdown	Redact. Use secure stores
	Stop child processes

Self-check Q&A

Q1. What are the three MCP roles? **A:** Host (LLM app), Client (1:1 connector), Server (exposes primitives).

Q2. Who typically decides to invoke a tool? **A:** The model (with host policy gates). Tools are model-controlled actions.

Q3. Who typically selects a prompt template? **A:** The user (user-controlled primitive), often through UI.

Q4. Direct vs. templated resource? **A:** Direct = fixed URI. Templated = URI pattern with parameters that the client fills.

Q5. Why use the Python SDK decorators? **A:** Derive tool schemas from types/Field descriptions. Do not maintain raw JSON schemas by hand.

Q6. What is the MCP Inspector for? **A:** Browser-based testing/debugging of server tools/resources before full host integration.

Q7. Name two transports and a use case for each. **A:** STDIO for local subprocess servers. Streamable HTTP (or HTTP/SSE family) for remote servers.

Q8. Where do you go next after this intro? **A:** [MCP Advanced Topics](#) for sampling, progress/roots, and production transport/scaling details.

Q9. What problem does MCP reduce mathematically? **A:** $M \times N$ custom integrations toward M hosts + N specialized servers.

Q10. What are the two MCP layers? **A:** Data layer (JSON-RPC semantics/primitives) and transport layer (STDIO / HTTP).

Q11. What must happen before ordinary tools/list calls? **A:** Transport connect + initialize capability negotiation (+ initialized signal).

Q12. Who owns backend API keys for a GitHub MCP server? **A:** Typically the server (or its secret store). Not every host hardcoding tokens.

- Q13. How should a client react to tools/list_changed?** A: Refresh tool lists and update what the model can call.
- Q14. Resource or tool: “fetch schema for context”?** A: Resource (read-oriented, app-controlled injection).
- Q15. Resource or tool: “insert a row”?** A: Tool (side effect / action).
- Q16. Why keep stdout clean on STDIO servers?** A: Protocol messages travel on stdio. Logging noise can break framing. Use stderr for logs.
- Q17. What is context injection?** A: The host reads resources (or other data) and attaches them into the model request with a clear plan.
- Q18. What is autocomplete/discovery in MCP UIs?** A: Listing tools/resources/prompts so users/hosts can pick capabilities without hardcoding.
- Q19. Does MCP replace Claude’s tool_result loop?** A: No. Hosts still feed tool results back into the model conversation.
- Q20. Name a client-exposed advanced primitive.** A: Sampling, elicitation, or logging (intro: awareness only).
- Q21. What is a good tool granularity rule?** A: Narrow, composable tools with clear schemas are better than one oversized tool.
- Q22. Why are Field descriptions important?** A: They become model-facing guidance for arguments. This is prompt engineering for tools.
- Q23. What is the security mistake in “local = safe”?** A: STDIO servers run with the user’s privileges. They can still exfiltrate or delete data.
- Q24. How do prompts help teams?** A: Share vetted instruction templates across hosts. Each user does not reinvent the wording.
- Q25. What MIME-related bug is common?** A: Parsing a text/plain resource as JSON (or ignoring content type entirely).
- Q26. Without notes: how many clients for 4 servers?** A: Four clients (1:1), managed by one host.
- Q27. What does transport-agnostic mean here?** A: The same data-layer methods can travel on different transports.
- Q28. What is a notification missing that requests have?** A: An id / expected response.
- Q29. When is plain inline tooling better than MCP?** A: Single-app prototypes with a few stable functions and no reuse need.
- Q30. What is the first tool you use when a new server does not work?** A: MCP Inspector (list/call/inspect payloads) before full host integration.
-

Quick review checklist

- ☐ I can draw Host/Client/Server with 1:1 links.
- ☐ I can explain $M \times N \rightarrow M+N$ motivation.
- ☐ I can define tools vs resources vs prompts by *who controls* them.

- ☐ I can outline initialize → list → call/read → shutdown.
- ☐ I know STDIO vs Streamable HTTP selection criteria.
- ☐ I can describe Inspector’s role.
- ☐ I handle MIME types on resource reads.
- ☐ I can sort example tasks into the triad correctly.
- ☐ I know MCP sits beside—not instead of—the model tool loop.
- ☐ I can list at least five security considerations.
- ☐ I know Advanced Topics exists for sampling/roots/progress/scaling.
- ☐ I can sketch a doc-management server using all three primitives.

Glossary

Term	Plain meaning
MCP	Open protocol connecting AI hosts to external tools/data
Host	User-facing AI application that coordinates clients + model
Client	In-host 1:1 session object talking to one server
Server	Process/service that exposes MCP primitives
Tool	Executable action, usually model-requested
Resource	URI-addressable data for app-controlled context
Prompt (MCP)	User-selectable instruction template
STDIO transport	Subprocess stdin/stdout messaging
Streamable HTTP	Remote HTTP transport with optional SSE
JSON-RPC 2.0	Request/response/notification message format
Initialize	Lifecycle handshake + capability negotiation
Capability	Declared feature support (tools, resources, ...)
Inspector	Dev UI to exercise a server
MIME type	Content type that guides how you parse resource bodies
Notification	One-way RPC message without response
Sampling	Server-requested LLM call via client (advanced)
Elicitation	Server-requested user input via host (advanced)
Roots	Boundary paths/scopes for server access (advanced)
listChanged	Capability that allows list update notifications
Context injection	Host inserts resource content into model context

Study drills (original practice ideas)

- Sketch without notes:** Host/Client/Server with two servers attached. Label which hop is 1:1.
- Primitive sorting:** Given a list like “load README”, “create issue”, “Apply ADR template”, tag each as resource, tool, or prompt.
- Fake doc server:** Two tools (list_docs, get_doc) + one templated resource. Exercise them in an Inspector-like mindset (even if you only print JSON).
- MIME handling:** Return JSON and plain text resources. Write client code that branches on MIME type.
- Failure path:** A tool throws. Ensure the host surfaces an error `tool_result` the model can recover from. Do not crash the session.

6. **Transport choice memo:** One paragraph that argues STDIO for a desktop filesystem server vs. HTTP for a shared company knowledge server.
7. **Lifecycle flashcards:** Order connect / initialize / initialized / list / call / shutdown.
8. **Security review:** For a proposed shell_exec tool, write three policy gates you would require in the host.
9. **Description rewrite:** Take a vague tool blurb (“manages docs”). Rewrite it with when-to-use / when-not / side effects.
10. **Bridge drill:** Explain in four sentences how an MCP tools/call becomes a Claude tool_use / tool_result round-trip inside a host.

Keep drills short. Depth on sampling, roots, and Streamable HTTP belongs in the Advanced Topics course linked above.

Source URLs

- Intro course: <https://academy.claude.com/courses/introduction-to-model-context-protocol>
- Advanced course (pointer): <https://academy.claude.com/courses/model-context-protocol-advanced-topics>
- MCP docs / architecture: <https://modelcontextprotocol.io>
- Spec & SDK indexes: linked from modelcontextprotocol.io (versions change—open live)

End of course notes. Prefer the official MCP specification and Anthropic Academy labs. Hands-on syntax may change between SDK versions.

CCAR-F Domain 2 mechanics supplement (added 2026-08-23)

Scope: These are the concrete mechanics that **CCAR-F exam guide Domain 2 (Tool Design & MCP Integration, 18%)** tests beyond this course conceptual intro. The tasks are 2.2 (structured errors), 2.4 (MCP in Claude Code / agent workflows), and 2.5 (built-in tools). §6 and 08 §Q&A cover tool-description craft (2.1). 03-building-with-claude-api.md covers tool_choice and tool distribution (2.3). Cross-check both. The author verified these mechanics against current Claude Code docs. Original synthesis.

S1. Structured error responses for MCP tools (task 2.2)

The isError flag is MCP’s mechanism for tool failure. It sends the failure back to the agent *inside a successful protocol response*. The tools/call result carries isError: true. Error details are in content. The **model** can then read the failure and reason about recovery. A protocol-level JSON-RPC error is for malformed calls. A *tool-domain* failure travels as a normal result with isError: true.

Why generic errors fail: a uniform "Operation failed" gives the agent no basis to choose. The agent cannot choose among retry, alternative approach, escalation, or asking the user. Wasted retries on permanent errors come from unstructured errors. Abandoned recoverable errors come from the same cause.

Error taxonomy the exam uses (memorize the four):

Category	Example	Retryable?	Agent's correct move
Transient	Timeout, service unavailable	Yes	Retry (with backoff)
Validation	Malformed input, bad ID format	No (as-is)	Fix the input, then retry
Business	Policy violation (refund cap)	No	Explain to the user / alternative workflow
Permission	Caller lacks access	No	Escalate / different credential path

Structured error metadata pattern: return `errorCategory` (transient / validation / permission / business). Return an **`isRetryable` boolean**. Return a **human-readable description**. For business-rule violations, set `retryable` **false**. Include a **customer-friendly explanation** the agent can relay (“refunds above \$500 require a supervisor”). The agent then communicates. It does not send many retries.

Multi-agent link (Domain 5.3): subagents recover **transient** failures locally. Only unresolvable errors propagate to the coordinator. They carry what was attempted and partial results. Also keep **access failure ≠ valid empty result**: an empty result set is a *successful* query with zero matches. It is never `isError: true`.

S2. MCP servers in Claude Code: scoping and configuration (task 2.4)

Two scopes, one exam distinction:

File	Scope	Shared?	Use for
<code>.mcp.json</code> (project root)	Project	Yes — committed to version control. Every teammate gets it on clone/pull	Shared team tooling (the team's Jira/GitHub/db servers)
<code>~/.claude.json</code>	User	No — personal machine only	Personal/experimental servers you try out

Environment-variable expansion in `.mcp.json` — e.g. `${GITHUB_TOKEN}` in an `env/header` value. Teams can commit server *configuration*. Each member supplies their own **credentials from their environment**. The exam framing: how do you share the config **without committing secrets**? Use `env-var` expansion in project-scoped `.mcp.json`.

Discovery model: tools from **all configured MCP servers** Claude Code discovers tools from **all configured MCP servers at connection time**. The agent can use them **simultaneously** to the agent. A project server and a personal server coexist in one session (official prep Exercise 2 has you verify exactly this).

MCP resources as content catalogs: expose issue summaries, documentation hierarchies, or database schemas as **resources**. Agents then get *visibility into what data exists* without extra turns on exploratory tool calls. Catalog = resource. Action = tool (consistent with the triad in §7–9 above).

Two more 2.4 skills:

- **Description competition with built-ins:** if your MCP tool description is weak, the agent may prefer a built-in (like Grep). It may ignore your more capable server tool. Fix this: **enrich the MCP tool description** with its capabilities and outputs. Description quality drives adoption. It is not only selection between similar tools.
- **Community vs custom servers:** prefer existing community MCP servers for standard integrations (e.g. Jira). Reserve custom server builds for team-specific workflows. (The exam does not cover how you deploy or host MCP server infrastructure.)

S3. Built-in tools: selection criteria (task 2.5)

The agent-side toolkit the exam names — know each tool’s *selection criterion*:

Tool	Selects when...	Canonical example
Grep	Search file contents for patterns	Find all callers of a function. Locate an error message. Find import statements
Glob	Match file paths/names by pattern	<code>**/*.test.tsx</code> — all test files by naming convention
Read	Load a file’s full contents	Follow an import found by Grep
Write	Write a full file	Pair with Read when Edit cannot find a unique anchor
Edit	Targeted modification via unique text matching	Replace one function body
Bash	Run commands (tests, builds, git)	Execute the test suite

The Edit-failure pattern (tested): Edit requires its anchor text to match **uniquely**. When it fails on non-unique matches, use this fallback: **Read the full file, then Write the modified version**. Do not repeat Edit attempts with longer anchors.

Incremental exploration pattern (tested): build codebase understanding **incrementally**. Grep for entry points first. Then Read to follow imports and trace flows. Do not read all files first (too much context. See Domain 5.4). Tracing usage across wrapper modules: first identify all exported names, then search for each name across the codebase.

S4. Supplement Q&A

SQ1. How does an MCP tool tell the agent it failed? **A.** `isError`: `true` on the tool result, with details in content. A domain failure travels as a readable result, not a protocol error, so the model can reason about recovery.

SQ2. Name the four error categories and which is retryable. **A.** Transient (retryable), validation (fix input first), business (not retryable — explain to user), permission (not retryable — escalate/credentials).

SQ3. What three fields make an error response “structured”? **A.** `errorCategory`, `isRetryable` boolean, human-readable description (plus a customer-friendly explanation for business violations).

SQ4. Why do uniform “Operation failed” responses hurt agents? **A.** They remove the basis for recovery decisions. This causes wasted retries on permanent errors and abandonment of recoverable ones.

SQ5. Where does a shared team MCP server go? A personal experiment? **A.** Project-scoped `.mcp.json` (version-controlled). User-scoped `~/.claude.json`.

SQ6. How do you commit MCP config without committing tokens? **A.** Environment-variable expansion (e.g. `${GITHUB_TOKEN}`) in `.mcp.json`. Each member supplies credentials through their environment.

SQ7. When are MCP tools from multiple servers available? **A.** Discovered at connection time. Tools from all configured servers are available simultaneously.

SQ8. Agents keep making exploratory calls to learn what data exists. MCP fix? **A.** Expose content catalogs (issue summaries, doc hierarchies, schemas) as **resources**.

SQ9. The agent prefers built-in Grep over your capable MCP search tool. Fix? **A.** Enrich the MCP tool's description with its capabilities and outputs. Descriptions drive adoption.

SQ10. Grep vs Glob in one line? **A.** Grep searches file *contents* for patterns. Glob matches file *paths/names*.

SQ11. Edit keeps failing on a non-unique match. Fallback? **A.** Read the full file, then Write the modified version.

SQ12. Right way to build understanding of a large codebase? **A.** Incrementally: Grep entry points → Read to follow imports/trace flows. Do not read everything first.

SQ13. A subagent's search times out once, then succeeds on retry. What reaches the coordinator? **A.** Nothing. Subagents recover transient failures locally. Only unresolvable errors propagate (with attempts + partials).

SQ14. Empty search results: `isError: true` or not? **A.** Not. A valid empty result is a successful query with zero matches. If you flag it as error, you corrupt retry logic.

S5. Supplement checklist

- ☐ I can state what `isError` does and where error detail lives.
- ☐ I can classify an error into the four categories and derive the agent's move.
- ☐ I know `.mcp.json` vs `~/.claude.json` and the env-var expansion method.
- ☐ I know tools from all servers are discovered at connection time, simultaneously.
- ☐ I can pick resource-as-catalog vs tool-as-action.
- ☐ I can select among Read/Write/Edit/Bash/Grep/Glob and recite the Edit→Read+Write fallback.

Introduction to Model Context Protocol — Exam Prep Study Notes

Disclaimer: These are **original study notes** for exam / certification prep. They are **not** official Anthropic course material or verbatim lesson text. Topics follow the public outline of [Introduction to Model Context Protocol](#). **Advanced follow-on (pointer only):** [Model Context Protocol: Advanced Topics](#) covers sampling, progress/logging, roots, JSON-RPC message details, STDIO handshake depth, Streamable HTTP / SSE, stateful vs. stateless scaling — take that after this intro course.

Cross-check live docs: modelcontextprotocol.io (architecture, primitives, transports, lifecycle). Spec versions evolve—prefer concepts over memorizing a single date stamp.

Overview

MCP (Model Context Protocol) is an open standard for connecting AI hosts (Claude Desktop, IDEs, custom agents) to external tools and data **without** hand-rolling a unique JSON tool schema for every app×service pair. The intro course’s public learning goals emphasize: architecture (shifting tool definition/execution to specialized servers); transport-agnostic messaging; the end-to-end request flow; building servers with the Python SDK (decorators vs. raw JSON schemas); document-style tools; the MCP Inspector; **resources** (direct + templated); client-side resource reading and MIME types; **prompts** as reusable workflows; choosing among tools / resources / prompts; and integration patterns like autocomplete and context injection.

Public module shape: Introduction → Hands-on MCP servers → Connecting with MCP clients.

Prereqs called out publicly: basic Python, async/await, API literacy.

How these notes relate to the API course: Building with the Claude API teaches the Messages/tool loop inside *one* app. MCP teaches how many apps can share the same external capabilities through a common protocol. On exams, if you see “reuse across hosts” or “Inspector,” think MCP.

Key concepts (cheat sheet)

Concept	One-liner
Host	The AI application the user sees (orchestrates clients, UI, model calls)
Client	In-host connector; 1:1 with a single MCP server
Server	Process/service exposing tools, resources, and/or prompts
Tools	Model-controlled actions (side effects allowed; like function calling)
Resources	App-controlled, mostly read-only data via URIs
Prompts	User-controlled, pre-crafted instruction templates
Transport	How JSON-RPC messages move (commonly STDIO locally; Streamable HTTP / SSE remotely)
Data layer	JSON-RPC methods, lifecycle, primitives, notifications
Inspector	Browser-based helper to list/call tools and debug a server
Capability negotiation	Initialize handshake declares what each side supports

Mental model: Host asks the model what to do → model may select a tool → client invokes the MCP server → server talks to real systems → results flow back into the model context.

Deep notes by topic

1. Why MCP exists

Without MCP, every AI app duplicates integration code for GitHub, Slack, databases, filesystems, etc.
With MCP:

- **N servers** (one per system or capability domain) + **M hosts/clients** beat **M×N** bespoke adapters.
- Servers own auth to the backend, input validation, and domain logic.
- Hosts stay focused on UX, policy, and model orchestration.

Exam phrase to remember: MCP **shifts** tool definition and execution burden from your monolithic app server toward **specialized MCP servers**.

Analogy: USB standard vs. proprietary cables. Hosts speak MCP; servers speak MCP; backends stay behind the server adapter.

What MCP is not

- Not a replacement for the Claude Messages API (the host still calls the model).
- Not a database or an embedding store by itself.
- Not a guarantee of safety—poorly scoped servers are still dangerous.
- Not “only for Anthropic”; it is an open protocol many hosts can implement.

2. Architecture: hosts, clients, servers

User ↔ Host (LLM app) ↔ Client₁ ↔ Server₁ (e.g. docs)
 ↔ Client₂ ↔ Server₂ (e.g. calendar)

- **Host responsibilities:** spawn/manage clients; present prompts/resources to users; decide what context reaches the model; enforce user consent and security policy; run or call the LLM.
- **Client responsibilities:** capability negotiation / session lifecycle; translate host needs into MCP requests; isolate one server from another.
- **Server responsibilities:** advertise and implement primitives; stay focused (single domain is easier to secure and test).

1:1 rule: each client talks to exactly one server. A host that uses three servers runs three clients. This isolation is an exam favorite.

Local vs. remote servers

	Local (often STDIO)	Remote (often Streamable HTTP)
Process	Host spawns subprocess	Independent service
Typical fan-out	One client per process	Many clients to one service
Auth story	OS user privileges	HTTP auth / OAuth / tokens
Failure mode	Crash kills that server	Network partitions, version skew

Security boundary reminder: a compromised or overly powerful server is dangerous — least privilege, clear roots/paths, and user approval for sensitive tools matter even in “local” STDIO setups.

3. Layers: data vs. transport

Official architecture splits MCP into:

1. **Data layer** — JSON-RPC 2.0 message semantics: lifecycle, tools/resources/prompts, notifications, client features like sampling/elicitation (advanced).
2. **Transport layer** — how bytes move: STDIO vs. Streamable HTTP (with optional SSE), framing, and transport auth.

Intro exam bar: you can explain that the **same** JSON-RPC methods ride on different transports. Advanced course goes deeper on production HTTP scaling.

4. Message flow (request → Claude → tools → back)

A typical turn:

1. User asks a question in the host.
2. Host assembles context (chat history, selected resources, chosen prompt template).
3. Host calls Claude with available tool definitions discovered from MCP servers.
4. Claude returns a tool call (name + arguments).
5. Host/client sends an MCP tools/call to the right server.
6. Server executes (read file, edit doc, query API) and returns structured results.
7. Host appends tool results into the conversation and calls Claude again.
8. Claude produces a user-visible answer (or more tool calls).

Know the difference between **request/result** pairs (RPC calls) and **notifications** (one-way events). Deeper message taxonomy appears in the Advanced Topics course.

Original sequence diagram (text)

```
User → Host: "Summarize Q3 plan doc"
Host → Client(docs): resources/read docs://q3-plan
Client → Server: resources/read
Server → Client: text/markdown body
Host → Claude: messages + tool defs + injected resource
Claude → Host: tool_use edit_section(...)
Host → Client: tools/call edit_section
Server → Client: ok
Host → Claude: tool_result
Claude → Host: final summary for user
```

5. Lifecycle (intro depth)

MCP sessions are not “fire random methods first.” Conceptual lifecycle:

1. **Connect** transport (spawn STDIO process or open HTTP session).
2. **Initialize** — exchange protocol version, clientInfo / serverInfo, and capabilities.
3. **Initialized notification** — client signals it is ready.
4. **Operate** — tools/list, tools/call, resources/list, resources/read, prompts/list, prompts/get, etc.
5. **Shut down** — close streams / terminate subprocess; don't leak child processes.

Capability negotiation intuition

- Server may declare support for tools, resources, prompts, and whether lists can change (listChanged).
- Client may declare support for features servers might request later (sampling, elicitation—advanced).
- Calling an unsupported method is a protocol smell—discover first.

Exam tip: if asked “what happens before tools/list?”, answer **initialize / negotiate**, not “immediately call the tool.”

6. Building MCP servers (Python SDK mindset)

Intro course emphasis: use the SDK so you **don’t** hand-author giant JSON schemas.

- Decorators / type hints / Field descriptions become tool metadata the model sees.
- Good descriptions dramatically improve tool selection — treat them like prompt engineering.
- Example domain from the public outline: **document management** — tools to read and edit documents.
- Keep tools narrow: `read_document(id)` and `update_section(id, section, text)` beat a vague `do_anything(payload)`.

Original decorator-style sketch (illustrative, not an SDK dump)

```
# Conceptual only – verify current SDK APIs before coding
@server.tool()
def read_document(doc_id: str) -> str:
    """Return the full text of a document by id. Use when the user asks what's inside a doc.
    ...

@server.tool()
def update_section(doc_id: str, section: str, text: str) -> str:
    """Replace one named section. Prefer this over rewriting the entire document."""
    ...
```

Description writing checklist

- What the tool does in one sentence.
- When to use it / when not to.
- Units, id formats, side effects.
- Failure modes the model should expect.

Inspector: run the server and open the MCP Inspector in a browser to list tools, fire calls, and inspect payloads before wiring a full host. Great for exam “how do you debug a server?” answers.

7. Resources: direct vs. templated

Resources expose read-oriented data the *application* (not the model alone) decides to fetch and inject.

- **Direct resources:** static URI, e.g. `docs://index` or `config://app-settings`.
- **Templated resources:** URI templates with parameters, e.g. `docs://{doc_id}` — client fills params.

Client reading checklist:

- Request resource by URI.
- Handle **MIME types** (e.g. `application/json` vs. `text/plain`) so you parse correctly.
- Inject content into the model context deliberately (whole doc vs. excerpt).

When resource vs. tool? If it's primarily "fetch data for context" with no meaningful side effect → resource. If it changes state or performs an action → tool.

URI design tips

- Use a clear scheme (`docs://`, `db://`, `config://`) per server domain.
- Keep ids URL-safe; document template parameters.
- Prefer immutable snapshots for auditability when injecting into prompts.

MIME handling examples

MIME	Client behavior
<code>text/plain</code>	Inject as text; maybe truncate
<code>text/markdown</code>	Same, preserve headings if possible
<code>application/json</code>	Parse; maybe pretty-print selected fields
<code>image/png</code>	Host may route to vision-capable message blocks

Exam trap: ignoring MIME and always `json.loads` — binary or plain text will blow up.

8. Prompts as MCP primitives

Prompts are reusable, high-quality instruction templates for common workflows (the course publicly mentions examples like document formatting).

- **User-controlled:** typically selected from a menu ("Format selection as RFC-style memo") rather than silently invented by the model.
- Can parameterize slots (title, tone, audience).
- Help teams share best-practice instructions across hosts.

Original prompt template sketch

Name: `format_as_adr`

Args: `title`, `status`, `context`

Body:

Write an Architecture Decision Record titled `{title}`.

Status: `{status}`.

Context: `{context}`.

Use sections: Context, Decision, Consequences.

Host UX: user picks prompt → fills args → host expands → sends to model (optionally with related resources).

9. Choosing tools vs. resources vs. prompts

Primitive	Who steers?	Typical use	Side effects?
Tool	Model	Search, edit, send, compute	Often yes
Resource	Application / host logic	Load file, config, dataset into context	Generally no
Prompt	User	Start a known workflow with vetted instructions	No (instructions only)

Exam trap: calling everything a “tool.” Graders look for the control-plane distinction above.

Sorting drill (memorize)

- “Load README into context” → **resource**
- “Create GitHub issue” → **tool**
- “Apply ADR template” → **prompt**
- “Search docs then edit” → **tools** (+ maybe resources for known URIs)
- “Show company style guide” → **resource** or **prompt** depending on whether it’s raw data vs. a workflow template

10. Transports (intro-level)

MCP is **transport-agnostic** at the message layer (JSON-RPC style messages). Common options:

- **STDIO**: client spawns server as a subprocess; communicate over stdin/stdout. Ideal for local desktop/IDE integrations. Requires a correct **initialization handshake** before ordinary calls (details stressed more in Advanced Topics).
- **HTTP + SSE / Streamable HTTP**: for remote/networked servers; servers can push events; think sessions, auth, and scaling tradeoffs. Modern docs emphasize **Streamable HTTP** (HTTP POST client→server, optional SSE for streaming) and often recommend OAuth for tokens.

Intro exam bar: name the transports and when you’d pick local STDIO vs. remote HTTP. Leave dual-connection SSE minutiae and stateless horizontal scaling to the advanced course.

Transport choice memo (original)

Use STDIO when the server must touch the local filesystem or a developer laptop secret store and the host can manage a child process. Use Streamable HTTP when multiple hosts across a company should share one managed integration (with real auth, audit logs, and central updates).

11. Practical integration patterns

- **Context injection**: host reads resources → attaches relevant text to the model request.
- **Autocomplete / discovery**: list tools, resources, and prompts so UIs can suggest actions.
- **Lifecycle**: connect → initialize/negotiate → operate → shut down cleanly (don’t leak subprocesses).
- **Composition**: one host, many servers (docs + issue tracker + DB), each behind its own client.
- **Notifications**: if a server declares list-changed capability, it may notify clients to refresh tools/list (and similar)—hosts should update the model’s available tools.
- **Error surfacing**: tool failures should become structured errors the model can see, not silent host crashes.

12. Client features (pointer — advanced)

Intro awareness only (depth in Advanced Topics):

- **Sampling:** server asks the host/client to run an LLM completion (model-agnostic servers).
- **Elicitation:** server asks the host to collect user input / confirmation.
- **Logging / progress:** server emits logs or progress for long work.

Don't overclaim these on an intro quiz—know they exist and which side exposes them.

13. Security considerations (intro must-pass)

1. **User consent:** hosts should approve sensitive tool calls, especially first use.
2. **Least privilege:** filesystem servers get a directory root, not \$HOME entire disk, when possible.
3. **Secret handling:** servers should use OS keychains / env / secret managers—not hardcode tokens in tool responses.
4. **Confused deputy:** a malicious document could try to coerce the model into calling delete_repo—policy gates still required.
5. **Supply chain:** treat third-party MCP servers like installing software; review permissions.
6. **Transport auth:** remote servers need real authentication; STDIO inherits the user's OS privileges—still not “safe by default.”
7. **Audit:** log tool name, args (redacted), actor, timestamp.

14. End-to-end document-management mental lab

Match the course's public project theme without copying lab prose:

1. Server exposes list_docs, read_document, update_section tools.
2. Server exposes docs://{doc_id} templated resource and docs://index direct resource.
3. Server exposes a format_document prompt.
4. Use Inspector to call read_document before wiring Claude.
5. Host lists prompts for a “Format” menu; on select, expand prompt + optionally inject resource.
6. Model may still call update_section as a tool when the user asks to edit.

This single domain exercises **all three primitives**.

15. MCP vs. plain Messages tool defs

Concern	Plain tool JSON in Messages	MCP
Discovery	Hardcoded in app	*/list from servers
Reuse	Copy schemas	Shared servers across hosts
Resources / prompts	Roll your own	First-class primitives
Ops	In-process functions	Local processes or remote services
Best when	One app, few tools	Ecosystem / many hosts

Bridge insight: hosts often **translate** MCP tool schemas into the model provider's tool format under the hood. MCP doesn't remove the Claude tool loop—it standardizes the outer integration.

16. JSON-RPC mental model (lightweight)

Enough for intro exams:

- **Request:** has `id`, `method`, `params` → expects **result** or **error** with same `id`.
- **Notification:** has `method` (and maybe `params`) but **no `id`** → no response.
- Methods look like `initialize`, `tools/list`, `tools/call`, `resources/read`, `prompts/get`.
- Unknown methods / bad `params` → structured JSON-RPC errors, not HTTP HTML pages (even on HTTP transports, the payload is still RPC-shaped).

17. Common failure modes & debugging

Symptom	Likely cause	Fix direction
Host shows no tools	Init failed / capabilities missing / list not called	Check initialize + tools/list in Inspector
Tool call 404-ish / unknown tool	Stale list after server change	Handle <code>list_changed</code> ; refresh
MIME parse error	Assumed JSON for text resource	Branch on MIME
Hung STDIO server	Handshake incomplete / stdout polluted with logs	Keep protocol on stdout; log to stderr
Remote 401	Missing bearer/OAuth	Fix transport auth
Model never calls your tool	Weak description / too many overlapping tools	Rewrite descriptions; reduce set
Subprocess zombies	Host didn't close lifecycle	Tear down clients on exit

18. Host implementation checklist (what “good” looks like)

When you build or review a host that speaks MCP, graders (and production reviewers) look for more than “it connected once.”

1. **Config surface:** user can add/remove servers (command + args for STDIO; URL + auth for HTTP).
2. **Consent UX:** first-time tool use prompts; distinguish read-only vs. mutating tools.
3. **Unified registry:** merge tools from all clients with namespaced collision handling (`docs.read` vs. `db.read`).
4. **Model translation:** map MCP tool schemas into the provider tool format your LLM API expects.
5. **Result hygiene:** truncate huge tool outputs; preserve enough detail for the model to continue.
6. **Cancellation:** user can abort long tool calls; client cancels in-flight RPC when supported.
7. **Observability:** per-server latency, error rates, and which prompts/resources were injected.
8. **Graceful degradation:** if one server dies, other servers keep working.

19. Server design patterns (document domain + beyond)

Pattern A — Thin adapter MCP server wraps an existing REST API. Tools ≈ endpoints. Resources ≈ GETs that are safe to inject.

Pattern B — Domain facade Fewer, smarter tools (`search_and_summarize_locally`) that hide backend chaos. Easier for models; harder to keep descriptions honest.

Pattern C — Read/write split Expose reads as resources; writes as tools requiring confirmation. Matches the control-plane triad cleanly.

Pattern D — Prompt pack Server mostly ships prompts + a few resources (style guides). Great for teams standardizing workflows without giving the model write power.

Anti-pattern — Kitchen sink One server that shells out arbitrarily, reads all files, and sends email. Fails least-privilege and frightens host permission UIs.

20. Mapping Claude Academy modules → study actions

Public module	What to practice tonight
Introduction	Draw architecture; recite triad
Hands-on servers	Build 2–3 tools; open Inspector
Connecting with clients	Read resource + MIME branch; list prompts; inject context

Tie-back to Building with the Claude API: after MCP returns tool output, you still perform the Messages `tool_result` continuation—MCP didn’t erase that loop.

21. Worked story: autocomplete + context injection

Autocomplete: Host calls `prompts/list` and `tools/list` on idle. UI shows “Format as ADR”, “Search docs”, “Create issue”. User picks “Format as ADR” → host calls `prompts/get` with args → expands text into the composer.

Context injection: User @-mentions a doc. Host resolves `docs://{id}` resource, checks MIME, injects a truncated body into the next user message inside tagged `<doc>` delimiters, and *separately* keeps write tools available if the user asks to edit.

Exam insight: autocomplete is **discovery UX**; injection is **context assembly**. Both are host responsibilities even though data comes from servers.

22. Versioning & compatibility mindset

- Protocol versions appear during initialize; incompatible versions should fail fast.
- Servers and hosts upgrade independently—capability flags exist so optional features don’t hard-break.
- For exams: prefer answering “negotiate capabilities and degrade gracefully” over inventing a single forever-version number.

Comparison tables (exam quick hits)

Control plane triad

Question	Tools	Resources	Prompts
Who decides to use it?	Model	App/host	User
Primary verb	Call / execute	Read / inject	Select / expand

Question	Tools	Resources	Prompts
Typical risk	Side effects	Data exfil via context	Instruction exfil / social engineering
Discovery method family	tools/*	resources/*	prompts/*

STDIO vs Streamable HTTP

Dimension	STDIO	Streamable HTTP
Where it shines	Desktop, IDE, local files	Team/shared integrations
Auth	OS user	Tokens / OAuth / headers
Scaling	Per-user processes	Central service
Ops complexity	Process mgmt	Network + auth + versioning

Host vs Client vs Server (blame chart)

Bug	First place to look
Wrong tool chosen	Tool descriptions + model prompt (host)
Tool not listed	Client discovery / server capabilities
Wrong file edited	Server implementation / authz roots
UI missing prompt menu	Host didn't list/present prompts
MIME crash	Client resource reader

Exam tips

1. Draw the **Host** → **Client** → **Server** triangle from memory; mark 1:1 client-server.
2. Memorize the triad: **tools = model-controlled**, **resources = app-controlled**, **prompts = user-controlled**.
3. SDK decorators exist to avoid brittle hand-written schemas — say that out loud once.
4. Inspector = first debugging stop for servers.
5. MIME type handling on resource reads is an easy “gotcha” question.
6. Know that an **Advanced Topics** course exists for sampling, roots, progress, and production transports — don't overclaim intro-course depth on those.
7. Security: local STDIO is not “no security”; it still runs with user privileges.
8. Initialize before list/call.
9. Notifications ≠ requests (no response id).
10. MCP complements Messages tool use; it does not delete the tool_result loop.

Exam traps

Trap	Fix
“Client can talk to many servers”	No— host has many clients; each client is 1:1
Everything is a tool	Use resource/prompt when control plane differs
Skip Inspector, debug only in prod host	Inspector first
Ignore MIME	Always branch
Assume STDIO is sandboxed	It inherits user privileges
Memorize one HTTP dialect forever	Concepts: Streamable HTTP / SSE exist; verify current spec
Put secrets in resource bodies casually	Redact; use secure stores
Forget shutdown	Reap subprocesses

Self-check Q&A

Q1. What are the three MCP roles? **A:** Host (LLM app), Client (1:1 connector), Server (exposes primitives).

Q2. Who typically decides to invoke a tool? **A:** The model (with host policy gates); tools are model-controlled actions.

Q3. Who typically selects a prompt template? **A:** The user (user-controlled primitive), often via UI.

Q4. Direct vs. templated resource? **A:** Direct = fixed URI; templated = URI pattern with parameters filled by the client.

Q5. Why use the Python SDK decorators? **A:** Derive tool schemas from types/Field descriptions instead of maintaining raw JSON schemas by hand.

Q6. What is the MCP Inspector for? **A:** Browser-based testing/debugging of server tools/resources before full host integration.

Q7. Name two transports and a use case for each. **A:** STDIO for local subprocess servers; Streamable HTTP (or HTTP/SSE family) for remote servers.

Q8. Where do you go next after this intro? **A:** [MCP Advanced Topics](#) for sampling, progress/roots, and production transport/scaling details.

Q9. What problem does MCP reduce mathematically? **A:** $M \times N$ custom integrations toward M hosts + N specialized servers.

Q10. What are the two MCP layers? **A:** Data layer (JSON-RPC semantics/primitives) and transport layer (STDIO / HTTP).

Q11. What must happen before ordinary tools/list calls? **A:** Transport connect + initialize capability negotiation (+ initialized signal).

Q12. Who owns backend API keys for a GitHub MCP server? **A:** Typically the server (or its secret store)—not every host hardcoding tokens.

Q13. How should a client react to tools/list_changed? **A:** Refresh tool lists and update what the model can call.

Q14. Resource or tool: “fetch schema for context”? **A:** Resource (read-oriented, app-controlled injection).

Q15. Resource or tool: “insert a row”? **A:** Tool (side effect / action).

Q16. Why keep stdout clean on STDIO servers? **A:** Protocol messages travel on stdio—logging noise can break framing; use stderr for logs.

Q17. What is context injection? **A:** Host reads resources (or other data) and attaches them into the model request deliberately.

Q18. What is autocomplete/discovery in MCP UIs? **A:** Listing tools/resources/prompts so users/hosts can pick capabilities without hardcoding.

Q19. Does MCP replace Claude’s tool_result loop? **A:** No. Hosts still feed tool results back into the model conversation.

Q20. Name a client-exposed advanced primitive. **A:** Sampling, elicitation, or logging (intro: awareness only).

Q21. What’s a good tool granularity rule? **A:** Narrow, composable tools with clear schemas beat one mega-tool.

Q22. Why are Field descriptions important? **A:** They become model-facing guidance for arguments—prompt engineering for tools.

Q23. What’s the security mistake in “local = safe”? **A:** STDIO servers run with the user’s privileges and can still exfiltrate or delete data.

Q24. How do prompts help teams? **A:** Share vetted instruction templates across hosts instead of each user reinventing wording.

Q25. What MIME-related bug is classic? **A:** Parsing a text/plain resource as JSON (or ignoring content type entirely).

Q26. Draw-from-memory: how many clients for 4 servers? **A:** Four clients (1:1), managed by one host.

Q27. What does transport-agnostic mean here? **A:** Same data-layer methods can ride different transports.

Q28. What’s a notification missing that requests have? **A:** An id / expected response.

Q29. When is plain inline tooling better than MCP? **A:** Single-app prototypes with a few stable functions and no reuse need.

Q30. What’s the first tool you use when a new server misbehaves? **A:** MCP Inspector (list/call/in-spect payloads) before full host integration.

Quick review checklist

- ☐ I can draw Host/Client/Server with 1:1 links.
- ☐ I can explain $M \times N \rightarrow M+N$ motivation.
- ☐ I can define tools vs resources vs prompts by *who controls* them.
- ☐ I can outline initialize → list → call/read → shutdown.

- ☐ I know STDIO vs Streamable HTTP selection criteria.
- ☐ I can describe Inspector’s role.
- ☐ I handle MIME types on resource reads.
- ☐ I can sort example tasks into the triad correctly.
- ☐ I know MCP sits beside—not instead of—the model tool loop.
- ☐ I can list at least five security considerations.
- ☐ I know Advanced Topics exists for sampling/roots/progress/scaling.
- ☐ I can sketch a doc-management server using all three primitives.

Glossary

Term	Plain meaning
MCP	Open protocol connecting AI hosts to external tools/data
Host	User-facing AI application coordinating clients + model
Client	In-host 1:1 session object talking to one server
Server	Process/service exposing MCP primitives
Tool	Executable action, usually model-requested
Resource	URI-addressable data for app-controlled context
Prompt (MCP)	User-selectable instruction template
STDIO transport	Subprocess stdin/stdout messaging
Streamable HTTP	Remote HTTP transport with optional SSE
JSON-RPC 2.0	Request/response/notification message format
Initialize	Lifecycle handshake + capability negotiation
Capability	Declared feature support (tools, resources, ...)
Inspector	Dev UI to exercise a server
MIME type	Content type guiding how to parse resource bodies
Notification	One-way RPC message without response
Sampling	Server-requested LLM call via client (advanced)
Elicitation	Server-requested user input via host (advanced)
Roots	Boundary paths/scopes for server access (advanced)
listChanged	Capability allowing list update notifications
Context injection	Host inserts resource content into model context

Study drills (original practice ideas)

1. **Sketch from memory:** Host/Client/Server with two servers attached; label which hop is 1:1.
2. **Primitive sorting:** Given a list like “load README”, “create issue”, “Apply ADR template”, tag each as resource, tool, or prompt.
3. **Fake doc server:** Two tools (list_docs, get_doc) + one templated resource; exercise them in an Inspector-like mindset (even if you only print JSON).
4. **MIME handling:** Return JSON and plain text resources; write client code that branches on MIME type.
5. **Failure path:** Tool throws; ensure the host surfaces an error tool_result the model can recover from instead of crashing the session.

6. **Transport choice memo:** One paragraph arguing STDIO for a desktop filesystem server vs. HTTP for a shared company knowledge server.
7. **Lifecycle flashcards:** Order connect / initialize / initialized / list / call / shutdown.
8. **Security review:** For a proposed `shell_exec` tool, write three policy gates you'd require in the host.
9. **Description rewrite:** Take a vague tool blurb ("manages docs") and rewrite it with when-to-use / when-not / side effects.
10. **Bridge drill:** Explain in four sentences how an MCP `tools/call` becomes a Claude `tool_use` / `tool_result` round-trip inside a host.

Keep drills short; depth on sampling, roots, and Streamable HTTP belongs in the Advanced Topics course linked above.

Source URLs

- Intro course: <https://academy.claude.com/courses/introduction-to-model-context-protocol>
- Advanced course (pointer): <https://academy.claude.com/courses/model-context-protocol-advanced-topics>
- MCP docs / architecture: <https://modelcontextprotocol.io>
- Spec & SDK indexes: linked from modelcontextprotocol.io (versions change—open live)

End of course notes. Prefer the official MCP specification and Anthropic Academy labs for hands-on syntax that may change between SDK versions.

CCAR-F Domain 2 mechanics supplement (added 2026-08-23)

Scope: the concrete mechanics the **CCAR-F exam guide Domain 2 (Tool Design & MCP Integration, 18%)** tests beyond this course's conceptual intro — task statements 2.2 (structured errors), 2.4 (MCP in Claude Code / agent workflows), 2.5 (built-in tools). Tool-description craft (2.1) is covered above §6 and in 08 §Q&A; `tool_choice` and tool distribution (2.3) are covered in 03-building-with-claude-api.md — cross-check both. Verified against current Claude Code docs; original synthesis.

S1. Structured error responses for MCP tools (task 2.2)

The `isError` flag is MCP's mechanism for communicating tool failure back to the agent *inside a successful protocol response*: the `tools/call` result carries `isError: true` and the error details in content, so the **model** can read the failure and reason about recovery. (A protocol-level JSON-RPC error is for malformed calls; a *tool-domain* failure travels as a normal result with `isError: true`.)

Why generic errors fail: a uniform "Operation failed" gives the agent no basis for choosing between retry, alternative approach, escalation, or asking the user. Wasted retries on permanent errors and abandoned recoverable ones both trace to unstructured errors.

Error taxonomy the exam uses (memorize the four):

Category	Example	Retryable?	Agent's correct move
Transient	Timeout, service unavailable	Yes	Retry (with backoff)
Validation	Malformed input, bad ID format	No (as-is)	Fix the input, then retry
Business	Policy violation (refund cap)	No	Explain to the user / alternative workflow
Permission	Caller lacks access	No	Escalate / different credential path

Structured error metadata pattern: return `errorCategory` (transient / validation / permission / business), an **`isRetryable` boolean**, and a **human-readable description**. For business-rule violations, set retryable **false** and include a **customer-friendly explanation** the agent can relay (“refunds above \$500 require a supervisor”) — the agent then communicates instead of hammering retries.

Multi-agent link (Domain 5.3): subagents recover **transient** failures locally; only unresolvable errors propagate to the coordinator, carrying what was attempted and partial results. And keep **access failure ≠ valid empty result**: an empty result set is a *successful* query with zero matches, never `isError: true`.

S2. MCP servers in Claude Code: scoping and configuration (task 2.4)

Two scopes, one exam distinction:

File	Scope	Shared?	Use for
<code>.mcp.json</code> (project root)	Project	Yes — committed to version control; every teammate gets it on clone/pull	Shared team tooling (the team's Jira/GitHub/db servers)
<code>~/.claude.json</code>	User	No — personal machine only	Personal/experimental servers you're trying out

Environment-variable expansion in `.mcp.json` — e.g. `${GITHUB_TOKEN}` in an `env/header` value — lets teams commit server *configuration* while each member supplies their own **credentials from their environment**. The exam framing: how do you share the config **without committing secrets**? → env-var expansion in project-scoped `.mcp.json`.

Discovery model: tools from **all configured MCP servers are discovered at connection time and are available simultaneously** to the agent — a project server and a personal server coexist in one session (official prep Exercise 2 has you verify exactly this).

MCP resources as content catalogs: expose issue summaries, documentation hierarchies, or database schemas as **resources** so agents get *visibility into what data exists* without burning turns on exploratory tool calls. Catalog = resource; action = tool (consistent with the triad in §7–9 above).

Two more 2.4 skills:

- **Description competition with built-ins:** if your MCP tool’s description is thin, the agent may prefer a built-in (like Grep) over your more capable server tool. Fix by **enriching the MCP tool description** with its capabilities and outputs — description quality drives adoption, not just selection between similar tools.
- **Community vs custom servers:** prefer existing community MCP servers for standard integrations (e.g. Jira); reserve custom server builds for team-specific workflows. (Deploying/hosting MCP server infrastructure is explicitly **out of scope** for the exam.)

S3. Built-in tools: selection criteria (task 2.5)

The agent-side toolkit the exam names — know each tool’s *selection criterion*:

Tool	Selects when...	Canonical example
Grep	Searching file contents for patterns	Find all callers of a function; locate an error message; find import statements
Glob	Matching file paths/names by pattern	<code>**/*.test.tsx</code> — all test files by naming convention
Read	Load a file’s full contents	Follow an import found by Grep
Write	Write a full file	Pair with Read when Edit can’t anchor
Edit	Targeted modification via unique text matching	Replace one function body
Bash	Run commands (tests, builds, git)	Execute the test suite

The Edit-failure pattern (tested): Edit requires its anchor text to match **uniquely**. When it fails on non-unique matches, the fallback is **Read the full file, then Write the modified version** — not repeated Edit attempts with longer anchors.

Incremental exploration pattern (tested): build codebase understanding **incrementally** — Grep for entry points first, then Read to follow imports and trace flows — rather than reading all files upfront (context bloat; see Domain 5.4). Tracing usage across wrapper modules: first identify all exported names, then search for each name across the codebase.

S4. Supplement Q&A

SQ1. How does an MCP tool tell the agent it failed? **A.** `isError`: `true` on the tool result, with details in content — a domain failure travels as a readable result, not a protocol error, so the model can reason about recovery.

SQ2. Name the four error categories and which is retryable. **A.** Transient (retryable), validation (fix input first), business (not retryable — explain to user), permission (not retryable — escalate/credentials).

SQ3. What three fields make an error response “structured”? **A.** `errorCategory`, `isRetryable` boolean, human-readable description (plus a customer-friendly explanation for business violations).

SQ4. Why do uniform “Operation failed” responses hurt agents? **A.** They remove the basis for recovery decisions — causing wasted retries on permanent errors and abandonment of recoverable ones.

SQ5. Where does a shared team MCP server go? A personal experiment? **A.** Project-scoped `.mcp.json` (version-controlled); user-scoped `~/.claude.json`.

SQ6. How do you commit MCP config without committing tokens? **A.** Environment-variable expansion (e.g. `${GITHUB_TOKEN}`) in `.mcp.json` — each member supplies credentials via their environment.

SQ7. When are MCP tools from multiple servers available? **A.** Discovered at connection time; all configured servers' tools are available simultaneously.

SQ8. Agents keep making exploratory calls to learn what data exists. MCP fix? **A.** Expose content catalogs (issue summaries, doc hierarchies, schemas) as **resources**.

SQ9. The agent prefers built-in Grep over your capable MCP search tool. Fix? **A.** Enrich the MCP tool's description with its capabilities and outputs — descriptions drive adoption.

SQ10. Grep vs Glob in one line? **A.** Grep searches file *contents* for patterns; Glob matches file *paths/names*.

SQ11. Edit keeps failing on a non-unique match. Fallback? **A.** Read the full file, then Write the modified version.

SQ12. Right way to build understanding of a large codebase? **A.** Incrementally: Grep entry points → Read to follow imports/trace flows — not reading everything upfront.

SQ13. A subagent's search times out once, then succeeds on retry. What reaches the coordinator? **A.** Nothing — transient failures are recovered locally; only unresolvable errors propagate (with attempts + partials).

SQ14. Empty search results: `isError: true` or not? **A.** Not — a valid empty result is a successful query with zero matches; flagging it as error corrupts retry logic.

S5. Supplement checklist

- ☐ I can state what `isError` does and where error detail lives.
- ☐ I can classify an error into the four categories and derive the agent's move.
- ☐ I know `.mcp.json` vs `~/.claude.json` and the `env-var` expansion trick.
- ☐ I know tools from all servers are discovered at connection time, simultaneously.
- ☐ I can pick resource-as-catalog vs tool-as-action.
- ☐ I can select among Read/Write/Edit/Bash/Grep/Glob and recite the Edit→Read+Write fallback.

Claude Code in Action — Primary Study Notes

Note: This edition follows the ASD-STE100 writing rules: simple present tense, active voice, one word = one meaning, sentences of 20–25 words or less. Technical names (Claude, Claude Code, MCP, CLAUDE.md, hooks, plan mode) are exceptions and stay as written. These notes are **original** exam-prep notes for Claude certification study. They are **not** official Anthropic material. They are **not** a lesson transcript.

Disclaimer: These notes align to the **public** course outline at [Claude Code in Action](#) (objectives and module themes only). They are not Claude Academy lesson transcripts or verbatim slide text. Completing the official course remains the source of truth for quizzes. Check CLI flags and UI labels in current Claude Code docs. Names change.

Who it is for: Developers who already use Claude Code for single prompts and want longer, less supervised, team-wide workflows.

Prerequisites (course): Claude Code for one-shot tasks. Basic Git and command line.

How to use these notes: Read one major section. Then answer the self-check. Do not scroll back. Treat tables as flashcards. The ladder **steer** → **configure** → **automate** → **verify/share** is the main structure of most scenario questions.

1. Course map (public objectives)

The public page frames four outcome clusters (about 9 lessons, 1 quiz):

1. **Steer the work** — plan mode, directed compaction, rewind, hands-on vs autonomous goal/loop runs, safe parallelism.
2. **Configure Claude** — lean `CLAUDE.md`, skills, permission modes, hooks for rules that you cannot skip.
3. **Automate repeat work** — routines on Anthropic infrastructure, headless mode in your pipeline, PR / GitHub Action wiring.
4. **Verify and share** — verify unsupervised runs in proportion to how little you watch them. Gate on real tests. Package a trusted setup as a plugin.

Mental model: Claude Code is an agentic coding loop with tools (file edit, bash, search, and others). Your job on exams is not “write a longer prompt.” Your job is to **select the correct control surface** for the risk of the job.

Layer	Question it answers	Strength
Steering	What should we do <i>this session</i> ?	Ephemeral control
Configuration	What should Claude <i>a/ways</i> know / be allowed?	Persistent
Automation	How does this run without me typing?	Operational
Verification / plugins	How do we trust and reuse the setup?	Assurance + portability

Remember: many exam-style questions ask which layer **advises** vs **enforces**.

2. Steering long sessions

2.1 Plan mode

For changes that are not small, start in **plan mode**. Claude may read the repo and run exploratory commands. Claude does **not edit source** until you approve a plan. Use it when:

- The codebase is unfamiliar
- The change touches many files or modules
- You need architectural decisions before code

- The scope of harm includes migrations, auth, billing, or deploy scripts

Cycle modes in the CLI (commonly **Shift+Tab**). Learn the *intent* of each mode more than any single UI string. Labels can change. The spectrum from “read-only exploration” to “auto-approve edits” to “bypass prompts” does not change.

Exam view: Plan mode is a *steering* choice. It is not a permanent project setting. After you approve a plan, you typically switch to a mode that allows implementation. Use permissions that you trust.

2.2 Compaction

Long sessions fill the context window. Claude Code **compacts** older turns. You can also start compaction by hand in many setups (for example `/compact`). Summaries drop detail.

What survives compaction poorly: single-use chat rules (“always use pytest”), temporary naming decisions, and failed experiment paths that you still need as negative examples.

What should survive by design:

- Durable conventions → **CLAUDE.md** / `.claude/rules/`
- On-demand procedures → **skills**
- Hard stops → **hooks** / deny rules

When you *direct* compaction (keep X, drop Y), prefer to keep: approved plan, open decisions, constraints, names of tests that fail. Drop verbose tool output, abandoned experiment paths, and duplicate file listings.

Common exam error: “Compaction lost our coding standards” → The root cause is standards that exist only in chat, not in CLAUDE.md.

2.3 Rewind

When a session drifts, **rewind** to an earlier clean point. Do not stack contradictory fixes on a mixed context. Then restate **one** clear goal.

Rewind vs compact:

Action	Best when	Risk if misused
Rewind	Wrong path / contradictory instructions	You lose useful later work if you rewind too far
Compact	Context full but direction still good	You lose detail. A messy chat may reinforce bad summaries

Pattern: Drift → rewind → single goal → (optional) plan mode again → implement.

2.4 Hands-on vs autonomous

Style	Use when	Control
Hands-on steering	Novel / high scope of harm	You approve edits and watch turns

Style	Use when	Control
Goal + loop / autonomous	Trusted, well-scoped tasks	Hooks, tests, tighter permission mode
Parallel agents	Independent subtasks	Git worktrees (or separate checkouts)

Exam fact: Parallel agents that share one dirty working tree are the wrong pattern. Isolation via worktrees is the safe answer.

2.5 Steering vs autonomy decision tree

Is the task novel or high blast radius?

YES → Hands-on + plan mode; default/manual or accept-edits only after reviewed plan

NO → Is the procedure repeated and already verified?

YES → Skill + hooks; consider routine

NO → Still interactive; encode learnings into CLAUDE.md as you go

Does it need your laptop secrets, private network, or custom piping?

YES → Headless in YOUR CI/agent host

NO → Routine on Anthropic infra may suffice

Must results be identical run-to-run in CI?

YES → Bare/deterministic headless flags + pinned permissions

NO → Interactive or auto modes OK with verification

Multiple agents useful?

YES → Split independent work; isolate with worktrees

NO → Single agent, clear goal

Session went sideways?

→ Rewind first; only then re-prompt

3. Configuration surfaces (advisory vs enforced)

Surface	Nature	Strength
Chat prompt	Temporary	Advisory
CLAUDE.md / rules	Always-on project context	Advisory but persistent
Skills	On-demand procedures	Advisory when loaded
Permission rules / modes	Tool allow / ask / deny	Enforced by Claude Code
Hooks	Lifecycle scripts	Enforced guarantees

Core distinction: “Never edit .env” in CLAUDE.md is a polite request. A **deny** rule or **PreToolUse hook** that blocks .env is a guarantee. Instructions shape intent. Rules and hooks change what is allowed.

3.1 Lean CLAUDE.md

Keep it short enough that Claude follows it. Public guidance and good exam answers agree on this:

- Project purpose, stack, layout (few bullets)
- Commands that matter (test, lint, build)
- Hard do/do not for secrets, migrations, force-push
- Pointers to skills/docs for long procedures — do **not** paste a very long document

Put each rule on the right surface:

Content type	Surface
Always-true facts and conventions	CLAUDE.md
Path-scoped conventions	.claude/rules/ (when available)
Multi-step / occasional workflows	Skills
Must-run-at-event guarantees	Hooks
Tool allow/deny	Permission rules

/init (or equivalent) maps the repo and starts CLAUDE.md. Then refine with what Claude cannot find from code alone.

Anti-pattern: A 2,000-line CLAUDE.md that duplicates the README, the wiki, and every coding standard PDF. Claude will not follow it fully. Exams reward *lean* memory.

CLAUDE.md maintenance loop: After a difficult session, ask: “What should never be found from scratch again?” Put that fact into CLAUDE.md. If it is a procedure, put it in a skill instead.

3.2 Skills

Skills package repeated procedures (start with a verification skill that runs real checks). Typical skill contents conceptually:

- When to apply
- Steps / commands
- Success criteria
- Optional allowed-tools grants for the invoking turn (still subject to baseline permissions)

Skills load **on demand** (invoked or auto-selected when relevant). That is why they are better than putting every procedure into CLAUDE.md.

Exam contrast:

- CLAUDE.md = always in context → keep short
- Skill = load when needed → detailed OK
- Hook = runs regardless of Claude’s “opinion” → use for rules you cannot skip

Good first skills for a team: “run verification suite,” “create PR with checklist,” “migrate DB safely,” “security review touchpoints.”

3.3 Plugins

Plugins bundle CLAUDE.md pieces + skills + hooks + related config (and sometimes MCP / subagents). The whole team installs one trusted setup. Plugins are about **portability**. They are not new model

capabilities.

Share the plugin when the setup is verified. Hooks must gate real tests. Permissions must be least-privilege. Onboarding then changes from “read a wiki” to “install plugin.”

Plugin hygiene checklist:

1. Document required env vars and secrets injection points
2. Fail closed if tests cannot run
3. Namespace skills so multiple plugins coexist
4. Version the plugin. Break changes need a changelog
- 5.

Never include bypass-by-default in a team plugin

4. Permission modes (match mode to risk)

Modes set the baseline before allow/ask/deny rules. Names change. Know the spectrum:

Mode (conceptual)	Behavior	Typical job
Default / manual	Prompt on first tool use	Interactive pairing
Accept edits	Auto-accept common workspace edits	Trusted local feature work
Plan	Explore. No source edits until plan approved	Design before implement
Auto (classifier)	Approves routine actions. Blocks risky ones	Faster interactive with guardrails
Do not ask	Strict. Often denies unless pre-allowed	Controlled non-interactive
Bypass / do not ask risky variant	Skips most prompts	Isolated CI / sandbox only

Evaluation order (mental model): deny → ask → allow. First match wins. Deny wins over allow. A PreToolUse hook that exits with a blocking decision can still stop a call. This is true even when an allow rule or bypass mode would otherwise pass. Hooks are the last hard gate in many designs.

Permission decision tree:

Interactive pairing / learning codebase?

→ Default/manual or plan

Trusted local feature after reviewed plan?

→ Accept edits (or auto with classifier)

CI / headless with pinned allowlist?

→ Don't ask + explicit allows + hooks

Throwaway container, no secrets, ephemeral FS?

→ Bypass may be acceptable

Production credentials on laptop?

→ Never bypass; least privilege + deny rules

Exam traps:

- Bypass-style modes as a laptop default with production credentials → wrong
- Assuming CLAUDE.md overrides a permission deny → wrong
- Using plan mode for a trusted, already-specified one-line rename → overly slow but not “unsafe”. Usually accept-edits or auto is fine
- Believing auto mode replaces deny rules for .env → wrong. The classifier is probabilistic. Deny is absolute

Admins may disable bypass in team settings. Know that organizational policy can remove the bypass option.

5. Hooks: unskippable control

Hooks fire at fixed lifecycle points. Examples worth memorizing (exact event names may change — learn the *jobs*):

Lifecycle job	Example use
Session start	Inject env facts, repo branch warnings
Pre tool use	Block .env, force ask on <code>rm -rf</code> , deny production deploys
Permission request	Auto-decide in non-interactive runs
Pre compact	Re-inject critical constraints into the summary
Stop / after turn	Run linters. Fail closed if tests are red
File / rules loaded	Audit what entered context

Patterns:

1. Block protected paths (.env, production configs, private keys)
2. Gate a turn on **real** test runners (fail closed if red)
3. Force a prompt for sensitive Bash even if Bash is broadly allowed
4. Re-inject critical context after compaction
5. Return permission-decision JSON / exit codes to allow, deny, or ask

Hooks enforce. Prompts advise.

Non-interactive note: In headless `-p` runs, interactive permission prompts may not exist. Rely on PreToolUse hooks and pre-declared allow/deny rules. Do not assume a human will click “allow.”

Hook design tips for exams:

- Prefer fail-closed for safety checks (missing test runner = deny)
- Keep hooks fast. Slow hooks make interactive use painful, and people disable them.
- Log denials with enough detail for audit without dumping secrets
- Do not put irreversible destructive capabilities behind soft asks alone

6. Automation spectrum

When you trust a task, stop starting it by hand:

1. **Routines** — schedule prompts on Anthropic-managed infrastructure (lowest operations effort).
2. **Headless** — `-p / --print`: one-shot, no TUI. Stdin/stdout for pipes and CI.
3. **Bare / deterministic flags** — prefer when CI needs repeatable, low-variance runs.
4. **Agent SDK** — embed Claude Code inside your TypeScript/Python product.
5. **PR / GitHub Action wiring** — managed review patterns. Claude in the pull-request loop.

Decision rule: start with routines. Change to headless when the job needs *your* environment or surrounding script logic. Use the Agent SDK when the agent *is* part of your product.

Need Anthropic-hosted schedule, little custom env?

→ Routine

Need local secrets, Docker, private packages, custom scripts?

→ Headless in your runner

Need agent embedded in product UX?

→ Agent SDK

Need PR comments / checks?

→ GitHub Action / managed review integration

Automation without proportional verification is incomplete. That sentence is an exam answer by itself.

Headless specifics worth remembering:

- Non-interactive means no human in the permission loop
- Pipe prompts via stdin or args. Capture stdout for logs
- Pair with explicit permission allowlists and PreToolUse hooks
- Prefer deterministic settings when CI flakes would cost a lot

7. Verify unsupervised work

When you watch less, you verify more.

How unsupervised	Minimum verification
Interactive pair	Spot-check diffs
Accept-edits local run	<code>git diff</code> + smoke tests
Auto mode long session	Diff review + targeted tests + hook gates
Headless / scheduled / bypass	Mandatory automated tests, deny rules for dangerous tools, human review of high-risk paths

Do not trust Claude’s claim that “tests passed.” Execute the runner in a hook or CI step.

Verification skill pattern: Package “run unit + lint + typecheck. Print summary. Non-zero exit on failure” as a skill. Also wire the same commands into a Stop/PreToolUse gate for unattended runs.

Risk-tiered review:

- Low: formatting, comments, docs → automated checks may suffice
- Medium: app logic → tests + a quick read of the diff
-

High: auth, payments, migrations, IAM → mandatory human review regardless of green CI

8. Practical end-to-end workflow (cert checklist)

1. Initialize / map the repo (/init or equivalent).
2. Feed relevant files / point at the subsystem before you ask for a feature.
3. Plan (no code) then approve then implement.
4. Prefer test-aware loops when quality matters.
5. Encode durable rules in CLAUDE.md. Procedures in skills. Hard stops in hooks.
6. Choose permission mode for the job risk.
7. Automate only after verification is reliable. Share via plugin.
8. If drift: rewind, then one goal, then continue.
9. Before you ship plugin: prove hooks fail closed on a deliberate red test.

Session hygiene habits:

- One goal per period of autonomous work
- Commit or stash before large autonomous runs so rewind or git restore is easy
- Name branches clearly when you use worktrees
- After compaction, restate the current plan in one sentence

9. Anti-patterns (high-yield exam recognition)

Anti-pattern	Why it fails	Prefer
Safety only in chat or CLAUDE.md	Advisory. Ignored under pressure	Deny plus PreToolUse hook
Bypass on laptop with prod creds	Huge scope of harm	Sandbox CI plus least privilege
Automate before tests exist	Unsupervised bad output at scale	Write gates first
Two agents, one dirty branch	Merge chaos or lost work	Worktrees
Trust model claim that tests passed	Hallucinated verification	Hook executes runner
Very long text in CLAUDE.md	Too much text. Claude does not follow it fully	Skills plus short CLAUDE.md
Compact instead of rewind after contradiction	A bad summary keeps the confusion	Rewind
Plan mode forever	The work never ships	Approve plan then implement mode
Plugin with bypass default	Spreads unsafe defaults	Least privilege defaults
Ignoring Stop-hook failures	Broken code shown as correct	Fail closed

10. Comparison: control surfaces deep dive

Think of Claude Code configuration as layered defense:

[Chat instructions] advisory, ephemeral
|
[CLAUDE.md / rules] advisory, durable
|
[Skills] advisory, on-demand depth
|
[Permission modes/rules] enforced allow/ask/deny
|
[Hooks] enforced lifecycle guarantees
|
[CI / human review] external verification

Exams often use scenarios where you select a higher layer than needed. You must have Example: you add a very long document to CLAUDE.md when a deny rule. Exams also use scenarios where you pick a lower layer than needed. Example: you rely on chat alone to stop force-push.

Worked scenario A — secret file protection

- Wrong: ask in chat not to touch secret files
- Better: CLAUDE.md note, still not enough alone
- Right: deny rule for edit/write on secret env files and a PreToolUse hook blocking those paths

Worked scenario B — weekly dependency bump

- Wrong: manually re-prompt every Monday with bypass on laptop
- Right: verified skill plus routine or headless CI job plus tests plus PR for human merge

Worked scenario C — unfamiliar payment refactor

- Wrong: auto mode overnight
- Right: plan mode, design review, accept-edits with tight scope, tests, human review of payment paths

11. Exam tips

- Map scenarios to steer / configure / automate / verify.
- Identify advisory vs enforced controls every time.
- Prefer plan mode for unfamiliar or multi-file work.
- Prefer worktrees for parallel agents.
- Prefer routines until you truly need headless or SDK.
- Compaction without CLAUDE.md loses conventions. That failure mode is testable.
- Bypass without isolation is the wrong safety answer.
- Hooks can block even when modes are permissive. Know the evaluation order.
- Plugins package trust. They do not replace verification.
-

Headless without hooks is an incomplete answer for unattended safety.

12. Self-check Q&A (with answers)

Q1. CLAUDE.md says never commit secrets, but Claude Code still stages a secret env file. What stops it? **A1.** Deny permission rules and/or a PreToolUse hook — not the markdown sentence alone.

Q2. When is plan mode the right default? **A2.** Before large or unclear edits: explore and propose without changing source until approval.

Q3. Session is confused after many failed attempts. Next step? **A3.** Rewind (or carefully compact), then one clear goal. Avoid contradictory stacked instructions.

Q4. Same prompt every Monday. First automation choice? **A4.** A routine. Change to headless if you need local env or custom piping.

Q5. Why CLAUDE.md instead of only chat? **A5.** Claude Code compacts chat content away. CLAUDE.md is the durable project instruction surface.

Q6. Two agents on one branch — safer pattern? **A6.** Isolate with git worktrees / separate checkouts.

Q7. What does headless -p provide? **A7.** Non-interactive, scriptable runs for pipelines and pipes.

Q8. How much verification for fully unattended runs? **A8.** Maximum: test gates, deny rules for dangerous tools, and review policy. Match the policy to zero human watching.

Q9. Skill vs CLAUDE.md for a 20-step release checklist? **A9.** Skill (on-demand procedure). Keep CLAUDE.md lean with a pointer.

Q10. Accept-edits vs bypass for trusted local feature work on a developer machine? **A10.** Accept-edits (or auto with classifier). Bypass belongs in isolated sandboxes/CI, not default laptop use.

Q11. What should a PreCompact hook often do? **A11.** Re-inject critical constraints so compaction summaries do not drop rules you cannot skip.

Q12. Plugin vs skill? **A12.** Skill = one packaged procedure/knowledge unit. Plugin = installable bundle of skills, hooks, config (portability layer).

Q13. Auto mode still does something dangerous. Your next hardening step? **A13.** Add an explicit deny rule and/or PreToolUse hook for that action class. Do not rely on the classifier alone for rules you cannot skip.

Q14. When choose Agent SDK over headless CLI? **A14.** When the coding agent is embedded inside your product TypeScript/Python application. Do not use it only as a call from a shell pipeline.

Q15. GitHub Action runs Claude on every PR. What must exist first? **A15.** Proportional verification: real tests, scoped permissions, and review of high-risk paths. Automation without gates is incomplete.

Q16. Evaluation order if allow and deny both match a tool path? **A16.** Deny wins (first-match mental model: deny before allow).

Q17. Why might hooks still matter in bypass mode? **A17.** PreToolUse hooks can deny regardless of permission mode. They enforce policy. Users cannot skip this policy by flipping mode.

Q18. Parallel agents editing the same file — correct? **A18.** No. Split independent work. Isolate trees. Merge with intent.

Q19. After /init, CLAUDE.md is huge and generic. What do you do? **A19.** Trim to always-true facts. Move procedures to skills. Add only conventions that Claude cannot find from code.

Q20. Unattended job needs private package registry credentials. Routine or headless? **A20.** Headless (or your CI), where the secrets and network are located. Do not use a cloud routine alone unless that infra has the secrets.

Q21. What is the ladder order for building team Claude Code maturity? **A21.** Steer, then configure, then automate, then verify/share.

Q22. Spot-check diffs enough for scheduled overnight refactors? **A22.** No. Require automated tests plus hooks plus human review of high-risk diffs.

Q23. Where do irreversible must-never rules belong? **A23.** Permission deny rules and hooks — not only CLAUDE.md.

Q24. Directed compaction should prioritize keeping what? **A24.** Approved plan, open decisions, constraints, and failing tests. Drop noisy output of failed experiments.

Q25. Why is a verification skill still valuable if CI already runs tests? **A25.** Local interactive loops catch failures earlier. The same commands should also gate unattended Stop/PreToolUse hooks for consistency. —

13. Review checklist (before exam)

- ☐ Can explain plan / compact / rewind and when to use each
 - ☐ Can sort CLAUDE.md vs skills vs hooks vs permissions by advisory/enforced
 - ☐ Can select permission mode for interactive, trusted local, CI, sandbox
 - ☐ Can choose routine vs headless vs Agent SDK
 - ☐ Can describe worktree isolation for parallel agents
 - ☐ Can state verification proportional to supervision
 - ☐ Can list five anti-patterns and the fix for each
 - ☐ Can describe what a plugin packages and why teams use it
 - ☐ Know hooks override permission mode for hard policy
 - ☐ Know compaction loses chat-only conventions
 - ☐ Can go through the steer → configure → automate → verify ladder
 - ☐ Can design a fail-closed test gate for unattended runs
-

14. Glossary

- **Plan mode** — research/propose without editing source until approval.
- **Compaction** — summarize older context to free window space.
- **Directed compaction** — steer what the summary must keep/drop.
- **Rewind** — restore earlier session state after drift.
- **CLAUDE.md** — persistent project memory loaded each session.
- **Rules** — modular / path-scoped instruction files (when used).
- **Skill** — on-demand packaged procedure or domain knowledge.
- **Hook** — enforced lifecycle script (allow/deny/ask/side effects).
- **Permission mode** — baseline for tool approval behavior.
- **Allow / ask / deny rules** — explicit tool permission policy.
- **Routine** — scheduled managed run on Anthropic infrastructure.
- **Headless** — print/one-shot CLI (-p) for pipelines.
- **Agent SDK** — embed Claude Code loops in your application.

- **Plugin** — shareable Claude Code setup bundle.
 - **Worktree** — separate Git working tree for isolated parallel agents.
 - **Verification gate** — real test/lint command that fails closed.
 - **Advisory vs enforced** — prompt/CLAUDE.md advise. Permissions/hooks enforce.
 - **Goal and loop** — autonomous style: clear goal, iterate until checks pass.
 - **Classifier auto mode** — model-assisted approval of routine actions with risk blocks.
 - **Fail closed** — if a safety check cannot run, treat as failure/deny.
-

15. Extra depth: subagents, MCP, and team rollout

Subagents (conceptual)

Claude Code can spawn specialized subagents for isolated work (explore, plan, or custom). Isolation prevents the main session from growing too large. For exams: use subagents when the subtask does not need the full conversation. Keep dangerous tools constrained. Remember: background subagents in non-interactive mode cannot show interactive prompts. Hooks and denies decide.

MCP inside Claude Code

MCP servers extend tools/resources/prompts available to the coding agent. Treat MCP tools like any other tools: permission rules and hooks still apply. Do not assume an MCP server is safe because it is “just config.” It can expose powerful side effects.

Team rollout sequence (practical)

1. One repo pilot: lean CLAUDE.md + verification skill + deny secrets
2. Add PreToolUse hooks for protected paths and test gates
3. Standardize permission defaults for interactive vs CI
4. Package as plugin. Install on second repo
5. Add routine/headless for one routine weekly task
6. Measure: fewer permission prompts that are noise, fewer incidents, faster PR checks
7. Only then expand autonomy (auto mode, more routines)

Metrics that matter (not empty metrics)

- Time from issue → green PR
- Rate of hook denials that prevented real mistakes
- Flake rate of verification gates
- Onboarding time for a new engineer with the plugin
- Incidents involving unsupervised edits

Study rhythm

Day 1: sections 1–5 (steer + configure). Flashcard advisory vs enforced. Day 2: sections 6–10 (automate + verify + anti-patterns). Speak the decision trees aloud. Day 3: all Q&A closed-book, then checklist. Re-read only missed items.

16. Reading tip

Each major heading is a flashcard front. The tables are the backs. Quiz yourself on advisory vs enforced. Then quiz on routines vs headless. Then quiz on plan/compact/rewind. Do this before you open the Academy outline again.

Aligned to public outline at <https://academy.claude.com/courses/claude-code-in-action>. Use for recall. Complete the official course for quizzes. Confirm live CLI details in Claude Code documentation.

17. Quick reference card (print-friendly)

Need	Reach for
Unfamiliar multi-file change	Plan mode
Context full, direction still good	Directed / compact
Contradictory / mixed session	Rewind
Always-true project facts	Lean CLAUDE.md
Occasional multi-step procedure	Skill
Must never happen	Deny rule + hook
Weekly routine job, little custom env	Routine
Needs your secrets/network/scripts	Headless -p
Agent inside your product	Agent SDK
Team-wide portable setup	Plugin
Parallel independent tasks	Worktrees
Unsupervised trust	Real test gates + human high-risk review

One-sentence exam rule: Advise with CLAUDE.md and skills. Enforce with permissions and hooks. Automate only after verification matches how little you watch.

Common distractors on multiple choice:

- “Add it to the prompt again” when the real fix is a hook
- “Use bypass for speed” when the environment is a developer laptop
- “Compact” when the session needs rewind
- You must have “One shared branch for two agents” when worktrees.
- “Claude said tests passed” when CI/hooks must execute the runner

CCAR-F Domain 3 mechanics supplement (added 2026-08-23)

Scope: the concrete configuration mechanics that the **CCAR-F exam guide Domain 3 (Claude Code Configuration & Workflows, 20%)** tests. The course notes above cover these only conceptually. Task statements: 3.1 (CLAUDE.md hierarchy), 3.2 (commands & skills), 3.3 (path-specific rules), 3.4 (plan mode / Explore), 3.6 (CI/CD flags). The author verified all file paths and frontmatter keys below against current Claude Code docs (2026-08). This is original synthesis. Volatile details are marked as possibly changing.

S1. CLAUDE.md hierarchy, @import, and /memory (task 3.1)

Three levels the exam names:

Level	Location	Shared?
User	~/ .claude/CLAUDE.md	No — personal, applies to all your projects, not in version control
Project	./CLAUDE.md (root) or ./ .claude/CLAUDE.md	Yes — via version control, whole team
Directory	CLAUDE.md in a subdirectory	Yes — loaded on demand when Claude works with files in that directory

Files **concatenate** (broader scope first, closer-to-work read last). They do not override each other. *(Current docs add a managed/org policy level above user. It is org-wide. You cannot exclude it. Current docs also add a gitignored CLAUDE.local.md for personal per-project notes. Know that these exist.)*

The diagnostic scenario that appears often on the exam: a new team member’s Claude ignores “the team’s” instructions. The instructions are in someone’s user-level ~/ .claude/CLAUDE.md instead of **project-level** config. Version control never delivers them to teammates.

@import syntax: reference external files from CLAUDE.md with @path/to/file. Claude Code expands the imported file into context at launch. Use it to keep CLAUDE.md modular. Example: each package’s CLAUDE.md imports only the standards files that are relevant to it. (Details verified: relative paths resolve against the *importing file*. Imports recurse to a maximum depth of **four hops** (per current official docs, verified 2026-08-23). Wrap a path in backticks to *mention* it without importing.)

/memory lists the memory files (CLAUDE.md and related memory files) across scopes and lets you open/edit them. Load Use it to **verify which memory files and diagnose inconsistent behavior across sessions**. *(Current docs: /context additionally confirms what actually loaded into this session.)*

S2. Custom slash commands and skills (task 3.2)

Commands — the scope rule (official sample Q4 depends on it):

Location	Scope
.claude/commands/ in the repo	Project — version-controlled, automatically available to every developer on clone/pull
~/ .claude/commands/	User — personal, not shared

A markdown file .claude/commands/review.md defines /review. Team-standard workflows → project scope. Personal helpers → user scope. *(Current docs note: Current docs merge custom commands into skills. .claude/commands/deploy.md and .claude/skills/deploy/SKILL.md both create /deploy. Existing commands keep working. The exam-facing answer for “team-shared /review command” remains .claude/commands/ in the project repository.)*

Skills are located at `.claude/skills/<name>/SKILL.md` (project) or `~/.claude/skills/` (personal). The frontmatter keys the exam names, verified current:

Frontmatter key	What it does
context: fork	Runs the skill in an isolated sub-agent context . This keeps verbose skill output (codebase analysis, brainstorming) from filling the main conversation
allowed-tools	Tools Claude may use without permission prompts during the skill's invoking turn . Restrict to what the skill needs (e.g. file-write-only to prevent destructive actions)
argument-hint	Autocomplete hint that shows expected arguments (e.g. <code>[issue-number]</code>) when a developer invokes the skill without them

Personal variants without team impact: create your own version under `~/.claude/skills/` **with a different name**. Leave the team's project skill untouched.

Skills vs CLAUDE.md (the exam's dividing line): skills = **on-demand invocation for task-specific workflows** (body loads only when used). CLAUDE.md = **always-loaded universal standards**. A CLAUDE.md section that becomes a procedure should move to a skill.

S3. Path-specific rules — `.claude/rules/` (task 3.3)

Rule files are markdown in `.claude/rules/`. The YAML frontmatter `paths:` field holds glob patterns:

```
---
paths:
- "src/api/**/*.ts"
- "**/*.test.tsx"
---
```

API + test conventions

- All endpoints validate input
- Tests use the team fixture factory

- A rule **loads only when Claude works with files that match its globs**. Irrelevant conventions stay out of context. This saves tokens. (Rules *without* paths load every session, like CLAUDE.md.)
- **Globs are better than directory-level CLAUDE.md when a convention spans directories** (official sample Q6). Test files sit beside sources throughout the tree, so `**/*.test.tsx` catches them all. Per-directory CLAUDE.md files cannot do this. Root-CLAUDE.md-with-headers relies on inference. Skills require invocation. Rules-with-globs is the deterministic, automatic answer.
- Example from the guide's own exercises: `paths: ["terraform/**/*.ts"]` so Terraform conventions load only when you edit Terraform.

S4. Plan mode, direct execution, and the Explore subagent (task 3.4)

Named decision rule (§2.1 above covers the workflow. The exam wants the *criteria* as written):

- **Plan mode** — use it for complex tasks: large-scale changes, **multiple valid approaches**, architectural decisions, multi-file modifications (monolith→microservices. A 45-file library migration). Safe exploration and design before you commit to changes prevent costly rework (sample Q5’s correct A).
- **Direct execution** — simple, well-scoped changes: single-file bug fix with a clear stack trace. Adding one validation conditional.
- **Combine them:** plan mode for investigation, then direct execution for the planned implementation.
- **The Explore subagent** isolates **verbose discovery output** and returns summaries. It **preserves main-conversation context** during multi-phase work. Name it when a question describes context exhaustion during codebase discovery. (Ties to Domain 5.4 subagent delegation.)

S4b. Iterative refinement techniques (task 3.5)

- **Concrete input/output examples are better than prose** when the model interprets natural language descriptions of a transformation in different ways. Provide **2–3 example pairs** showing input and expected output.
- **Test-driven iteration:** write the test suite first (expected behavior, edge cases, performance). Then iterate: **share test failures** to guide progressive improvement. Specific failing cases with example input/expected output fix edge-case handling (e.g. null values in migration scripts).
- **The interview pattern:** in an unfamiliar domain, have Claude **ask you questions first**. This shows considerations you did not anticipate (cache invalidation strategy, failure modes) *before* you implement.
- **Interacting vs independent issues:** fixes that **interact** → one single detailed message that addresses all of them together. **Independent** issues → sequential iteration is fine.

S5. CI/CD integration flags (task 3.6)

Mechanic	Exam fact
<code>-p / --print</code>	Runs Claude Code non-interactively : processes the prompt, writes to stdout, exits. This is the fix when a pipeline job “hangs waiting for interactive input” (sample Q10. Distractors like <code>CLAUDE_HEADLESS=true</code> or <code>--batch</code> are features that do not exist)
<code>--output-format json</code>	Machine-parseable output in print mode (text, json, stream-json)
<code>--json-schema</code>	Print-mode flag that produces validated JSON which matches your schema. Structured findings are ready for automated posting as PR comments
CLAUDE.md in CI	The mechanism for giving CI-invoked Claude project context: testing standards, fixture conventions, review criteria. Result: better test generation, fewer low-value cases

Mechanic	Exam fact
Session context isolation	The session that generated code is less effective at reviewing it. It retains its own reasoning context and will not question its decisions. Run review as an independent instance (ties to Domain 4.6 multi-instance review)
Re-review without duplicates	Include prior review findings in context when you re-run after new commits. Instruct Claude to report only new or still-unaddressed issues
Test generation without duplicates	Provide existing test files in context so generated tests skip already-covered scenarios

S6. Supplement Q&A

SQ1. (= official sample Q4) A /review command must reach every developer on clone/pull. Where does the file go? **A1.** `.claude/commands/` in the project repository — version-controlled project scope. `~/.claude/commands/` is personal. `CLAUDE.md` holds instructions, not command definitions. `.claude/config.json` with a `commands` array does not exist.

SQ2. (= official sample Q6) React, API, and DB conventions differ. Test files sit beside sources everywhere and share one convention. Most maintainable mechanism? **A2.** `.claude/rules/` files with YAML frontmatter glob patterns (`**/*.test.tsx`) — conditional, automatic, location-independent. Root-`CLAUDE.md` headers rely on inference. Per-directory `CLAUDE.md` cannot span the tree. Skills need invocation.

SQ3. New teammate does not get “the team’s” instructions. Likely cause? **A3.** Instructions are in user-level `~/.claude/CLAUDE.md` (not shared via version control) instead of project-level `CLAUDE.md/.claude/`.

SQ4. How do you keep `CLAUDE.md` modular per package? **A4.** `@import` syntax (`@docs/standards.md`). Each package’s `CLAUDE.md` imports only relevant standards files.

Load **SQ5.** Which command verifies which memory files? **A5.** `/memory` (and `/context` confirms what loaded this session).

SQ6. A codebase-analysis skill fills the main conversation with output. Frontmatter fix? **A6.** `context: fork` — runs the skill in an isolated sub-agent context.

SQ7. Restrict a skill to file writes to prevent destructive actions — which key? **A7.** `allowed-tools` in `SKILL.md` frontmatter (grants scoped, prompt-free tool use for the invoking turn).

SQ8. Developers invoke `/deploy` without required parameters. Which key prompts them? **A8.** `argument-hint` (e.g. `[environment] [version]`).

SQ9. You want a personal variant of a team skill without affecting teammates. **A9.** Create it in `~/.claude/skills/` under a **different name**.

SQ10. Skill or `CLAUDE.md`: a 20-step release procedure vs “we use 2-space indent”? **A10.** Procedure → skill (on-demand). Universal standard → `CLAUDE.md` (always loaded).

SQ11. When do `paths:-`scoped rules load? **A11.** Only when Claude works with files matching the globs. Rules without `paths` load every session.

SQ12. Monolith→microservices restructuring: plan mode or direct? **A12.** Plan mode — large-scale, multiple valid approaches, architectural decisions (sample Q5). Single-file bug fix with a stack trace → direct execution.

SQ13. Multi-phase task exhausts context during discovery. Named remedy? **A13.** The Explore subagent — verbose discovery isolated, summaries returned.

SQ14. CI job hangs: “Claude Code is waiting for interactive input.” Fix? **A14.** `claude -p "..."` — the `-p/--print` flag is the documented non-interactive mode (sample Q10. `CLAUDE_HEADLESS/--batch` do not exist).

SQ15. Structured, machine-postable review findings from CI — which flags? **A15.** `--output-format json` with `--json-schema` (print mode).

SQ16. Why should the session that wrote the code not review it? **A16.** Session context isolation: it retains generation reasoning and will not question its own decisions. Use an independent review instance.

SQ17. Re-running review after new commits posts the same comments again. Fix? **A17.** Include prior findings in context and instruct: report only new or still-unaddressed issues.

SQ18. Generated tests duplicate existing coverage. Fix? **A18.** Provide existing test files in context so generation avoids covered scenarios.

S7. Supplement checklist

- ☐ I can place an instruction at user / project / directory level and predict who receives it.
- ☐ I can use `@import` and explain what `/memory` shows.
- ☐ I know `.claude/commands/` vs `~/ .claude/commands/` without notes (sample Q4).
- ☐ I can write a `.claude/rules/` file with paths: globs and argue it over per-directory `CLAUDE.md` (sample Q6).
- ☐ I can recite context: `fork`, `allowed-tools`, `argument-hint`.
- ☐ I can classify plan mode vs direct execution and name the Explore subagent.
- ☐ I know `-p`, `--output-format json`, `--json-schema`, and the three CI context tricks (`CLAUDE.md`, prior findings, existing tests).

Claude Code in Action — Primary Study Notes

Disclaimer: These are **original** exam-prep notes for Claude certification study. They align to the **public** course outline at [Claude Code in Action](#) (objectives and module themes only). They are **not** Claude Academy lesson dumps, transcripts, or verbatim slide text. Completing the official course remains the source of truth for quizzes. Verify CLI flag and UI labels in current Claude Code docs — names evolve.

Who it is for: Developers who already use Claude Code for single prompts and want longer, less supervised, team-wide workflows.

Prerequisites (course): Claude Code for one-shot tasks; basic Git and command line.

How to use these notes: Read one major section, then answer the self-check without scrolling back. Treat tables as flashcards. The ladder **steer** → **configure** → **automate** → **verify/share** is the spine of most scenario questions.

1. Course map (public objectives)

The public page frames four outcome clusters (~9 lessons, 1 quiz):

1. **Steer the work** — plan mode, directed compaction, rewind, hands-on vs autonomous goal/loop runs, safe parallelism.
2. **Configure Claude** — lean CLAUDE.md, skills, permission modes, hooks for rules that cannot be skipped.
3. **Automate repeat work** — routines on Anthropic infrastructure, headless mode in your pipeline, PR / GitHub Action wiring.
4. **Verify and share** — verify unsupervised runs in proportion to how little you watched them; gate on real tests; package a trusted setup as a plugin.

Mental model: Claude Code is an agentic coding loop with tools (file edit, bash, search, etc.). Your job on exams is not “write a longer prompt,” but **choose the right control surface** for the risk of the job.

Layer	Question it answers	Strength
Steering	What should we do <i>this session</i> ?	Ephemeral control
Configuration	What should Claude <i>always</i> know / be allowed?	Persistent
Automation	How does this run without me typing?	Operational
Verification / plugins	How do we trust and reuse the setup?	Assurance + portability

Remember: many exam-style questions ask which layer **advises** vs **enforces**.

2. Steering long sessions

2.1 Plan mode

For non-trivial changes, start in **plan mode**. Claude may read the repo and run exploratory commands, but it should **not edit source** until you approve a plan. Use it when:

- The codebase is unfamiliar
- The change touches many files or modules
- You need architectural decisions before code
- Blast radius includes migrations, auth, billing, or deploy scripts

Cycle modes in the CLI (commonly **Shift+Tab**). Learn the *intent* of each mode more than any single UI string — labels can change; the spectrum from “read-only exploration” to “auto-approve edits” to “bypass prompts” does not.

Exam framing: Plan mode is a *steering* choice, not a permanent project setting. After you approve a plan, you typically switch to a mode that allows implementation under permissions you trust.

2.2 Compaction

Long sessions fill the context window. Claude Code **compacts** older turns (and you can trigger compaction manually in many setups, e.g. /compact). Summaries drop detail.

What survives compaction poorly: one-off chat rules (“always use pytest”), temporary naming decisions, failed experiment paths that you still need as negative examples.

What should survive by design:

- Durable conventions → **CLAUDE.md** / `.claude/rules/`
- On-demand procedures → **skills**
- Hard stops → **hooks** / deny rules

When you *direct* compaction (keep X, drop Y), prefer keeping: approved plan, open decisions, constraints, failing test names. Drop: verbose tool dumps, abandoned rabbit holes, duplicate file listings.

Exam trap: “Compaction lost our coding standards” → root cause is standards living only in chat, not in CLAUDE.md.

2.3 Rewind

When a session drifts, **rewind** to an earlier clean point instead of stacking contradictory fixes on a polluted context. Then restate **one** clear goal.

Rewind vs compact:

Action	Best when	Risk if misused
Rewind	Wrong path / contradictory instructions	Lose useful later work if you rewind too far
Compact	Context full but direction still good	Lose detail; may reinforce bad summaries if chat was messy

Pattern: Drift → rewind → single goal → (optional) plan mode again → implement.

2.4 Hands-on vs autonomous

Style	Use when	Control
Hands-on steering	Novel / high blast radius	You approve edits and watch turns
Goal + loop / autonomous	Trusted, well-scoped tasks	Hooks, tests, tighter permission mode
Parallel agents	Independent subtasks	Git worktrees (or separate checkouts)

Exam nugget: Parallel agents sharing one dirty working tree is the wrong pattern; isolation via worktrees is the safe answer.

2.5 Steering vs autonomy decision tree

Is the task novel or high blast radius?

YES → Hands-on + plan mode; default/manual or accept-edits only after reviewed plan

NO → Is the procedure repeated and already verified?

YES → Skill + hooks; consider routine

NO → Still interactive; encode learnings into CLAUDE.md as you go

Does it need your laptop secrets, private network, or custom piping?

YES → Headless in YOUR CI/agent host

NO → Routine on Anthropic infra may suffice

Must results be identical run-to-run in CI?

YES → Bare/deterministic headless flags + pinned permissions

NO → Interactive or auto modes OK with verification

Multiple agents useful?

YES → Split independent work; isolate with worktrees

NO → Single agent, clear goal

Session went sideways?

→ Rewind first; only then re-prompt

3. Configuration surfaces (advisory vs enforced)

Surface	Nature	Strength
Chat prompt	Temporary	Advisory
CLAUDE.md / rules	Always-on project context	Advisory but persistent
Skills	On-demand procedures	Advisory when loaded
Permission rules / modes	Tool allow / ask / deny	Enforced by Claude Code
Hooks	Lifecycle scripts	Enforced guarantees

Core distinction: “Never edit .env” in CLAUDE.md is a polite request. A **deny** rule or **PreToolUse hook** that blocks .env is a guarantee. Instructions shape intent; rules and hooks change what is allowed.

3.1 Lean CLAUDE.md

Keep it short enough that Claude follows it. Public guidance and good exam answers converge on:

- Project purpose, stack, layout (few bullets)
- Commands that matter (test, lint, build)
- Hard do/don't for secrets, migrations, force-push
- Pointers to skills/docs for long procedures — do **not** paste a novel

Put each rule on the right surface:

Content type	Surface
Always-true facts and conventions	CLAUDE.md
Path-scoped conventions	.claude/rules/ (when available)
Multi-step / occasional workflows	Skills
Must-run-at-event guarantees	Hooks
Tool allow/deny	Permission rules

`/init` (or equivalent) maps the repo and seeds CLAUDE.md; refine with what Claude cannot discover from code alone.

Anti-pattern: A 2,000-line CLAUDE.md that duplicates the README, the wiki, and every coding standard PDF. Claude will under-follow it; exams reward *lean* memory.

CLAUDE.md maintenance loop: After a painful session, ask: “What should never be rediscovered from scratch?” Promote that fact into CLAUDE.md. If it is a procedure, promote to a skill instead.

3.2 Skills

Skills package repeated procedures (start with a verification skill that runs real checks). Typical skill contents conceptually:

- When to apply
- Steps / commands
- Success criteria
- Optional allowed-tools grants for the invoking turn (still subject to baseline permissions)

Skills load **on demand** (invoked or auto-selected when relevant). That is why they beat stuffing every procedure into CLAUDE.md.

Exam contrast:

- CLAUDE.md = always in context → keep short
- Skill = load when needed → detailed OK
- Hook = runs regardless of Claude’s “opinion” → use for non-negotiables

Good first skills for a team: “run verification suite,” “create PR with checklist,” “migrate DB safely,” “security review touchpoints.”

3.3 Plugins

Plugins bundle CLAUDE.md pieces + skills + hooks + related config (and sometimes MCP / subagents) so the whole team installs one trusted setup. Plugins are about **portability**, not new model capabilities.

Share when: the setup is verified, hooks gate real tests, permissions are least-privilege, and onboarding cost drops from “read a wiki” to “install plugin.”

Plugin hygiene checklist:

1. Document required env vars and secrets injection points
2. Fail closed if tests cannot run
3. Namespace skills so multiple plugins coexist

4. Version the plugin; break changes need a changelog
- 5.

Never ship bypass-by-default in a team plugin

4. Permission modes (match mode to risk)

Modes set the baseline before allow/ask/deny rules. Names evolve; know the spectrum:

Mode (conceptual)	Behavior	Typical job
Default / manual	Prompt on first tool use	Interactive pairing
Accept edits	Auto-accept common workspace edits	Trusted local feature work
Plan	Explore; no source edits until plan approved	Design before implement
Auto (classifier)	Approves routine actions; blocks risky ones	Faster interactive with guardrails
Don't ask	Strict; often denies unless pre-allowed	Controlled non-interactive
Bypass / don't ask risky variant	Skips most prompts	Isolated CI / sandbox only

Evaluation order (mental model): deny → ask → allow; first match wins. Deny beats allow. A PreToolUse hook that exits with a blocking decision can still stop a call even when an allow rule or bypass mode would otherwise pass — hooks are the last hard gate in many designs.

Permission decision tree:

```
Interactive pairing / learning codebase?
→ Default/manual or plan
Trusted local feature after reviewed plan?
→ Accept edits (or auto with classifier)
CI / headless with pinned allowlist?
→ Don't ask + explicit allows + hooks
Throwaway container, no secrets, ephemeral FS?
→ Bypass may be acceptable
Production credentials on laptop?
→ Never bypass; least privilege + deny rules
```

Exam traps:

- Bypass-style modes as a laptop default with production credentials → wrong
- Assuming CLAUDE.md overrides permission deny → wrong
- Using plan mode for a trusted, already-specified one-line rename → overly slow but not “unsafe”; usually accept-edits or auto is fine
- Believing auto mode replaces deny rules for .env → wrong; classifier is probabilistic, deny is absolute

Admins may disable bypass in team settings — know that organizational policy can remove the escape hatch.

5. Hooks: unskippable control

Hooks fire at fixed lifecycle points. Examples worth memorizing (exact event names may evolve — learn the *jobs*):

Lifecycle job	Example use
Session start	Inject env facts, repo branch warnings
Pre tool use	Block <code>.env</code> , force ask on <code>rm -rf</code> , deny production deploys
Permission request	Auto-decide in non-interactive runs
Pre compact	Re-inject critical constraints into summary
Stop / after turn	Run linters; fail closed if tests red
File / rules loaded	Audit what entered context

Patterns:

1. Block protected paths (`.env`, production configs, private keys)
2. Gate a turn on **real** test runners (fail closed if red)
3. Force a prompt for sensitive Bash even if Bash is broadly allowed
4. Re-inject critical context after compaction
5. Return permission-decision JSON / exit codes to allow, deny, or ask

Hooks enforce; prompts advise.

Non-interactive note: In headless `-p` runs, interactive permission prompts may not exist. Rely on `PreToolUse` hooks and pre-declared allow/deny rules — do not assume a human will click “allow.”

Hook design tips for exams:

- Prefer fail-closed for safety checks (missing test runner = deny)
- Keep hooks fast; slow hooks frustrate interactive use and get disabled
- Log denials with enough detail for audit without dumping secrets
- Do not put irreversible destructive capabilities behind soft asks alone

6. Automation spectrum

Once a task is trusted, stop kicking it off by hand:

1. **Routines** — schedule prompts on Anthropic-managed infrastructure (least ops).
2. **Headless** — `-p` / `--print`: one-shot, no TUI; stdin/stdout for pipes and CI.
3. **Bare / deterministic flags** — prefer when CI needs repeatable, low-variance runs.
4. **Agent SDK** — embed Claude Code inside your TypeScript/Python product.
5. **PR / GitHub Action wiring** — managed review patterns; Claude in the pull-request loop.

Decision rule: start with routines; drop to headless when the job needs *your* environment or surround-script logic; use the Agent SDK when the agent *is* part of your product.

Need Anthropic-hosted schedule, little custom env?

→ Routine

Need local secrets, Docker, private packages, custom scripts?

→ Headless in your runner

Need agent embedded in product UX?

→ Agent SDK

Need PR comments / checks?

→ GitHub Action / managed review integration

Automation without proportional verification is incomplete — that sentence is an exam answer by itself.

Headless specifics worth remembering:

- Non-interactive means no human in the permission loop
- Pipe prompts via stdin or args; capture stdout for logs
- Pair with explicit permission allowlists and PreToolUse hooks
- Prefer deterministic settings when CI flakes would be costly

7. Verify unsupervised work

The less you watched, the more you verify.

How unsupervised	Minimum verification
Interactive pair	Spot-check diffs
Accept-edits local run	<code>git diff</code> + smoke tests
Auto mode long session	Diff review + targeted tests + hook gates
Headless / scheduled / bypass	Mandatory automated tests, deny rules for dangerous tools, human review of high-risk paths

Never trust Claude's claim that "tests passed" — execute the runner in a hook or CI step.

Verification skill pattern: Package "run unit + lint + typecheck; print summary; non-zero exit on failure" as a skill, and also wire the same commands into a Stop/PreToolUse gate for unattended runs.

Risk-tiered review:

- Low: formatting, comments, docs → automated checks may suffice
- Medium: app logic → tests + diff skim
-

High: auth, payments, migrations, IAM → mandatory human review regardless of green CI

8. Practical end-to-end workflow (cert checklist)

1. Initialize / map the repo (/init or equivalent).
2. Feed relevant files / point at the subsystem before asking for a feature.
3. Plan (no code) then approve then implement.
4. Prefer test-aware loops when quality matters.
5. Encode durable rules in CLAUDE.md; procedures in skills; hard stops in hooks.
6. Choose permission mode for the job risk.
7. Automate only after verification is solid; share via plugin.
8. If drift: rewind, then one goal, then continue.

- Before shipping plugin: prove hooks fail closed on a deliberate red test.

Session hygiene habits:

- One goal per stretch of autonomous work
- Commit or stash before large autonomous runs so rewind or git restore is easy
- Name branches clearly when using worktrees
- After compaction, restate the current plan in one sentence

9. Anti-patterns (high-yield exam recognition)

Anti-pattern	Why it fails	Prefer
Safety only in chat or CLAUDE.md	Advisory; ignored under pressure	Deny plus PreToolUse hook
Bypass on laptop with prod creds	Huge blast radius	Sandbox CI plus least privilege
Automate before tests exist	Unsupervised garbage at scale	Write gates first
Two agents, one dirty branch	Merge chaos or lost work	Worktrees
Trust model claim that tests passed	Hallucinated verification	Hook executes runner
Novel in CLAUDE.md	Dilution; under-followed	Skills plus short CLAUDE.md
Compact instead of rewind after contradiction	Bad summary locks in confusion	Rewind
Plan mode forever	Never ships	Approve plan then implement mode
Plugin with bypass default	Spreads unsafe defaults	Least privilege defaults
Ignoring Stop-hook failures	Greenwashed broken code	Fail closed

10. Comparison: control surfaces deep dive

Think of Claude Code configuration as layered defense:

```
[Chat instructions] advisory, ephemeral
|
[CLAUDE.md / rules] advisory, durable
|
[Skills] advisory, on-demand depth
|
[Permission modes/rules] enforced allow/ask/deny
|
[Hooks] enforced lifecycle guarantees
|
[CI / human review] external verification
```

Exams love scenarios where the student picks a higher layer than needed (for example, adding a novel to CLAUDE.md when a deny rule was required) or a lower layer than needed (hoping chat alone stops force-push).

Worked scenario A — secret file protection

- Wrong: ask in chat not to touch secret files
- Better: CLAUDE.md note, still not enough alone
- Right: deny rule for edit/write on secret env files and a PreToolUse hook blocking those paths

Worked scenario B — weekly dependency bump

- Wrong: manually re-prompt every Monday with bypass on laptop
- Right: verified skill plus routine or headless CI job plus tests plus PR for human merge

Worked scenario C — unfamiliar payment refactor

- Wrong: auto mode overnight
 - Right: plan mode, design review, accept-edits with tight scope, tests, human review of payment paths
-

11. Exam tips

- Map scenarios to steer / configure / automate / verify.
- Identify advisory vs enforced controls every time.
- Prefer plan mode for unfamiliar or multi-file work.
- Prefer worktrees for parallel agents.
- Prefer routines until you truly need headless or SDK.
- Compaction without CLAUDE.md loses conventions — that failure mode is testable.
- Bypass without isolation is the wrong safety answer.
- Hooks can block even when modes are permissive — know that ordering story.
- Plugins package trust; they do not replace verification.
-

Headless without hooks is an incomplete answer for unattended safety.

12. Self-check Q&A (with answers)

Q1. CLAUDE.md says never commit secrets, but a secret env file still gets staged. What stops it? **A.** Deny permission rules and/or a PreToolUse hook — not the markdown sentence alone.

Q2. When is plan mode the right default? **A.** Before large or unclear edits: explore and propose without changing source until approval.

Q3. Session is confused after many failed attempts. Next step? **A.** Rewind (or carefully compact), then one clear goal — avoid contradictory stacked instructions.

Q4. Same prompt every Monday. First automation choice? **A.** A routine; drop to headless if you need local env or custom piping.

Q5. Why CLAUDE.md instead of only chat? **A.** Chat is compacted away; CLAUDE.md is the durable project instruction surface.

- Q6.** Two agents on one branch — safer pattern? **A.** Isolate with git worktrees / separate checkouts.
- Q7.** What does headless -p provide? **A.** Non-interactive, scriptable runs for pipelines and pipes.
- Q8.** How much verification for fully unattended runs? **A.** Maximum: test gates, deny rules for dangerous tools, and review policy proportional to zero human watching.
- Q9.** Skill vs CLAUDE.md for a 20-step release checklist? **A.** Skill (on-demand procedure); keep CLAUDE.md lean with a pointer.
- Q10.** Accept-edits vs bypass for trusted local feature work on a developer machine? **A.** Accept-edits (or auto with classifier). Bypass belongs in isolated sandboxes/CI, not default laptop use.
- Q11.** What should a PreCompact hook often do? **A.** Re-inject critical constraints so compaction summaries do not drop non-negotiables.
- Q12.** Plugin vs skill? **A.** Skill = one packaged procedure/knowledge unit. Plugin = installable bundle of skills, hooks, config (portability layer). **Q13.** Auto mode still does something dangerous. Your next hardening step? **A.** Add an explicit deny rule and/or PreToolUse hook for that action class — do not rely on the classifier alone for non-negotiables.
- Q14.** When choose Agent SDK over headless CLI? **A.** When the coding agent is embedded inside your product TypeScript/Python application, not merely invoked from a shell pipeline.
- Q15.** GitHub Action runs Claude on every PR. What must exist first? **A.** Proportional verification: real tests, scoped permissions, and review of high-risk paths — automation without gates is incomplete.
- Q16.** Evaluation order if allow and deny both match a tool path? **A.** Deny wins (first-match mental model: deny before allow).
- Q17.** Why might hooks still matter in bypass mode? **A.** PreToolUse hooks can deny regardless of permission mode — they enforce policy users cannot skip by flipping mode.
- Q18.** Parallel agents editing the same file — correct? **A.** No. Split independent work; isolate trees; merge deliberately.
- Q19.** After /init, CLAUDE.md is huge and generic. What do you do? **A.** Trim to always-true facts; move procedures to skills; add only non-discoverable conventions.
- Q20.** Unattended job needs private package registry credentials. Routine or headless? **A.** Headless (or your CI) where secrets and network live — not a cloud routine alone unless that infra has the secrets.
- Q21.** What is the ladder order for building team Claude Code maturity? **A.** Steer, then configure, then automate, then verify/share.
- Q22.** Spot-check diffs enough for scheduled overnight refactors? **A.** No — require automated tests plus hooks plus human review of high-risk diffs.
- Q23.** Where do irreversible must-never rules belong? **A.** Permission deny rules and hooks — not only CLAUDE.md.
- Q24.** Directed compaction should prioritize keeping what? **A.** Approved plan, open decisions, constraints, and failing tests — drop noisy failed experiment dumps.
- Q25.** Why is a verification skill still valuable if CI already runs tests? **A.** Local interactive loops catch failures earlier; the same commands should also gate unattended Stop/PreToolUse hooks for consistency. —

13. Review checklist (before exam)

- ☐ Can explain plan / compact / rewind and when to use each
 - ☐ Can sort CLAUDE.md vs skills vs hooks vs permissions by advisory/enforced
 - ☐ Can pick permission mode for interactive, trusted local, CI, sandbox
 - ☐ Can choose routine vs headless vs Agent SDK
 - ☐ Can describe worktree isolation for parallel agents
 - ☐ Can state verification proportional to supervision
 - ☐ Can list five anti-patterns and the fix for each
 - ☐ Can describe what a plugin packages and why teams use it
 - ☐ Know hooks beat permission mode for hard policy
 - ☐ Know compaction loses chat-only conventions
 - ☐ Can walk the steer → configure → automate → verify ladder
 - ☐ Can design a fail-closed test gate for unattended runs
-

14. Glossary

- **Plan mode** — research/propose without editing source until approval.
 - **Compaction** — summarize older context to free window space.
 - **Directed compaction** — steer what the summary must keep/drop.
 - **Rewind** — restore earlier session state after drift.
 - **CLAUDE.md** — persistent project memory loaded each session.
 - **Rules** — modular / path-scoped instruction files (when used).
 - **Skill** — on-demand packaged procedure or domain knowledge.
 - **Hook** — enforced lifecycle script (allow/deny/ask/side effects).
 - **Permission mode** — baseline for tool approval behavior.
 - **Allow / ask / deny rules** — explicit tool permission policy.
 - **Routine** — scheduled managed run on Anthropic infrastructure.
 - **Headless** — print/one-shot CLI (-p) for pipelines.
 - **Agent SDK** — embed Claude Code loops in your application.
 - **Plugin** — shareable Claude Code setup bundle.
 - **Worktree** — separate Git working tree for isolated parallel agents.
 - **Verification gate** — real test/lint command that fails closed.
 - **Advisory vs enforced** — prompt/CLAUDE.md advise; permissions/hooks enforce.
 - **Goal and loop** — autonomous style: clear goal, iterate until checks pass.
 - **Classifier auto mode** — model-assisted approval of routine actions with risk blocks.
 - **Fail closed** — if a safety check cannot run, treat as failure/deny.
-

15. Extra depth: subagents, MCP, and team rollout

Subagents (conceptual)

Claude Code can spawn specialized subagents for isolated work (explore, plan, or custom). Isolation prevents bloating the main session. For exams: use subagents when the subtask does not need the full conversation; keep dangerous tools constrained; remember background subagents in non-interactive mode cannot show interactive prompts — hooks and denies decide.

MCP inside Claude Code

MCP servers extend tools/resources/prompts available to the coding agent. Treat MCP tools like any other tools: permission rules and hooks still apply. Do not assume an MCP server is safe because it is “just config” — it can expose powerful side effects.

Team rollout sequence (practical)

1. One repo pilot: lean CLAUDE.md + verification skill + deny secrets
2. Add PreToolUse hooks for protected paths and test gates
3. Standardize permission defaults for interactive vs CI
4. Package as plugin; install on second repo
5. Add routine/headless for one boring weekly task
6. Measure: fewer permission prompts that are noise, fewer incidents, faster PR checks
7. Only then expand autonomy (auto mode, more routines)

Metrics that matter (not vanity)

- Time from issue → green PR
- Rate of hook denials that prevented real mistakes
- Flake rate of verification gates
- Onboarding time for a new engineer with the plugin
- Incidents involving unsupervised edits

Study rhythm

Day 1: sections 1–5 (steer + configure). Flashcard advisory vs enforced. Day 2: sections 6–10 (automate + verify + anti-patterns). Decision trees aloud. Day 3: all Q&A closed-book, then checklist. Re-read only missed items.

16. Reading tip

Each major heading is a flashcard front; the tables are the backs. Quiz yourself on advisory vs enforced, then on routines vs headless, then on plan/compact/rewind, before re-opening the Academy outline.

Aligned to public outline at <https://academy.claude.com/courses/claude-code-in-action>. Use for recall; complete the official course for quizzes. Confirm live CLI details in Claude Code documentation.

17. Quick reference card (print-friendly)

Need	Reach for
Unfamiliar multi-file change	Plan mode
Context full, direction still good	Directed / compact
Contradictory / polluted session	Rewind

Need	Reach for
Always-true project facts	Lean CLAUDE.md
Occasional multi-step procedure	Skill
Must never happen	Deny rule + hook
Weekly boring job, little custom env	Routine
Needs your secrets/network/scripts	Headless -p
Agent inside your product	Agent SDK
Team-wide portable setup	Plugin
Parallel independent tasks	Worktrees
Unsupervised trust	Real test gates + human high-risk review

One-sentence exam mantra: Advise with CLAUDE.md and skills; enforce with permissions and hooks; automate only after verification matches how little you watch.

Common distractors on multiple choice:

- “Add it to the prompt again” when the real fix is a hook
- “Use bypass for speed” when the environment is a developer laptop
- “Compact” when the session needs rewind
- “One shared branch for two agents” when worktrees are required
- “Claude said tests passed” when CI/hooks must execute the runner

CCAR-F Domain 3 mechanics supplement (added 2026-08-23)

Scope: the concrete configuration mechanics the **CCAR-F exam guide Domain 3 (Claude Code Configuration & Workflows, 20%)** tests, which the course notes above cover only conceptually — task statements 3.1 (CLAUDE.md hierarchy), 3.2 (commands & skills), 3.3 (path-specific rules), 3.4 (plan mode / Explore), 3.6 (CI/CD flags). All file paths and frontmatter keys below verified against current Claude Code docs (2026-08); original synthesis, volatile details hedged.

S1. CLAUDE.md hierarchy, @import, and /memory (task 3.1)

Three levels the exam names:

Level	Location	Shared?
User	~/ .claude/CLAUDE.md	No — personal, applies to all your projects, not in version control
Project	./CLAUDE.md (root) or ./ .claude/CLAUDE.md	Yes — via version control, whole team
Directory	CLAUDE.md in a subdirectory	Yes — loaded on demand when Claude works with files in that directory

Files **concatenate** (broader scope first, closer-to-work read last); they do not override each other. (Current docs add a *managed/org policy level above user — org-wide, cannot be excluded — and a gitignored CLAUDE.local.md for personal per-project notes; know they exist.*)

The diagnostic scenario the exam likes: a new team member’s Claude ignores “the team’s” instructions → the instructions live in someone’s **user-level** `~/ .claude/CLAUDE.md` instead of **project-level** config, so version control never delivered them.

@import syntax: reference external files from CLAUDE.md with `@path/to/file` — the imported file is expanded into context at launch. Use it to keep CLAUDE.md modular, e.g. each package’s CLAUDE.md imports only the standards files relevant to it. (Details verified: relative paths resolve against the *importing file*; imports recurse to a maximum depth of **four hops** (per current official docs, verified 2026-08-23); wrap a path in backticks to *mention* it without importing.)

/memory lists the memory files (CLAUDE.md and friends) across scopes and lets you open/edit them — use it to **verify which memory files are loaded and diagnose inconsistent behavior across sessions**. (Current docs: */context additionally confirms what actually loaded into this session.*)

S2. Custom slash commands and skills (task 3.2)

Commands — the scope rule (official sample Q4 hinges on it):

Location	Scope
<code>.claude/commands/</code> in the repo	Project — version-controlled, automatically available to every developer on clone/pull
<code>~/ .claude/commands/</code>	User — personal, not shared

A markdown file `.claude/commands/review.md` defines `/review`. Team-standard workflows → project scope; personal helpers → user scope. (Current docs note: *custom commands have merged into skills — .claude/commands/deploy.md and .claude/skills/deploy/SKILL.md both create /deploy; existing commands keep working. The exam-facing answer for “team-shared /review command” remains .claude/commands/ in the project repository.*)

Skills live at `.claude/skills/<name>/SKILL.md` (project) or `~/ .claude/skills/` (personal). The frontmatter keys the exam names, verified current:

Frontmatter key	What it does
context: fork	Runs the skill in an isolated sub-agent context , keeping verbose skill output (codebase analysis, brainstorming) from polluting the main conversation
allowed-tools	Tools Claude may use without permission prompts during the skill’s invoking turn — restrict to what the skill needs (e.g. file-write-only to prevent destructive actions)
argument-hint	Autocomplete hint showing expected arguments (e.g. <code>[issue-number]</code>) when a developer invokes the skill without them

Personal variants without team impact: create your own version under `~/ .claude/skills/` with a different name, leaving the team's project skill untouched.

Skills vs CLAUDE.md (the exam's dividing line): skills = **on-demand invocation for task-specific workflows** (body loads only when used); CLAUDE.md = **always-loaded universal standards**. A growing CLAUDE.md section that has become a *procedure* should move to a skill.

S3. Path-specific rules — `.claude/rules/` (task 3.3)

Rule files are markdown in `.claude/rules/` with **YAML frontmatter** whose **paths:** field holds **glob patterns**:

```
---
paths:
- "src/api/**/*.ts"
- "**/*.test.tsx"
---
```

API + test conventions

- All endpoints validate input
- Tests use the team fixture factory

- A rule **loads only when Claude works with files matching its globs** — irrelevant conventions stay out of context, saving tokens (rules *without* paths load every session, like CLAUDE.md).
- **Globs beat directory-level CLAUDE.md when a convention spans directories** (official sample Q6): test files sit beside sources throughout the tree, so `**/*.test.tsx` catches them all — per-directory CLAUDE.md files can't, root-CLAUDE.md-with-headers relies on inference, and skills require invocation. Rules-with-globs is the deterministic, automatic answer.
- Example from the guide's own exercises: `paths: ["terraform/**/*.ts"]` so Terraform conventions load only when editing Terraform.

S4. Plan mode, direct execution, and the Explore subagent (task 3.4)

Named decision rule (§2.1 above covers the workflow; the exam wants the *criteria* verbatim):

- **Plan mode** — complex tasks: large-scale changes, **multiple valid approaches**, architectural decisions, multi-file modifications (monolith→microservices; a 45-file library migration). Safe exploration and design **before committing to changes** prevents costly rework (sample Q5's correct A).
- **Direct execution** — simple, well-scoped changes: single-file bug fix with a clear stack trace; adding one validation conditional.
- **Combine them:** plan mode for investigation, then direct execution for the planned implementation.
- **The Explore subagent** isolates **verbose discovery output** and returns summaries, **preserving main-conversation context** during multi-phase work — name it when a question describes context exhaustion during codebase discovery. (Ties to Domain 5.4 subagent delegation.)

S4b. Iterative refinement techniques (task 3.5)

- **Concrete input/output examples beat prose** when natural-language descriptions of a transformation are interpreted inconsistently — provide **2–3 example pairs** showing input and expected output.

- **Test-driven iteration:** write the test suite first (expected behavior, edge cases, performance), then iterate by **sharing test failures** to guide progressive improvement; specific failing cases with example input/expected output fix edge-case handling (e.g. null values in migration scripts).
- **The interview pattern:** in an unfamiliar domain, have Claude **ask you questions first** to surface considerations you hadn't anticipated (cache invalidation strategy, failure modes) *before* implementing.
- **Interacting vs independent issues:** fixes that **interact** → one single detailed message addressing all of them together; **independent** issues → sequential iteration is fine.

S5. CI/CD integration flags (task 3.6)

Mechanic	Exam fact
-p / --print	Runs Claude Code non-interactively : processes the prompt, writes to stdout, exits — the fix when a pipeline job “hangs waiting for interactive input” (sample Q10; distractors like CLAUDE_HEADLESS=true or --batch are non-existent features)
--output-format json	Machine-parseable output in print mode (text, json, stream-json)
--json-schema	Print-mode flag producing validated JSON matching your schema — structured findings ready for automated posting as PR comments
CLAUDE.md in CI	The mechanism for giving CI-invoked Claude project context: testing standards, fixture conventions, review criteria — better test generation, fewer low-value cases
Session context isolation	The session that generated code is less effective at reviewing it (it retains its own reasoning context and won't question its decisions) — run review as an independent instance (ties to Domain 4.6 multi-instance review)
Re-review without duplicates	Include prior review findings in context when re-running after new commits; instruct Claude to report only new or still-unaddressed issues
Test generation without duplicates	Provide existing test files in context so generated tests skip already-covered scenarios

S6. Supplement Q&A

SQ1. (= official sample Q4) A /review command must reach every developer on clone/pull. Where does the file go? **A.** .claude/commands/ in the project repository — version-controlled project scope. ~/.claude/commands/ is personal; CLAUDE.md holds instructions, not command definitions; .claude/config.json with a commands array doesn't exist.

SQ2. (= official sample Q6) React, API, and DB conventions differ; test files sit beside sources everywhere and share one convention. Most maintainable mechanism? **A.** .claude/rules/ files with YAML frontmatter glob patterns (**/*.test.tsx) — conditional, automatic, location-independent.

Root-CLAUDE.md headers rely on inference; per-directory CLAUDE.md can't span the tree; skills need invocation.

SQ3. New teammate doesn't get "the team's" instructions. Likely cause? **A.** Instructions live in user-level `~/.claude/CLAUDE.md` (not shared via version control) instead of project-level `CLAUDE.md/.claude/`.

SQ4. How do you keep CLAUDE.md modular per package? **A.** `@import` syntax (`@docs/standards.md`) — each package's CLAUDE.md imports only relevant standards files.

SQ5. Which command verifies which memory files are loaded? **A.** `/memory` (and `/context` confirms what loaded this session).

SQ6. A codebase-analysis skill floods the main conversation with output. Frontmatter fix? **A.** `context: fork` — runs the skill in an isolated sub-agent context.

SQ7. Restrict a skill to file writes to prevent destructive actions — which key? **A.** `allowed-tools` in SKILL.md frontmatter (grants scoped, prompt-free tool use for the invoking turn).

SQ8. Developers invoke `/deploy` without required parameters. Which key prompts them? **A.** `argument-hint` (e.g. `[environment] [version]`).

SQ9. You want a personal variant of a team skill without affecting teammates. **A.** Create it in `~/.claude/skills/` under a **different name**.

SQ10. Skill or CLAUDE.md: a 20-step release procedure vs "we use 2-space indent"? **A.** Procedure → skill (on-demand); universal standard → CLAUDE.md (always loaded).

SQ11. When do `paths:-scoped` rules load? **A.** Only when Claude works with files matching the globs; rules without paths load every session.

SQ12. Monolith→microservices restructuring: plan mode or direct? **A.** Plan mode — large-scale, multiple valid approaches, architectural decisions (sample Q5). Single-file bug fix with a stack trace → direct execution.

SQ13. Multi-phase task exhausts context during discovery. Named remedy? **A.** The Explore subagent — verbose discovery isolated, summaries returned.

SQ14. CI job hangs: "Claude Code is waiting for interactive input." Fix? **A.** `claude -p "..."` — the `-p/--print` flag is the documented non-interactive mode (sample Q10; `CLAUDE_HEADLESS/--batch` don't exist).

SQ15. Structured, machine-postable review findings from CI — which flags? **A.** `--output-format json` with `--json-schema` (print mode).

SQ16. Why shouldn't the session that wrote the code review it? **A.** Session context isolation: it retains generation reasoning and won't question its own decisions — use an independent review instance.

SQ17. Re-running review after new commits spams duplicate comments. Fix? **A.** Include prior findings in context and instruct: report only new or still-unaddressed issues.

SQ18. Generated tests duplicate existing coverage. Fix? **A.** Provide existing test files in context so generation avoids covered scenarios.

S7. Supplement checklist

- ☐ I can place an instruction at user / project / directory level and predict who receives it.

- ☐ I can use `@import` and explain what `/memory` shows.
- ☐ I know `.claude/commands/` vs `~/ .claude/commands/` cold (sample Q4).
- ☐ I can write a `.claude/rules/` file with paths: globs and argue it over per-directory `CLAUDE.md` (sample Q6).
- ☐ I can recite context: `fork`, `allowed-tools`, `argument-hint`.
- ☐ I can classify plan mode vs direct execution and name the Explore subagent.
- ☐ I know `-p`, `--output-format json`, `--json-schema`, and the three CI context tricks (`CLAUDE.md`, prior findings, existing tests).

Claude with Amazon Bedrock — Primary Study Notes

Note: This edition follows the ASD-STE100 writing rules: simple present tense, active voice, one word = one meaning, sentences of 20–25 words or less. Technical names (Claude, Bedrock, Amazon Bedrock, Converse, InvokeModel, IAM, RAG, MCP, Vertex, Anthropic API, inference profile, boto3, Region, ARN, JSON, schema, streaming, prompt caching, temperature) are exceptions. They stay as written. Model IDs, Regions, and API fields change. Check the current AWS Bedrock and Anthropic docs before the exam.

Disclaimer: These notes are **original** cert-prep notes. They align to the **public** Claude Academy course outline at [Claude with Amazon Bedrock](#). They are **not** Academy lesson dumps. They are **not** verbatim course prose. Use current AWS Bedrock and Anthropic docs with these notes for live model IDs, Regions, and API fields. The official course is the source of truth for quizzes.

Who it is for: Developers who add Claude features to apps with Amazon Bedrock.

Prerequisites (course): Python, JSON basics, and an AWS account with Bedrock model access enabled.

How to use: Platform-specific sections are the most valuable differences from the Anthropic API and Vertex tracks. These sections cover enablement, IDs, IAM, Converse vs Invoke, and residency. Shared Claude skills still appear. These skills include prompting, tools, RAG, MCP, and agents. Know those skills in the Bedrock request shape.

1. Course map (public modules)

The public page has about 65 lessons and multiple quizzes. It groups topics roughly as:

1. **Working with the API** — auth, basic requests, conversation management, system prompts, structured output
2. **Prompt engineering** — strategies, evaluation frameworks, systematic testing
3. **Tool use** — JSON Schema tools, multi-turn tool loops, batch tool calling, built-ins
4. **RAG** — chunking, embeddings, BM25 hybrid search, multi-index, reranking, contextual retrieval
5. **Model Context Protocol (MCP)** — tools, resources, prompts. Servers and clients
6. **Agents** — Claude Code / computer-use style automation patterns on top of Bedrock access
7. **Advanced Claude features** — extended thinking, vision, prompt caching, streaming, temperature, structured extraction

For certification-style prep, give first priority to **access patterns, model IDs, IAM, Regions, and Converse vs InvokeModel**. Then know how shared Claude skills look when the transport is Bedrock.

Main sequence for scenario questions: Enable the model, then select the Region or profile → use IAM least privilege → use Converse (default). Alternatively, Invoke (native) → manage history yourself → add tools/RAG/eval as needed → select a residency-aware profile.

2. Access patterns on Bedrock

2.1 Enablement and credentials

1. In the Bedrock console (per account/Region), **request/enable** the Anthropic Claude models you need.
2. Call **Bedrock Runtime** with the standard AWS credential chain (env vars, shared config, instance role, SSO, and similar).
3. Create a runtime client in your Region. Example: `boto3.client("bedrock-runtime", region_name="us-east-1")`.

You do **not** send an Anthropic API key for common Bedrock Runtime calls. Billing and tenancy use AWS. Separate offerings like Claude Platform on AWS can differ. Know which product the question names.

Common enablement failures:

- You enable the model in Region A, but the client points at Region B
- You must have You enable the model, but you call a raw foundation ID when an inference profile.
- Credentials are valid for AWS in general, but the role does not have Bedrock invoke on the correct ARNs

2.2 Model IDs and inference profiles

Bedrock identifies Claude with IDs such as:

- Foundation-style: `anthropic.claude-...`
- **Cross-Region inference profiles:** often prefixed by geography, e.g. `us.anthropic.claude-...`, `eu.anthropic...`, `global.anthropic...`
- **Application inference profiles:** customer-created profiles (often for cost attribution or single-Region pinning)

Exam-critical facts:

- Not every model exists in every Region. If you call a missing model, you get confusing errors. The errors often say “does not exist” or “on-demand not supported.”
- Newer Claude models often need an **inference profile ID**. A raw foundation model ID is not enough.
- Inference profiles route to Regions where the model is hosted. Check the Bedrock console under cross-Region inference or the model detail page for the exact ID.
- IAM must allow the resources you actually invoke. This includes foundation model ARNs **and** inference profile ARNs when you use both. Sample policies often use wildcards on Region in ARNs. Still prefer least privilege.

Always confirm current IDs on AWS model cards. Strings change when new Claude versions launch.

Decision tree — which ID to pass:

Does the model require an inference profile for on-demand?
 YES → Use system geo profile (us./eu./global.) OR your application profile ARN/ID
 NO → Foundation model ID may work in that Region (still verify)

Need single-Region residency?
 → Do NOT use multi-Region geo profiles; use application profile pointing at one Region P (or other in-Region options documented by AWS for that Region)

Need cost attribution per team/app?
 → Application inference profile with tags

3. Converse API vs InvokeModel (high-yield)

	Converse / ConverseStream	InvokeModel / InvokeModel-WithResponseStream
Shape	Unified Bedrock message schema	Model-native JSON body
Portability	Same code shape across many Bedrock chat models	Provider-specific payloads
Multi-turn / tools	Direct support for messages, system, toolConfig	You build the Anthropic Messages-style body yourself
Claude extras	Use additionalModelRequestFields for the parameters that Converse does not show	Full native Anthropic request fields
When to use	Default for chat, agents, tools	Embeddings/image gens, legacy, or native-only features

Default for exams: Prefer **Converse** for conversational Claude on Bedrock. Use **InvokeModel** when you need the native Anthropic Messages payload. Use it also for certain tool types/fields or non-chat modalities.

AWS public docs emphasize this: Converse gives a consistent API for Bedrock models that support messages. Unique parameters still pass through a model-specific structure.

3.1 Converse mental model

```
client.converse(
  modelId=...,
  messages=[{role, content:[{text|...}]}],
  system=[...], # optional
  inferenceConfig={maxTokens, temperature, topP,...},
  toolConfig={...}, # optional
  additionalModelRequestFields={...} # Claude-specific extras
)
```

Read text from a path like `response["output"]["message"]["content"][i]["text"]`. Exact indexing depends on content blocks.

Statelessness: Bedrock does not remember prior turns for you. You must send the full alternating user/assistant history again on each call.

3.2 InvokeModel mental model (Claude)

The body typically includes:

- `anthropic_version` (Bedrock Claude Messages value such as `bedrock-2023-05-31` — confirm current docs)
- `max_tokens` (required in native Messages style)
- `messages` array
- optional `system`, `temperature`, `tools`, etc.

You parse the Anthropic-shaped response yourself. Content-Type is usually `application/json`. In `boto3` the body is bytes (`json.dumps(...).encode()`).

3.3 Streaming

Non-stream	Stream
<code>converse</code>	<code>converse_stream</code>
<code>invoke_model</code>	<code>invoke_model_with_response_stream</code>

Stream calls use the same IAM family as non-stream, plus the **stream-specific** action. If you grant only `bedrock:InvokeModel`, streaming often fails with `AccessDenied`. Non-stream still works. This is a common exam error.

Stream responses arrive as event sequences. Your code puts the events back together (content deltas, message stop, and similar). Streaming changes delivery. It does not change the need for correct tools and history.

3.4 When Converse is not enough

Select `InvokeModel` when:

- You need embeddings or image generation models (not chat `Converse`)
- A Claude field is easier, or docs only show it in native Messages form
- You need to see the exact Anthropic-shaped body for debug
- You migrate existing Anthropic API code with minimal mapping (still change auth, version field, and endpoint)

Select `Converse` when:

- You build portable Bedrock chat/agents across model families
- You want native `toolConfig` and Guardrails integration patterns
- The team uses one Bedrock conversation API as the standard

4. IAM essentials (least privilege)

Both Converse and Invoke authorize with Bedrock inference actions:

- `bedrock:InvokeModel` — required for `InvokeModel` **and** `Converse`
- `bedrock:InvokeModelWithResponseStream` — required for stream variants

Surprise exam fact: An `AccessDeniedException` on `Converse` is often fixed when you grant `bedrock:InvokeModel`. Common policies do not use a separate `bedrock:Converse` permission. AWS docs state that `Converse` needs `bedrock:InvokeModel`. `ConverseStream` needs `bedrock:InvokeModelWithResponseStream`.

Also grant these when you use profiles or console selection:

- `bedrock:GetInferenceProfile` — run inference with an inference profile
- `bedrock:ListInferenceProfiles` — select profiles in the console.
- Access to the correct **model / inference profile ARNs**
- Any Guardrails / knowledge-base actions if the question includes those features

Inference profile IAM trap: When you use a cross-Region profile, the policy must allow invoke on:

1. The inference profile ARN (entry point)
2. The underlying foundation model ARNs in **each destination Region** the profile can route to

If you allow only the profile ARN, Lambda roles often fail. This is a frequent failure mode.

Least privilege checklist:

- Limit scope by account and Region where possible
- List specific model family ARNs rather than *
- Use separate roles for production and for experiments.
- You must have Deny dangerous adjacent actions your app does not need (model customization, marketplace purchase, and similar) unless they.
-

For residency: combine profile choice with IAM Region conditions where policy requires it

5. Regions and data residency

- Select a **source Region** for the runtime client that supports your access pattern.
- Use **geo inference profiles** (`us.`, `eu.`, ...) when you need availability across a geography.
- Use **global** profiles when they are allowed and residency is flexible.
- For **single-Region residency**, follow current AWS guidance. Use application inference profiles that point at one Region foundation model ARN. Use other in-Region options where they apply. Do **not** assume that cross-Region profiles keep data in one AZ/Region.

For compliance questions, match the profile geography to the policy. A model ID that worked in a tutorial is not enough.

Residency decision tree:

Flexible residency, maximize availability?

→ global inference profile (if model supports)

Must stay in US or EU broadly?

→ us. or eu. geo profile

Must stay in ONE Region?

→ application inference profile pinned to that Region FM ARN
(+ IAM Region conditions as required by your org)

Unsure model available?

→ Check Bedrock console model card / ListFoundationModels / profile docs

Claude Code on Bedrock note (conceptual): If an agentic coding tool uses cross-Region profiles by default, single-Region residency can need an application profile. Point the tool at that profile ID. Public AWS blogs name this as a common enterprise error.

6. Claude features through Bedrock (course themes)

6.1 Conversations and system prompts

The ideas are the same as the Anthropic Messages API. Roles alternate. The system prompt sets durable behavior. Keep history tidy. Bedrock Converse uses field names like the system list and inferenceConfig.

History care: Trim old turns. Summarize when you need to. Never drop unpaired tool results in the middle of a loop.

6.2 Prompt engineering and evaluation

The course emphasis is structured prompting, systematic test sets, **model-based grading**, and **code-based grading**. For exams: evaluation is a workflow (cases → run → score → iterate). It is not a one-time informal check.

Grading style	What it checks	Strength
Code-based	Exact match, regex, JSON schema, unit asserts	Deterministic and cheap
Model-based	Rubric scored by another model call	Flexible for open answers
Human spot	Sample review	Calibration

6.3 Tool use

Define tools with JSON Schema. Claude returns tool calls. Your app runs the tools and returns tool results in the next turn. Support multi-turn and batch tool calling patterns. On Converse, tools are set under toolConfig. Native-only Anthropic tool types can need InvokeModel or additional fields.

Tool loop correctness:

1. The model returns an assistant message with tool use block(s)
2. The app runs the tools

3. The app appends tool result(s) in the required role/shape
4. Call the model again with the full history
5. Repeat until a final text answer or a stop condition

Never invent tool results. Never change the order of tool_use / tool_result pairs without care.

6.4 RAG

Production RAG topics in the outline:

- Chunking strategies
- Embeddings (often a separate Bedrock embedding model via **Invoke**, not Converse)
- Lexical search (**BM25**) plus vector hybrid
- Multi-index architectures, reranking
- **Contextual retrieval** (add context to chunks before you embed or index)

Know *why* hybrid search exists. It covers exact keywords vs semantic match. This matters more than any single library.

Bedrock Knowledge Bases can appear in AWS-centric questions. Still map them to the same RAG concepts: ingest, chunk, embed, retrieve, generate, cite.

6.5 Advanced features

- **Extended thinking / reasoning** — extra tokens and cost. Current models use **adaptive thinking**. The model decides when and how deeply to think. You tune depth with **effort levels**. Claude 4.6 deprecates fixed budget_tokens thinking budgets. Newer frontier models remove them and return a 400 error. Check the specific model card on Bedrock before you use either shape.
- **Vision** — image content blocks
- **Prompt caching** — lower cost and latency for stable prefixes when the model/platform offers it
- **Streaming** — better UX for long answers
- **Structured extraction** — limit outputs (schemas / careful prompting). Temperature applies **only on older models**. Current frontier models remove the sampling parameters. Schemas give you determinism.

6.6 MCP and agents

MCP standardizes tools, resources, and prompts across clients and servers. The Agents section connects Bedrock-backed Claude to automation patterns (Claude Code, computer use). For cert prep: MCP is a **protocol for modular tool/resource exposure**. It is not a Bedrock-only API.

Agent patterns still apply: parallelize, chain, route, and debug with traces.

7. Comparison snapshot: Bedrock vs Anthropic API vs Vertex

Topic	Anthropic API	Amazon Bedrock	Google Vertex (Claude)
Auth	Anthropic API key	AWS IAM / credentials	GCP ADC / service account
Client	Anthropic	bedrock-runtime (Converse/Invoke)	AnthropicVertex

Topic	Anthropic API	Amazon Bedrock	Google Vertex (Claude)
Model ID style	Anthropic IDs	anthropic... / us.anthropic...	Vertex publisher IDs (e.g. claude-sonnet- 4-6)
Version field	Headers / SDK	anthropic_version in the Invoke body. Converse covers much of this	anthropic_version: vertex-2023-10-16 on raw HTTP
Unified multi-model chat	Messages	Converse (Bedrock-wide)	Messages via Vertex backend
Billing / contracts	Anthropic	AWS	Google Cloud
Residency knobs	Anthropic policies	Regions + inference profiles	global / us / eu / regional endpoints
Enablement	API key access	Bedrock model access per account/Region	Model Garden enablement

Select Bedrock when: You need AWS-centric IAM, Converse portability across Bedrock models, an AWS compliance boundary. Alternatively, existing AWS networking/VPC patterns.

Pick Vertex when: GCP is the control plane. You need Google billing/residency or the Model Garden partner path.

Pick Anthropic API when: You need the fastest path to newest Anthropic-only features and the simplest key-based auth.

Do not mix ID formats across clouds. That fact alone answers several exam distractors.

8. Exam traps (Bedrock-specific)

Trap	Reality
You look for a bedrock:Converse permission	Converse uses bedrock:InvokeModel
You must have You use a foundation ID when a profile.	Switch to an inference profile ID/ARN.
You grant the profile ARN only	Also grant destination FM ARNs for geo profiles
Store You assume history.	Runtime is stateless. Send messages again.
You use Converse for embeddings	Invoke embedding models separately
You use a cross-Region profile for a single-Region policy	Wrong residency tool. Pin with an application profile
You copy Vertex model IDs into Bedrock	Catalog strings differ
Streaming AccessDenied with Invoke granted	The policy misses InvokeModelWithResponseStream
“Enable once globally”	Enablement follows per account/Region patterns
You treat Guardrails as optional when the question requires them	Put Guardrails config on Converse when the question asks

9. Exam tips

- Enable model access **and** use a Region/profile where the model exists.
 - Prefer **Converse** unless the question forces native Invoke.
 - Remember **stateless** history management.
 - IAM for Converse $\approx$ `bedrock:InvokeModel` (plus the stream variant).
 - For newer models, check **inference profile** IDs.
 - For RAG answers, mention chunking and retrieval quality. Do not put the full PDF in the prompt for production.
 - Eval = systematic grading. It is not a single anecdotal prompt.
 - Distinguish advisory prompting from tool/MCP connections.
 - For residency, name the profile strategy explicitly.
 - For tool loops, insist on correct message ordering.
-

10. Minimal practice sketch (conceptual)

1. Enable Claude in the Bedrock console for your account/Region.
2. Create a runtime client with AWS creds.
3. Call `converse` with an inference profile model ID and one user message.
4. Add a second turn. Append the prior assistant output.
5. Add a tiny tool (e.g. calculator) via `toolConfig` and complete one tool loop.
6. Stream the same prompt with `converse_stream`.
7. Use a wrong Region/model ID once so you recognize the failure mode.
8. Sketch an IAM policy that includes profile + destination FM ARNs.
- 9.

Write five eval cases. Grade with code asserts plus one model rubric.

11. Self-check Q&A (with answers)

Q1. A Converse call fails with `AccessDenied`. Invoke works with the same role. What is the likely cause? **A.** The role misses `bedrock:InvokeModel` (or the resource ARN for the new model/profile). Converse uses the same invoke permission family. If only streaming fails, check the stream action.

Q2. A model ID from the catalog works in docs but fails in your Region. What next? **A.** Confirm Regional availability. Switch to the correct **inference profile** ID for cross-Region routing.

Q3. Why does Claude not remember the previous turn on Bedrock? **A.** Runtime is stateless. You must send the full message history again.

Q4. When do you select `InvokeModel` over `Converse` for Claude? **A.** You need native Anthropic request/response fields or features that Converse does not show cleanly. Or you use non-chat models (embeddings, and similar).

Q5. What belongs in `inferenceConfig` on Converse? **A.** Shared knobs like `maxTokens`, `temperature`, and `topP`. Claude-specific extras often go in `additionalModelRequestFields`.

Q6. Name two RAG improvements beyond naive top-k embedding search. **A.** BM25 hybrid search. Reranking. Contextual retrieval. Better chunking / multi-index.

Q7. What is MCP in one sentence? **A.** A protocol that exposes tools, resources, and prompts to AI clients in a modular, reusable way.

Q8. How do you keep multi-turn tool use correct? **A.** Append assistant tool-call messages and user/tool-result messages in order. Never drop results in the middle of a loop.

Q9. Cross-Region profile invoke: IAM allows the profile ARN only. You still get AccessDenied. Why? **A.** You also need invoke permission on foundation model ARNs in each destination Region the profile routes to.

Q10. Which IAM action does ConverseStream require? **A.** `bedrock:InvokeModelWithResponseStream` (not only `InvokeModel`).

Q11. How do you pursue single-Region residency with newer Claude models that require profiles? **A.** Create an application inference profile that points at one Region foundation model ARN. Avoid multi-Region geo prefixes.

Q12. Why might embeddings use Invoke while chat uses Converse in the same app? **A.** Embeddings are not conversational Converse workloads. Invoke is the correct modality API.

Q13. What is the Bedrock Claude Invoke body version field commonly set to? **A.** `anthropic_version` such as `bedrock-2023-05-31` (confirm current docs).

Q14. Code-based vs model-based eval — when do you prefer code-based? **A.** When success is objectively checkable (JSON schema, exact fields, unit asserts).

Q15. Name three differences vs Vertex Claude. **A.** AWS IAM vs GCP ADC. `anthropic.` / geo profile IDs vs publisher IDs. Converse vs Messages-on-Vertex.

Q16. Guardrails on Converse — conceptual role? **A.** Platform safety/filters that you apply around model calls when you configure them. They are not a substitute for app authz.

Q17. What does batch tool calling mean at a high level? **A.** The model can request multiple tools in one turn. The app returns multiple results before the next model call.

Q18. Prompt caching helps most when? **A.** Stable long prefixes (system, tools, large docs) are reused across calls — when the model/platform supports it.

Q19. You enabled Claude in us-east-1 but call eu-west-1. What happens? **A.** Failure or an unavailable model. Enablement and availability are Region-sensitive. Fix the Region or use the appropriate profile.

Q20. What does contextual retrieval add? **A.** It adds surrounding document context to chunks before you embed or index. This improves retrieval quality.

Q21. Agent pattern that classifies then sends to specialized prompts/tools? **A.** Routing.

Q22. Why is temperature usually lower for tool argument generation? **A.** You need more deterministic structured arguments for reliable execution.

Q23. Application inference profile primary benefits? **A.** Cost attribution/tags and/or pinning of routing behavior (including single-Region sources) vs system geo profiles.

Q24. Is billing via Anthropic API key on common Bedrock Runtime? **A.** No. Common Bedrock Runtime Claude calls use AWS credentials and AWS billing. —

12. Review checklist (before exam)

- ☐ You distinguish enablement vs credentials vs the correct Region
 - ☐ Foundation model ID vs geo profile vs application profile
 - ☐ You memorize Converse vs Invoke decision criteria
 - ☐ IAM: InvokeModel for Converse. Stream action for streams
 - ☐ Profile IAM includes destination FM ARNs
 - ☐ Stateless multi-turn history
 - ☐ toolConfig loop ordering
 - ☐ RAG hybrid / rerank / contextual retrieval vocabulary
 - ☐ Eval = systematic cases + code/model grading
 - ☐ Residency decision tree (global / geo / single-Region pin)
 - ☐ Bedrock vs Vertex vs Anthropic API comparison table
 - ☐ How you put stream events back together (conceptual)
-

13. Glossary

- **Bedrock Runtime** — API surface that invokes foundation models (bedrock-runtime client).
 - **Converse** — Unified conversational inference API across Bedrock chat models.
 - **ConverseStream** — Streaming variant of Converse.
 - **InvokeModel** — Model-native body invocation API.
 - **Inference profile** — Resource that routes inference (system geo or application).
 - **Foundation model (FM) ARN** — Underlying model resource in a Region.
 - **Application inference profile** — Customer-created profile for attribution and/or pinned routing.
 - **Geo profile prefix** — e.g. us., eu., global. on profile IDs.
 - **additionalModelRequestFields** — Field for model-specific Converse params that Converse does not show.
 - **inferenceConfig** — Shared sampling/max token knobs on Converse.
 - **toolConfig** — Converse tool definitions and tool choice settings.
 - **Stateless inference** — No server-side chat memory between calls.
 - **BM25** — Lexical ranking function used in hybrid RAG.
 - **Contextual retrieval** — Prepend context to chunks before you embed or index.
 - **MCP** — Model Context Protocol for tools/resources/prompts.
 - **Guardrails** — Bedrock safety filters that you can configure around calls.
 - **Knowledge Base** — Managed RAG retrieval component in the AWS Bedrock ecosystem.
 - **Model-based grading** — You use a model to score outputs against a rubric.
 - **Code-based grading** — Programmatic asserts on outputs.
 - **AccessDeniedException** — IAM authorization failure from Bedrock APIs.
-

14. Deeper decision trees

14.1 API shape picker

Is the workload conversational messages with optional tools?

YES → Converse / ConverseStream

NO → Is it embeddings / image / native-only payload?

YES → InvokeModel / stream variant
NO → Re-check modality docs

14.2 Failure triage

AccessDenied?

- Check action (Invoke vs InvokeWithResponseStream)
- Check resource ARNs (profile + destination FMs)
- Check account model enablement

Validation / model does not exist / on-demand not supported?

- Wrong Region or need inference profile ID

Weird empty/partial answer?

- maxTokens too low; stop reasons; tool loop incomplete

Tool loop spinning?

- Missing/malformed tool results; unbounded max turns

14.3 Production readiness mini-rubric

1. Correct profile for residency and availability
2. Least-privilege IAM that you test with deny cases
3. Timeouts/retries for stream and non-stream
4. Eval suite with fixed cases before prompt tweaks
5. Logging that does not leak secrets
6. Cost monitors on tokens and profile tags
7. Fallback model or Region strategy that you document

15. Shared skill themes mapped to Bedrock fields

Claude skill theme	Bedrock expression
System prompt	system on Converse
Temperature	inferenceConfig.temperature
Tools	toolConfig (or native tools on Invoke)
Multi-turn	App-managed messages array
Streaming	converse_stream / invoke stream
Structured output	Careful prompting + schema validation. Platform features where offered
Vision	Image content blocks in messages
Caching	Model/platform-specific cache fields when available
Agents	Your orchestration + optional Claude Code with Bedrock backend

16. Study rhythm

Day 1: Sections 1–5 (access, Converse/Invoke, IAM, residency). Draw the comparison table from memory. Day 2: Sections 6–10 (features, traps, practice sketch). Run the failure triage aloud. Day 3: All Q&A closed-book. Checklist. Check glossary gaps only.

Rule to memorize: enable, profile, IAM, Converse by default, you manage the history, residency with purpose.

Aligned to <https://academy.claude.com/courses/claude-with-amazon-bedrock>. Check live model IDs, Regions, and IAM examples in AWS Bedrock documentation before production use.

17. Worked mini-scenarios (exam style)

Scenario 1 — New Sonnet in prod Lambda The team copies an old foundation model ID into converse. The platform does not support The error says on-demand throughput / use an inference profile. **Answer path:** Look up the system or application inference profile ID for that Claude version. Update modelId. Extend IAM for profile + destination FM ARNs. Retest in the Lambda Region.

Scenario 2 — EU data policy Legal requires processing in the EU. An engineer uses a global... profile because it worked in a tutorial. **Answer path:** Wrong. Prefer an eu. geo profile for EU multi-Region residency. Or use a single-Region EU application profile if policy demands one Region. Document which guarantee you actually have.

Scenario 3 — Streaming chat widget Non-stream chat works. Stream fails with AccessDenied. **Answer path:** Add bedrock:InvokeModelWithResponseStream on the same resources. Keep putting stream events back together in the client.

Scenario 4 — Tool-using support agent The model asks for two tools. The app returns only one result. The next call confuses roles. **Answer path:** Return all tool results paired correctly. Keep the assistant tool-use message. Then continue.

Scenario 5 — RAG “hallucinated policy” Naive top-k embedding retrieval misses exact clause IDs. **Answer path:** Add BM25/hybrid. Consider rerank. Improve chunking. Try contextual retrieval. Evaluate with fixed cases that contain those clause IDs.

Scenario 6 — Multi-cloud migration Port from Anthropic API to Bedrock. An engineer pastes x-api-key and the Anthropic base URL into AWS code. **Answer path:** Use AWS credentials + Bedrock Runtime. Map Messages fields to Converse, or keep Invoke with anthropic_version for Bedrock. Change model IDs to Bedrock catalog forms.

Scenario 7 — Cost attribution Finance cannot see which product line spent Bedrock Claude tokens. **Answer path:** Use application inference profiles with tags per product. Invoke via those profile ARNs. Build dashboards by tag.

Scenario 8 — Eval disagreement A prompt performs well on three selected examples. It fails silently in production. **Answer path:** Build a held-out eval set. Combine code-based checks for schema with model-based rubrics for quality. Gate deploys on eval thresholds.

18. Quick reference card

Need	Bedrock move
Default chat/agent	Converse
Stream tokens	ConverseStream + stream IAM
Native Anthropic body / embeddings	InvokeModel
Cross-Region availability	us. / eu. / global. profile
Single-Region pin	Application profile → one FM ARN
Auth	AWS credential chain, not Anthropic key
Tools	toolConfig (+ loop)
Claude-only knobs	additionalModelRequestFields
Portable vs Vertex	Different IDs and auth entirely

One-sentence rule: On Bedrock, enable the model, call the correct profile with Invoke-family IAM, prefer Converse, manage your history yourself, and select residency with purpose.

19. Extra Q&A (stretch)

Q25. Can you use the same boto3 Converse code for Claude and another Bedrock chat model? **A.** Often yes for the message shape. That portability is a main benefit of Converse. Model-specific fields and capabilities still differ.

Q26. What breaks if you omit max_tokens on native Invoke Messages-style Claude? **A.** Request validation errors. You must have max_tokens in native Messages-style bodies.

Q27. Where do system prompts go on Converse vs a mistaken role: system inside messages? **A.** Use the Converse system parameter (list of system blocks). Do not invent a system role inside messages the way some other APIs do.

Q28. Why list GetInferenceProfile in some IAM policies? **A.** The AWS inference prerequisites docs require this action to run inference with an inference profile.

Q29. Hybrid search in one line? **A.** Combine lexical (BM25) and vector retrieval so exact tokens and semantic matches both surface.

Q30. Computer use / Claude Code themes in this course — what to remember? **A.** They are agentic automation patterns that can run with Bedrock-backed Claude. You still need AWS auth, model access, and verification discipline.

20. Study close

Redraw these items without notes: (1) Converse vs Invoke, (2) IAM actions for stream/non-stream, (3) profile types for residency, (4) the three-cloud comparison table. If you can do this, you are ready for Bedrock-differentiating exam items. Then refresh shared Claude skills (tools, RAG, MCP, eval). Use the same vocabulary as the Anthropic API and Vertex tracks. Only the transport and ID formats change.

Claude with Amazon Bedrock — Primary Study Notes

Disclaimer: These are **original** cert-prep notes aligned to the **public** Claude Academy course outline at [Claude with Amazon Bedrock](#). They are **not** Academy lesson dumps or verbatim course prose. Pair with current AWS Bedrock and Anthropic docs for live model IDs, Regions, and API fields. Completing the official course remains the source of truth for quizzes.

Who it is for: Developers adding Claude features to apps via Amazon Bedrock.

Prerequisites (course): Python, JSON basics, AWS account with Bedrock model access enabled.

How to use: Platform-specific sections (enablement, IDs, IAM, Converse vs Invoke, residency) are the highest-yield differentiators vs the Anthropic API and Vertex tracks. Shared Claude skills (prompting, tools, RAG, MCP, agents) still appear — know them in Bedrock's request shape.

1. Course map (public modules)

Public page (~65 lessons, multiple quizzes) groups topics roughly as:

1. **Working with the API** — auth, basic requests, conversation management, system prompts, structured output
2. **Prompt engineering** — strategies, evaluation frameworks, systematic testing
3. **Tool use** — JSON Schema tools, multi-turn tool loops, batch tool calling, built-ins
4. **RAG** — chunking, embeddings, BM25 hybrid search, multi-index, reranking, contextual retrieval
5. **Model Context Protocol (MCP)** — tools, resources, prompts; servers and clients
6. **Agents** — Claude Code / computer-use style automation patterns on top of Bedrock access
7. **Advanced Claude features** — extended thinking, vision, prompt caching, streaming, temperature, structured extraction

For certification-style prep, prioritize **access patterns, model IDs, IAM, Regions, Converse vs InvokeModel**, then know how shared Claude skills look when the transport is Bedrock.

Spine for scenario questions: Enable model → pick Region/profile → IAM least privilege → Converse (default) or Invoke (native) → manage history yourself → tools/RAG/eval as needed → residency-aware profile choice.

2. Access patterns on Bedrock

2.1 Enablement and credentials

1. In the Bedrock console (per account/Region), **request/enable** Anthropic Claude models you need.
2. Call **Bedrock Runtime** with the standard AWS credential chain (env vars, shared config, instance role, SSO, etc.).
3. Create a runtime client in your Region, for example `boto3.client("bedrock-runtime", region_name="us-east-1")`.

You do **not** send an Anthropic API key for classic Bedrock Runtime calls. Billing and tenancy ride on AWS. (Separate offerings like Claude Platform on AWS may differ — know which product the question names.)

Common enablement failures:

- Model enabled in Region A, client pointed at Region B
- Model enabled, but calling raw foundation ID when an inference profile is required
- Credentials valid for AWS generally, but role lacks Bedrock invoke on the right ARNs

2.2 Model IDs and inference profiles

Bedrock identifies Claude with IDs such as:

- Foundation-style: `anthropic.claude-...`
- **Cross-Region inference profiles**: often prefixed by geography, e.g. `us.anthropic.claude-...`, `eu.anthropic.claude-...`, `global.anthropic.claude-...`
- **Application inference profiles**: customer-created profiles (often for cost attribution or single-Region pinning)

Exam-critical facts:

- Not every model exists in every Region. Calling a missing model yields confusing “does not exist” / on-demand not supported style errors.
- Newer Claude models frequently require an **inference profile ID**, not only a raw foundation model ID.
- Inference profiles route to Regions where the model is hosted. Check the Bedrock console under cross-Region inference / the model detail page for the exact ID.
- IAM must allow the resources you actually invoke (foundation model ARNs **and** inference profile ARNs when both are in play). Wildcards on Region in ARNs are common in sample policies — still prefer least privilege.

Always confirm current IDs on AWS model cards — strings change as new Claude versions launch.

Decision tree — which ID to pass:

Does the model require an inference profile for on-demand?

YES → Use system geo profile (`us./eu./global.`) OR your application profile ARN/ID

NO → Foundation model ID may work in that Region (still verify)

Need single-Region residency?

→ Do NOT use multi-Region geo profiles; use application profile pointing at one Region F (or other in-Region options documented by AWS for that Region)

Need cost attribution per team/app?

→ Application inference profile with tags

3. Converse API vs InvokeModel (high-yield)

	Converse / ConverseStream	InvokeModel / InvokeModel- WithResponseStream
Shape	Unified Bedrock message schema	Model-native JSON body
Portability	Same code shape across many Bedrock chat models	Provider-specific payloads
Multi-turn / tools	First-class messages, system, toolConfig	You build Anthropic Messages-style body yourself
Claude extras	Use additionalModelRequestFields for knobs Converse does not surface	Full native Anthropic request fields
When to use	Default for chat, agents, tools	Embeddings/image gens, legacy, or native-only features

Default for exams: prefer **Converse** for conversational Claude on Bedrock. Drop to **InvokeModel** when you need the native Anthropic Messages payload (certain tool types / fields) or non-chat modalities.

AWS public docs emphasize: Converse provides a consistent API for Bedrock models that support messages; unique parameters still pass through a model-specific structure.

3.1 Converse mental model

```
client.converse(
    modelId=...,
    messages=[{role, content:[{text|...}]}],
    system=[...], # optional
    inferenceConfig={maxTokens, temperature, topP,...},
    toolConfig={...}, # optional
    additionalModelRequestFields={...} # Claude-specific extras
)
```

Read text from a path like `response["output"]["message"]["content"][i]["text"]` (exact indexing depends on content blocks).

Statelessness: Bedrock does not remember prior turns for you. You must resend the full alternating user/assistant history each call.

3.2 InvokeModel mental model (Claude)

Body typically includes:

- `anthropic_version` (Bedrock Claude Messages value such as `bedrock-2023-05-31` — confirm current docs)
- `max_tokens` (required in native Messages style)
- `messages` array
- optional `system`, `temperature`, `tools`, etc.

You parse the Anthropic-shaped response yourself. Content-Type is usually application/json; body is bytes in `boto3(json.dumps(...).encode())`.

3.3 Streaming

Non-stream	Stream
<code>converse</code>	<code>converse_stream</code>
<code>invoke_model</code>	<code>invoke_model_with_response_stream</code>

Same IAM family as non-stream, with the **stream-specific** action. Granting only `bedrock:InvokeModel` is a classic reason streaming fails with `AccessDenied` while non-stream works.

Stream responses arrive as event sequences your code reassembles (content deltas, message stop, etc.). Streaming changes delivery, not the need for tools/history correctness.

3.4 When Converse is not enough

Choose `InvokeModel` when:

- You need embeddings or image generation models (not chat `Converse`)
- A Claude field is easier or only documented in native Messages form
- Debugging requires seeing the exact Anthropic-shaped body
- Migrating existing Anthropic API code with minimal mapping (still change auth, version field, endpoint)

Choose `Converse` when:

- Building portable Bedrock chat/agents across model families
- You want first-class `toolConfig` and Guardrails integration patterns
- Team standardizes on one Bedrock conversation API

4. IAM essentials (least privilege)

Both `Converse` and `Invoke` ultimately authorize with Bedrock inference actions:

- `bedrock:InvokeModel` — required for `InvokeModel` **and** `Converse`
- `bedrock:InvokeModelWithResponseStream` — required for stream variants

Surprise exam fact: An `AccessDeniedException` on `Converse` is often fixed by granting `bedrock:InvokeModel`, not a mythical separate `bedrock:Converse` permission in classic policies. AWS docs state `Converse` requires `bedrock:InvokeModel`; `ConverseStream` requires `bedrock:InvokeModelWithResponseStream`.

Also grant when using profiles / console selection:

- `bedrock:GetInferenceProfile` — run inference with an inference profile
- `bedrock:ListInferenceProfiles` — choose profiles in console
- Access to the correct **model / inference profile ARNs**
- Any Guardrails / knowledge-base actions if the question includes those features

Inference profile IAM trap: When using a cross-Region profile, policy must allow invoke on:

1. The inference profile ARN (entry point)
2. The underlying foundation model ARNs in **each destination Region** the profile can route to

Allowing only the profile ARN is a frequent Lambda role failure mode.

Least privilege checklist:

- Scope by account and Region where possible
- List specific model family ARNs rather than *
- Separate roles for prod vs experiment
- Deny dangerous adjacent actions your app does not need (model customization, marketplace purchase, etc.) unless required
-

For residency: combine profile choice with IAM Region conditions where policy requires it

5. Regions and data residency

- Pick a **source Region** for the runtime client that supports your access pattern.
- Use **geo inference profiles** (us., eu., ...) when you need availability across a geography.
- Use **global** profiles when allowed and residency is flexible.
- For **single-Region residency**, follow current AWS guidance: application inference profiles pointing at one Region foundation model ARN, or other in-Region options where applicable. Do **not** assume cross-Region profiles keep data in one AZ/Region.

Compliance questions: matching profile geography to policy beats “whatever model ID worked in the tutorial.”

Residency decision tree:

Flexible residency, maximize availability?

→ global inference profile (if model supports)

Must stay in US or EU broadly?

→ us. or eu. geo profile

Must stay in ONE Region?

→ application inference profile pinned to that Region FM ARN
(+ IAM Region conditions as required by your org)

Unsure model available?

→ Check Bedrock console model card / ListFoundationModels / profile docs

Claude Code on Bedrock note (conceptual): if an agentic coding tool defaults to cross-Region profiles, single-Region residency may require creating an application profile and pointing the tool at that profile ID — a common enterprise gotcha called out in public AWS blogs.

6. Claude features through Bedrock (course themes)

6.1 Conversations and system prompts

Same ideas as Anthropic Messages API: roles alternate; system sets durable behavior; keep history tidy. Bedrock Converse uses field names like `system` list and `inferenceConfig`.

History hygiene: trim old turns, summarize when needed, never drop unpaired tool results mid-loop.

6.2 Prompt engineering and evaluation

Course emphasis: structured prompting, systematic test sets, **model-based grading** and **code-based grading**. For exams: evaluation is a workflow (cases → run → score → iterate), not a one-off vibe check.

Grading style	What it checks	Strength
Code-based	Exact match, regex, JSON schema, unit asserts	Deterministic, cheap
Model-based	Rubric scored by another model call	Flexible for open answers
Human spot	Sample review	Calibration

6.3 Tool use

Define tools with JSON Schema; Claude returns tool calls; your app executes and returns tool results in the next turn. Support multi-turn and batch tool calling patterns. On Converse, tools live under `toolConfig`; native-only Anthropic tool flavors may need `InvokeModel` / additional fields.

Tool loop correctness:

1. Model returns assistant message with tool use block(s)
2. App runs tools
3. App appends tool result(s) in the required role/shape
4. Call model again with full history
5. Repeat until final text answer or stop condition

Never invent tool results. Never reorder `tool_use` / `tool_result` pairing carelessly.

6.4 RAG

Production RAG topics in the outline:

- Chunking strategies
- Embeddings (often a separate Bedrock embedding model via **Invoke**, not Converse)
- Lexical search (**BM25**) + vector hybrid
- Multi-index architectures, reranking
- **Contextual retrieval** (enrich chunks with context before embed/index)

Know *why* hybrid search exists (exact keywords vs semantic match) more than any single library.

Bedrock Knowledge Bases may appear in AWS-centric questions — still map to the same RAG concepts: ingest, chunk, embed, retrieve, generate, cite.

6.5 Advanced features

- **Extended thinking / reasoning** — extra tokens and cost. Current models use **adaptive thinking** (the model decides when/how deeply to think; depth tuned via **effort levels**) — fixed `budget_tokens` thinking budgets are **deprecated on 4.6 and removed (400 error) on newer frontier models**; verify the specific model card on Bedrock before using either shape
- **Vision** — image content blocks
- **Prompt caching** — reduce cost/latency for stable prefixes when available on the model/platform
- **Streaming** — better UX for long answers
- **Structured extraction** — constrain outputs (schemas / careful prompting); temperature applies **only on older models** — sampling params are removed on current frontier models, so schemas are the determinism lever

6.6 MCP and agents

MCP standardizes tools, resources, and prompts across clients/servers. Agents section connects Bedrock-backed Claude to automation patterns (Claude Code, computer use). For cert prep: MCP is a **protocol for modular tool/resource exposure**, not a Bedrock-only API.

Agent patterns still apply: parallelize, chain, route, debug with traces.

7. Comparison snapshot: Bedrock vs Anthropic API vs Vertex

Topic	Anthropic API	Amazon Bedrock	Google Vertex (Claude)
Auth	Anthropic API key	AWS IAM / credentials	GCP ADC / service account
Client	Anthropic	bedrock-runtime (Converse/Invoke)	AnthropicVertex
Model ID style	Anthropic IDs	anthropic... / us.anthropic...	Vertex publisher IDs (e.g. claude-sonnet-4-6)
Version field	Headers / SDK	anthropic_version in Invoke body; Converse abstracts much	anthropic_version: vertex-2023-10-16 on raw HTTP
Unified multi-model chat	Messages	Converse (Bedrock-wide)	Messages via Vertex backend
Billing / contracts	Anthropic	AWS	Google Cloud
Residency knobs	Anthropic policies	Regions + inference profiles	global / us / eu / regional endpoints
Enablement	API key access	Bedrock model access per account/Region	Model Garden enablement

Pick Bedrock when: AWS-centric IAM, Converse portability across Bedrock models, AWS compliance boundary, existing AWS networking/VPC patterns.

Pick Vertex when: GCP is the control plane, Google billing/residency, Model Garden partner path.

Pick Anthropic API when: fastest path to newest Anthropic-only features and simplest key-based auth.

Do not mix ID formats across clouds — that alone answers several exam distractors.

8. Exam traps (Bedrock-specific)

Trap	Reality
Looking for <code>bedrock:Converse</code> permission	Converse uses <code>bedrock:InvokeModel</code>
Using foundation ID when profile required	Switch to inference profile ID/ARN
Granting profile ARN only	Also grant destination FM ARNs for geo profiles
Assuming history is stored	Stateless; resend messages
Using Converse for embeddings	Invoke embedding models separately
Cross-Region profile for single-Region policy	Wrong residency tool; pin with application profile
Copying Vertex model IDs into Bedrock	Different catalog strings
Streaming <code>AccessDenied</code> with <code>Invoke</code> granted	Missing <code>InvokeModelWithResponseStream</code>
“Enable once globally”	Enablement is per account/Region patterns
Treating Guardrails as optional when question requires them	Wire Guardrails config on Converse when asked

9. Exam tips

- Enable model access **and** use a Region/profile where the model exists.
 - Prefer **Converse** unless the question forces native `Invoke`.
 - Remember **stateless** history management.
 - IAM for Converse $\approx$ `bedrock:InvokeModel` (+ stream variant).
 - Newer models $\rightarrow$ check **inference profile** IDs.
 - RAG answers should mention chunking + retrieval quality, not “just stuff the PDF in the prompt” for production.
 - Eval = systematic grading, not single anecdotal prompts.
 - Distinguish advisory prompting from tool/MCP plumbing.
 - For residency, name the profile strategy explicitly.
 - For tool loops, insist on correct message ordering.
-

10. Minimal practice sketch (conceptual)

1. Enable Claude in Bedrock console for your account/Region.
2. Create runtime client with AWS creds.
3. Call converse with an inference profile model ID and one user message.
4. Add a second turn by appending prior assistant output.
5. Add a tiny tool (e.g. calculator) via `toolConfig` and complete one tool loop.

6. Stream the same prompt with `converse_stream`.
7. Intentionally use a wrong Region/model ID once to recognize the failure mode.
8. Sketch an IAM policy that includes profile + destination FM ARNs.
- 9.

Write five eval cases and grade with code asserts + one model rubric.

11. Self-check Q&A (with answers)

- Q1.** Converse call fails with `AccessDenied` though `Invoke` worked yesterday with same role. Likely cause? **A.** Missing `bedrock:InvokeModel` (or resource ARN for the new model/profile) — Converse uses the same invoke permission family. If only streaming fails, check stream action.
- Q2.** Model ID from the catalog works in docs but fails in your Region. What next? **A.** Confirm Regional availability; switch to the correct **inference profile** ID for cross-Region routing.
- Q3.** Why doesn't Claude remember the previous turn on Bedrock? **A.** Runtime is stateless; you must resend full message history.
- Q4.** When choose `InvokeModel` over Converse for Claude? **A.** Need native Anthropic request/response fields or features Converse does not expose cleanly; or non-chat models (embeddings, etc.).
- Q5.** What belongs in `inferenceConfig` on Converse? **A.** Shared knobs like `maxTokens`, `temperature`, `topP` — Claude-specific extras often go in `additionalModelRequestFields`.
- Q6.** Name two RAG improvements beyond naive top-k embedding search. **A.** BM25 hybrid search; reranking; contextual retrieval; better chunking / multi-index.
- Q7.** What is MCP in one sentence? **A.** A protocol for exposing tools, resources, and prompts to AI clients in a modular, reusable way.
- Q8.** How do you keep multi-turn tool use correct? **A.** Append assistant tool-call messages and user/tool-result messages in order; never drop results mid-loop.
- Q9.** Cross-Region profile invoke: IAM allows the profile ARN only. Still `AccessDenied`. Why? **A.** Also need invoke permission on foundation model ARNs in each destination Region the profile routes to.
- Q10.** Which IAM action does `ConverseStream` require? **A.** `bedrock:InvokeModelWithResponseStream` (not only `InvokeModel`).
- Q11.** How do you pursue single-Region residency with newer Claude models that require profiles? **A.** Create an application inference profile pointing at one Region foundation model ARN; avoid multi-Region geo prefixes.
- Q12.** Why might embeddings use `Invoke` while chat uses Converse in the same app? **A.** Embeddings are not conversational Converse workloads; `Invoke` is the right modality API.
- Q13.** What is the Bedrock Claude `Invoke` body version field commonly set to? **A.** `anthropic_version` such as `bedrock-2023-05-31` (confirm current docs).
- Q14.** Code-based vs model-based eval — when prefer code-based? **A.** When success is objectively checkable (JSON schema, exact fields, unit asserts).
- Q15.** Name three differences vs Vertex Claude. **A.** AWS IAM vs GCP ADC; `anthropic.` / geo profile IDs vs publisher IDs; Converse vs Messages-on-Vertex.

- Q16.** Guardrails on Converse — conceptual role? **A.** Platform safety/filters applied around model calls when configured — not a substitute for app authz.
- Q17.** Batch tool calling means what at a high level? **A.** Model can request multiple tools in one turn; app returns multiple results before the next model call.
- Q18.** Prompt caching helps most when? **A.** Stable long prefixes (system, tools, large docs) reused across calls — when supported for that model/platform.
- Q19.** You enabled Claude in us-east-1 but call eu-west-1. What happens? **A.** Failure or unavailable model — enablement and availability are Region-sensitive; fix Region or use appropriate profile.
- Q20.** What does contextual retrieval add? **A.** Enrich chunks with surrounding document context before embedding/indexing to improve retrieval quality.
- Q21.** Agent pattern that classifies then sends to specialized prompts/tools? **A.** Routing.
- Q22.** Why is temperature usually lower for tool argument generation? **A.** Need more deterministic structured arguments for reliable execution.
- Q23.** Application inference profile primary benefits? **A.** Cost attribution/tags and/or pinning routing behavior (including single-Region sources) vs system geo profiles.
- Q24.** Is billing via Anthropic API key on classic Bedrock Runtime? **A.** No — AWS credentials and AWS billing for classic Bedrock Runtime Claude calls. —

12. Review checklist (before exam)

- ☐ Enablement vs credentials vs correct Region all distinguished
 - ☐ Foundation model ID vs geo profile vs application profile
 - ☐ Converse vs Invoke decision criteria memorized
 - ☐ IAM: InvokeModel for Converse; stream action for streams
 - ☐ Profile IAM includes destination FM ARNs
 - ☐ Stateless multi-turn history
 - ☐ toolConfig loop ordering
 - ☐ RAG hybrid / rerank / contextual retrieval vocabulary
 - ☐ Eval = systematic cases + code/model grading
 - ☐ Residency decision tree (global / geo / single-Region pin)
 - ☐ Bedrock vs Vertex vs Anthropic API comparison table
 - ☐ Streaming event reassembly conceptual understanding
-

13. Glossary

- **Bedrock Runtime** — API surface for invoking foundation models (bedrock-runtime client).
- **Converse** — Unified conversational inference API across Bedrock chat models.
- **ConverseStream** — Streaming variant of Converse.
- **InvokeModel** — Model-native body invocation API.
- **Inference profile** — Resource that routes inference (system geo or application).
- **Foundation model (FM) ARN** — Underlying model resource in a Region.
- **Application inference profile** — Customer-created profile for attribution and/or pinned routing.
- **Geo profile prefix** — e.g. us., eu., global. on profile IDs.

- **additionalModelRequestFields** — Escape hatch for model-specific Converse params.
 - **inferenceConfig** — Shared sampling/max token knobs on Converse.
 - **toolConfig** — Converse tool definitions and tool choice settings.
 - **Stateless inference** — No server-side chat memory between calls.
 - **BM25** — Lexical ranking function used in hybrid RAG.
 - **Contextual retrieval** — Prepend context to chunks before embed/index.
 - **MCP** — Model Context Protocol for tools/resources/prompts.
 - **Guardrails** — Bedrock safety filters configurable around calls.
 - **Knowledge Base** — Managed RAG retrieval component in AWS Bedrock ecosystem.
 - **Model-based grading** — Using a model to score outputs against a rubric.
 - **Code-based grading** — Programmatic asserts on outputs.
 - **AccessDeniedException** — IAM authorization failure from Bedrock APIs.
-

14. Deeper decision trees

14.1 API shape picker

Is the workload conversational messages with optional tools?

YES → Converse / ConverseStream

NO → Is it embeddings / image / native-only payload?

YES → InvokeModel / stream variant

NO → Re-check modality docs

14.2 Failure triage

AccessDenied?

→ Check action (Invoke vs InvokeWithResponseStream)

→ Check resource ARNs (profile + destination FMs)

→ Check account model enablement

Validation / model does not exist / on-demand not supported?

→ Wrong Region or need inference profile ID

Weird empty/partial answer?

→ maxTokens too low; stop reasons; tool loop incomplete

Tool loop spinning?

→ Missing/malformed tool results; unbounded max turns

14.3 Production readiness mini-rubric

1. Correct profile for residency and availability
2. Least-privilege IAM tested with deny cases
3. Timeouts/retries for stream and non-stream
4. Eval suite with fixed cases before prompt tweaks
5. Logging without secret leakage
6. Cost monitors on tokens and profile tags
7. Fallback model or Region strategy documented

15. Shared skill themes mapped to Bedrock fields

Claude skill theme	Bedrock expression
System prompt	system on Converse
Temperature	inferenceConfig.temperature
Tools	toolConfig (or native tools on Invoke)
Multi-turn	App-managed messages array
Streaming	converse_stream / invoke stream
Structured output	Careful prompting + schema validation; platform features where offered
Vision	Image content blocks in messages
Caching	Model/platform-specific cache fields when available
Agents	Your orchestration + optional Claude Code with Bedrock backend

16. Study rhythm

Day 1: Sections 1–5 (access, Converse/Invoke, IAM, residency). Draw the comparison table from memory. Day 2: Sections 6–10 (features, traps, practice sketch). Run the failure triage aloud. Day 3: All Q&A closed-book; checklist; skim glossary gaps only.

Mantra: Enable, profile, IAM, Converse-by-default, history-is-yours, residency-on-purpose.

Aligned to <https://academy.claude.com/courses/claude-with-amazon-bedrock>. Verify live model IDs, Regions, and IAM examples in AWS Bedrock documentation before production use.

17. Worked mini-scenarios (exam style)

Scenario 1 — New Sonnet in prod Lambda Team copies an old foundation model ID into converse. Error says on-demand throughput is not supported / use an inference profile. **Answer path:** Look up the system or application inference profile ID for that Claude version; update modelId; extend IAM for profile + destination FM ARNs; retest in the Lambda Region.

Scenario 2 — EU data policy Legal requires processing in the EU. Engineer uses global... profile because it “just worked” in a tutorial. **Answer path:** Wrong. Prefer eu_ geo profile for EU multi-Region residency, or a single-Region EU application profile if policy demands one Region. Document which guarantee you actually have.

Scenario 3 — Streaming chat widget Non-stream chat works; stream fails AccessDenied. **Answer path:** Add bedrock:InvokeModelWithResponseStream on the same resources; keep reassembling stream events in the client.

Scenario 4 — Tool-using support agent Model asks for two tools; app returns only one result; next call confuses roles. **Answer path:** Return all tool results paired correctly; preserve assistant tool-use message; then continue.

Scenario 5 — RAG “hallucinated policy” Naive top-k embedding retrieval misses exact clause IDs. **Answer path:** Add BM25/hybrid, consider rerank, improve chunking, try contextual retrieval; evaluate with fixed cases containing those clause IDs.

Scenario 6 — Multi-cloud migration Port from Anthropic API to Bedrock. Engineer pastes x-api-key and Anthropic base URL into AWS code. **Answer path:** Use AWS credentials + Bedrock Runtime; map Messages fields to Converse or keep Invoke with anthropic_version for Bedrock; change model IDs to Bedrock catalog forms.

Scenario 7 — Cost attribution Finance cannot see which product line spent Bedrock Claude tokens. **Answer path:** Application inference profiles with tags per product; invoke via those profile ARNs; dashboards by tag.

Scenario 8 — Eval disagreement Prompt looks great on three cherry-picked examples, fails silently in prod. **Answer path:** Build a held-out eval set; combine code-based checks for schema with model-based rubrics for quality; gate deploys on eval thresholds.

18. Quick reference card

Need	Bedrock move
Default chat/agent	Converse
Stream tokens	ConverseStream + stream IAM
Native Anthropic body / embeddings	InvokeModel
Cross-Region availability	us. / eu. / global. profile
Single-Region pin	Application profile → one FM ARN
Auth	AWS credential chain, not Anthropic key
Tools	toolConfig (+ loop)
Claude-only knobs	additionalModelRequestFields
Portable vs Vertex	Different IDs and auth entirely

One-sentence mantra: On Bedrock, enable the model, call the right profile with Invoke-family IAM, prefer Converse, own your history, and pick residency deliberately.

19. Extra Q&A (stretch)

Q25. Can you use the same boto3 Converse code for Claude and another Bedrock chat model? **A.** Often yes for the message shape — that portability is a Converse selling point — but model-specific fields and capabilities still differ.

Q26. What breaks if you omit max_tokens on native Invoke Messages-style Claude? **A.** Request validation errors — max_tokens is required in native Messages-style bodies.

Q27. Where do system prompts go on Converse vs a mistaken role: system inside messages?
A. Use the Converse system parameter (list of system blocks); do not invent a system role inside messages the way some other APIs do.

Q28. Why list GetInferenceProfile in some IAM policies? **A.** Required to run inference with an inference profile per AWS inference prerequisites docs.

Q29. Hybrid search in one line? **A.** Combine lexical (BM25) and vector retrieval so exact tokens and semantic matches both surface.

Q30. Computer use / Claude Code themes in this course — what to remember? **A.** They are agentic automation patterns that can ride on Bedrock-backed Claude; still need AWS auth, model access, and verification discipline.

20. Study close

If you can redraw (1) Converse vs Invoke, (2) IAM actions for stream/non-stream, (3) profile types for residency, and (4) the three-cloud comparison table without notes, you are ready for the Bedrock-differentiating exam items. Then refresh shared Claude skills (tools, RAG, MCP, eval) using the same vocabulary as the Anthropic API and Vertex tracks — only the transport and ID formats change.

Claude with Google Cloud's Vertex AI — Primary Study Notes

Note: This edition follows the ASD-STE100 writing rules: simple present tense, active voice, one word = one meaning, sentences of 20–25 words or less. Technical names (Claude, Vertex, Model Garden, ADC, MCP, prompting, caching, RAG) are exceptions and stay as written. Model IDs, endpoints, and feature matrices change. Check the current Google Cloud and Anthropic docs before the exam.

Disclaimer: These notes are **original** cert-prep notes. They align to the **public** Claude Academy course at [Claude with Google Cloud's Vertex AI](#). They are **not** Academy lesson dumps. They are **not** verbatim course prose. Confirm live model IDs, endpoints, and feature matrices in Google Cloud and Anthropic docs. The official course is the source of truth for quizzes.

Who it is for: Developers who add Claude features with Google Cloud Vertex AI / Agent Platform.

Prerequisites (course): Python, JSON basics, and a Google Cloud project with Vertex AI access.

How to use: Platform-specific material is the most valuable difference. This includes Model Garden enablement, ADC, global/us/eu/regional endpoints, publisher model IDs, and feature parity caveats. Shared Claude skills overlap the Bedrock track. Know those skills in the Vertex Messages-shaped client.

1. Course map (public modules)

The public page has about 66 lessons and multiple quizzes. It organizes roughly as:

1. **Accessing Claude with the API** — auth, requests, multi-turn, system prompts, temperature, streaming, structured output
2. **Prompt engineering techniques** — strategies, evaluation, automated grading
3. **Tool use with Claude** — function calling, multi-turn tools, batch tools, built-ins
4. **RAG** — chunking, embeddings, BM25 hybrid, multi-index, reranking, contextual retrieval
5. **Model Context Protocol** — tools, resources, prompts. Server/client lifecycle
6. **Agents and workflows** — parallelization, chaining, routing, debugging
7. Related themes: vision, PDF, citations, prompt caching, Claude Code / computer-use style apps

Shared Claude skills overlap heavily with the Bedrock course. What is **distinct** for exams is **GCP setup, auth, regions/endpoints, model ID format, and API differences vs Anthropic and Bedrock**.

Main sequence: Enable in Model Garden, then set ADC. Alternatively, gcloud, then select the endpoint class (global/us/eu/regional) → AnthropicVertex Messages call → own history → tools/RAG/eval → know parity gaps vs Anthropic API.

2. Setup and authentication

2.1 Enable Claude in Model Garden

1. Open Vertex AI / Model Garden in Google Cloud Console. Select the correct project.
2. Search **Anthropic** / Claude.
3. **Enable** the model(s) you need for the project (if they are not already enabled).
4. Confirm the model is available for the location/endpoint style you plan to use.

If you skip enablement, the docs work, but your project fails. This is a common failure.

2.2 gcloud and Application Default Credentials (ADC)

Typical local setup:

```
gcloud init
gcloud auth login
gcloud config set project YOUR_PROJECT_ID
gcloud auth application-default login
```

The Anthropic **Vertex** SDK uses Google auth (ADC / service account / workload identity). You do **not** pass an Anthropic API key for Vertex-backed calls.

Production: prefer **service accounts** with least-privilege Vertex AI permissions. Do not use long-lived user ADC on servers. Use workload identity on GKE/Cloud Run where possible.

2.3 Install SDK

Python example: `pip install -U "anthropic[vertex]"`. TypeScript: `@anthropic-ai/vertex-sdk`.

Client pattern:

```
from anthropic import AnthropicVertex
```

```
client = AnthropicVertex(project_id="YOUR_PROJECT_ID", region="global")
```



```
message = client.messages.create(
    model="claude-sonnet-4-6", # example – check current IDs
    max_tokens=1024,
    messages=[{"role": "user", "content": "Hello"}],
)
```

Env vars commonly involved: project id and region (ANTHROPIC_VERTEX_PROJECT_ID, CLOUD_ML_REGION, etc., depending on SDK docs).

Auth decision tree:

Local laptop prototyping?

→ gcloud auth application-default login (user ADC)

Cloud Run / GKE / GCE prod?

→ Service account + workload identity / attached SA

CI from GitHub without long-lived keys?

→ Workload identity federation patterns

Still tempted to set ANTHROPIC_API_KEY for Vertex?

→ Wrong product path – that is Anthropic API, not Vertex

3. How the Vertex Claude API differs from Anthropic API

The **Messages** shape is nearly the same. The platform differences are important:

Topic	Anthropic API	Claude on Vertex / Agent Platform
Auth	x-api-key / Anthropic key	Google Cloud credentials
Model location	In request body model	Often in URL path for raw HTTP. SDK still takes model=
Version marker	API version headers	Body field anthropic_version = vertex-2023-10-16 on raw HTTP
Endpoint	api.anthropic.com	...- aiplatform.googleapis.com or multi-region/global hosts
Billing / ToS	Anthropic	Google Cloud partner model terms
Some platform features	Batches, Files, certain server tools, etc.	Check “supported / not supported” lists — they differ

Raw HTTP reminder: the model goes in the publisher path /v1/projects/{PROJECT}/locations/{LOCATION}/ (and streamPredict for streaming where applicable) and the JSON body includes anthropic_version, messages, max_tokens, ...

SDK reminder: AnthropicVertex lets you keep writing messages.create(...) with familiar fields. The SDK handles Vertex routing.

Parity caveat rule: Same Claude family. The feature surface is not always the same. If a question says “on Vertex,” do not assume every Anthropic-only admin/files/batches/server-tool feature exists.

4. Regions and endpoints (high-value exam topic)

Agent Platform / Vertex offers three endpoint styles (public Google/Anthropic docs):

4.1 Global

- `region="global"`
- Dynamic routing for availability across supported regions with capacity
- Usually **standard pricing** (no residency premium)
- Best default when data residency is flexible
- Pay-as-you-go oriented. Provisioned throughput needs regional capacity

Hosts conceptually use global AI Platform endpoints (confirm current host strings in docs).

4.2 Multi-region (us or eu)

- Routes within that geography
- Google docs often cite these example hosts: `aiplatform.us.rep.googleapis.com` and `aiplatform.eu.rep.googleapis.com`.
- Data residency at **geo** granularity with better availability than a single region
- Typically a **pricing premium** vs global (Google commonly cites a premium of about 10 percent for newer models from 4.5 onward. Confirm the pricing pages).

4.3 Specific regional (e.g. us-east5, europe-west1)

- Strict single-region routing
- Needed for hard residency, some enterprise controls, **provisioned throughput**
- Also typically premium vs global
- Model availability can lag. Newest models may appear first on global / key regions

Decision flow for exams:

1. Flexible residency → `global`
2. Must stay in US or EU broadly → `us` / `eu`
3. Must pin one region or use provisioned throughput → specific region
4. Always verify the chosen Claude version is enabled and available in that location

Common exam error: You use `global` when the compliance question requires EU residency. Availability is not the same as residency.

Common exam error: You expect provisioned throughput on global. Public materials emphasize provisioned capacity on regional endpoints.

5. Model IDs on Vertex

Vertex publisher IDs look like Anthropic names. They follow **Google's catalog strings**. Examples (illustrative — always check live docs):

- `claude-sonnet-4-6`
- `claude-haiku-4-5@20251001` (some IDs include date suffixes)
- `claude-opus-4-6`

They are **not** Bedrock's `anthropic.claude-...` or `us.anthropic....` inference profile strings. They can also differ slightly from Anthropic API aliases.

Lifecycle (deprecated/retired) dates on partner platforms can differ from Anthropic's own schedule. Exam answers must say "check Google's Claude model page." Do not invent dates.

ID comparison cheat:

Platform	Example flavor
Anthropic API	Anthropic model aliases / versioned IDs
Bedrock	<code>anthropic.claude-...</code> or <code>us.anthropic.claude-...</code>
Vertex	<code>claude-sonnet-4-6</code> publisher ID

Copy-paste across clouds is an intentional distractor.

6. Feature support (what to expect)

Typically available on Vertex Claude (confirm per model):

- Messages API (chat)
- Tool use (including several Anthropic tool types, with caveats)
- Prompt caching, thinking/extended reasoning where offered
- Vision and documents (check the payload size limits. Vertex may cap the request size, for example tens of MB).
- Citations, structured outputs where documented
- Streaming via SDK/stream APIs

The surface is often **not** the same as the full Anthropic API. Public docs identify certain Files and URL helpers, server-side tools, Message Batches, and Admin APIs, and some managed-agent infrastructure. If a question says "on Vertex," do not assume every Anthropic-only feature exists.

Context windows: newer Claude generations may offer large windows (e.g. 1M-class on listed models). Older ones may be smaller. Prefer "check model card" over memorizing every number.

Parity decision rule:

Feature required by product?

→ Check Vertex/Agent Platform support matrix for THAT model

→ If missing, either redesign, use Anthropic API (if policy allows), or wait for parity

Never answer "Claude can do X on Anthropic API, therefore Vertex too" without checking

7. Course skill themes (shared with Bedrock track)

7.1 Multi-turn and system prompts

Alternate user/assistant. Keep history coherent. Put durable behavior in system. HTTP calls are stateless. Your app stores history.

7.2 Prompt engineering and eval

Use XML-ish structure, examples, and output control. Build **test sets**. Grade with model-based and code-based scorers. Production quality is an iteration loop. It is not a single clever prompt.

7.3 Tool use

JSON Schema tools → model tool calls → your execution → tool results back into messages. Multi-turn and batch patterns matter for agents.

7.4 RAG

Chunk → embed → retrieve (vector + **BM25** hybrid) → optionally rerank → contextual retrieval techniques → answer with citations when available. The conceptual pipeline is the same as other Claude cloud courses. Storage/embeddings may use GCP services (Vertex AI embeddings, Vector Search, etc.).

7.5 MCP

Define tools, resources, prompt templates. Run MCP servers/clients. Integrate into apps. Protocol knowledge transfers across Anthropic API, Bedrock, and Vertex transports.

7.6 Agents and workflows

Patterns to memorize:

Pattern	Idea	Use when
Parallelization	Fan-out independent subtasks	Embarrassingly parallel work
Chaining	Output of step N feeds step N+1	Pipelines with dependencies
Routing	Classify then send to specialized prompts/tools	Mixed intent traffic
Debugging	Log traces, constrain tools, evaluate intermediate steps	Agents that behave without consistency

8. Vertex vs Bedrock vs Anthropic API (comparison sheet)

Dimension	Anthropic API	Bedrock	Vertex AI Claude
Cloud identity	Anthropic account	AWS IAM	GCP IAM / ADC
Primary SDK	Anthropic	AWS SDK + Converse	AnthropicVertex

Dimension	Anthropic API	Bedrock	Vertex AI Claude
Model ID flavor	Anthropic IDs	anthropic. / geo profiles	Publisher IDs (claude-...)
Unified multi-model chat API	Messages	Converse (Bedrock-wide)	Messages via Vertex backend
Version marker	Anthropic headers	bedrock-2023-05-31 style on Invoke	vertex-2023-10-16 on raw HTTP
Residency control	Anthropic options	Regions + inference profiles	global / us / eu / regional
Enterprise packaging	Anthropic contracts	AWS Marketplace / Bedrock	Google Cloud / Model Garden
Enablement	API access	Bedrock model access	Model Garden enable

Select Vertex when: GCP is already the control plane, you need Google billing/residency, or org policy mandates Vertex. **Pick Bedrock when:** AWS-centric IAM, Converse portability across Bedrock models, AWS compliance boundary. **Pick Anthropic API when:** fastest path to newest Anthropic-only features and simplest key-based auth.

9. Exam traps (Vertex-specific)

Trap	Reality
Using Anthropic API key with AnthropicVertex	Wrong auth — need GCP ADC/SA
Pasting Bedrock model IDs	Wrong catalog
Choosing global under EU residency mandate	Wrong endpoint class
Assuming provisioned throughput on global	Typically regional
Assuming full Anthropic feature parity	Check support matrix
Forgetting Model Garden enable	Project-level enablement required
Putting model only in body for raw HTTP	Model often in URL path. Body needs anthropic_version
Believing Vertex stores chat history	Stateless. App resends messages
Ignoring request size limits for PDFs/images	Vertex may enforce payload caps
Using europe-west1 for a brand-new model only on global	Regional lag

10. Exam tips

- Setup order: enable model → gcloud/ADC → AnthropicVertex(project, region).
- Memorize **anthropic_version: vertex-2023-10-16** for raw HTTP.
- Region choice is a **compliance and availability** question. It is not a minor style question.
- Do not paste Bedrock model IDs into Vertex or vice versa.
- Stateless multi-turn history is still your job.
- For “feature X on Vertex?” — prefer “supported if listed for Agent Platform / that model.” Do not use Anthropic-API assumptions.

- Agent workflow vocabulary: parallelize, chain, route.
 - RAG answers must cite hybrid retrieval and evaluation. Do not cite embeddings only.
 - Pricing premium for us/eu/regional vs global is a known exam-adjacent fact. Confirm current numbers on pricing pages.
 - Service accounts in production are better than user ADC on servers.
-

11. Minimal practice sketch (conceptual)

1. Enable a Claude model in Model Garden.
 2. Configure gcloud project + ADC.
 3. Call AnthropicVertex with `region="global"` and a short Messages request.
 4. Repeat with `region="us"` and note endpoint/pricing implications.
 5. Build a two-turn chat. Append the assistant output.
 6. Add one custom tool and complete a tool loop.
 7. Sketch a tiny RAG path (chunk → retrieve → answer) and an eval rubric with 5 fixed cases.
 8. Attempt a raw HTTP `rawPredict` once to see `anthropic_version` and path-based model ID.
 9. Intentionally use a Bedrock-style model ID once so you recognize the failure mode.
 10. Document which features you rely on. Verify each against the Vertex support list.
-

12. Self-check Q&A (with answers)

Q1. How does local auth usually work for AnthropicVertex? **A.** Google Application Default Credentials after `gcloud auth application-default login` (or a service account in prod). It is not an Anthropic API key.

Q2. What goes in the raw request body instead of/in addition to Anthropic headers? **A.** `anthropic_version` set to `vertex-2023-10-16`. The model often lives in the URL path.

Q3. When do you select `region="eu"` instead of `global`? **A.** When you need EU geographic data residency with multi-region availability inside the EU. You accept possible premium pricing.

Q4. Why might a brand-new Claude model fail in `eu-west1` but work with `global`? **A.** Regional availability lags. Newest models often appear on global or select regions first.

Q5. Name one difference vs Bedrock model IDs. **A.** Vertex uses publisher IDs like `claude-sonnet-4-6`. Bedrock uses `anthropic.claude-...` or `us.anthropic....` profiles.

Q6. Is conversation history stored by Vertex between calls? **A.** No. Your application must resend messages each turn (unless you build your own session store).

Q7. What agent pattern sends different query types to different tools/prompts? **A.** Routing.

Q8. Give two reasons Converse knowledge still helps even if you deploy on Vertex. **A.** Shared Claude concepts (tools, system prompts, eval). Comparison questions on cloud exams often contrast Bedrock's Converse with Vertex's Messages-on-GCP approach.

Q9. When is a specific regional endpoint required? **A.** Hard single-region residency and/or provisioned throughput (per public guidance).

Q10. What is Model Garden's role? **A.** Discovery and enablement of partner models (including Claude) for your GCP project.

- Q11.** Why prefer service accounts in production? **A.** Least privilege, rotatable, no interactive user ADC on servers, fits workload identity.
- Q12.** Streaming on Vertex — does it change tool-loop correctness rules? **A.** No. You still assemble the final message and keep tool_use/tool_result pairing correct.
- Q13.** Name two features that may differ from Anthropic API on Vertex. **A.** Examples: Message Batches, some Files/URL helpers, some server tools, Admin APIs. Always verify current lists.
- Q14.** Hybrid RAG means what? **A.** Combining lexical (BM25) and vector retrieval before generation.
- Q15.** Parallelization vs chaining? **A.** Parallelization fans out independent work. Chaining sequences dependent steps.
- Q16.** What host flavor is associated with US multi-region? **A.** Often `aiplatform.us.rep.googleapis.com` style hosts (confirm docs).
- Q17.** Can you use Claude on Vertex without enabling it in the project? **A.** No. You must have Enablement in Model Garden / project access.
- Q18.** Where does max_tokens go? **A.** In the Messages request body (SDK parameter). It is the required upper bound on generation.
- Q19.** Temperature for structured extraction? **A.** On older models (4.6 and earlier): lower for more deterministic outputs. Current frontier models have removed sampling params entirely. Rely on schemas/structured outputs for determinism.
- Q20.** What should you check before assuming 1M context? **A.** The specific model card on Vertex. Not all models share the same window.
- Q21.** Workload identity purpose in one line? **A.** Let cloud workloads obtain GCP credentials without long-lived downloaded keys.
- Q22.** Citations feature — exam caution? **A.** Available where documented for that model/platform. Do not invent citations support.
- Q23.** Why might PDF/image requests fail despite valid auth? **A.** Payload size limits or unsupported media handling on Vertex path.
- Q24.** Bedrock additionalModelRequestFields analog on Vertex? **A.** Not the same API. On Vertex you generally pass Anthropic Messages fields the SDK/API supports. Extras follow Anthropic/Vertex docs, not Bedrock Converse naming. —

13. Review checklist (before exam)

- ☐ Model Garden enablement steps
- ☐ ADC vs service account vs Anthropic API key (which belongs where)
- ☐ global vs us/eu vs regional decision tree
- ☐ Raw HTTP: path model ID + anthropic_version: vertex-2023-10-16
- ☐ Publisher ID format vs Bedrock IDs
- ☐ Feature parity caveats vs Anthropic API
- ☐ Stateless history + tool loop ordering
- ☐ Agent patterns: parallelize, chain, route
- ☐ RAG hybrid / contextual retrieval vocabulary
- ☐ Eval = systematic code + model grading

- ❑ Three-cloud comparison table from memory
 - ❑ Provisioned throughput tied to regional endpoints
-

14. Glossary

- **Model Garden** — GCP catalog to discover/enable partner and Google models.
 - **Agent Platform** — Google Cloud surface for agentic / partner model serving (naming in docs evolves).
 - **ADC** — Application Default Credentials for Google auth.
 - **AnthropicVertex** — Official Anthropic SDK client for Vertex-backed Claude.
 - **Publisher model ID** — e.g. `claude-sonnet-4-6` under publisher `anthropic`.
 - **rawPredict** — Vertex raw prediction endpoint for Anthropic-shaped JSON bodies.
 - **anthropic_version** — Body field. Vertex uses `vertex-2023-10-16` on raw HTTP.
 - **Global endpoint** — Dynamic multi-region routing for availability. Flexible residency.
 - **Multi-region endpoint** — `us` or `eu` geo routing with geo residency.
 - **Regional endpoint** — Single-region routing. Residency pin. Provisioned throughput.
 - **Provisioned throughput** — Reserved capacity (typically regional).
 - **Workload identity** — Cloud workload auth without static keys.
 - **Feature parity** — Whether Vertex supports the same capabilities as Anthropic API for a feature.
 - **BM25** — Lexical retrieval component in hybrid RAG.
 - **Contextual retrieval** — Context-enriched chunks before embedding.
 - **Routing / chaining / parallelization** — Core agent workflow patterns.
 - **Code-based grading** — Programmatic eval asserts.
 - **Model-based grading** — Rubric scoring via another model call.
-

15. Worked mini-scenarios

Scenario 1 — Wrong key An engineer sets `ANTHROPIC_API_KEY` and wonders why `AnthropicVertex` fails in a VPC-SC GCP project. **Answer:** Use GCP credentials (ADC/SA). Anthropic API keys are a different product path.

Scenario 2 — Compliance Policy: process in EU. Code uses `region="global"` for cheaper tokens. **Answer:** Switch to `eu` multi-region or a specific EU regional endpoint. Accept premium if applicable. Verify model availability.

Scenario 3 — New model lag `Opus` is brand-new on global. The regional endpoint 404s/unavailable. **Answer:** Use global (if residency allows) or wait/enable in the needed region. Do not invent Bedrock-style profile prefixes.

Scenario 4 — Raw HTTP debugging SDK works. Hand-rolled curl fails. **Answer:** Check bearer token from `gcloud`, URL location segment, publisher path model ID, and `anthropic_version` in JSON.

Scenario 5 — Parity miss App depends on an Anthropic-only batch API. Port to Vertex breaks. **Answer:** Check the support matrix. Redesign with your own queue, or stay on Anthropic API if policy allows.

Scenario 6 — Agent design Mixed intents: reset password vs order status vs FAQ. **Answer:** Routing pattern to specialized prompts/tools. Evaluate each branch.

Scenario 7 — RAG on GCP Exact SKU strings missed by vector-only search. **Answer:** Hybrid BM25 + vectors. Rerank. Use contextual retrieval. Use fixed eval cases with those SKUs.

Scenario 8 — Prod auth hardening A developer commits user ADC JSON to the repo for Cloud Run. **Answer:** Stop. Use the attached service account or workload identity. Rotate the exposed credentials.

16. Deeper endpoint comparison table

	Global	Multi-region us/eu	Specific regional
Residency	Flexible / none guaranteed	Geo-level	Single region
Availability	Highest (dynamic)	High within geo	Depends on one region
Typical price	Baseline	Premium (often ~10% on newer models)	Premium
Provisioned throughput	Generally no	Check docs	Yes (typical home)
Best for	Most apps without residency rules	US/EU compliance + HA	Hard pin / reserved capacity

17. Study rhythm

Day 1: Setup, auth, endpoints, model IDs. Draw residency decision tree. Day 2: Feature parity, shared skills, comparison vs Bedrock/Anthropic. Day 3: Q&A closed-book + scenarios + checklist.

Rule to memorize: Enable in Garden, authenticate with Google, select the endpoint for residency, use publisher IDs, assume Messages-not-Converse, verify parity before you promise features.

18. Mapping shared Claude skills to Vertex request fields

Skill theme	Vertex expression
System prompt	top-level system on Messages
Temperature	temperature request field
Tools	tools / tool choice fields (Anthropic shape)
Multi-turn	App-managed messages
Streaming	SDK stream helpers / streamPredict
Structured output	Schema prompting + validation. Platform features if listed
Vision / PDF	Content blocks. Watch size limits
Caching	Prompt cache fields where supported on Vertex model
Thinking	Extended thinking params where supported
Agents	Your orchestration (parallel/chain/route) + optional Claude Code

Contrast with Bedrock naming: Bedrock Converse uses `inferenceConfig`, `toolConfig`, `additionalModelRequestFields`. Vertex stays closer to Anthropic Messages names via `AnthropicVertex`.

19. Failure triage tree

Auth error / 401 / 403?

- ADC present? Correct project? SA roles include Vertex AI user permissions?
- Model enabled in Model Garden for this project?

404 / model not found?

- Wrong publisher ID string?
- Model not available in chosen location?
- Accidentally used Bedrock-style ID?

Empty/truncated output?

- `max_tokens` too low; stop reasons; tool loop incomplete

Tool loop errors?

- Missing `tool_result`; role ordering; schema mismatch

Residency audit fail?

- `global` used under geo mandate → move to `us/eu/regional`
-

20. Extra self-check Q&A

Q25. What does `locations/global` mean in a Vertex path? **A.** Use of the global endpoint location rather than a single region like `us-east5`.

Q26. Why might finance care about `us/eu` endpoints? **A.** Premium pricing vs global for newer models. Cost and compliance both matter.

Q27. Chaining example in one line? **A.** Extract entities → fetch records → draft answer. Each step feeds the next.

Q28. Is MCP Vertex-specific? **A.** No. MCP is transport-agnostic. Vertex is one place you might host or consume tools alongside Claude.

Q29. Computer use / Claude Code in this course — takeaway? **A.** Agentic apps can run with Vertex-backed Claude. You still need GCP auth, enablement, and verification.

Q30. Best first endpoint for a startup with no residency rules? **A.** Global — availability and baseline pricing. Then revisit if compliance appears.

Q31. How do you pass project id to the SDK? **A.** `AnthropicVertex(project_id=..., region=...)` and/or documented env vars.

Q32. What is wrong with storing only the last user message each turn? **A.** You lose assistant/tool history. This breaks multi-turn coherence and tool loops.

Q33. Code-based grading example for a Vertex app? **A.** Assert JSON schema, required fields, and forbidden PII patterns on outputs in CI.

Q34. Model-based grading example? **A.** Rubric prompt scoring helpfulness/groundedness on a 1–5 scale with a fixed judge prompt.

Q35. Why compare Bedrock Converse on an exam for a Vertex course? **A.** Cloud choice questions test whether you know auth, IDs, and API shapes differ even when Claude skills transfer. —

21. Production readiness mini-rubric (Vertex)

1. Model enabled in the correct project. ID verified against Model Garden card
 2. Endpoint class matches residency policy (global / us / eu / regional)
 3. Prod uses service account or workload identity — not laptop user ADC
 4. IAM roles least-privilege for prediction only where possible
 5. Timeouts/retries for stream and non-stream paths
 6. Eval suite gates prompt/model changes
 7. Logging redacts secrets and sensitive prompts as required
 8. Cost alerts on token usage. Understand premium endpoint pricing
 9. Documented fallback if regional capacity fails (only if policy allows)
 10. Feature dependencies checked against Vertex support matrix
-

22. Side-by-side request anatomy (conceptual)

Anthropic API: host `api.anthropic.com` + API key header + body `{model, messages, max_tokens, ...}`

Bedrock Converse: AWS SigV4 to `bedrock-runtime` + `{modelId, messages, inferenceConfig, toolConfig, ...}`

Vertex rawPredict: Google bearer token to `...aiplatform.../publishers/anthropic/models/{MODEL}:rawPredict` + body `{anthropic_version, messages, max_tokens, ...}` (model in path)

If you can rewrite one hello-world request across all three shapes, you can answer the cloud comparison questions.

23. Team onboarding one-pager (what to paste in a wiki)

1. Enable Claude models X/Y in Model Garden for project P
2. Install `anthropic[vertex]`. Set project. Prefer `region=global` unless compliance says otherwise
3. Local: `gcloud auth application-default login`
4. Prod: service account with documented roles
5. Never commit credentials
6. Use publisher model IDs from the model card — do not copy Bedrock IDs
7. Own conversation history in our session store service
8. Tools must return results for every `tool_use`
9. Before you use an advanced Anthropic-only feature, check the Vertex support list
10. Run the shared eval pack before you promote prompt changes.

24. Quick reference card

Need	Vertex move
Auth	ADC / service account
Enable	Model Garden
Default location	global if residency flexible
US/EU residency + HA	us / eu
Hard pin / provisioned	Specific region
SDK	AnthropicVertex
Raw version field	vertex-2023-10-16
Model ID	Publisher claude-...
History	App-managed
vs Bedrock	Messages not Converse. Different IDs/auth

One-sentence rule: On Vertex, enable in Garden and authenticate with Google. Select the endpoint for residency, call Messages with publisher IDs, and verify feature parity before you promise capabilities.

25. Study close

Redraw these four items without notes. The list also includes (1) the three endpoint classes, (2) ADC vs API key, (3) publisher vs Bedrock ID examples, (4) the three-cloud comparison table. If you can, you are ready for Vertex-differentiating items. Then refresh shared Claude skills (tools, RAG, MCP, agents, eval). The ideas are the same as Bedrock/Anthropic tracks. Vertex transport and constraints differ.

Aligned to <https://academy.claude.com/courses/claude-with-google-cloud-s-vertex-ai>. Confirm model IDs, regions, pricing premiums, and feature matrices on Google Cloud and Anthropic docs before shipping.

26. Final flash drills

Say out loud without notes:

1. Enable → ADC → endpoint class → publisher model ID → Messages call.
2. Global vs us/eu vs regional in one sentence each.
3. Three reasons Vertex differs from Anthropic API (auth, version field/path, feature parity).
4. Three reasons Vertex differs from Bedrock (IAM cloud, Converse vs Messages, ID format).
5. Name parallelization, chaining, and routing with a one-line example each.

If any drill fails, re-read only that section's tables, then retry. Prefer short closed-book loops over re-reading the entire file end-to-end.

Claude with Google Cloud's Vertex AI — Primary Study Notes

Disclaimer: These are **original** cert-prep notes aligned to the **public** Claude Academy course at [Claude with Google Cloud's Vertex AI](#). They are **not** Academy lesson dumps or verbatim course prose. Confirm live model IDs, endpoints, and feature matrices in Google Cloud and Anthropic docs. Completing the official course remains the source of truth for quizzes.

Who it is for: Developers adding Claude features via Google Cloud Vertex AI / Agent Platform.

Prerequisites (course): Python, JSON basics, Google Cloud project with Vertex AI access.

How to use: Platform-specific material (Model Garden enablement, ADC, global/us/eu/regional endpoints, publisher model IDs, feature parity caveats) is the highest-yield differentiator. Shared Claude skills overlap the Bedrock track — know them in Vertex's Messages-shaped client.

1. Course map (public modules)

Public page (~66 lessons, multiple quizzes) organizes roughly as:

1. **Accessing Claude with the API** — auth, requests, multi-turn, system prompts, temperature, streaming, structured output
2. **Prompt engineering techniques** — strategies, evaluation, automated grading
3. **Tool use with Claude** — function calling, multi-turn tools, batch tools, built-ins
4. **RAG** — chunking, embeddings, BM25 hybrid, multi-index, reranking, contextual retrieval
5. **Model Context Protocol** — tools, resources, prompts; server/client lifecycle
6. **Agents and workflows** — parallelization, chaining, routing, debugging
7. Related themes: vision, PDF, citations, prompt caching, Claude Code / computer-use style apps

Shared Claude skills overlap heavily with the Bedrock course. What is **distinct** for exams is **GCP setup, auth, regions/endpoints, model ID format, and API differences vs Anthropic and Bedrock**.

Spine: Enable in Model Garden → ADC/gcloud → pick endpoint class (global/us/eu/regional) → AnthropicVertex Messages call → own history → tools/RAG/eval → know parity gaps vs Anthropic API.

2. Setup and authentication

2.1 Enable Claude in Model Garden

1. Open Vertex AI / Model Garden in Google Cloud Console (correct project selected).
2. Search **Anthropic** / Claude.
3. **Enable** the model(s) you need for the project (if not already enabled).
4. Confirm the model is available for the location/endpoint style you plan to use.

Skipping enablement is a classic “works in docs, fails in my project” failure.

2.2 gcloud and Application Default Credentials (ADC)

Typical local setup:

```
gcloud init
gcloud auth login
gcloud config set project YOUR_PROJECT_ID
gcloud auth application-default login
```

The Anthropic **Vertex** SDK uses Google auth (ADC / service account / workload identity). You do **not** pass an Anthropic API key for Vertex-backed calls.

Production: prefer **service accounts** with least-privilege Vertex AI permissions over long-lived user ADC on servers. Use workload identity on GKE/Cloud Run where possible.

2.3 Install SDK

Python example: `pip install -U "anthropic[vertex]"`. TypeScript: `@anthropic-ai/vertex-sdk`.

Client pattern:

```
from anthropic import AnthropicVertex

client = AnthropicVertex(project_id="YOUR_PROJECT_ID", region="global")
message = client.messages.create(
    model="claude-sonnet-4-6", # example - check current IDs
    max_tokens=1024,
    messages=[{"role": "user", "content": "Hello"}],
)
```

Env vars commonly involved: project id and region (ANTHROPIC_VERTEX_PROJECT_ID, CLOUD_ML_REGION, etc., depending on SDK docs).

Auth decision tree:

Local laptop prototyping?

→ `gcloud auth application-default login` (user ADC)

Cloud Run / GKE / GCE prod?

→ Service account + workload identity / attached SA

CI from GitHub without long-lived keys?

→ Workload identity federation patterns

Still tempted to set ANTHROPIC_API_KEY for Vertex?

→ Wrong product path – that is Anthropic API, not Vertex

3. How the Vertex Claude API differs from Anthropic API

Nearly the same **Messages** shape, with important platform differences:

Topic	Anthropic API	Claude on Vertex / Agent Platform
Auth	<code>x-api-key</code> / Anthropic key	Google Cloud credentials
Model location	In request body <code>model</code>	Often in URL path for raw HTTP; SDK still takes <code>model=</code>

Topic	Anthropic API	Claude on Vertex / Agent Platform
Version marker	API version headers	Body field anthropic_version = vertex-2023-10-16 on raw HTTP
Endpoint	api.anthropic.com	...- aiplatform.googleapis.com or multi-region/global hosts
Billing / ToS	Anthropic	Google Cloud partner model terms
Some platform features	Batches, Files, certain server tools, etc.	Check “supported / not supported” lists — they differ

Raw HTTP reminder: model goes in the publisher path /v1/projects/{PROJECT}/locations/{LOCATION}/pub (and streamPredict for streaming where applicable) and the JSON body includes anthropic_version, messages, max_tokens, ...

SDK reminder: AnthropicVertex lets you keep writing messages.create(...) with familiar fields while it handles Vertex routing.

Parity caveat mantra: Same Claude family, not always the same feature surface. If a question says “on Vertex,” do not assume every Anthropic-only admin/files/batches/server-tool feature exists.

4. Regions and endpoints (high-value exam topic)

Agent Platform / Vertex offers three endpoint styles (public Google/Anthropic docs):

4.1 Global

- region="global"
- Dynamic routing for availability across supported regions with capacity
- Usually **standard pricing** (no residency premium)
- Best default when data residency is flexible
- Pay-as-you-go oriented; provisioned throughput needs regional capacity

Hosts conceptually use global AI Platform endpoints (confirm current host strings in docs).

4.2 Multi-region (us or eu)

- Routes within that geography
- Example hosts often cited: aiplatform.us.rep.googleapis.com/aiplatform.eu.rep.googleapis.com
- Data residency at **geo** granularity with better availability than a single region
- Typically a **pricing premium** vs global (commonly cited ~10% for newer models from 4.5 onward — confirm pricing pages)

4.3 Specific regional (e.g. us-east5, europe-west1)

- Strict single-region routing
- Needed for hard residency, some enterprise controls, **provisioned throughput**
- Also typically premium vs global
- Model availability can lag; newest models may appear first on global / key regions

Decision flow for exams:

1. Flexible residency → global
2. Must stay in US or EU broadly → us / eu
3. Must pin one region or use provisioned throughput → specific region
4. Always verify the chosen Claude version is enabled and available in that location

Exam trap: Using global when the compliance question requires EU residency — availability is not the same as residency.

Exam trap: Expecting provisioned throughput on global — public materials emphasize provisioned capacity on regional endpoints.

5. Model IDs on Vertex

Vertex publisher IDs look like Anthropic names but follow **Google’s catalog strings**, for example (illustrative — always check live docs):

- claude-sonnet-4-6
- claude-haiku-4-5@20251001 (some IDs include date suffixes)
- claude-opus-4-6

They are **not** Bedrock’s anthropic.claude-... or us.anthropic.... inference profile strings, and they may differ slightly from Anthropic API aliases.

Lifecycle (deprecated/retired) dates on partner platforms can differ from Anthropic’s own schedule — exam answers should say “check Google’s Claude model page,” not invent dates.

ID comparison cheat:

Platform	Example flavor
Anthropic API	Anthropic model aliases / versioned IDs
Bedrock	anthropic.claude-... or us.anthropic.claude-...
Vertex	claude-sonnet-4-6 publisher ID

Copy-paste across clouds is an intentional distractor.

6. Feature support (what to expect)

Typically available on Vertex Claude (confirm per model):

- Messages API (chat)
- Tool use (including several Anthropic tool types, with caveats)
- Prompt caching, thinking/extended reasoning where offered
- Vision / documents (watch **payload size limits** — Vertex may cap request size, e.g. tens of MB)

- Citations, structured outputs where documented
- Streaming via SDK/stream APIs

Often **not** the same as full Anthropic API surface — examples called out in public docs over time include certain Files/URL input helpers, some server-side tools, Message Batches, Admin APIs, and some managed-agent infrastructure. If a question says “on Vertex,” do not assume every Anthropic-only feature exists.

Context windows: newer Claude generations may offer large windows (e.g. 1M-class on listed models); older ones may be smaller. Prefer “check model card” over memorizing every number.

Parity decision rule:

Feature required by product?

→ Check Vertex/Agent Platform support matrix for THAT model

→ If missing, either redesign, use Anthropic API (if policy allows), or wait for parity

Never answer "Claude can do X on Anthropic API, therefore Vertex too" without checking

7. Course skill themes (shared with Bedrock track)

7.1 Multi-turn and system prompts

Alternate user/assistant; keep history coherent; put durable behavior in system. Stateless across HTTP calls — your app stores history.

7.2 Prompt engineering and eval

XML-ish structure, examples, output control; build **test sets** and grade with model-based and code-based scorers. Production quality = iteration loop, not a single clever prompt.

7.3 Tool use

JSON Schema tools → model tool calls → your execution → tool results back into messages. Multi-turn and batch patterns matter for agents.

7.4 RAG

Chunk → embed → retrieve (vector + **BM25** hybrid) → optionally rerank → contextual retrieval techniques → answer with citations when available. Same conceptual pipeline as other Claude cloud courses; storage/embeddings may use GCP services (Vertex AI embeddings, Vector Search, etc.).

7.5 MCP

Define tools, resources, prompt templates; run MCP servers/clients; integrate into apps. Protocol knowledge transfers across Anthropic API, Bedrock, and Vertex transports.

7.6 Agents and workflows

Patterns to memorize:

Pattern	Idea	Use when
Parallelization	Fan-out independent subtasks	Embarrassingly parallel work
Chaining	Output of step N feeds step N+1	Pipelines with dependencies
Routing	Classify then send to specialized prompts/tools	Mixed intent traffic
Debugging	Log traces, constrain tools, evaluate intermediate steps	Flaky agents

8. Vertex vs Bedrock vs Anthropic API (comparison sheet)

Dimension	Anthropic API	Bedrock	Vertex AI Claude
Cloud identity	Anthropic account	AWS IAM	GCP IAM / ADC
Primary SDK	Anthropic	AWS SDK + Converse	AnthropicVertex
Model ID flavor	Anthropic IDs	anthropic. / geo profiles	Publisher IDs (claude-...)
Unified multi-model chat API	Messages	Converse (Bedrock-wide)	Messages via Vertex backend
Version marker	Anthropic headers	bedrock-2023-05-31 style on Invoke	vertex-2023-10-16 on raw HTTP
Residency control	Anthropic options	Regions + inference profiles	global / us / eu / regional
Enterprise packaging	Anthropic contracts	AWS Marketplace / Bedrock	Google Cloud / Model Garden
Enablement	API access	Bedrock model access	Model Garden enable

Pick Vertex when: GCP is already the control plane, you need Google billing/residency, or org policy mandates Vertex. **Pick Bedrock when:** AWS-centric IAM, Converse portability across Bedrock models, AWS compliance boundary. **Pick Anthropic API when:** fastest path to newest Anthropic-only features and simplest key-based auth.

9. Exam traps (Vertex-specific)

Trap	Reality
Using Anthropic API key with AnthropicVertex	Wrong auth — need GCP ADC/SA
Pasting Bedrock model IDs	Wrong catalog
Choosing global under EU residency mandate	Wrong endpoint class
Assuming provisioned throughput on global	Typically regional
Assuming full Anthropic feature parity	Check support matrix
Forgetting Model Garden enable	Project-level enablement required
Putting model only in body for raw HTTP	Model often in URL path; body needs anthropic_version

Trap	Reality
Believing Vertex stores chat history	Stateless; app resends messages
Ignoring request size limits for PDFs/images	Vertex may enforce payload caps
Using europe-west1 for a brand-new model only on global	Regional lag

10. Exam tips

- Setup order: enable model → gcloud/ADC → AnthropicVertex(project, region).
- Memorize **anthropic_version: vertex-2023-10-16** for raw HTTP.
- Region choice is a **compliance and availability** question, not cosmetics.
- Do not paste Bedrock model IDs into Vertex or vice versa.
- Stateless multi-turn history is still your job.
- For “feature X on Vertex?” — prefer “supported if listed for Agent Platform / that model,” not Anthropic-API assumptions.
- Agent workflow vocabulary: parallelize, chain, route.
- RAG answers should cite hybrid retrieval and evaluation, not only embeddings.
- Pricing premium for us/eu/regional vs global is a known exam-adjacent fact — confirm current numbers on pricing pages.
- Service accounts in prod beat user ADC on servers.

11. Minimal practice sketch (conceptual)

1. Enable a Claude model in Model Garden.
2. Configure gcloud project + ADC.
3. Call AnthropicVertex with region="global" and a short Messages request.
4. Repeat with region="us" and note endpoint/pricing implications.
5. Build a two-turn chat by appending assistant output.
6. Add one custom tool and complete a tool loop.
7. Sketch a tiny RAG path (chunk → retrieve → answer) and an eval rubric with 5 fixed cases.
8. Attempt a raw HTTP rawPredict once to see anthropic_version and path-based model ID.
9. Intentionally use a Bedrock-style model ID once to recognize the failure mode.
10. Document which features you rely on and verify each against the Vertex support list.

12. Self-check Q&A (with answers)

Q1. How does local auth usually work for AnthropicVertex? **A.** Google Application Default Credentials after gcloud auth application-default login (or a service account in prod) — not an Anthropic API key.

Q2. What goes in the raw request body instead of/in addition to Anthropic headers? **A.** anthropic_version set to vertex-2023-10-16; model often lives in the URL path.

- Q3.** When choose `region="eu"` over `global`? **A.** When you need EU geographic data residency with multi-region availability inside the EU, accepting possible premium pricing.
- Q4.** Why might a brand-new Claude model fail in `europa-west1` but work with `global`? **A.** Regional availability lags; newest models often land on `global` / select regions first.
- Q5.** Name one difference vs Bedrock model IDs. **A.** Vertex uses publisher IDs like `claude-sonnet-4-6`; Bedrock uses `anthropic.claude-...` or `us.anthropic....` profiles.
- Q6.** Is conversation history stored by Vertex between calls? **A.** No — your application must resend messages each turn (unless you build your own session store).
- Q7.** What agent pattern sends different query types to different tools/prompts? **A.** Routing.
- Q8.** Give two reasons Converse knowledge still helps even if you deploy on Vertex. **A.** Shared Claude concepts (tools, system prompts, eval); and comparison questions on cloud exams often contrast Bedrock's Converse with Vertex's Messages-on-GCP approach.
- Q9.** When is a specific regional endpoint required? **A.** Hard single-region residency and/or provisioned throughput (per public guidance).
- Q10.** What is Model Garden's role? **A.** Discovery and enablement of partner models (including Claude) for your GCP project.
- Q11.** Why prefer service accounts in production? **A.** Least privilege, rotatable, no interactive user ADC on servers, fits workload identity.
- Q12.** Streaming on Vertex — does it change tool-loop correctness rules? **A.** No — still assemble the final message and keep `tool_use/tool_result` pairing correct.
- Q13.** Name two features that may differ from Anthropic API on Vertex. **A.** Examples: Message Batches, some Files/URL helpers, some server tools, Admin APIs — always verify current lists.
- Q14.** Hybrid RAG means what? **A.** Combining lexical (BM25) and vector retrieval before generation.
- Q15.** Parallelization vs chaining? **A.** Parallelization fans out independent work; chaining sequences dependent steps.
- Q16.** What host flavor is associated with US multi-region? **A.** Often `aiplatform.us.rep.googleapis.com` style hosts (confirm docs).
- Q17.** Can you use Claude on Vertex without enabling it in the project? **A.** No — enablement in Model Garden / project access is required.
- Q18.** Where does `max_tokens` go? **A.** In the Messages request body (SDK parameter) — required upper bound on generation.
- Q19.** Temperature for structured extraction? **A.** On older models (4.6 and earlier): lower for more deterministic outputs. Current frontier models have removed sampling params entirely — rely on schemas/structured outputs for determinism.
- Q20.** What should you check before assuming 1M context? **A.** The specific model card on Vertex — not all models share the same window.
- Q21.** Workload identity purpose in one line? **A.** Let cloud workloads obtain GCP credentials without long-lived downloaded keys.

Q22. Citations feature — exam caution? **A.** Available where documented for that model/platform; do not invent citations support.

Q23. Why might PDF/image requests fail despite valid auth? **A.** Payload size limits or unsupported media handling on Vertex path.

Q24. Bedrock `additionalModelRequestFields` analog on Vertex? **A.** Not the same API — on Vertex you generally pass Anthropic Messages fields the SDK/API supports; extras follow Anthropic/Vertex docs, not Bedrock Converse naming. —

13. Review checklist (before exam)

- ☐ Model Garden enablement steps
 - ☐ ADC vs service account vs Anthropic API key (which belongs where)
 - ☐ global vs us/eu vs regional decision tree
 - ☐ Raw HTTP: path model ID + `anthropic_version: vertex-2023-10-16`
 - ☐ Publisher ID format vs Bedrock IDs
 - ☐ Feature parity caveats vs Anthropic API
 - ☐ Stateless history + tool loop ordering
 - ☐ Agent patterns: parallelize, chain, route
 - ☐ RAG hybrid / contextual retrieval vocabulary
 - ☐ Eval = systematic code + model grading
 - ☐ Three-cloud comparison table from memory
 - ☐ Provisioned throughput tied to regional endpoints
-

14. Glossary

- **Model Garden** — GCP catalog to discover/enable partner and Google models.
 - **Agent Platform** — Google Cloud surface for agentic / partner model serving (naming in docs evolves).
 - **ADC** — Application Default Credentials for Google auth.
 - **AnthropicVertex** — Official Anthropic SDK client for Vertex-backed Claude.
 - **Publisher model ID** — e.g. `claude-sonnet-4-6` under publisher `anthropic`.
 - **rawPredict** — Vertex raw prediction endpoint for Anthropic-shaped JSON bodies.
 - **anthropic_version** — Body field; Vertex uses `vertex-2023-10-16` on raw HTTP.
 - **Global endpoint** — Dynamic multi-region routing for availability; flexible residency.
 - **Multi-region endpoint** — us or eu geo routing with geo residency.
 - **Regional endpoint** — Single-region routing; residency pin; provisioned throughput.
 - **Provisioned throughput** — Reserved capacity (typically regional).
 - **Workload identity** — Cloud workload auth without static keys.
 - **Feature parity** — Whether Vertex supports the same capabilities as Anthropic API for a feature.
 - **BM25** — Lexical retrieval component in hybrid RAG.
 - **Contextual retrieval** — Context-enriched chunks before embedding.
 - **Routing / chaining / parallelization** — Core agent workflow patterns.
 - **Code-based grading** — Programmatic eval asserts.
 - **Model-based grading** — Rubric scoring via another model call.
-

15. Worked mini-scenarios

Scenario 1 — Wrong key Engineer sets ANTHROPIC_API_KEY and wonders why AnthropicVertex fails in a VPC-SC GCP project. **Answer:** Use GCP credentials (ADC/SA). Anthropic API keys are a different product path.

Scenario 2 — Compliance Policy: process in EU. Code uses region="global" for cheaper tokens. **Answer:** Switch to eu multi-region or a specific EU regional endpoint; accept premium if applicable; verify model availability.

Scenario 3 — New model lag Opus brand-new on global; regional endpoint 404s/unavailable. **Answer:** Use global (if residency allows) or wait/enable in the needed region; do not invent Bedrock-style profile prefixes.

Scenario 4 — Raw HTTP debugging SDK works; hand-rolled curl fails. **Answer:** Check bearer token from gcloud, URL location segment, publisher path model ID, and anthropic_version in JSON.

Scenario 5 — Parity miss App depends on an Anthropic-only batch API. Port to Vertex breaks. **Answer:** Check support matrix; redesign with your own queue or stay on Anthropic API if policy allows.

Scenario 6 — Agent design Mixed intents: reset password vs order status vs FAQ. **Answer:** Routing pattern to specialized prompts/tools; evaluate each branch.

Scenario 7 — RAG on GCP Exact SKU strings missed by vector-only search. **Answer:** Hybrid BM25 + vectors; rerank; contextual retrieval; fixed eval cases with those SKUs.

Scenario 8 — Prod auth hardening User ADC JSON committed to repo for Cloud Run. **Answer:** Stop; use attached service account / workload identity; rotate exposed creds.

16. Deeper endpoint comparison table

	Global	Multi-region us/eu	Specific regional
Residency	Flexible / none guaranteed	Geo-level	Single region
Availability	Highest (dynamic)	High within geo	Depends on one region
Typical price	Baseline	Premium (often ~10% on newer models)	Premium
Provisioned throughput	Generally no	Check docs	Yes (typical home)
Best for	Most apps without residency rules	US/EU compliance + HA	Hard pin / reserved capacity

17. Study rhythm

Day 1: Setup, auth, endpoints, model IDs. Draw residency decision tree. Day 2: Feature parity, shared skills, comparison vs Bedrock/Anthropic. Day 3: Q&A closed-book + scenarios + checklist.

Mantra: Enable in Garden, auth with Google, pick endpoint for residency, use publisher IDs, assume Messages-not-Converse, verify parity before promising features.

18. Mapping shared Claude skills to Vertex request fields

Skill theme	Vertex expression
System prompt	top-level system on Messages
Temperature	temperature request field
Tools	tools / tool choice fields (Anthropic shape)
Multi-turn	App-managed messages
Streaming	SDK stream helpers / streamPredict
Structured output	Schema prompting + validation; platform features if listed
Vision / PDF	Content blocks; watch size limits
Caching	Prompt cache fields where supported on Vertex model
Thinking	Extended thinking params where supported
Agents	Your orchestration (parallel/chain/route) + optional Claude Code

Contrast with Bedrock naming: Bedrock Converse uses `inferenceConfig`, `toolConfig`, `additionalModelRequestFields`. Vertex stays closer to Anthropic Messages names via `AnthropicVertex`.

19. Failure triage tree

Auth error / 401 / 403?

- ADC present? Correct project? SA roles include Vertex AI user permissions?
- Model enabled in Model Garden for this project?

404 / model not found?

- Wrong publisher ID string?
- Model not available in chosen location?
- Accidentally used Bedrock-style ID?

Empty/truncated output?

- `max_tokens` too low; stop reasons; tool loop incomplete

Tool loop errors?

- Missing `tool_result`; role ordering; schema mismatch

Residency audit fail?

- global used under geo mandate → move to `us/eu/regional`
-

20. Extra self-check Q&A

Q25. What does `locations/global` mean in a Vertex path? **A.** Use of the global endpoint location rather than a single region like `us-east5`.

Q26. Why might finance care about `us/eu` endpoints? **A.** Premium pricing vs global for newer models — cost and compliance both matter.

Q27. Chaining example in one line? **A.** Extract entities → fetch records → draft answer, each step feeding the next.

Q28. Is MCP Vertex-specific? **A.** No — MCP is transport-agnostic; Vertex is one place you might host or consume tools alongside Claude.

Q29. Computer use / Claude Code in this course — takeaway? **A.** Agentic apps can run with Vertex-backed Claude; still need GCP auth, enablement, and verification.

Q30. Best first endpoint for a startup with no residency rules? **A.** Global — availability and baseline pricing, then revisit if compliance appears.

Q31. How do you pass project id to the SDK? **A.** `AnthropicVertex(project_id=..., region=...)` and/or documented env vars.

Q32. What is wrong with storing only the last user message each turn? **A.** Loses assistant/tool history; breaks multi-turn coherence and tool loops.

Q33. Code-based grading example for a Vertex app? **A.** Assert JSON schema, required fields, and forbidden PII patterns on outputs in CI.

Q34. Model-based grading example? **A.** Rubric prompt scoring helpfulness/groundedness on a 1–5 scale with a fixed judge prompt.

Q35. Why compare Bedrock Converse on an exam for a Vertex course? **A.** Cloud choice questions test whether you know auth, IDs, and API shapes differ even when Claude skills transfer. —

21. Production readiness mini-rubric (Vertex)

1. Model enabled in the correct project; ID verified against Model Garden card
2. Endpoint class matches residency policy (global / us / eu / regional)
3. Prod uses service account or workload identity — not laptop user ADC
4. IAM roles least-privilege for prediction only where possible
5. Timeouts/retries for stream and non-stream paths
6. Eval suite gates prompt/model changes
7. Logging redacts secrets and sensitive prompts as required
8. Cost alerts on token usage; understand premium endpoint pricing
9. Documented fallback if regional capacity fails (only if policy allows)
10. Feature dependencies checked against Vertex support matrix

22. Side-by-side request anatomy (conceptual)

Anthropic API: `host api.anthropic.com + API key header + body {model, messages, max_tokens, ...}`

Bedrock Converse: `AWS SigV4 to bedrock-runtime + {modelId, messages, inferenceConfig, toolConfig, ...}`

Vertex rawPredict: `Google bearer token to ...aiplatform.../publishers/anthropic/models/{MODEL}:rawPredict + body {anthropic_version, messages, max_tokens, ...}` (model in path)

If you can rewrite one hello-world request across all three shapes, you own the cloud comparison questions.

23. Team onboarding one-pager (what to paste in a wiki)

- 1. Enable Claude models X/Y in Model Garden for project P
- 2. Install anthropic[vertex]; set project; prefer region=global unless compliance says otherwise
- 3. Local: gcloud auth application-default login
- 4. Prod: service account with documented roles
- 5. Never commit credentials
- 6. Use publisher model IDs from the model card — do not copy Bedrock IDs
- 7. Own conversation history in our session store service
- 8. Tools must return results for every tool_use
- 9. Before using a fancy Anthropic-only feature, check Vertex support
- 10. Run the shared eval pack before promoting prompt changes

24. Quick reference card

Need	Vertex move
Auth	ADC / service account
Enable	Model Garden
Default location	global if residency flexible
US/EU residency + HA	us / eu
Hard pin / provisioned	Specific region
SDK	AnthropicVertex
Raw version field	vertex-2023-10-16
Model ID	Publisher claude-...
History	App-managed
vs Bedrock	Messages not Converse; different IDs/auth

One-sentence mantra: On Vertex, enable in Garden, authenticate with Google, choose the endpoint for residency, call Messages with publisher IDs, and verify feature parity before you promise Anthropic-API capabilities.

25. Study close

If you can redraw (1) the three endpoint classes, (2) ADC vs API key, (3) publisher vs Bedrock ID examples, and (4) the three-cloud comparison table without notes, you are ready for Vertex-differentiating items. Then refresh shared Claude skills (tools, RAG, MCP, agents, eval) — same ideas as Bedrock/Anthropic tracks, Vertex transport and constraints.

Aligned to <https://academy.claude.com/courses/claude-with-google-cloud-s-vertex-ai>. Confirm model IDs, regions, pricing premiums, and feature matrices on Google Cloud and Anthropic docs before shipping.

26. Final flash drills

Say out loud without notes:

1. Enable → ADC → endpoint class → publisher model ID → Messages call.
2. Global vs us/eu vs regional in one sentence each.
3. Three reasons Vertex differs from Anthropic API (auth, version field/path, feature parity).
4. Three reasons Vertex differs from Bedrock (IAM cloud, Converse vs Messages, ID format).
5. Name parallelization, chaining, and routing with a one-line example each.

If any drill fails, re-read only that section's tables, then retry. Prefer short closed-book loops over re-reading the entire file end-to-end.

Agentic Architecture & Orchestration — Domain 1 (27%)

Note: This edition follows the ASD-STE100 writing rules: simple present tense, active voice, one word = one meaning, sentences of 20–25 words or less. Technical names (Claude, Agent SDK, Claude Code, MCP, Task, AgentDefinition, stop_reason, hooks) are exceptions and stay as written.

Disclaimer: Original exam-prep study synthesis, aligned to the **official CCAR-F Exam Guide v1.0 (July 2026)** task statements 1.1–1.7. Also, grounded in public Anthropic docs (Claude Agent SDK, Claude Code). Verify live SDK option names against code.claude.com/docs/en/agent-sdk before exam day. Names evolve.

Why this file exists: Domain 1 is the **heaviest domain on the exam (27%)**. It drives three of the six exam scenarios (Customer Support Resolution Agent, Multi-Agent Research System, Developer Productivity). Everything here maps to a numbered task statement in the official guide.

Overview

The exam's Domain 1 tests whether you can design and reason about seven tasks. The tasks are **agentic loops** (1.1), **coordinator–subagent orchestration** (1.2), and **subagent spawning and context passing** (1.3). They also include **workflow enforcement and handoff** (1.4), **Agent SDK hooks** (1.5), **task decomposition** (1.6), and **session state, resumption, and forking** (1.7). Questions are

scenario-shaped. Production logs show a failure pattern. You select the architecturally proportionate fix.

Master mental model: an agent = model + tools + loop. The *model* decides which tool to call next from context (model-driven decision-making). The loop only executes tools and feeds the results back. *Hooks* are the deterministic policy layer wrapped around that plumbing. *Subagents* are extra isolated loops the coordinator spawns for focused work.

Key map (task statement ↔ exam verbs)

Task	You must be able to...
1.1 Agentic loops	Implement stop_reason-driven control flow. Name the anti-patterns
1.2 Coordinator–subagent	Design hub-and-spoke. Diagnose narrow decomposition. Iterative refinement
1.3 Spawning & context	Task tool + allowedTools. AgentDefinition. Explicit context passing. Parallel spawning
1.4 Workflows & handoff	Programmatic prerequisites vs prompts. Structured escalation handoffs
1.5 Hooks	PostToolUse normalization. Tool-call interception. Deterministic vs probabilistic
1.6 Decomposition	Prompt chaining vs adaptive decomposition. Per-file + integration passes
1.7 Sessions	–resume, fork_session. Stale-context judgment

Deep notes

1. The agentic loop (task 1.1)

The lifecycle every loop implements:

1. Send the request (messages + tools) to Claude.
2. Inspect **stop_reason**: "tool_use" → Claude wants tools executed. "end_turn" → Claude is done.
3. Execute the requested tool(s).
4. Append the assistant message **and** the tool_result blocks to the conversation history.
5. Send the updated history back. Repeat until end_turn.

loop:

```
response = call_claude(history, tools)
history.append(response)
if response.stop_reason == "tool_use":
    results = execute(response.tool_calls)
    history.append(user_message(tool_results=results))
    continue
if response.stop_reason == "end_turn":
    break # final answer is in response content
```

Why appending tool results matters: the model is stateless between requests. It can only reason about a tool’s output if that output is *in the conversation it receives next*. If you drop or truncate a tool_result, you silently change what the model “knows.”

Model-driven vs pre-configured: the exam distinguishes an agent from a decision tree or a fixed tool sequence that you code. An agent means Claude reasons about which tool to call next from accumulated context. Agents work well on high-ambiguity inputs (returns, billing disputes) where you cannot enumerate the path in advance. Fixed pipelines work well when the path is known (see task 1.6).

Loop termination anti-patterns (the guide names these — memorize):

Anti-pattern	Why it fails	Correct signal
Parsing natural-language signals (“I am done!”) to decide termination	Brittle string matching. Models phrase completion many ways	stop_reason == "end_turn"
Arbitrary iteration cap as the primary stopping mechanism	Cuts off legitimate long tasks. Masks real bugs	stop_reason. Keep a cap only as a safety limit
Checking whether the assistant message contains text as a completion indicator	Assistant turns can contain text <i>and</i> tool_use blocks together	stop_reason, never content shape

2. Coordinator–subagent orchestration (task 1.2)

Hub-and-spoke: one coordinator agent owns task decomposition, delegation, result aggregation, and error handling. Subagents never talk to each other directly. **All inter-subagent communication routes through the coordinator.** Why: observability (one place to log), consistent error handling (one recovery policy), controlled information flow (no unreviewed cross-talk).

Context isolation (the exam tests it heavily): subagents do **not** automatically inherit the coordinator’s conversation history. They do not share memory between invocations. Each spawned subagent starts a fresh context. That context contains only its own system prompt plus the prompt string the coordinator passed. If the synthesis subagent needs the web-search findings, the coordinator must **put those findings in the synthesis subagent’s prompt**. Nothing flows implicitly.

The coordinator’s job list:

- Analyze the query and **dynamically select** which subagents to invoke. Do not always run the full pipeline. A simple lookup should not start four specialists.
- **Partition scope** across subagents to minimize duplication (assign distinct subtopics or source types per agent).
- **Aggregate** results and evaluate the synthesis for gaps.
- **Iterative refinement loop:** if the synthesis has coverage gaps, re-delegate to search/analysis subagents with *targeted* queries, then re-invoke synthesis. Repeat until coverage is sufficient.

The signature Domain 1 failure — overly narrow decomposition (official sample Q7). The research report on “creative industries” only covers visual arts. The coordinator decomposes the topic into digital art, graphic design, and photography. Every subagent executes its assignment correctly. **The root cause is what they were assigned.** Exam reflex: when subagent logs look clean but coverage is bad, blame the coordinator’s decomposition, not the downstream agents.

Coordinator prompt design: specify **research goals and quality criteria** (“cover every major creative-industry sector. Each claim cited”) rather than step-by-step procedural instructions. Goal-shaped prompts let subagents adapt. Procedure-shaped prompts break on the first unanticipated case.

3. Spawning, AgentDefinition, and context passing (task 1.3)

The **Task tool** is the mechanism a coordinator uses to spawn subagents. The coordinator’s **allowedTools must include "Task"** or it cannot delegate at all. A common exam distractor is a delegation failure whose root cause is a missing Task entry. (Current-docs note: Claude Code renames the tool to Agent in v2.1.63. The exam guide and appendix use Task. Answer with the guide’s terminology. Know the rename exists.)

AgentDefinition configures each subagent type. These fields pass verification against the current SDK docs. The agents option of query() maps names to definitions:

Field	Purpose
description	When the coordinator should use this subagent — this is how Claude matches tasks to agents
prompt	The subagent’s system prompt : role, expertise, constraints
tools	Tool restriction list — a synthesis agent gets no web tools. A reviewer gets read-only ["Read", "Grep", "Glob"]
model	Optional per-subagent model override (e.g. cheaper model for bulk reading)

Explicit context passing skills the guide lists:

- Include **complete findings from prior agents directly in the subagent’s prompt** (e.g. pass search results and document analyses into the synthesis prompt). Use **structured data formats that separate content from metadata** (claim text vs source URL / document name / page number). The list also includes thus, attribution survives hand-offs. This arrives again in Domain 5.6.
- **Parallel spawning:** emit **multiple Task tool calls in a single coordinator response**, not one per turn. One-per-turn serializes what could run concurrently. The single-response pattern is the documented way to parallelize (official prep Exercise 4 measures the latency improvement).

Fork-based exploration belongs here conceptually (details in §7): fork a session to explore divergent approaches from a shared analysis baseline.

4. Multi-step workflows: enforcement and handoff (task 1.4)

Programmatic enforcement vs prompt guidance — the single most-tested distinction in Domain 1 (official sample Q1):

12% of cases skip get_customer before process_refund. Fix? **A programmatic prerequisite that blocks the downstream call until the prerequisite has completed.** Prompt wording and few-shot examples are probabilistic. They lower the failure rate. They never reach zero. When errors have financial or compliance consequences (“identity verification before financial operations”), only deterministic gates are acceptable.

Enforcement levels (weakest to strongest): system-prompt instruction → few-shot examples → **hook / prerequisite gate in code**. The exam rewards the proportionate level: precision problems in review prompts get criteria and few-shots (Domain 4). Compliance-critical ordering gets code.

Multi-concern requests: decompose a message that contains several distinct issues into items. Investigate each item in parallel where the items are independent, and share common context like the verified customer ID. Then synthesize **one unified resolution**. Do not answer concern 1 and silently drop concern 3.

Structured handoff protocol for escalation: the human agent who receives an escalation typically **cannot see the conversation transcript**. A compliant handoff summary carries customer ID and the concern(s). It also carries **root cause analysis so far**, amounts/order numbers involved, what the agent already attempted, and the **recommended action**. “Escalating to human” with no structured payload forces the human to restart the investigation. This is an anti-pattern. Scenario stems punish it.

5. Agent SDK hooks (task 1.5)

Hooks are callbacks the SDK fires at fixed lifecycle points. They are the **deterministic guarantee layer**. Prompt instructions give **probabilistic compliance**. This is the same advisory-versus-enforced difference as file 05’s Claude Code section, now inside the SDK.

The two hook patterns the exam names:

- 1. **PostToolUse — result transformation before the model sees it.** Different MCP backends return heterogeneous formats. One tool emits Unix epoch timestamps. Another emits ISO 8601. A third emits numeric status codes. A PostToolUse hook **normalizes tool results into one canonical format before the agent processes them**. The model never has to handle three date formats during reasoning. *(Current SDK mechanics, verified: You register hooks via the hooks option. Each event name keys its matchers. A PostToolUse callback can replace the tool’s output — updatedToolOutput — or append additionalContext.)*
- 2. **Tool-call interception — enforcing compliance rules on outgoing calls.** Intercept the call *before execution*. If it violates policy (refund > \$500), **block it and redirect to an alternative workflow** (human escalation). *(Current SDK mechanics, verified: a PreToolUse callback returns permissionDecision: "deny" with a permissionDecisionReason the model sees, or updatedInput to rewrite arguments. Deny always wins when multiple hooks fire.)*

When hooks, when prompts: business rules that require **guaranteed** compliance → hooks. Style, tone, soft preferences → prompts. If a question says “must never,” “compliance,” “regulatory,” or names a dollar threshold — the answer is a hook or gate. It is not a better sentence in the system prompt.

6. Task decomposition strategies (task 1.6)

Two families:

Strategy	Shape	Use when
Prompt chaining (fixed sequential pipeline)	Predefined step 1 → step 2 → step 3	Predictable multi-aspect work: the steps are known in advance
Dynamic adaptive decomposition	Plan emerges from intermediate findings	Open-ended investigation: what to do next depends on what you discover

Prompt-chaining exemplar (also official sample Q12 / Domain 4.6): a 14-file PR that you review in one pass produces inconsistent depth and contradictory findings. This is **attention dilution**. Fix: **per-file local analysis passes** plus a **separate cross-file integration pass**. Splitting is a decomposition answer. It is not a bigger-context-window answer (option C in Q12 is wrong precisely because larger windows do not fix attention dilution).

Adaptive exemplar: “add comprehensive tests to a legacy codebase” — you cannot enumerate the steps in advance. First **map the structure**, identify **high-impact areas**, create a prioritized plan that adapts as the agent discovers dependencies. The plan is a document that changes during the work. You regenerate it as findings arrive.

Choosing: ask: could I write the full step list before I start? Yes → chain. No → adaptive. Hybrid is common: adaptive outer plan, chained inner passes.

7. Session state, resumption, and forking (task 1.7)

- **--resume <session-name-or-id>** continues a specific prior conversation with its full context (files read, analyses done, decisions made). Verified current CLI: **--resume** accepts a session **ID or name** (or shows a picker). **--continue** selects the most recent session in the directory. In the SDK, pass `resume=<session_id>` (Python) / `resume: sessionId` (TypeScript) in the query options.
- **fork_session** creates an **independent branch from a shared analysis baseline**. The fork copies the original’s history and gets its own session ID. **The original stays unchanged**. Use it to explore divergent approaches — e.g. compare two testing strategies or two refactoring designs from one expensive codebase analysis, one fork each. Verified SDK spelling: `fork_session=True` (Python) or `forkSession: true` (TypeScript). Pass it together with `resume`.
- **Resuming after code changed:** a resumed session’s tool results describe the files as *they were*. **Tell the agent which files changed** so it re-analyzes those specifically instead of trusting stale reads. Targeted re-analysis is better than full re-exploration.
- **Resume vs fresh-with-summary:** if prior context is *mostly valid* → **resume** (cheap, keeps nuance). If prior tool results are *substantially stale* → **start a new session and inject a structured summary** of the still-valid conclusions. A fresh session with clean, current facts is more reliable than a resumed session that drags contradicted file reads which the model may still trust.

Decision trees

“The agent did not do X” triage:

Did it skip a REQUIRED step (compliance/order)?

→ Programmatic prerequisite or PreToolUse-style interception hook (1.4/1.5)

Did it pick the WRONG TOOL between similar tools?

→ Tool descriptions first (Domain 2.1), few-shots second

Did it produce INCOMPLETE COVERAGE with clean subagent logs?

→ Coordinator decomposition too narrow (1.2)

Did it FAIL TO DELEGATE at all?

→ Is "Task" in the coordinator's allowedTools? (1.3)

Did it terminate early / loop forever?

→ Check stop_reason handling; remove NL-parsing / text-presence checks (1.1)

Session continuation:

Prior context still valid?

YES → --resume (add a note listing any changed files)

MOSTLY, want to try two directions → resume + fork_session per direction

NO (stale tool results) → new session + injected structured summary

Exam traps

Trap	Fix
“Strengthen the system prompt” for a compliance-critical ordering bug	Programmatic prerequisite / hook — prompts are probabilistic
Assuming subagents see the coordinator’s conversation	They see only their system prompt + the prompt passed at spawn
Blaming downstream agents for coverage gaps	Check the coordinator’s decomposition first (sample Q7)
Spawning parallel subagents across separate turns	Emit multiple Task calls in one response
Iteration cap as the primary loop terminator	stop_reason end_turn. Cap only as backstop
Escalating with a bare “needs human” flag	Structured handoff: ID, root cause, amounts, recommended action
Resuming a session after heavy code changes without saying so	Name the changed files, or start fresh with a summary
Fork mutates the original session	It does not. Fork copies the history into a new session ID
Using hooks for tone/style preferences	Hooks for guarantees. Prompts for preferences
One giant review pass “because the context window fits it”	Attention dilution — split per-file + integration pass

Self-check Q&A

Q1. The loop should continue when stop_reason is ___ and terminate when it is ___. **A.** "tool_use". "end_turn".

Q2. Why must tool results be appended to history before the next request? **A.** The model is stateless per request. It can only incorporate results present in the conversation it receives.

Q3. Name the three loop-termination anti-patterns the exam guide lists. **A.** Parsing natural-language completion signals. Arbitrary iteration caps as the primary stop. Checking for assistant text content as a completion indicator.

Q4. Agent vs pre-configured pipeline — the defining difference? **A.** Model-driven decision-making: the agent reasons about the next tool from context. A pipeline follows a coded sequence.

Q5. In hub-and-spoke, who talks to whom? **A.** All subagent communication routes through the coordinator. This gives observability, consistent error handling, and controlled information flow.

- Q6.** Research report covers only a slice of the topic. Every subagent's log looks correct. Root cause? **A.** Coordinator's task decomposition was too narrow. Subagent assignments did not cover the topic (sample Q7 pattern).
- Q7.** What must be true of `allowedTools` for a coordinator to spawn subagents? **A.** It must include "Task" (the subagent-spawning tool. Renamed Agent in current Claude Code — exam uses Task).
- Q8.** What context does a freshly spawned subagent start with? **A.** Its own system prompt (Agent-Definition prompt) plus the prompt string the coordinator passed. No parent history. No shared memory.
- Q9.** The synthesis agent needs the search agents' findings. How does it get them? **A.** The coordinator includes the complete findings directly in the synthesis agent's prompt, ideally as structured data that separates content from source metadata.
- Q10.** How do you run three subagents in parallel? **A.** The coordinator emits three Task tool calls in a single response.
- Q11.** AgentDefinition: name its two required fields and what each does. **A.** `description` (tells the coordinator when to use this agent type) and `prompt` (the subagent's system prompt). `tools` and `model` are optional restrictions/overrides.
- Q12.** 12% of refunds skip identity verification. Best fix? **A.** A programmatic prerequisite that blocks `process_refund` until `get_customer` returns a verified ID. This is deterministic, unlike prompt/few-shot fixes (sample Q1).
- Q13.** What belongs in a structured escalation handoff? **A.** Customer ID, concern(s), root-cause analysis, amounts/order numbers, what was attempted, recommended action. The human lacks transcript access.
- Q14.** Three MCP tools return dates as epoch, ISO 8601, and strings. Where do you normalize? **A.** A `PostToolUse` hook that transforms tool results into one canonical format before the model processes them.
- Q15.** Refunds over \$500 must never execute automatically. Mechanism? **A.** A tool-call interception (`PreToolUse`-style) hook that blocks the call and redirects to human escalation. Hooks give deterministic guarantees. Prompts are probabilistic.
- Q16.** Prompt chaining vs adaptive decomposition — one-line rule? **A.** Steps enumerable in advance → chain. Plan depends on discoveries → adaptive.
- Q17.** Large multi-file review produces contradictory, inconsistent findings. Restructure how? **A.** Per-file local analysis passes plus a separate cross-file integration pass (attention dilution fix. Sample Q12).
- Q18.** "Add comprehensive tests to a legacy codebase" — first three moves? **A.** Map the structure, identify high-impact areas, build a prioritized plan that adapts as dependencies surface.
- Q19.** `fork_session` — what does the fork share with, and change about, the original? **A.** It starts with a copy of the original's history under a new session ID. The original session is untouched.
- Q20.** When is fresh-session-plus-summary better than `-resume`? **A.** When prior tool results are stale (code changed heavily). Injected summaries of valid conclusions are better than resumed contexts that contain contradicted reads.
- Q21.** You resume an investigation session after editing two files. What should your first message

include? **A.** Which files changed, so the agent re-analyzes those specifically rather than trusting stale reads or re-exploring everything.

Q22. Coordinator prompts should specify ___ rather than ___. **A.** Research goals and quality criteria. Step-by-step procedural instructions (adaptability).

Q23. When may an iteration cap legitimately exist in a loop? **A.** Only as a safety limit against loops that never stop. Never as the primary termination mechanism.

Q24. The coordinator routes every query through all four subagents, even trivial ones. Critique? **A.** Coordinators should dynamically select subagents by query complexity. Full-pipeline routing wastes latency/cost and adds failure surface.

Q25. Iterative refinement loop — describe it. **A.** Coordinator evaluates synthesis output for gaps → re-delegates targeted queries to search/analysis agents → re-invokes synthesis → repeats until coverage is sufficient.

Pre-exam checklist

- ☐ I can write the stop_reason loop from memory and name all three termination anti-patterns.
 - ☐ I can explain why subagent context isolation forces explicit context passing.
 - ☐ I know Task must appear in allowedTools (and about the Task→Agent rename).
 - ☐ I can list AgentDefinition's fields and what tool restriction is for.
 - ☐ I default to programmatic gates for compliance ordering, prompts for preferences.
 - ☐ I can specify a structured escalation handoff.
 - ☐ I can pick PostToolUse (normalize results) vs interception (block policy violations).
 - ☐ I can choose prompt chaining vs adaptive decomposition and justify per-file + integration passes.
 - ☐ I can choose between -resume, fork_session, and fresh-with-summary.
-

Glossary

Term	Meaning
Agentic loop	Request → stop_reason check → execute tools → append results → repeat
stop_reason	API field driving loop control: tool_use continue, end_turn stop
Coordinator	Hub agent owning decomposition, delegation, aggregation, error handling
Hub-and-spoke	Topology where all subagent communication passes through the coordinator
Task tool	The subagent-spawning tool (renamed Agent in current Claude Code)
allowedTools	SDK option auto-approving listed tools. Must include Task to delegate
AgentDefinition	Per-subagent config: description, prompt, tools, model

Term	Meaning
Context isolation	Subagents start fresh. Only the spawn prompt carries context in
Programmatic prerequisite	Code gate blocking a downstream tool until a prerequisite completed
PostToolUse hook	Callback transforming/normalizing a tool result before the model sees it
Tool-call interception	Pre-execution hook blocking or rewriting policy-violating calls
Prompt chaining	Fixed sequential pipeline of focused passes
Adaptive decomposition	Plan generated and revised from intermediate findings
Attention dilution	Quality degradation when one pass covers too many files/aspects
–resume	CLI/SDK option continuing a named/ID'd session with full context
fork_session	Branch a session's history into a new independent session ID
Structured handoff	Escalation payload: ID, root cause, amounts, attempts, recommendation

Companion files: 09 (Domain 5 context & reliability), 04 supplement (Domain 2 MCP mechanics), 05 supplement (Domain 3 Claude Code mechanics). Verify SDK spellings at code.claude.com/docs/en/agent-sdk before exam day.

Agentic Architecture & Orchestration — Domain 1 (27%)

Disclaimer: Original exam-prep study synthesis, aligned to the **official CCAR-F Exam Guide v1.0 (July 2026)** task statements 1.1–1.7 and grounded in public Anthropic docs (Claude Agent SDK, Claude Code). Verify live SDK option names against code.claude.com/docs/en/agent-sdk before exam day — names evolve.

Why this file exists: Domain 1 is the **heaviest domain on the exam (27%)** and drives three of the six exam scenarios (Customer Support Resolution Agent, Multi-Agent Research System, Developer Productivity). Everything here maps to a numbered task statement in the official guide.

Overview

The exam's Domain 1 tests whether you can design and reason about **agentic loops** (task 1.1), **coordinator–subagent orchestration** (1.2), **subagent spawning and context passing** (1.3), **workflow enforcement and handoff** (1.4), **Agent SDK hooks** (1.5), **task decomposition** (1.6), and **session state, resumption, and forking** (1.7). Questions are scenario-shaped: production logs show a failure pattern, and you pick the architecturally proportionate fix.

Master mental model: an agent = model + tools + loop. The *model* decides which tool to call next from context (model-driven decision-making); the *loop* is dumb plumbing that executes tools and feeds results back; *hooks* are the deterministic policy layer wrapped around that plumbing; *subagents* are extra isolated loops the coordinator spawns for focused work.

Key map (task statement ↔ exam verbs)

Task	You must be able to...
1.1 Agentic loops	Implement stop_reason-driven control flow; name the anti-patterns
1.2 Coordinator–subagent	Design hub-and-spoke; diagnose narrow decomposition; iterative refinement
1.3 Spawning & context	Task tool + allowedTools; AgentDefinition; explicit context passing; parallel spawning
1.4 Workflows & handoff	Programmatic prerequisites vs prompts; structured escalation handoffs
1.5 Hooks	PostToolUse normalization; tool-call interception; deterministic vs probabilistic
1.6 Decomposition	Prompt chaining vs adaptive decomposition; per-file + integration passes
1.7 Sessions	–resume, fork_session; stale-context judgment

Deep notes

1. The agentic loop (task 1.1)

The lifecycle every loop implements:

1. Send the request (messages + tools) to Claude.
2. Inspect **stop_reason**: "tool_use" → Claude wants tools executed; "end_turn" → Claude is done.
3. Execute the requested tool(s).
4. Append the assistant message **and** the tool_result blocks to the conversation history.
5. Send the updated history back; repeat until end_turn.

loop:

```
response = call_claude(history, tools)
history.append(response)
if response.stop_reason == "tool_use":
    results = execute(response.tool_calls)
    history.append(user_message(tool_results=results))
    continue
if response.stop_reason == "end_turn":
    break # final answer is in response content
```

Why appending tool results matters: the model is stateless between requests. It can only reason about a tool’s output if that output is *in the conversation it receives next*. Dropping or truncating a tool_result silently changes what the model “knows.”

Model-driven vs pre-configured: the exam distinguishes an agent (Claude reasons about which tool to call next based on accumulated context) from a decision tree or fixed tool sequence coded by you. Agents shine on high-ambiguity inputs (returns, billing disputes) where the path can’t be enumerated in advance; fixed pipelines shine when the path is known (see task 1.6).

Loop termination anti-patterns (the guide names these — memorize):

Anti-pattern	Why it fails	Correct signal
Parsing natural-language signals (“I’m done!”) to decide termination	Brittle string matching; models phrase completion many ways	stop_reason == "end_turn"
Arbitrary iteration cap as the primary stopping mechanism	Cuts off legitimate long tasks; masks real bugs	stop_reason; keep a cap only as a <i>safety backstop</i>
Checking whether the assistant message contains text as a completion indicator	Assistant turns can contain text <i>and</i> tool_use blocks together	stop_reason, never content shape

2. Coordinator–subagent orchestration (task 1.2)

Hub-and-spoke: one coordinator agent owns task decomposition, delegation, result aggregation, and error handling. Subagents never talk to each other directly — **all inter-subagent communication routes through the coordinator**. Why: observability (one place to log), consistent error handling (one recovery policy), controlled information flow (no unreviewed cross-talk).

Context isolation (heavily tested): subagents do **not** automatically inherit the coordinator’s conversation history or share memory between invocations. Each spawned subagent starts a fresh context containing only its own system prompt plus the prompt string the coordinator passed. If the synthesis subagent needs the web-search findings, the coordinator must **put those findings in the synthesis subagent’s prompt** — nothing flows implicitly.

The coordinator’s job list:

- Analyze the query and **dynamically select** which subagents to invoke — not always the full pipeline. A simple lookup shouldn’t wake four specialists.
- **Partition scope** across subagents to minimize duplication (assign distinct subtopics or source types per agent).
- **Aggregate** results and evaluate the synthesis for gaps.
- **Iterative refinement loop:** if the synthesis has coverage gaps, re-delegate to search/analysis subagents with *targeted* queries, then re-invoke synthesis; repeat until coverage is sufficient.

The signature Domain 1 failure — overly narrow decomposition (official sample Q7): the research report on “creative industries” only covers visual arts because the coordinator decomposed the topic into digital art / graphic design / photography. Every subagent executed its assignment correctly; **the root cause is what they were assigned**. Exam reflex: when subagent logs look clean but coverage is bad, blame the coordinator’s decomposition, not the downstream agents.

Coordinator prompt design: specify **research goals and quality criteria** (“cover every major creative-industry sector; each claim cited”) rather than step-by-step procedural instructions. Goal-shaped

prompts let subagents adapt; procedure-shaped prompts break on the first unanticipated case.

3. Spawning, AgentDefinition, and context passing (task 1.3)

The Task tool is the mechanism a coordinator uses to spawn subagents, and the coordinator's **allowedTools must include "Task"** or it cannot delegate at all — a classic exam distractor is a delegation failure whose root cause is a missing Task entry. *(Current-docs note: Claude Code renamed the tool to Agent in v2.1.63; the exam guide and appendix use Task — answer with the guide's terminology, know the rename exists.)*

AgentDefinition configures each subagent type (verified against current SDK docs — the agents option of query() maps names to definitions):

Field	Purpose
description	When the coordinator should use this subagent — this is how Claude matches tasks to agents
prompt	The subagent's system prompt : role, expertise, constraints
tools	Tool restriction list — a synthesis agent gets no web tools; a reviewer gets read-only ["Read", "Grep", "Glob"]
model	Optional per-subagent model override (e.g. cheaper model for bulk reading)

Explicit context passing skills the guide lists:

- Include **complete findings from prior agents directly in the subagent's prompt** (e.g. pass search results and document analyses into the synthesis prompt).
- Use **structured data formats that separate content from metadata** (claim text vs source URL / document name / page number) so attribution survives hand-offs — arrives again in Domain 5.6.
- **Parallel spawning**: emit **multiple Task tool calls in a single coordinator response**, not one per turn. One-per-turn serializes what could run concurrently; the single-response pattern is the documented way to parallelize (official prep Exercise 4 measures the latency win).

Fork-based exploration belongs here conceptually (details in §7): fork a session to explore divergent approaches from a shared analysis baseline.

4. Multi-step workflows: enforcement and handoff (task 1.4)

Programmatic enforcement vs prompt guidance — the single most-tested distinction in Domain 1 (official sample Q1):

12% of cases skip get_customer before process_refund. Fix? **A programmatic pre-requisite that blocks the downstream call until the prerequisite has completed.** Prompt wording and few-shot examples are probabilistic — they lower the failure rate, never to zero. When errors have financial or compliance consequences ("identity verification before financial operations"), only deterministic gates are acceptable.

Enforcement ladder (weakest → strongest): system-prompt instruction → few-shot examples → **hook / prerequisite gate in code**. The exam rewards choosing the *proportionate* rung: precision problems in review prompts get criteria and few-shots (Domain 4); compliance-critical ordering gets code.

Multi-concern requests: decompose a message containing several distinct issues into items, investigate each (in parallel where independent, sharing common context like the verified customer ID), then synthesize **one unified resolution** — don’t answer concern 1 and silently drop concern 3.

Structured handoff protocol for escalation: the human agent receiving an escalation typically **cannot see the conversation transcript**. A compliant handoff summary carries: customer ID, the concern(s), **root cause analysis so far**, amounts/order numbers involved, what was already attempted, and the **recommended action**. “Escalating to human” with no structured payload forces the human to restart the investigation — an anti-pattern the scenario stems punish.

5. Agent SDK hooks (task 1.5)

Hooks are callbacks the SDK fires at fixed lifecycle points; they are the **deterministic guarantee layer**, in contrast to prompt instructions’ **probabilistic compliance** — the same advisory-vs-enforced axis as file 05’s Claude Code section, now inside the SDK.

The two hook patterns the exam names:

- 1. **PostToolUse — result transformation before the model sees it.** Different MCP backends return heterogeneous formats: one tool emits Unix epoch timestamps, another ISO 8601, a third numeric status codes. A PostToolUse hook **normalizes tool results into one canonical format before the agent processes them**, so the model never has to juggle three date formats mid-reasoning. *(Current SDK mechanics, verified: hooks are registered via the hooks option keyed by event name with matchers; a PostToolUse callback can replace the tool’s output — updatedToolOutput — or append additionalContext.)*
- 2. **Tool-call interception — enforcing compliance rules on outgoing calls.** Intercept the call *before execution*; if it violates policy (refund > \$500), **block it and redirect to an alternative workflow** (human escalation). *(Current SDK mechanics, verified: a PreToolUse callback returns permissionDecision: "deny" with a permissionDecisionReason the model sees, or updatedInput to rewrite arguments; deny always wins when multiple hooks fire.)*

When hooks, when prompts: business rules that require **guaranteed** compliance → hooks. Style, tone, soft preferences → prompts. If a question says “must never,” “compliance,” “regulatory,” or names a dollar threshold — the answer is a hook or gate, not a better sentence in the system prompt.

6. Task decomposition strategies (task 1.6)

Two families:

Strategy	Shape	Use when
Prompt chaining (fixed sequential pipeline)	Predefined step 1 → step 2 → step 3	Predictable multi-aspect work: the steps are known in advance
Dynamic adaptive decomposition	Plan emerges from intermediate findings	Open-ended investigation: what to do next depends on what you discover

Prompt-chaining exemplar (also official sample Q12 / Domain 4.6): a 14-file PR reviewed in one pass produces inconsistent depth and contradictory findings — **attention dilution**. Fix: **per-file local analysis passes** plus a **separate cross-file integration pass**. Splitting is a decomposition answer, not a bigger-context-window answer (option C in Q12 is wrong precisely because larger windows don't fix attention dilution).

Adaptive exemplar: “add comprehensive tests to a legacy codebase” — you can't enumerate steps upfront. First **map the structure**, identify **high-impact areas**, create a **prioritized plan that adapts as dependencies are discovered**. The plan is a living artifact, regenerated as findings arrive.

Choosing: ask “could I write the full step list before starting?” Yes → chain. No → adaptive. Hybrid is common: adaptive outer plan, chained inner passes.

7. Session state, resumption, and forking (task 1.7)

- **--resume <session-name-or-id>** continues a specific prior conversation with its full context (files read, analyses done, decisions made). Verified current CLI: **--resume** accepts a session **ID or name** (or shows a picker); **--continue** grabs the most recent session in the directory. In the SDK, pass `resume=<session_id>` (Python) / `resume: sessionId` (TypeScript) in the query options.
- **fork_session** creates an **independent branch from a shared analysis baseline**: the fork copies the original's history and gets its own session ID; **the original stays unchanged**. Use it to explore divergent approaches — e.g. compare two testing strategies or two refactoring designs from one expensive codebase analysis, one fork each. Verified SDK spelling: `fork_session=True` (Python) / `forkSession: true` (TypeScript), passed *together with* `resume`.
- **Resuming after code changed:** a resumed session's tool results describe the files as *they were*. **Tell the agent which files changed** so it re-analyzes those specifically instead of trusting stale reads — targeted re-analysis beats full re-exploration.
- **Resume vs fresh-with-summary:** if prior context is *mostly valid* → **resume** (cheap, keeps nuance). If prior tool results are *substantially stale* → **start a new session and inject a structured summary** of the still-valid conclusions. A fresh session with clean, current facts is more reliable than a resumed session dragging contradicted file reads the model may still trust.

Decision trees

“The agent didn't do X” triage:

Did it skip a REQUIRED step (compliance/order)?

→ Programmatic prerequisite or PreToolUse-style interception hook (1.4/1.5)

Did it pick the WRONG TOOL between similar tools?

→ Tool descriptions first (Domain 2.1), few-shots second

Did it produce INCOMPLETE COVERAGE with clean subagent logs?

→ Coordinator decomposition too narrow (1.2)

Did it FAIL TO DELEGATE at all?

→ Is "Task" in the coordinator's allowedTools? (1.3)

Did it terminate early / loop forever?

→ Check stop_reason handling; remove NL-parsing / text-presence checks (1.1)

Session continuation:

Prior context still valid?

YES → --resume (add a note listing any changed files)

MOSTLY, want to try two directions → resume + fork_session per direction

NO (stale tool results) → new session + injected structured summary

Exam traps

Trap	Fix
“Strengthen the system prompt” for a compliance-critical ordering bug	Programmatic prerequisite / hook — prompts are probabilistic
Assuming subagents see the coordinator’s conversation	They see only their system prompt + the prompt passed at spawn
Blaming downstream agents for coverage gaps	Check the coordinator’s decomposition first (sample Q7)
Spawning parallel subagents across separate turns	Emit multiple Task calls in one response
Iteration cap as the primary loop terminator	stop_reason end_turn; cap only as backstop
Escalating with a bare “needs human” flag	Structured handoff: ID, root cause, amounts, recommended action
Resuming a session after heavy code changes without saying so	Name the changed files, or start fresh with a summary
Fork mutates the original session	It doesn’t — fork copies history into a new session ID
Using hooks for tone/style preferences	Hooks for guarantees; prompts for preferences
One giant review pass “because the context window fits it”	Attention dilution — split per-file + integration pass

Self-check Q&A

Q1. The loop should continue when stop_reason is ___ and terminate when it is ___. **A.** "tool_use"; "end_turn".

Q2. Why must tool results be appended to history before the next request? **A.** The model is stateless per request — it can only incorporate results present in the conversation it receives.

Q3. Name the three loop-termination anti-patterns the exam guide lists. **A.** Parsing natural-language completion signals; arbitrary iteration caps as the primary stop; checking for assistant text content as a completion indicator.

Q4. Agent vs pre-configured pipeline — the defining difference? **A.** Model-driven decision-making: the agent reasons about the next tool from context; a pipeline follows a coded sequence.

Q5. In hub-and-spoke, who talks to whom? **A.** All subagent communication routes through the coordinator — for observability, consistent error handling, and controlled information flow.

- Q6.** Research report covers only a slice of the topic; every subagent's log looks correct. Root cause? **A.** Coordinator's task decomposition was too narrow — subagent assignments didn't cover the topic (sample Q7 pattern).
- Q7.** What must be true of `allowedTools` for a coordinator to spawn subagents? **A.** It must include "Task" (the subagent-spawning tool; renamed Agent in current Claude Code — exam uses Task).
- Q8.** What context does a freshly spawned subagent start with? **A.** Its own system prompt (Agent-Definition prompt) plus the prompt string the coordinator passed — no parent history, no shared memory.
- Q9.** The synthesis agent needs the search agents' findings. How does it get them? **A.** The coordinator includes the complete findings directly in the synthesis agent's prompt, ideally as structured data separating content from source metadata.
- Q10.** How do you run three subagents in parallel? **A.** The coordinator emits three Task tool calls in a single response.
- Q11.** AgentDefinition: name its two required fields and what each does. **A.** `description` (tells the coordinator when to use this agent type) and `prompt` (the subagent's system prompt); `tools` and `model` are optional restrictions/overrides.
- Q12.** 12% of refunds skip identity verification. Best fix? **A.** Programmatic prerequisite blocking `process_refund` until `get_customer` returns a verified ID — deterministic, unlike prompt/few-shot fixes (sample Q1).
- Q13.** What belongs in a structured escalation handoff? **A.** Customer ID, concern(s), root-cause analysis, amounts/order numbers, what was attempted, recommended action — the human lacks transcript access.
- Q14.** Three MCP tools return dates as epoch, ISO 8601, and strings. Where do you normalize? **A.** A `PostToolUse` hook that transforms tool results into one canonical format before the model processes them.
- Q15.** Refunds over \$500 must never execute automatically. Mechanism? **A.** A tool-call interception (`PreToolUse`-style) hook that blocks the call and redirects to human escalation — hooks give deterministic guarantees; prompts are probabilistic.
- Q16.** Prompt chaining vs adaptive decomposition — one-line rule? **A.** Steps enumerable in advance → chain; plan depends on discoveries → adaptive.
- Q17.** Large multi-file review produces contradictory, inconsistent findings. Restructure how? **A.** Per-file local analysis passes plus a separate cross-file integration pass (attention dilution fix; sample Q12).
- Q18.** "Add comprehensive tests to a legacy codebase" — first three moves? **A.** Map the structure, identify high-impact areas, build a prioritized plan that adapts as dependencies surface.
- Q19.** `fork_session` — what does the fork share with, and change about, the original? **A.** It starts with a copy of the original's history under a new session ID; the original session is untouched.
- Q20.** When is fresh-session-plus-summary better than `-resume`? **A.** When prior tool results are stale (code changed heavily) — injected summaries of valid conclusions beat resumed contexts containing contradicted reads.
- Q21.** You resume an investigation session after editing two files. What should your first message

include? **A.** Which files changed, so the agent re-analyzes those specifically rather than trusting stale reads or re-exploring everything.

Q22. Coordinator prompts should specify ___ rather than ___. **A.** Research goals and quality criteria; step-by-step procedural instructions (adaptability).

Q23. When may an iteration cap legitimately exist in a loop? **A.** As a safety backstop against runaways — never as the primary termination mechanism.

Q24. The coordinator routes every query through all four subagents, even trivial ones. Critique? **A.** Coordinators should dynamically select subagents by query complexity — full-pipeline routing wastes latency/cost and adds failure surface.

Q25. Iterative refinement loop — describe it. **A.** Coordinator evaluates synthesis output for gaps → re-delegates targeted queries to search/analysis agents → re-invokes synthesis → repeats until coverage is sufficient.

Pre-exam checklist

- ☐ I can write the stop_reason loop from memory and name all three termination anti-patterns.
 - ☐ I can explain why subagent context isolation forces explicit context passing.
 - ☐ I know Task must appear in allowedTools (and about the Task→Agent rename).
 - ☐ I can list AgentDefinition's fields and what tool restriction is for.
 - ☐ I default to programmatic gates for compliance ordering, prompts for preferences.
 - ☐ I can spec a structured escalation handoff.
 - ☐ I can pick PostToolUse (normalize results) vs interception (block policy violations).
 - ☐ I can choose prompt chaining vs adaptive decomposition and justify per-file + integration passes.
 - ☐ I can choose between -resume, fork_session, and fresh-with-summary.
-

Glossary

Term	Meaning
Agentic loop	Request → stop_reason check → execute tools → append results → repeat
stop_reason	API field driving loop control: tool_use continue, end_turn stop
Coordinator	Hub agent owning decomposition, delegation, aggregation, error handling
Hub-and-spoke	Topology where all subagent communication passes through the coordinator
Task tool	The subagent-spawning tool (renamed Agent in current Claude Code)
allowedTools	SDK option auto-approving listed tools; must include Task to delegate
AgentDefinition	Per-subagent config: description, prompt, tools, model

Term	Meaning
Context isolation	Subagents start fresh; only the spawn prompt carries context in
Programmatic prerequisite	Code gate blocking a downstream tool until a prerequisite completed
PostToolUse hook	Callback transforming/normalizing a tool result before the model sees it
Tool-call interception	Pre-execution hook blocking or rewriting policy-violating calls
Prompt chaining	Fixed sequential pipeline of focused passes
Adaptive decomposition	Plan generated and revised from intermediate findings
Attention dilution	Quality degradation when one pass covers too many files/aspects
-resume	CLI/SDK option continuing a named/ID'd session with full context
fork_session	Branch a session's history into a new independent session ID
Structured handoff	Escalation payload: ID, root cause, amounts, attempts, recommendation

Companion files: 09 (Domain 5 context & reliability), 04 supplement (Domain 2 MCP mechanics), 05 supplement (Domain 3 Claude Code mechanics). Verify SDK spellings at code.claude.com/docs/en/agent-sdk before exam day.

Context Management & Reliability — Domain 5 (15%)

Note: This edition follows the ASD-STE100 writing rules: simple present tense, active voice, one word = one meaning, sentences of 20–25 words or less. Technical names (Claude, Claude Code, MCP, /compact, case-facts, scratchpad, manifest, RAG, eval, p95, TTL, JSON, QPS, SLA, Task) are exceptions and stay as written.

Disclaimer: Original exam-prep study synthesis. It aligns to the **official CCAR-F Exam Guide v1.0 (July 2026)** task statements 5.1–5.6 and public Anthropic docs.

Why this file exists: Domain 5 appears in **four of the six exam scenarios**. The scenarios are customer support, Claude Code sessions, multi-agent research, and structured extraction. The domain tests the judgment layer. You decide what to keep in context. You decide when to escalate. You decide how errors travel between agents. You decide how humans stay calibrated on model output.

Overview

The exam has six task statements.

5.1 preserve critical information across long interactions.

- 5.2 escalation and ambiguity resolution.
 - 5.3 error propagation across multi-agent systems.
 - 5.4 context in large-codebase exploration.
 - 5.5 human review workflows and confidence calibration.
 - 5.6 information provenance and uncertainty in multi-source synthesis.
- Master mental model:** context is a lossy, position-sensitive, finite buffer.

Reliability engineering here means two things.

- (a) You decide *what must never be lossy*. Those items are case facts, provenance, and error detail. You move them into **structured layers** outside free-flowing history.
- (b) You never trust a single unstructured signal to make a routing decision. Unstructured signals include model confidence, sentiment, and aggregate accuracy.

Key map

Task	Core skill
5.1	Case-facts blocks. Trim verbose tool output. Use position-aware ordering.
5.2	Explicit escalation criteria. Honor human requests. Detect policy gaps.
5.3	Structured error context. Access-failure vs empty-result. Local recovery.
5.4	Scratchpad files, subagent delegation, crash-recovery manifests, /compact
5.5	Stratified sampling. Field-level confidence calibrated on labeled sets.
5.6	Claim–source mappings. Conflict annotation. Temporal metadata.

Deep notes

1. Preserving critical information in long interactions (task 5.1)

Progressive summarization risk: each summarization pass tends to condense **numerical values, percentages, dates, and customer-stated expectations** into vague prose. Example: “a partial refund was discussed” loses “\$127.50 by Friday”. Exact transactional facts are the first things that summaries destroy.

Fix — the persistent “case facts” block: extract transactional facts into a structured block. The facts include amounts, dates, order numbers, and statuses. Include that block **in each prompt, outside the summarized history**. Summaries can stay lossy because the facts layer is lossless. For multi-issue

sessions, store structured issue data as its own context layer. That issue data includes order IDs, amounts, and per-issue status.

Lost in the middle: models process the **beginning and end** of long inputs with high reliability. They may omit findings from **middle sections**. Mitigations: put **key-findings summaries at the beginning** of aggregated inputs. Organize detail with **explicit section headers**. Do not put the decisive constraint at position 60 percent of a 100K-token prompt.

Tool-result bloat: tool results add tokens that often exceed their relevance. An order lookup returns 40+ fields when 5 fields matter. **Trim verbose tool outputs to return-relevant fields before they enter context.** In multi-agent systems, do this trim in the upstream subagents. Subagents that serve limited-context consumers downstream must return **structured data**. Return key facts, citations, and relevance scores. Do not return verbose content and reasoning chains. Also include metadata (dates, source locations, methodological context) so synthesis stays accurate.

Conversational coherence basics: pass the complete conversation history on subsequent API requests. The API is stateless. If “the agent forgot the last turn,” the client usually does not resend the history.

2. Escalation and ambiguity resolution (task 5.2)

Legitimate escalation triggers (the guide’s list):

1. The customer **explicitly requests a human**. Escalate **immediately**. Do not investigate first. If the issue is simple, you may acknowledge frustration and *offer* the fix. Escalate if they repeat the request.
2. **Policy exceptions or gaps**. The policy is ambiguous or silent on the specific request. Example: competitor price-matching when policy only covers own-site adjustments. Note: this is about *policy coverage*, not case difficulty.
3. **Inability to make meaningful progress**.

Unreliable escalation signals (tested as wrong answers):

- **Self-reported confidence scores**. These scores are poorly calibrated. The agent is confidently wrong on the hard cases (official sample Q3’s wrong option B).
- **Sentiment analysis**. Frustration does not correlate with case complexity (Q3’s wrong option D).
- The first proportionate fix for bad escalation calibration is **explicit escalation criteria with few-shot examples in the system prompt**. The examples show escalate-vs-resolve (Q3’s correct A). Apply this fix before any classifier infrastructure.

Ambiguity resolution: when a lookup returns **multiple customer matches**, instruct the agent to **request additional identifiers**. Never select by heuristic (probably the most recent account).

3. Error propagation across multi-agent systems (task 5.3)

Structured error context lets a coordinator make intelligent recovery decisions. On failure, a subagent returns **failure type, the attempted query, any partial results, and potential alternative approaches** (official sample Q8’s correct A).

The two anti-patterns (both named by the guide):

- **Silent suppression**. You return empty results that you mark as success. This hides the failure. Incomplete output can then appear as complete.

- **Terminating the whole workflow on a single failure.** This stops recovery strategies that can still succeed. Those strategies include retry-with-modified-query, alternative source, and proceed-with-partial.

Access failure vs valid empty result: a timeout (“could not reach the source”) needs a retry decision. A successful query with no matches is a real answer. These two cases are different. Error reporting must distinguish them. If you do not distinguish them, the coordinator retries empty results and accepts outages.

Local recovery first: subagents handle **transient** failures themselves (retry with backoff). They propagate only errors they **cannot resolve locally**. The propagated error includes what they attempted and any partial results. Generic statuses (“search unavailable”) hide the context that recovery needs.

Coverage annotation: when synthesis proceeds despite gaps, the output must **annotate which findings are well-supported**. It must also annotate which topic areas have gaps due to unavailable sources. Degraded coverage must be visible. Do not hide it.

4. Context in large-codebase exploration (task 5.4)

Degradation symptoms: in extended sessions the model starts to give **inconsistent answers**. It refers to “typical patterns” instead of the **specific classes from earlier in the session**. Generic-knowledge fallback is the sign that the session diluted or evicted the discovered facts.

Countermeasures (all four are exam-listed skills):

1. **Scratchpad files.** The agent maintains a file of key findings. It **references that file for subsequent questions**. This stores knowledge across context boundaries. Files are lossless. Context is not.
2. **Subagent delegation.** Spawn subagents for verbose investigations (“find all test files,” “trace refund-flow dependencies”). Exploration output stays in the subagent’s context. The main agent keeps high-level coordination clean.
3. **Phase summaries.** Summarize key findings from one exploration phase before you spawn the next phase’s subagents. **Inject the summary into their initial context.**
4. **/compact.** Reduce context during extended sessions when it fills with verbose discovery output. In Claude Code, this pairs with directed compaction — see file 05.

Crash recovery with manifests: use structured state persistence. Each agent **exports its state to a known location**. On resume, the coordinator **loads a manifest and injects it into agent prompts**. A crash then loses only the last part of one phase, not the whole run. This is the same files-outrank-context principle as scratchpads. You apply it to whole-system recovery.

5. Human review workflows and confidence calibration (task 5.5)

Aggregate metrics mask segment failure: “97% overall accuracy” can hide 60% accuracy on one document type or one field. **Validate accuracy by document type and field segment before you automate** high-confidence extractions.

Stratified random sampling: you still apply **ongoing random sampling per stratum** (document type × field) even to extractions the model marks high-confidence. This measures true error rates. It also catches **novel error patterns**. Without this sampling, drift stays invisible until an incident.

Calibrated field-level confidence: have the model output **per-field confidence scores**. Then **calibrate review thresholds on labeled validation sets**. Raw self-reported confidence is not trustworthy

(same lesson as 5.2). Calibration against labels is what makes the scores usable for routing.

Routing rule: send low-confidence extractions and **ambiguous/contradictory source documents** to human review. Put limited reviewer capacity where error probability is genuinely highest.

6. Provenance and uncertainty in multi-source synthesis (task 5.6)

Summarization destroys attribution: if you compress findings without **claim–source mappings**, the final report cannot say which source supports which claim. Fix: subagents output **structured claim–source mappings**. Each mapping holds the claim, the source URL or document name, and the relevant excerpt. Downstream agents **preserve and merge those mappings through synthesis**. Provenance is a data contract, not a prose habit.

Conflicting statistics from credible sources: **annotate the conflict with source attribution**. Never select one value arbitrarily. Never average. Document analysis completes **with conflicting values included and explicitly annotated**. The coordinator then decides reconciliation before synthesis.

Temporal metadata: require **publication/collection dates in structured outputs**. Then a 2024 figure and a 2026 figure read as a time series, not a contradiction.

Report structure: use explicit sections that distinguish **well-established findings from contested ones**. Preserve original source characterizations and methodological context. Render content by type. Use **tables for financial data, prose for news, and structured lists for technical findings**. Do not flatten everything to uniform prose.

Decision trees

“Where should this information live?”

Exact transactional fact (amount, date, ID)?

→ Case-facts block in every prompt, outside summarized history

Discovered codebase/system fact needed later?

→ Scratchpad file (re-read on demand)

Verbose raw tool output?

→ Trim to relevant fields BEFORE it enters context

Cross-phase / cross-crash state?

→ Structured state export + coordinator-loaded manifest

Who-said-what across sources?

→ Claim-source mapping preserved through every hand-off

“Escalate or resolve?”

Customer explicitly asks for a human → escalate NOW (no investigation first)

Policy silent/ambiguous on the request → escalate (policy gap)

No meaningful progress possible → escalate with structured handoff (file 08 §4)

Merely complex but in-policy and progressing → keep resolving

Multiple identity matches → ask for more identifiers, never guess

“A subagent failed — what flows upward?”

Transient? → local retry first; nothing propagates if recovered

Unrecoverable? → failure type + attempted query + partial results + alternatives

Empty result set? → report as SUCCESS with zero matches (not an error)
Synthesis proceeding anyway? → coverage-gap annotations in the output

Exam traps

Trap	Fix
Summarize everything to save tokens	Put numbers, dates, and IDs in a lossless case-facts layer first
Key constraint buried mid-prompt	Place it at the beginning or end. Use section headers (lost-in-the-middle)
Full 40-field tool payloads in history	Trim to relevant fields upstream
Route escalation on sentiment or self-confidence	Use explicit criteria plus few-shots. Calibrate confidence on labels before you trust it
Investigate first when the customer demanded a human	Escalate immediately
“Search unavailable” as the error message	Use structured error context (type, attempt, partials, alternatives)
Empty result treated as failure (or failure as empty success)	Distinguish access failure from valid empty result
One subagent timeout stops the run	Use local recovery, then partial-results propagation plus coverage notes
Trust “97% accurate” and automate	Segment by doc type × field. Use stratified sampling of high-confidence strata
Average or pick between conflicting source stats	Annotate both with sources. The coordinator reconciles
Dates dropped from findings	Require publication/collection dates in structured outputs
Session cites “typical patterns” not discovered classes	Context degradation → scratchpad + subagent delegation + /compact

Self-check Q&A

Q1. What kinds of information does progressive summarization destroy first? **A.** Exact numbers, percentages, dates, order numbers, and customer-stated expectations. Summaries condense these into vague prose.

Q2. Define the case-facts pattern. **A.** Extract transactional facts into a structured block. Include the block in every prompt outside summarized history. This is a lossless layer beneath lossy summaries.

Q3. What is lost-in-the-middle and two mitigations? **A.** Models process the beginning and end of long inputs with high reliability. They omit content from the middle. Put key-findings summaries first. Use explicit section headers.

Q4. An order lookup returns 40+ fields, 5 relevant. What do you do and when? **A.** Trim to return-relevant fields *before* the result enters context.

- Q5.** What should upstream subagents return when downstream context is tight? **A.** Structured data: key facts, citations, relevance scores, plus dates and source metadata. Do not return verbose content and reasoning chains.
- Q6.** The three legitimate escalation triggers? **A.** Explicit customer request for a human. Policy exceptions/gaps. Inability to make meaningful progress.
- Q7.** Customer demands a human on a trivially fixable issue. Sequence? **A.** Honor the request immediately. At most, acknowledge and offer the fix once. Escalate if they repeat the request. Never investigate first against their request.
- Q8.** Why are sentiment and self-reported confidence bad escalation routers? **A.** Neither correlates with case complexity. Confidence is uncalibrated. The agent is confidently wrong on hard cases (sample Q3).
- Q9.** Tool returns three customers for “John Smith.” Correct behavior? **A.** Request additional identifiers. Never select by heuristic.
- Q10.** A search subagent times out. What does the coordinator need to recover intelligently? **A.** Structured error context: failure type, attempted query, partial results, alternative approaches (sample Q8).
- Q11.** Name the two error-handling anti-patterns and the harm of each. **A.** Silent suppression (failure presented as empty success → incomplete output looks complete). Whole-workflow termination on one failure (stops viable recovery).
- Q12.** Access failure vs valid empty result — why distinguish? **A.** One needs a retry decision. The other is a real answer. If you mix them, you get wrong retries and silently accepted outages.
- Q13.** When does an error leave a subagent? **A.** Only when local recovery (for example retry for transients) fails. Propagate the error with what the subagent attempted and any partial results.
- Q14.** Synthesis ran with one source down. What must the report contain? **A.** Coverage annotations: which findings are well-supported, and which topic areas have gaps from unavailable sources.
- Q15.** Symptom that a long exploration session’s context has degraded? **A.** Inconsistent answers. The model refers to “typical patterns” instead of the specific classes from earlier in the session.
- Q16.** Four countermeasures for large-codebase context management? **A.** Scratchpad files. Subagent delegation for verbose investigation. Phase summaries injected into next-phase context. /compact.
- Q17.** Design crash recovery for a multi-agent run. **A.** Each agent exports structured state to a known location. On resume the coordinator loads the manifest and injects it into agent prompts.
- Q18.** Why is 97% overall accuracy insufficient to automate? **A.** Aggregates mask segment failure. Validate accuracy per document type and field before you reduce human review.
- Q19.** Role of stratified random sampling in a review workflow? **A.** Ongoing error-rate measurement of high-confidence extractions per stratum. This catches novel error patterns and drift.
- Q20.** How does model confidence become usable for review routing? **A.** The model outputs field-level scores. You calibrate thresholds against labeled validation sets. Do not trust raw self-report.
- Q21.** Two credible sources give different market sizes. Handling? **A.** Include both, annotated with source attribution. The coordinator decides reconciliation. Never use arbitrary selection or averaging.

- Q22.** Why require publication dates in structured outputs? **A.** So temporal differences read as change over time rather than contradiction.
- Q23.** What keeps attribution alive through multi-agent synthesis? **A.** Structured claim–source mappings (claim, source, excerpt) that every downstream agent preserves and merges.
- Q24.** How should a synthesis report treat contested findings? **A.** Separate explicit sections for well-established vs contested. Keep original source characterizations and methodology context.
- Q25.** Financial data, news, and technical findings in one report — format? **A.** Tables, prose, and structured lists respectively. Render per content type. Do not flatten.

Pre-exam checklist

- ☐ I can specify a case-facts block and say what summarization destroys.
- ☐ I can state lost-in-the-middle and its two mitigations.
- ☐ I trim tool outputs upstream. I know what subagents should return downstream.
- ☐ I can recite the three escalation triggers and the two unreliable signals.
- ☐ I can spec structured error context and both error anti-patterns.
- ☐ I distinguish access failures from empty results immediately.
- ☐ I can name all four codebase-context countermeasures plus manifest recovery.
- ☐ As a first habit, I segment accuracy and stratify sampling before I automate.
- ☐ I preserve claim–source mappings and annotate conflicts with dates.

Glossary

Term	Meaning
Case-facts block	Structured lossless facts layer included in every prompt outside history
Progressive summarization	Repeated compression of history. It destroys exact values first
Lost in the middle	Position effect: the middle of long inputs gets omitted
Tool-result trimming	Reduce payloads to relevant fields before they enter context
Escalation trigger	Human request, policy gap, or no-progress — not sentiment or confidence
Policy gap	Request the policy is silent or ambiguous on → escalate
Structured error context	Failure type + attempted query + partials + alternatives
Silent suppression	Failure returned as empty success — anti-pattern
Local recovery	Subagent retries transients before it propagates the error
Coverage annotation	Mark well-supported vs gap areas in synthesis output

Term	Meaning
Scratchpad file	Agent-maintained findings file that stores knowledge across context limits
Manifest	Coordinator-loaded structured state export that enables crash resume
Stratified sampling	Random QA sampling per doc-type/field stratum, including high-confidence
Calibration	Map model confidence to real error rates via labeled sets
Claim–source mapping	Structured claim + source + excerpt preserved through synthesis
Temporal annotation	Publication/collection dates attached to findings

Companion files: 08 (Domain 1 orchestration — handoff structure, subagent mechanics), 05 (+ supplement) for /compact and scratchpad practice in Claude Code.

Context Management & Reliability – Domain 5 (15%)

Disclaimer: Original exam-prep study synthesis, aligned to the **official CCAR-F Exam Guide v1.0 (July 2026)** task statements 5.1–5.6 and public Anthropic docs.

Why this file exists: Domain 5 threads through **four of the six exam scenarios** (customer support, Claude Code sessions, multi-agent research, structured extraction). It tests the judgment layer: what to keep in context, when to escalate, how errors travel between agents, and how humans stay calibrated on model output.

Overview

Six task statements: **5.1** preserve critical information across long interactions; **5.2** escalation and ambiguity resolution; **5.3** error propagation across multi-agent systems; **5.4** context in large-codebase exploration; **5.5** human review workflows and confidence calibration; **5.6** information provenance and uncertainty in multi-source synthesis.

Master mental model: context is a lossy, position-sensitive, finite buffer. Reliability engineering here means (a) deciding *what must never be lossy* (case facts, provenance, error detail) and moving it into **structured layers** outside free-flowing history, and (b) never trusting a single unstructured signal (model confidence, sentiment, aggregate accuracy) to make a routing decision.

Key map

Task	Core skill
5.1	Case-facts blocks, trimming verbose tool output, position-aware ordering
5.2	Explicit escalation criteria; honor human requests; policy-gap detection
5.3	Structured error context; access-failure vs empty-result; local recovery
5.4	Scratchpad files, subagent delegation, crash-recovery manifests, /compact
5.5	Stratified sampling, field-level confidence calibrated on labeled sets
5.6	Claim–source mappings, conflict annotation, temporal metadata

Deep notes

1. Preserving critical information in long interactions (task 5.1)

Progressive summarization risk: every summarization pass tends to condense **numerical values, percentages, dates, and customer-stated expectations** into vague prose (“a partial refund was discussed” losing “\$127.50 by Friday”). Exact transactional facts are precisely the things summaries destroy first.

Fix — the persistent “case facts” block: extract transactional facts (amounts, dates, order numbers, statuses) into a structured block that is included **in each prompt, outside the summarized history**. Summaries can stay lossy because the facts layer is lossless. For multi-issue sessions, persist structured issue data (order IDs, amounts, per-issue status) as its own context layer.

Lost in the middle: models reliably process the **beginning and end** of long inputs but may omit findings from **middle sections**. Mitigations: put **key-findings summaries at the beginning** of aggregated inputs; organize detail with **explicit section headers**; don’t bury the decisive constraint at position 60% of a 100K-token prompt.

Tool-result bloat: tool results accumulate tokens disproportionately to their relevance — an order lookup returns 40+ fields when 5 matter. **Trim verbose tool outputs to return-relevant fields before they enter context.** In multi-agent systems, push this upstream: subagents with limited-context consumers downstream should return **structured data (key facts, citations, relevance scores), not verbose content and reasoning chains**, and include metadata (dates, source locations, methodological context) for accurate synthesis.

Conversational coherence basics: pass the complete conversation history on subsequent API requests — the API is stateless; “the agent forgot the last turn” usually means the client didn’t resend it.

2. Escalation and ambiguity resolution (task 5.2)

Legitimate escalation triggers (the guide’s list):

1. The customer **explicitly requests a human** — escalate **immediately**, without first attempting investigation. (If the issue is simple, you may acknowledge frustration and *offer* the fix, but escalate if they reiterate.)
2. **Policy exceptions or gaps** — the policy is ambiguous or silent on the specific request (competitor price-matching when policy only covers own-site adjustments). Note: this is about *policy coverage*, not case difficulty.
3. **Inability to make meaningful progress.**

Unreliable escalation signals (tested as wrong answers):

- **Self-reported confidence scores** — poorly calibrated; the agent is confidently wrong exactly on the hard cases (official sample Q3's wrong option B).
- **Sentiment analysis** — frustration doesn't correlate with case complexity (Q3's wrong option D).
- The proportionate first fix for bad escalation calibration is **explicit escalation criteria with few-shot examples in the system prompt** demonstrating escalate-vs-resolve (Q3's correct A) — before any classifier infrastructure.

Ambiguity resolution: when a lookup returns **multiple customer matches**, instruct the agent to **request additional identifiers** — never pick heuristically ("probably the most recent account").

3. Error propagation across multi-agent systems (task 5.3)

Structured error context is what lets a coordinator make intelligent recovery decisions. On failure, a subagent returns: **failure type, the attempted query, any partial results, and potential alternative approaches** (official sample Q8's correct A).

The two anti-patterns (both named by the guide):

- **Silent suppression** — returning empty results marked as success hides the failure and risks incomplete output presented as complete.
- **Terminating the whole workflow on a single failure** — kills recovery strategies that could have succeeded (retry-with-modified-query, alternative source, proceed-with-partial).

Access failure vs valid empty result: a timeout ("couldn't reach the source" → retry decision needed) is fundamentally different from a successful query with no matches (a real answer). Error reporting must distinguish them, or the coordinator retries empty results and accepts outages.

Local recovery first: subagents handle **transient** failures themselves (retry with backoff); they propagate only errors they **cannot resolve locally**, including what was attempted and partial results. Generic statuses ("search unavailable") hide the context recovery needs.

Coverage annotation: when synthesis proceeds despite gaps, the output must **annotate which findings are well-supported and which topic areas have gaps due to unavailable sources** — degraded coverage must be visible, not silent.

4. Context in large-codebase exploration (task 5.4)

Degradation symptoms: in extended sessions the model starts giving **inconsistent answers** and referencing "typical patterns" instead of the **specific classes it discovered earlier** — generic-knowledge fallback is the tell that discovered facts have been diluted or evicted.

Countermeasures (all four are exam-listed skills):

1. **Scratchpad files** — the agent maintains a file of key findings and **references it for subsequent questions**, persisting knowledge across context boundaries. Files are lossless; context is not.
2. **Subagent delegation** — spawn subagents for verbose investigations (“find all test files,” “trace refund-flow dependencies”); exploration output stays in the subagent’s context, the main agent keeps high-level coordination clean.
3. **Phase summaries** — summarize key findings from one exploration phase before spawning the next phase’s subagents, and **inject the summary into their initial context**.
4. **/compact** — reduce context during extended sessions when it fills with verbose discovery output (in Claude Code, pairs with directed compaction — see file 05).

Crash recovery with manifests: structured state persistence = each agent **exports its state to a known location**; on resume, the coordinator **loads a manifest and injects it into agent prompts**. A crash then costs the tail of one phase, not the whole run. (Same files-outrank-context principle as scratchpads, applied to whole-system recovery.)

5. Human review workflows and confidence calibration (task 5.5)

Aggregate metrics mask segment failure: “97% overall accuracy” can hide 60% accuracy on one document type or one field. **Validate accuracy by document type and field segment before automating** high-confidence extractions.

Stratified random sampling: even extractions the model marks high-confidence get **ongoing random sampling per stratum** (document type × field) to measure true error rates and catch **novel error patterns**. Without it, drift is invisible until an incident.

Calibrated field-level confidence: have the model output **per-field confidence scores**, then **calibrate review thresholds using labeled validation sets** — raw self-reported confidence is not trustworthy (same lesson as 5.2); calibration against labels is what makes it usable for routing.

Routing rule: low-confidence extractions and **ambiguous/contradictory source documents** go to human review, prioritizing limited reviewer capacity where error probability is genuinely highest.

6. Provenance and uncertainty in multi-source synthesis (task 5.6)

Attribution dies in summarization: when findings are compressed without preserving **claim–source mappings**, the final report can’t say which source supports which claim. Fix: subagents output **structured claim–source mappings (claim, source URL/document name, relevant excerpt)** that downstream agents **preserve and merge through synthesis** — provenance is a data contract, not a prose habit.

Conflicting statistics from credible sources: **annotate the conflict with source attribution** — never arbitrarily pick one value, and never average. Document analysis completes **with conflicting values included and explicitly annotated**, letting the coordinator decide reconciliation before synthesis.

Temporal metadata: require **publication/collection dates in structured outputs** so a 2024 figure and a 2026 figure read as a time series, not a contradiction.

Report structure: explicit sections distinguishing **well-established findings from contested ones**, preserving original source characterizations and methodological context. Render content appropriately — **financial data as tables, news as prose, technical findings as structured lists** — instead of flattening everything to uniform prose.

Decision trees

“Where should this information live?”

Exact transactional fact (amount, date, ID)?

→ Case-facts block in every prompt, outside summarized history

Discovered codebase/system fact needed later?

→ Scratchpad file (re-read on demand)

Verbose raw tool output?

→ Trim to relevant fields BEFORE it enters context

Cross-phase / cross-crash state?

→ Structured state export + coordinator-loaded manifest

Who-said-what across sources?

→ Claim-source mapping preserved through every hand-off

“Escalate or resolve?”

Customer explicitly asks for a human → escalate NOW (no investigation first)

Policy silent/ambiguous on the request → escalate (policy gap)

No meaningful progress possible → escalate with structured handoff (file 08 §4)

Merely complex but in-policy and progressing → keep resolving

Multiple identity matches → ask for more identifiers, never guess

“A subagent failed — what flows upward?”

Transient? → local retry first; nothing propagates if recovered

Unrecoverable? → failure type + attempted query + partial results + alternatives

Empty result set? → report as SUCCESS with zero matches (not an error)

Synthesis proceeding anyway? → coverage-gap annotations in the output

Exam traps

Trap	Fix
Summarize everything to save tokens	Numbers/dates/IDs go in a lossless case-facts layer first
Key constraint buried mid-prompt	Beginning/end placement + section headers (lost-in-the-middle)
Full 40-field tool payloads in history	Trim to relevant fields upstream
Route escalation on sentiment or self-confidence	Explicit criteria + few-shots; calibrate confidence on labels before trusting it
Investigate first when the customer demanded a human	Escalate immediately
“Search unavailable” as the error message	Structured error context (type, attempt, partials, alternatives)
Empty result treated as failure (or failure as empty success)	Distinguish access failure from valid empty result
One subagent timeout kills the run	Local recovery, then partial-results propagation + coverage notes

Trap	Fix
Trusting “97% accurate” to automate	Segment by doc type × field; stratified sampling of high-confidence strata
Averaging or picking between conflicting source stats	Annotate both with sources; coordinator reconciles
Dates dropped from findings	Publication/collection dates required in structured outputs
Session cites “typical patterns” not discovered classes	Context degradation → scratchpad + subagent delegation + /compact

Self-check Q&A

Q1. What kinds of information does progressive summarization destroy first? **A.** Exact numbers, percentages, dates, order numbers, and customer-stated expectations — condensed into vague prose.

Q2. Define the case-facts pattern. **A.** Extract transactional facts into a structured block included in every prompt outside summarized history — a lossless layer beneath lossy summaries.

Q3. What is lost-in-the-middle and two mitigations? **A.** Reliable processing at beginning/end of long inputs but omissions from the middle; put key-findings summaries first and use explicit section headers.

Q4. An order lookup returns 40+ fields, 5 relevant. What do you do and when? **A.** Trim to return-relevant fields *before* the result enters context.

Q5. What should upstream subagents return when downstream context is tight? **A.** Structured data — key facts, citations, relevance scores, plus dates/source metadata — not verbose content and reasoning chains.

Q6. The three legitimate escalation triggers? **A.** Explicit customer request for a human; policy exceptions/gaps; inability to make meaningful progress.

Q7. Customer demands a human on a trivially fixable issue. Sequence? **A.** Honor it immediately; at most acknowledge and offer the fix once, escalating if they reiterate — never investigate first against their request.

Q8. Why are sentiment and self-reported confidence bad escalation routers? **A.** Neither correlates with case complexity; confidence is uncalibrated — the agent is confidently wrong on hard cases (sample Q3).

Q9. Tool returns three customers for “John Smith.” Correct behavior? **A.** Request additional identifiers; never select heuristically.

Q10. A search subagent times out. What does the coordinator need to recover intelligently? **A.** Structured error context: failure type, attempted query, partial results, alternative approaches (sample Q8).

Q11. Name the two error-handling anti-patterns and the harm of each. **A.** Silent suppression (failure dressed as empty success → incomplete output looks complete) and whole-workflow termination on one failure (kills viable recovery).

- Q12.** Access failure vs valid empty result — why distinguish? **A.** One needs a retry decision, the other is a real answer; conflating them causes wrong retries and silently accepted outages.
- Q13.** When does an error leave a subagent? **A.** Only when local recovery (e.g. retry for transients) failed — propagated with what was attempted and partial results.
- Q14.** Synthesis ran with one source down. What must the report contain? **A.** Coverage annotations: which findings are well-supported, which topic areas have gaps from unavailable sources.
- Q15.** Symptom that a long exploration session's context has degraded? **A.** Inconsistent answers and appeals to “typical patterns” instead of the specific classes discovered earlier.
- Q16.** Four countermeasures for large-codebase context management? **A.** Scratchpad files; subagent delegation for verbose investigation; phase summaries injected into next-phase context; /compact.
- Q17.** Design crash recovery for a multi-agent run. **A.** Each agent exports structured state to a known location; on resume the coordinator loads the manifest and injects it into agent prompts.
- Q18.** Why is 97% overall accuracy insufficient to automate? **A.** Aggregates mask segment failure — accuracy must be validated per document type and field before reducing human review.
- Q19.** Role of stratified random sampling in a review workflow? **A.** Ongoing error-rate measurement of high-confidence extractions per stratum, catching novel error patterns and drift.
- Q20.** How does model confidence become usable for review routing? **A.** Model outputs field-level scores; thresholds are calibrated against labeled validation sets — raw self-report is not trusted.
- Q21.** Two credible sources give different market sizes. Handling? **A.** Include both, annotated with source attribution; the coordinator decides reconciliation — never arbitrary selection or averaging.
- Q22.** Why require publication dates in structured outputs? **A.** So temporal differences read as change over time rather than contradiction.
- Q23.** What keeps attribution alive through multi-agent synthesis? **A.** Structured claim–source mappings (claim, source, excerpt) that every downstream agent preserves and merges.
- Q24.** How should a synthesis report treat contested findings? **A.** Separate explicit sections for well-established vs contested, keeping original source characterizations and methodology context.
- Q25.** Financial data, news, and technical findings in one report — format? **A.** Tables, prose, and structured lists respectively — render per content type, don't flatten.
-

Pre-exam checklist

- ☐ I can spec a case-facts block and say what summarization destroys.
 - ☐ I can state lost-in-the-middle and its two mitigations.
 - ☐ I trim tool outputs upstream and know what subagents should return downstream.
 - ☐ I can recite the three escalation triggers and the two unreliable signals.
 - ☐ I can spec structured error context and both error anti-patterns.
 - ☐ I distinguish access failures from empty results on sight.
 - ☐ I can name all four codebase-context countermeasures + manifest recovery.
 - ☐ I reflexively segment accuracy and stratify sampling before automating.
 - ☐ I preserve claim–source mappings and annotate conflicts with dates.
-

Glossary

Term	Meaning
Case-facts block	Structured lossless facts layer included in every prompt outside history
Progressive summarization	Repeated compression of history; destroys exact values first
Lost in the middle	Position effect: middle of long inputs gets omitted
Tool-result trimming	Reducing payloads to relevant fields before context entry
Escalation trigger	Human request, policy gap, or no-progress — not sentiment/confidence
Policy gap	Request the policy is silent/ambiguous on → escalate
Structured error context	Failure type + attempted query + partials + alternatives
Silent suppression	Failure returned as empty success — anti-pattern
Local recovery	Subagent retries transients before propagating
Coverage annotation	Marking well-supported vs gap areas in synthesis output
Scratchpad file	Agent-maintained findings file persisting across context limits
Manifest	Coordinator-loaded structured state export enabling crash resume
Stratified sampling	Random QA sampling per doc-type/field stratum, incl. high-confidence
Calibration	Mapping model confidence to real error rates via labeled sets
Claim–source mapping	Structured claim + source + excerpt preserved through synthesis
Temporal annotation	Publication/collection dates attached to findings

Companion files: 08 (Domain 1 orchestration — handoff structure, subagent mechanics), 05 (+ supplement) for /compact and scratchpad practice in Claude Code.

CCAR-F Pro Tips — how to take this exam well

Companion to files 03–05 (+ supplements), 08, 09; distilled 2026-08-23 from the official Exam Guide v1.0 and this pack. Exam craft only.

What this exam actually rewards: diagnosing agentic-system failures from production evidence and choosing the *proportionate* fix. The recurring axis is **deterministic vs probabilistic**: prompts and few-shots lower failure rates; gates, hooks, and code make them zero. Every time a stem mentions compliance, ordering, money, or “must never,” the graded answer lives on the deterministic side.

The numbers that drive strategy

- **60 items / 120 min → 2 min each**, but the items cluster into **4 scenarios drawn from a bank of 6 — and all 6 are printed in your exam guide**. This is the biggest legal advantage on any of the four exams: study the scenarios themselves (Customer Support Resolution Agent, Code Generation with Claude Code, Multi-Agent Research System, Developer Productivity, Claude Code for CI, Structured Data Extraction). Walk into the room already knowing the four-ish systems you'll be debugging.
- **Domain 1 is 27%** — the Agent SDK orchestration file (08) carries more weight than any two other files combined. D3 + D4 are 20% each.
- **720 scaled, no guessing penalty, multiple-response all-or-nothing.**
- **The out-of-scope list is a weapon:** cloud-provider config, streaming implementation, pricing/rate limits, embeddings, prompt-caching internals, computer use, vision are all excluded. An option resting on one of these is a distractor by definition — kill it without deliberation.

If the stem says X → think Y

Stem signal	Reflex
Loop won't stop / stops early / parses "I'm done!"	stop_reason (tool_use continue, end_turn stop) — never string matching, never an iteration cap as primary
Report has clean subagent logs but bad coverage	Coordinator decomposition too narrow — blame the assignments, not the workers (sample Q7)
Agent "can't delegate"	"Task" missing from the coordinator's allowedTools
Subagent "doesn't know" something the coordinator learned	Context isolation — findings must be passed in the spawn prompt ; nothing flows implicitly
Steps skipped in a compliance/financial sequence	Programmatic prerequisite / PreToolUse-style gate , never "strengthen the prompt" (sample Q1)
Heterogeneous tool-result formats confusing the agent	PostToolUse hook normalizing results before the model sees them
"Must never execute above \$N"	Tool-call interception → block + redirect to human escalation
Inconsistent depth / contradictions in a big multi-file review	Attention dilution → per-file passes + separate integration pass — NOT a bigger context window (sample Q12)
Resume after heavy code changes	Tell it which files changed, or fresh session + injected summary — stale tool results are poison
Explore two designs from one expensive analysis	fork_session per direction; the original is untouched
Team's Claude Code behavior differs per developer	Project-level config (.claude/commands/, project CLAUDE.md, rules) vs someone's user-level file — version control is the tell
Convention spans scattered files	.claude/rules/ with paths: globs — beats per-directory CLAUDE.md and inference from headers

Stem signal	Reflex
Escalating to a human	Structured handoff payload (ID, root cause, amounts, attempts, recommendation) — the human can't see the transcript

Terminology discipline

The guide (and therefore the item bank) says **Task tool** — answer with the guide's vocabulary even though current Claude Code renamed it Agent. Same principle everywhere: this exam is graded against Exam Guide v1.0 terms; file 08 flags the current-docs deltas so you're never surprised, but don't "correct" the exam.

Distractor archetypes to expect

- **The prompt patch** for a determinism problem ("add to the system prompt that it must always..."). Lowers, never eliminates — wrong when the stem says must/never/compliance.
- **The bigger window** for attention dilution or coverage problems.
- **The out-of-scope lure:** Bedrock IAM, pricing tiers, embedding stores — plausible-sounding, explicitly excluded.
- **The blame-shift:** fixing the subagent when the coordinator's decomposition or the missing context hand-off is the root cause.
- **The transcript assumption:** answers that assume a human or subagent can see conversation history it was never given.

Freshness watchlist (re-verify the week of your exam)

Exact SDK option spellings (agents, fork_session/forkSession, hook callback fields) at code.claude.com/docs/en/agent-sdk — file 08's footer lists them. Five minutes; the concepts won't move, the spellings might.

The 48-hour plan

1. Re-read 08 end-to-end, then 09 (together = 42% of the form).
2. Re-read the two supplements (04's Domain-2 mechanics, 05's Domain-3 mechanics) — these are dense fact sections; recency helps.
3. Walk each of the 6 printed scenarios and predict its failure modes from the reflex table above — this is the closest thing to seeing the paper early.
4. Re-do the 12 sample questions cold; every one should now feel mechanical.
5. Skip files 06–07 entirely (out of scope) and 01–02 (background).

Day-of (online proctored)

ID, clean desk, room scan, stable connection. When a scenario block opens, spend 30 seconds identifying which of the 6 published scenarios it is — your pre-walked failure-mode map then prices every question in it. Deterministic beats probabilistic; coordinator before workers; guide terminology over current docs. Answer everything.